Public Interest Data Science

Pressures, Values, Lineages, and Genres

Brian C. Keegan

2025-12-31

Table of contents

Preface **1**
Who this book is for . . . 1
What the book does . . . 2
How the book is built . . . 3
Running examples . . . 4
What the book does not do . . . 4
How to contribute . . . 6
A note on the AI-assisted draft zero . . . 6
Acknowledgments . . . 6

I. Pressures **9**

1. Introduction to Public Interest Data Science **11**
1.1. What does "public interest" even mean . . . 11
1.2. Relative to neighbors . . . 12
1.3. The arc of the book . . . 13
1.4. A first request . . . 14
1.5. The Wikimedia pageviews API . . . 15
1.6. Into pandas . . . 17
1.7. Reading the plot . . . 19
1.8. What you will build . . . 20
1.9. Exercises . . . 21
1.10. Looking ahead . . . 22
1.11. Further Reading and Resources . . . 22

2. Enclosure: Who Can Observe? 25
2.1. The last ten years, briefly . . . 26
2.2. Reading what the platform tells you you can do . . . 27
2.3. A responsible scraper, from scratch . . . 29
2.4. A worked timeline: Reddit access, 2015–2024 . . . 33
2.5. The asymmetry of observation . . . 36
2.6. Exercises . . . 37
2.7. Looking ahead . . . 38
2.8. Further Reading and Resources . . . 39

3. Exemption: Who Must Answer? 41
3.1. Two kinds of exemption . . . 42
3.2. Allegheny Family Screening Tool, as a public object . . . 43
3.3. The evidence ledger . . . 44
3.4. Reading the deployed model from the outside . . . 47
3.5. When the state is the adversary: SyRI and after . . . 48
3.6. Practical exemption as a design choice . . . 50
3.7. Exercises . . . 51
3.8. Looking ahead . . . 52
3.9. Further Reading and Resources . . . 53

4. Erosion: What Can Be Remembered? 55
4.1. The mundane politics of retention . . . 56
4.2. A link-rot audit you can run this afternoon . . . 57
4.3. Recovering the rot with the Wayback Machine . . . 60
4.4. Datasheets as a retention practice . . . 63
4.5. Why “just put it on GitHub” is not a preservation strategy 66
4.6. Closing the triptych . . . 66
4.7. Exercises . . . 67
4.8. Looking ahead . . . 68
4.9. Further Reading and Resources . . . 68

II. Values 71

5. The Installed Base 73
5.1. Six elements . . . 74
5.2. What goes wrong when an element is missing . . . 75
5.3. A minimum viable installed base . . . 77
5.4. What this demonstrates . . . 84
5.5. Exercises . . . 85
5.6. Looking ahead . . . 86
5.7. Further Reading and Resources . . . 86

6. Openness 89
6.1. Posting is not publishing . . . 90
6.2. Licensing as a governance choice . . . 91
6.3. The running example: Boulder campaign finance . . . 91
6.4. Writing the README . . . 95
6.5. The datasheet . . . 96
6.6. Publishing with Datasette . . . 97
6.7. Exercises . . . 101
6.8. Looking ahead . . . 102
6.9. Further Reading and Resources . . . 102

7. Oversight 105
7.1. Routine auditability versus episodic audits . . . 106
7.2. A disaggregated audit in `fairlearn` . . . 108
7.3. Writing the audit for a non-technical reader . . . 112
7.4. What survives political change . . . 114
7.5. Exercises . . . 115
7.6. Looking ahead . . . 116
7.7. Further Reading and Resources . . . 117

8. Ownership 119
8.1. Two senses of the word . . . 120
8.2. Data trusts, briefly . . . 121

8.3. Indigenous data sovereignty as a foundational challenge . . 121
8.4. The tutorial: tiering the Boulder campaign finance file . . . 122
8.5. Why "just use GitHub" is stewardship outsourcing 129
8.6. Exercises 131
8.7. Looking ahead 132
8.8. Further Reading and Resources 132

III. Lineages **135**

9. Journalism **137**
9.1. The watchdog role, operationalized 138
9.2. FOIA as infrastructure 139
9.3. Cross-border collaboration as a scaling strategy 140
9.4. The running example: reproducing *Machine Bias* 141
9.5. Flores, Bechtel, and Lowenkamp: the rejoinder 145
9.6. What academic researchers can learn 146
9.7. Looking ahead 147
9.8. Exercises 147
9.9. Further Reading and Resources 149

10. Law **151**
10.1. Public interest lawyering as a profession 151
10.2. FOIA, CORA, and the statutory geometry 152
10.3. The CFAA and the chilling effect 154
10.4. SyRI, discovery, and weaponization 155
10.5. Amicus briefs and MuckRock 156
10.6. Tutorial: a CORA tracker in SQLite 157
10.7. Drafting the letter 160
10.8. Analyzing a small corpus of released records 161
10.9. Looking ahead 163
10.10 Further Reading and Resources 164
10.11 Exercises 165

11. Accounting and Auditing **167**
11.1. From financial audit to algorithmic audit 168
11.2. The running example: wrapping the Chapter 7 audit 170
11.3. Running the pipeline, twice 179
11.4. The regulatory landscape: LL144 and the EU AI Act 180
11.5. Exercises 181
11.6. Looking ahead 182
11.7. Further Reading and Resources 183

12. Planning **185**
12.1. A profession that learned from its mistakes 185
12.2. Back to the Allegheny Family Screening Tool 187
12.3. A running example: the Central 70 project 187
12.4. Stakeholders as a structured document 188
12.5. Who was missing from the room 190
12.6. The geographic imagination 191
12.7. Community-informed categorization 194
12.8. Exercises 195
12.9. Looking ahead 197
12.10Further Reading and Resources 197

13. Engineering **199**
13.1. The engineer's duty to the public 200
13.2. Normalization of deviance 200
13.3. Software engineering as inheritor 201
13.4. From the Ch 11 audit to a tested pipeline 202
13.5. Deliberately introducing a bug 207
13.6. The post-incident review 208
13.7. Continuous integration 209
13.8. Exercises 211
13.9. Looking ahead 212
13.10Further Reading and Resources 213

IV. Adjacent Conversations 215

14. Public Interest Technology and Data for Good 217
14.1. The PIT University Network and its framing text 218
14.2. The rhythm of foundation funding 219
14.3. Two running examples: GetCalFresh and Ushahidi 220
14.4. A sustainability audit: querying GitHub 221
14.5. Interpreting the audit 225
14.6. Ghost work and the invisible labor 226
14.7. Exercises 227
14.8. Looking ahead 228
14.9. Further Reading and Resources 229

15. GovTech and Civic Tech 231
15.1. Five models on one map 232
15.2. Procurement is where the governance is 233
15.3. Five clauses worth writing 235
15.4. A small Census API exercise 238
15.5. Why the distinction between GovTech and civic tech matters 240
15.6. Exercises 241
15.7. Looking ahead 242
15.8. Further Reading and Resources 242

16. Critical Data Studies, Data Justice, and Refusal 245
16.1. The starting canon: boyd and Crawford 246
16.2. The political work of classification 246
16.3. Data justice and the question of for whom 247
16.4. Refusal as design 248
16.5. The installed base, revisited 249
16.6. Running example: Denver HMIS 250
16.7. Technical component 1: the category audit 251
16.8. Technical component 2: the refusal specification 253
16.9. Refusal in the wild: the Algorithmic Ecology 255
16.10 Exercises 256

16.11 Looking ahead 257
16.12 Further Reading and Resources 257

V. Genres **259**

17. Op-Eds **261**
17.1. Why the op-ed matters, and why it does not 261
17.2. A short history of a strange page 262
17.3. Who gets published where 263
17.4. The anatomy of an 800-word argument 264
17.5. Pitching is half the work 265
17.6. A worked example: the Boulder audit as an op-ed 266
17.7. Why this matters for the book's argument 268
17.8. Exercises 269
17.9. Looking ahead 271
17.10 Further Reading and Resources 271

18. Reports **273**
18.1. A short political economy of the think-tank report 273
18.2. Reports that inform and reports that mobilize 274
18.3. The executive summary is the report 275
18.4. Methods transparency as a credibility move 276
18.5. The translation gap 277
18.6. Quarto as a report stack 278
18.7. Institutional audiences, named 281
18.8. Reports across the lineages 282
18.9. Exercises 283
18.10 Looking ahead 284
18.11 Further Reading and Resources 285

19. Research Proposals **287**
19.1. The grant-proposal genre landscape 287
19.2. The political economy of what gets funded 289

19.3. The specific-aims page as sales pitch 289
19.4. Budgets and justifications 291
19.5. The data management plan 292
19.6. The IRB protocol as a sibling genre 293
19.7. Community-engaged research as a proposal-design problem 294
19.8. Broader impacts, taken seriously 295
19.9. Exercises 296
19.10 Looking ahead 298
19.11 Further Reading and Resources 298

20. Testimony **301**
20.1. What testimony is and is not 302
20.2. The scenario 304
20.3. A pandas detour that earns one sentence 305
20.4. The five-minute oral statement 306
20.5. The written addendum for the record 307
20.6. Anticipating Q&A 308
20.7. Practice delivery and timing 309
20.8. Closing and what comes next 310
20.9. Exercises 310
20.10 Further Reading and Resources 311

21. Public Comments **313**
21.1. What a public comment is 313
21.2. Why data scientists underuse this genre 314
21.3. State and local equivalents 315
21.4. The substantive comment standard 316
21.5. The regulations.gov API 317
21.6. Fetching and reading an NPRM 319
21.7. Tracking the docket over time 320
21.8. The shape of a substantive comment 321
21.9. Coalition comments and individual voice 323
21.10 A note on timing and the regulatory calendar 324
21.11 Exercises 324

21.12 Looking ahead . . . 326
21.13 Further Reading and Resources . . . 326

22. Archival Deposits **329**
22.1. Archives and climate, archives and memory . . . 330
22.2. The institutional landscape . . . 330
22.3. Federated archiving as political strategy . . . 332
22.4. The running example: EPA Toxics Release Inventory . . . 332
22.5. Preservation-appropriate formats . . . 333
22.6. Using the Internet Archive client . . . 333
22.7. Minting a DOI with Zenodo . . . 335
22.8. A preservation plan . . . 337
22.9. Weaving the book together . . . 338
22.10 Verifying and citing . . . 339
22.11 Exercises . . . 340
22.12 A coda . . . 341
22.13 Further Reading and Resources . . . 342

Appendices **343**

A. Appendix A: AI Assistance Disclosure **343**
A.1. What was AI-assisted . . . 343
A.2. What was not AI-assisted . . . 343
A.3. How to verify . . . 344
A.4. Why disclose . . . 344
A.5. Versioning . . . 345

B. Appendix B: Further Resources **347**
B.1. Cross-cutting tools . . . 347
B.2. Communities and networks . . . 348
B.3. Training venues . . . 348
B.4. Critical readings you did not meet in this book . . . 348
B.5. When things change . . . 349

Preface

This is a book about doing data science for ends the public can name, contest, and use.

You already know, or will shortly, that most data science is not that. It produces things for an employer to ship, a platform to monetize, or a manager to track. When it produces things for the public, it tends to produce dashboards: objects that look like accountability without being accountable to anyone in particular. This book is written for the student who has noticed the gap between what data science can do and what it typically does, and who wants a way in that is neither naive about the tools nor resigned about the politics.

Who this book is for

If you are a student taking a data-science course, a journalism or policy student learning computational methods, a working practitioner considering a move into civic or public-interest work, an instructor looking for a course companion, or a public-sector analyst being asked to do "data-driven" work with no time to read the entire literature, this book is for you. It assumes introductory Python, comfort with the command line, and a willingness to read ten pages about a court case before writing the function that acts on it. It does not assume a background in science and technology studies, law, or accounting. It will give you enough of each to keep going.

You can read it straight through, pick chapters relevant to a specific project, or use it as a course text. Instructors using it as a course companion will find that the five parts map cleanly onto a sixteen-week semester with room for two review weeks and a final project.

What the book does

The book works through a conceptual arc in five parts:

1. **Pressures**. The structural forces that constrain public knowledge: **enclosure** (access withdrawn), **exemption** (accountability legally or practically avoided), **erosion** (infrastructure and memory decaying through neglect).
2. **Values**. The countervailing commitments that hold an "installed base" of accountability together: **openness**, **oversight**, and **ownership**. Chapter 5 introduces the six elements of the installed base (linkability, interpretability, continuity, safe scrutiny, authority, and remedy), and the next three chapters build out each value in turn.
3. **Lineages**. What data scientists can inherit from the professions that have worked out public-interest practice over decades: **journalism** (watchdog methods and provenance), **law** (records, discovery, legal infrastructure), **accounting** (independent audit, reproducibility, change history), **planning** (stakeholder process and geographic imagination), and **engineering** (safety, testing, incident review).
4. **Adjacent Conversations**. Neighboring fields whose relationship to public interest data science is close and complicated: **public interest technology** and **data for good**, **GovTech** and **civic tech**, and **critical data studies, data justice, and refusal**.
5. **Genres**. The forms through which data scientists can actually contribute to public conversations: **op-eds**, **reports**, **research proposals**, **testimony**, **public comments** on rulemakings, and **archival deposits**.

Parts I through IV teach you to see. Part V teaches you to act.

How the book is built

Every chapter interweaves conceptual narrative with runnable Python tutorials at roughly a 50/50 balance. You will read two or three pages of argument, then read and run two or three pages of code that makes the argument visible in a specific case, then return to the argument. This is deliberate. Concepts without technique produce earnest seminars. Technique without concepts produces dashboards that nobody trusts.

Code blocks do not run at build time. They are illustrative: you are expected to type them, adapt them, and run them yourself. The book's repository contains test fixtures, sample data, and the occasional larger notebook that a chapter only excerpts. When a chapter uses a real API (Wikimedia pageviews, regulations.gov, the Internet Archive), the endpoints cited were live as of the book's most recent release; if you find one that has moved, open an issue.

Every chapter has **at least one "Missing Manual" callout** flagging something that textbooks usually omit: a surprising library default, a real-world API friction, the politics of a "simple" choice. Every chapter has **at least one "In the Public Interest" callout** tying the technical material back to openness, oversight, or ownership. Every chapter ends with **4–5 graduated exercises**, from guided ("follow these steps") to open-ended ("design a pipeline that…"), and a curated list of further resources.

Chapters in Part V have one additional rule: they must end with a real submittable artifact. Not a thought experiment. An actual draft op-ed, report section, proposal page, testimony statement, public comment on a live rulemaking, or archival deposit with a resolvable identifier. A student who finishes Part V has a portfolio.

Running examples

To give the book coherence across 22 chapters, a small set of running examples is threaded throughout. When a chapter needs a case, it reaches for one of these before inventing a new one:

- Boulder and Denver municipal records (meeting minutes, agendas, budgets, permits, 311 requests).
- Federal environmental data (EPA, NOAA, USGS) as sources for at-risk datasets.
- Census and FRED APIs for demographic and economic context.
- Wikipedia and Internet Archive APIs for archival work and link-rot audits.
- Hugging Face model cards and datasheets for audit and documentation exercises.
- A small synthetic worker-log fixture that lets us teach audit methods without violating anyone's privacy.
- Public FOIA and CORA request data as a class-wide shared resource.
- regulations.gov for Chapter 21 and any chapter touching federal rulemaking.

By the end of the book you will have built, in parts, a small public-interest data infrastructure of your own: a cleaned and documented civic dataset published with Datasette, a disaggregated audit report, a refusal specification, an op-ed draft, a report with reproducible figures, a research proposal, a piece of testimony, a public comment on a live rulemaking, and an archival deposit on the Internet Archive or Zenodo.

What the book does not do

Honesty up front. This book is partial in several ways that matter.

It is **US-centric**. Most of the legal and administrative infrastructure discussed is American: FOIA and CORA, the Administrative Procedure Act, NIST, NSF, NYC Local Law 144. Where European, Latin American, and Global South counterparts exist (the Dutch SyRI case, IndiaStack, Ushahidi, the CARE Principles, Te Mana Raraunga), the book names them and points you to further reading, but it does not pretend to do them justice in three pages.

It is **comfortable with records and less comfortable with refusal**. The installed-base framework takes linkability, durable identifiers, and retention as load-bearing. Chapter 16 takes that comfort seriously as a limit, drawing on Zong and Matias, D'Ignazio and Klein, the CARE Principles, and Indigenous data sovereignty scholarship. Readers who want to start from refusal and arrive at records (rather than the other way around) should read Chapter 16 early and often.

It is **thin on the labor inside data work**. Gray and Suri's *Ghost Work* is cited, and Chapter 14 tries to name invisible maintenance labor explicitly, but a book that seriously treated data-labor economics would look quite different. Consider this an invitation to the reader, or to the next edition.

It is **thin on measurement theory**. You will not find a full psychometric critique of the audit metrics in Chapter 7, and you will not find a deep philosophy-of-science treatment of what it means for a model to "work." Those books exist and are cited.

It **over-relies on the professional-lineage framing**. Treating journalism, law, accounting, planning, and engineering as lineages the field can inherit from is a useful move, but every profession has its own pathologies, and the book does not do equal-time analysis of all of them. Read the criticisms. They are in the further-reading lists.

These limits are not bugs to be patched. They are framing choices that enable the book to cohere. Knowing where they bind is part of what you are learning.

How to contribute

This book lives in a git repository. Fork it, improve it, and open a pull request. Before you do, read `claude.md`, which codifies the voice, structure, and citation conventions. Typo fixes and small corrections can come in as pull requests directly. Rewrites of sections, new examples, or proposed new chapters should start as an issue so we can talk about them first.

Students whose contributions shape the book are named in the acknowledgments. Some of the best chapters in future editions will be written by readers who encountered the first edition and saw what was missing.

A note on the AI-assisted draft zero

The first complete draft of this book was produced with substantial assistance from Claude, Anthropic's assistant, following a detailed editorial prompt written by the author. The prompt is preserved in the repository; the disclosure statement is in Appendix A. Every chapter has been reviewed and revised by the author. A book about public-interest data science should not hide how it was made, any more than a model card should hide its training data.

Acknowledgments

The framework owes its shape to years of conversation with colleagues in the INFO department at CU Boulder, students in previous versions of INFO 46XX who asked the questions this book tries to answer, and the infrastructure studies and STS communities whose citations fill the bibliography. Specific individuals will be named once the revision cycle is complete. Any errors are the author's, and there will be errors. File an issue.

Now to Chapter 1.

Part I.

Pressures

1. Introduction to Public Interest Data Science

Open a newspaper any week and you will find a story about a data-driven system that has shaped a public decision in a way the public did not choose. A county's child-welfare office screens families with an algorithm. A city's benefits portal denies claims for reasons the claimants cannot discover. A federal agency's climate database is quietly reorganized and three years of comparison data vanish. These are not exotic cases. They are the texture of governance in the 2020s.

This book is about the practice that takes those systems seriously as objects of public concern. Not as opportunities to build a better dashboard, though dashboards have their place. Not as abstract ethics problems, though ethics are unavoidable. As infrastructure: settled arrangements of data, code, authority, and labor that shape what the public can know and what it can demand. Public interest data science is the work of making that infrastructure observable, contestable, and, where necessary, reshaped.

1.1. What does "public interest" even mean

The phrase "public interest" has been doing enormous rhetorical work for a long time, and most of it is contested. Robert Moses invoked it to justify expressways that cut Black neighborhoods in half. Jane Jacobs (1961) used it against him to defend the street life those expressways destroyed. Civil rights lawyers used it to secure access to housing, education, and the

franchise. Cause lawyers on the other side used it to dismantle affirmative action. "Public" is a word that asks: which public? Whose interests?

This book does not pretend to settle that question. It takes a position and names the position explicitly. Following Keegan (2026), it treats the public interest as operational rather than abstract: the public interest is served when the six elements of what Chapter 5 calls the **installed base** are maintained. Records can be linked. Categories can be interpreted. Evidence persists across political cycles. Researchers and journalists can scrutinize systems without being sued or arrested. Someone has authority to act on what is learned. And the people affected have a path to remedy. When those six elements are in place, "public interest" is not a slogan. It is a working arrangement. When they are missing, even the most benevolent data practice can produce what Eubanks (2018) calls *automating inequality*.

That is a particular theory. It is not the only one available, and Chapter 16 takes seriously the traditions (Indigenous data sovereignty, abolitionist critique, feminist refusal) that question whether "records" and "linkability" are the right starting places at all. But you have to start somewhere, and "making the installed base visible" is a useful place to start.

1.2. Relative to neighbors

Public interest data science sits next to, and overlaps with, several fields you may already know.

Data journalism investigates public questions through computational methods and publishes findings in the press. It is older and, at its best, more politically mature than data science. Chapter 9 treats it as a lineage.

Civic technology and **public interest technology** (PIT) build tools for government and civil society: usable benefits portals, court-filing inter-

faces, participatory mapping platforms. Chapters 14 and 15 sort out the differences.

Data for good describes, usually, a philanthropic or industry-led application of commercial methods to socially valued problems. Chapter 14 engages it skeptically.

Critical data studies and **data justice** theorize the politics of data without necessarily building anything. Chapter 16 argues that the practice you will learn in this book is stronger when it can absorb the critique, including the parts of the critique that undercut the book's own framing.

Public interest data science is none of these in isolation. It borrows from all of them. It aims for the practical footprint of data journalism, the critical self-awareness of critical data studies, the civic focus of civic tech, and the professional-infrastructure commitment that "data for good" often lacks.

1.3. The arc of the book

The book moves through five parts.

Part I: Pressures diagnoses the structural forces that constrain public knowledge in the current moment. Chapter 2 treats enclosure (who can observe), Chapter 3 treats exemption (who must answer), Chapter 4 treats erosion (what can be remembered). These are not the only pressures, but they are the three that organize most of the empirical material through the rest of the book.

Part II: Values names the countervailing commitments that hold accountability together. Chapter 5 introduces the installed base and its six elements. Chapters 6–8 build out openness, oversight, and ownership in turn. These values are not abstract ideals. They are practical configurations, and each chapter asks what it takes to sustain them.

Part III: Lineages looks at the professions that have worked out public-interest practice for longer than data science has existed: journalism, law, accounting, planning, engineering. Each chapter extracts a set of methods the reader can import.

Part IV: Adjacent Conversations situates the book in the fields it is not and could be confused with: PIT and data for good, GovTech and civic tech, critical data studies. The treatment is comparative rather than polemical. Each of these fields has something you want, and each has something you should be wary of.

Part V: Genres is the pivot from seeing to doing. Chapters 17–22 teach the forms through which data scientists contribute to public conversations: op-eds, reports, research proposals, testimony, public comments, archival deposits. Each chapter ends with a real artifact you can submit.

Twenty-two chapters, roughly 3,000 words each. You can read them straight through or pick from the table of contents. Cross-references will tell you when a later chapter depends on an earlier one.

1.4. A first request

Before we talk any more about the book, let us do something with it. The rest of this chapter is a short tour of the toolkit you will use throughout, with a first public data retrieval exercise that takes perhaps twenty minutes. If you have Python and git already, you can follow along now. If not, the **Environment setup** sidebar below points you at the installers.

> **Environment setup**
>
> You need Python 3.11 or newer, git, and a working command line. On macOS and Linux, `python3 --version` and `git --version` should return something reasonable. On Windows, WSL2 works well, as

> does the official Python installer with the "Add Python to PATH" box ticked. You will also want `pip install pandas requests matplotlib jupyter` before continuing, and a GitHub account if you do not have one already.

We will retrieve pageview data for the Wikipedia article titled "Public interest," across three different language editions, and look at how the concept's visibility has varied across linguistic publics over time. The example is small, but it puts a handful of habits in place that will recur: fetching from a real public API, handling the response thoughtfully, moving it into a `pandas` DataFrame, plotting it, and asking what the data can and cannot tell you.

1.5. The Wikimedia pageviews API

Wikimedia Foundation operates a pageviews API that returns view counts for any article on any of their projects, back to 2015. The endpoints are documented at `wikimedia.org/api/rest_v1`. The relevant endpoint for our purposes takes the form:

```
GET /metrics/pageviews/per-article/{project}/{access}/{agent}/{article}/{granularity}/{start
```

You plug in the project (e.g., `en.wikipedia`), access method (`all-access` is fine), agent type (`user` filters out bots and spiders, which you want), article title, and a start and end date in `YYYYMMDD` or `YYYYMMDDHH` format.

Here is a first retrieval:

```
import requests

HEADERS = {
```

```
    "User-Agent": "PublicInterestDataScience/0.1 (your-email@example.edu)",
    "Accept": "application/json",
}

url = (
    "https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/"
    "en.wikipedia/all-access/user/Public_interest/monthly/"
    "2020010100/2024123100"
)

response = requests.get(url, headers=HEADERS, timeout=30)
response.raise_for_status()
data = response.json()
print(data["items"][:2])
# => [{'project': 'en.wikipedia', 'article': 'Public_interest',
#      'granularity': 'monthly', 'timestamp': '2020010100',
#      'access': 'all-access', 'agent': 'user', 'views': 8442}, ...]
```

Three things happened there that you will keep doing. You sent a `User-Agent` header naming yourself and your purpose. You called `raise_for_status()` to turn HTTP errors into Python exceptions. You set a timeout so that if the server hangs, your script does not. The next chapter will make these habits formal as part of a "responsible scraper" function. For now, notice that `requests.get()` does not hand you the response body directly. It hands you a `Response` object, and `.json()` is one of several methods you can call on it.

> **The Missing Manual**
>
> Most tutorials teach you `requests.get(url).json()` as a single chained call. That works until it does not. The `Response` object carries the status code, headers, cookies, and error context separately

from the body. When a request fails (and it will), you need the status code and the headers to understand why; calling `.json()` on a 403 will give you an exception that hides the 403. Write your code so the `Response` survives long enough to inspect. You will thank yourself the first time an API silently starts returning an HTML error page in place of JSON.

1.6. Into pandas

A single JSON response is interesting. Three parallel time series for three language editions are more so. We will collect "Public interest" pageviews for English, Spanish, and Hindi Wikipedia and plot them side by side.

```
import pandas as pd

def fetch_pageviews(project, article, start="2020010100", end="2024123100"):
    url = (
        f"https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/"
        f"{project}/all-access/user/{article}/monthly/{start}/{end}"
    )
    response = requests.get(url, headers=HEADERS, timeout=30)
    response.raise_for_status()
    items = response.json()["items"]
    df = pd.DataFrame(items)
    df["timestamp"] = pd.to_datetime(df["timestamp"], format="%Y%m%d%H")
    return df[["timestamp", "project", "views"]]

en = fetch_pageviews("en.wikipedia", "Public_interest")
es = fetch_pageviews("es.wikipedia", "Interés_público")
hi = fetch_pageviews("hi.wikipedia", "  _  ")
```

```
combined = pd.concat([en, es, hi], ignore_index=True)
combined.groupby("project")["views"].describe()
# => project-level descriptive stats across 60 months
```

Already the choices are political. The English article is titled "Public interest." The Spanish equivalent is "Interés público." The Hindi article is " ," which translates roughly as "people's good" rather than "interest." These are not translations in the sense a machine would call translations; they are conceptually distinct articles that Wikipedia's interlanguage links happen to join. The fact that all three exist tells you something. The fact that their pageview trajectories differ tells you something else.

Plot them:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 4))
for project, group in combined.groupby("project"):
    ax.plot(group["timestamp"], group["views"], label=project)
ax.set_xlabel("Month")
ax.set_ylabel("Views")
ax.set_title("Wikipedia pageviews: 'Public interest' across three editions")
ax.legend()
fig.tight_layout()
fig.savefig("ch01_pageviews.png", dpi=120)
```

If your plot looks reasonable, congratulations: you have just run an end-to-end public-interest data pipeline. Not a large one, but a real one. It fetches from a public API, handles errors, transforms a JSON response into a DataFrame, and produces a figure that would be legible on the front page of a campus paper or in a policy brief.

1.7. Reading the plot

Three things to notice, because they recur.

First, the English series has different seasonality than the Spanish or Hindi series. English "Public interest" spikes in specific months that tend to correspond to U.S. news cycles: data-broker legislation debates, antitrust hearings, election seasons. Spanish and Hindi series have their own spikes that do not align. The concept of a single global "public interest" is a fiction; there are many publics, and their interests peak on different calendars.

Second, all three series are small relative to any major news article. "Public interest" is a reference concept, not a subject of mass curiosity. The size of the audience for public-interest *work* is not large. It is, however, possible to reach it if you know where it is.

Third, the data will drift. Wikipedia articles get renamed, redirected, merged. The endpoint you called today may return a different trajectory next year because the article itself has changed. Chapter 4 takes up the problem of *what can be remembered*; this exercise is your first tiny encounter with it.

> **In the Public Interest**
>
> This chapter's running example is, on the surface, a technical trifle: three line plots of Wikipedia pageviews. But notice what it required. A public API, documented and open. Endpoints that return consistent, structured data. A retention policy that preserves history for at least a decade. A `User-Agent` expectation that asks you to identify yourself rather than hiding. Wikipedia built these as deliberate commitments to **openness** (Chapter 6). If you were trying to do the same exercise against a commercial platform in 2026, you would find most of those commitments have been withdrawn. The pressure of

> **enclosure** (Chapter 2) makes the simple thing you just did unusually precious. Part of this book's work is to help you notice when the simple thing is actually rare.

1.8. What you will build

By the end of the book, you will have produced:

- A published, documented civic dataset hosted as a Datasette instance (Chapter 6).
- A refusal specification for a dataset where refusal is the right answer (Chapter 16).
- A disaggregated audit of a public model (Chapter 7), wrapped as a reproducible pipeline (Chapter 11).
- A stakeholder map and geographic visualization of a local decision (Chapter 12).
- A tested data pipeline with a deliberately-introduced bug and a written post-incident review (Chapter 13).
- A class-wide FOIA/CORA tracker with at least one filed request of your own (Chapter 10).
- An op-ed draft pitched to a named outlet (Chapter 17).
- A report with reproducible figures (Chapter 18).
- A research proposal with a data management plan and mock IRB protocol (Chapter 19).
- A five-minute testimony statement with written addendum (Chapter 20).
- A substantive comment on a live federal rulemaking (Chapter 21).
- An archival deposit on the Internet Archive and Zenodo with full metadata (Chapter 22).

A portfolio, in other words. Not an assignment dump. A collection of things you can point to when you apply for a job, a fellowship, a grant,

or a seat on a commission, and say: I did this, I published it, here is the permanent link.

1.9. Exercises

Exercise 1.1 (Guided). Install Python 3.11+, git, and the packages listed in the environment setup callout. Clone this book's repository. Create a new branch named `yourname/ch01-helloworld`. Add a single line to `exercises/ch01_helloworld.txt` that reads "Hello from ." Commit with the message `exercise 1.1: hello world` and push the branch. Open a pull request against the course's fork of the repository. Submit the PR URL to the course instructor. The point of this exercise is not the content. It is that the work mechanism (branch, commit, PR) is your default mode from here forward.

Exercise 1.2 (Guided). Adapt the pageviews script to retrieve monthly views for three Wikipedia articles you care about, across three language editions each. Produce one plot comparing them. Write 150 words interpreting what the plot tells you. Save the plot and writeup to `exercises/ch01_pageviews/`. Note at least one choice (which article, which language, which date range) where you had to decide something, and record what you decided and why.

Exercise 1.3 (Reflective). Write a 300-word "data obituary" for an API, dataset, or data portal that has been withdrawn, paywalled, or substantially restricted. Examples to consider: the old Twitter API, Reddit's Pushshift, the federal climate portal's pre-2017 snapshots, a city open-data portal that shut down. Your obituary should name the data source, say what it was useful for, describe what happened to it, and identify who has been worse off since. Cite two sources.

Exercise 1.4 (Analytic). Find an op-ed or published column about AI or data policy from the last six months. Read it twice. Identify its explicit or implicit *theory of change*: who is the audience, what action is the piece

trying to produce, and what has to be true about that audience for the argument to land? Write 250 words. Do not pick an op-ed you agree with; pick one whose argument you think is doing something politically.

Exercise 1.5 (Open-ended). What is a public-interest data question you want to be able to answer by the end of this book? Write two to three paragraphs: what is the question, who would benefit from the answer, what data would be needed, and what obstacles do you already see. Save it as `exercises/ch01_project_sketch.md`. You will revisit it every few chapters.

1.10. Looking ahead

Chapter 2 begins Part I's diagnosis of the pressures on public knowledge. It takes the polite `User-Agent` habit we established here and turns it into a full "responsible scraper" function. More importantly, it asks why polite habits are necessary at all: how the ground of what public-interest work can observe has shifted over the last decade, and what that shift implies for what you should try to do next.

1.11. Further Reading and Resources

- Brian C. Keegan (2026), "Public interest data infrastructuring," *Information, Communication & Society*. The paper this book extends. Start here.
- Virginia Eubanks (2018), *Automating Inequality*. The canonical contemporary text on how data-driven systems reshape welfare, policing, and child services, and why "the public interest" requires attention to whose lives those systems touch.
- Tara Dawson McGuinness and Hana Schank (2021), *Power to the Public*. The book-length articulation of public interest technology,

useful as a framing text and as a target for critique (Chapter 14 will take up both).

- Mike Ananny and Kate Crawford (2018), "Seeing without knowing: Limitations of the transparency ideal and its application to algorithmic accountability," *New Media & Society*, 20(3): 973–989. A useful counterweight to the assumption that transparency is self-executing.
- Wikimedia Foundation's Pageviews API documentation: https://wikimedia.org/api/rest_v1/. The endpoint you used in this chapter, plus siblings you will find handy later.
- *The Data Journalism Handbook 2*, ed. Jonathan Gray and Liliana Bounegru (2021), open-access at https://datajournalism.com/read/handbook/two. A reader's companion to many of the methods this book will extend.
- Hugh Gusterson (2017), "From Brexit to Trump: Anthropology and the rise of nationalist populism," *American Ethnologist* 44(2): 209–214. For readers who want a frame for why "the public" has fragmented.
- The course syllabus and pull-request template for INFO 46XX, in this book's repository. Your first assignment is to open one.

2. Enclosure: Who Can Observe?

In the sixteenth and seventeenth centuries, across England and Scotland, common pastures and woodlands that had been used collectively for generations were fenced off by landowners and Parliament. The legal instruments were workmanlike. The consequences were not. Villagers who had grazed sheep on the commons, gathered firewood from shared stands, and gleaned fallen grain from open fields found themselves dependent on wage labor, migrating to cities, or both. Enclosure was not a failure of the commons. It was a project, by those with power, to convert shared resources into private ones, often with the language of improvement attached.

James Boyle (2017) argued two decades ago that a "second enclosure movement" was underway, this time over ideas, code, and data. Patents expanded. Copyright terms stretched. Public-domain materials were fenced behind paywalls and click-through agreements. That framing remains useful, but the enclosure most relevant to this book has moved past ideas and into observation itself. The question public interest data science asks is not only *what can I license?* but *what can I see in the first place?* The answer, for a researcher in the mid-2020s, is less than it was a decade ago, and the trajectory is not accidental. Following Keegan (2026), you can read the withdrawal of research access as an attack on the installed base introduced in Chapter 1: when linkability, interpretability, and safe scrutiny are removed, "public interest" becomes a slogan without an operational grip.

2.1. The last ten years, briefly

The easiest way to feel enclosure as a data scientist is to list retrievals you could have done ten years ago and try them today. A decent starter list:

- Pull one percent of public tweets through the Twitter streaming API, filtered by keyword, for a dissertation on political discourse.
- Fetch the complete comment history of a subreddit from Pushshift, with timestamps and scores, for a study of moderation.
- Download the pre-2017 version of the EPA's environmental-justice screening data, with the variable definitions as of that date.
- Query Facebook's Graph API for public-page posts about a municipal election.
- Retrieve CrowdTangle data on viral content during a disaster.

Each was a routine graduate-student assignment within the last decade. None is routine now. Some are technically possible through partnerships most researchers cannot access. Some are impossible. Some have been replaced by expensive commercial products whose terms forbid the reuse the original data supported. Bruns (2019) called this shift the **APIcalypse**: platforms that depended on researcher goodwill in their growth phase have, in their extraction phase, decided that goodwill is an expensive luxury.

Twitter (now X) is the sharpest case because the timeline is easiest to reconstruct. From 2006 through early 2023, the platform's API included a credible research tier, and an enormous empirical literature grew out of it. In February 2023 the company announced that free API access would end; the replacement pricing tiers start in the tens of thousands of dollars per month for the volumes a real research project requires. Academics who had built longitudinal panels over a decade watched them become unrenewable.

Reddit's trajectory followed, faster and with more open advocacy against researchers. Pushshift, a volunteer-run archive that had collected and served Reddit data since 2015, shut down its public API in April 2023

after Reddit revoked its credentials. Reddit's own API pricing, announced weeks later, pushed third-party clients (including accessibility tools like Apollo) out of the ecosystem and put researcher access behind tiers that, for any nontrivial use, meant six-figure bills. Gaffney and Matias (2018) had already documented silent gaps in Pushshift's mirror, which compounded the loss: not only is the new data expensive, the old data you thought you had is incomplete, and you cannot fill it in.

Government data portals are a quieter case and, for that reason, one you should watch more carefully. When a federal agency reorganizes, variable names change. When a political appointee prefers different framings, dataset landing pages are rewritten. When budgets contract, legacy portals stop updating and older vintages are "archived," which is sometimes a euphemism for delisted. The 2017 withdrawal of climate-change pages from the EPA website was the loud case; the quiet cases happen constantly, and Chapter 4 treats the failure mode in detail. Enclosure and erosion are siblings: one restricts access, the other dissolves the object of access, and both shift the ground under public-interest work.

The political economy underneath these cases is not mysterious. Wu (2021) describes the current configuration as a new Gilded Age of concentrated platform power. A handful of firms hold the data that describes contemporary public life. Their incentives run toward monetizing that data through advertising, model training, and partnerships, and away from letting outsiders audit it. "Partnership" is doing a lot of work in that sentence. It means the firms will sometimes share data with other firms, with favored labs, and with governments, under NDAs. It almost never means they will share it with journalists, civil-society researchers, or the public.

2.2. Reading what the platform tells you you can do

Before you write any code, read what the platform says about your presence. There are two documents every serious scraper consults: `robots.txt` and

the site's Terms of Service. Neither is a law. Both are signals, and you ignore them at your peril, both ethically and practically.

A `robots.txt` file is served at the root of a domain and tells automated agents which paths are and are not welcome. The Robots Exclusion Protocol dates to 1994 and was standardized as RFC 9309 in 2022. It lives at `https://example.com/robots.txt`. Fetching it is a one-line task:

```
import urllib.robotparser

rp = urllib.robotparser.RobotFileParser()
rp.set_url("https://en.wikipedia.org/robots.txt")
rp.read()

rp.can_fetch("PublicInterestDataScience/0.1", "/wiki/Public_interest")
# => True

rp.can_fetch("PublicInterestDataScience/0.1",
             "/w/index.php?title=Special:Export")
# => False (or True; check what Wikipedia's file actually says today)
```

Python's standard library parser returns `True` or `False` for a given path and user agent. It will also surface the crawl-delay the site suggests, if one is present, and the sitemap URL if the file advertises one.

Terms of Service are the harder document because they are written by lawyers who assume you will not read them. Researchers have lived in the shadow of the U.S. Computer Fraud and Abuse Act (CFAA) on this point, because for a long time it was unclear whether violating a ToS could be prosecuted as "unauthorized access" under the statute. The *Sandvig v. Barr* litigation (American Civil Liberties Union 2019) established in 2020 that academic research violating a ToS is not by that fact alone a criminal CFAA violation. That is a meaningful ruling and not a blanket license. You can still be sued in civil court, banned from the platform, served with

a cease-and-desist, or (if your institution values its industry relationships) quietly asked to stop. Read the ToS, document what you read, and know which clauses you are relying on and which you are departing from and why.

> **The Missing Manual**
>
> A request from `requests.get(url)` with no `User-Agent` header does not arrive at a modern server as "anonymous." It arrives as `python-requests/2.31.0`, which is the default and is the single fastest way to get 403'd by a CDN that has blocklisted it by policy. Cloudflare, Fastly, and Akamai maintain lists of default agents used by common scraping libraries. Set a descriptive `User-Agent` that names your project and a contact email. If the server operator needs to tell you to stop, you want them to be able to email you rather than silently ban your subnet. A polite scraper is also a scraper that keeps running.

2.3. A responsible scraper, from scratch

You will reuse the function below, in one form or another, across the remaining chapters. The design goals are plain: every request identifies itself, respects `robots.txt` by default, rate-limits, retries on transient errors, and fails loudly on permanent ones.

```
import time
import logging
from urllib.parse import urlparse
from urllib.robotparser import RobotFileParser

import requests

log = logging.getLogger("responsible_scraper")
```

```python
DEFAULT_UA = (
    "PublicInterestDataScience/0.1 "
    "(+https://github.com/yourname/pids-book; you@example.edu)"
)


def check_robots(url, user_agent=DEFAULT_UA):
    """Return True if robots.txt allows user_agent to fetch url."""
    parsed = urlparse(url)
    robots_url = f"{parsed.scheme}://{parsed.netloc}/robots.txt"
    rp = RobotFileParser()
    try:
        rp.set_url(robots_url)
        rp.read()
    except Exception as exc:
        log.warning("Could not read %s: %s", robots_url, exc)
        # Conservative default: if we can't read robots.txt, assume we
        # should not fetch. Override explicitly if you know better.
        return False
    return rp.can_fetch(user_agent, url)


def polite_get(
    url,
    headers=None,
    params=None,
    user_agent=DEFAULT_UA,
    min_interval=1.0,
    max_retries=3,
    backoff=2.0,
    timeout=30,
    respect_robots=True,
):
```

```python
    """Fetch a URL with rate limiting, retries, and a named User-Agent.

    Returns a requests.Response on success.
    Raises requests.HTTPError on non-retryable failure.
    """
    if respect_robots and not check_robots(url, user_agent=user_agent):
        raise PermissionError(f"robots.txt disallows fetching {url}")

    hdrs = {"User-Agent": user_agent, "Accept": "application/json"}
    if headers:
        hdrs.update(headers)

    last_err = None
    for attempt in range(1, max_retries + 1):
        try:
            response = requests.get(
                url, headers=hdrs, params=params, timeout=timeout
            )
        except requests.RequestException as exc:
            last_err = exc
            log.warning("Attempt %d: network error %s", attempt, exc)
        else:
            if response.status_code in (429, 500, 502, 503, 504):
                log.warning(
                    "Attempt %d: status %s from %s",
                    attempt, response.status_code, url,
                )
                last_err = requests.HTTPError(response=response)
            else:
                response.raise_for_status()
                time.sleep(min_interval)
                return response
        time.sleep(backoff ** attempt)
```

```
    raise last_err or RuntimeError("polite_get exhausted retries")
```

A few design choices are worth naming. The function accepts a `respect_robots` flag that defaults to `True`. You should need a reason to flip it, and you should write that reason into a comment in the calling code, not just your head. The function treats 429 and 5xx responses as retryable with exponential backoff, and 4xx responses as permanent failures. It sleeps a minimum interval after every successful call, which is the crudest form of rate limiting and the one that gets you the furthest. A production version would honor the `Retry-After` header, cache responses to disk, and emit structured logs. Your exercises will push you toward those features.

Using it looks like any other `requests` call, with a politer footprint:

```
response = polite_get(
    "https://wikimedia.org/api/rest_v1/metrics/pageviews/per-article/"
    "en.wikipedia/all-access/user/Enclosure/monthly/"
    "2015010100/2024123100"
)
items = response.json()["items"]
```

If you want to scrape a site whose `robots.txt` disallows you, stop and think. Sometimes there is a legitimate scholarly reason to proceed (a platform has disallowed a path that is uncontroversially public, a journalism project has counsel's blessing, a court order is in hand). More often the disallow signals a norm you should respect. The function makes respecting it the default. Refusal, as Chapter 16 will develop, runs in both directions: platforms can refuse researchers, and researchers can refuse requests even when a platform invites them. Zong and Matias (2024) treat refusal by data subjects as a legitimate counter-move within an asymmetric information environment.

2.4. A worked timeline: Reddit access, 2015–2024

Pick one platform and write its access history as a structured timeline. The exercise forces you to treat enclosure as a series of concrete decisions rather than a diffuse climate. Reddit is useful because the dates are public and the consequences are well-documented. The table below is a first pass; Exercise 2.4 asks you to extend and verify it.

Year	Event	Consequence for researchers
2015	Pushshift begins continuous archiving of Reddit submissions and comments.	A reasonably complete corpus of Reddit becomes queryable through a free API.
2018	Gaffney and Matias (2018) document silent gaps in the Pushshift mirror.	Researchers learn the archive is incomplete; many published studies rest on the incomplete slice.

Year	Event	Consequence for researchers
2020	COVID-19 drives a surge in computa-tional social science using Reddit data.	Demand on Pushshift grows; the corpus becomes de facto infrastructure for a subfield.
April 2023	Reddit revokes Pushshift's API credentials. Reddit announces API pricing.	The public Pushshift API goes dark. Third-party clients (Apollo, Reddit Enhancement Suite) exit or retreat.
June 2023	Large subreddits go private in protest ("the blackout").	Moderator labor, the free substrate of Reddit's data, becomes briefly legible as labor.
Late 2023	Reddit signs data-licensing deal with Google for model training.	Data refused to researchers is sold, as training data, to a firm that can afford it.

Year	Event	Consequence for researchers
2024	Reddit files for IPO; discloses API licensing as revenue.	The economic logic of enclosure is explicit in the S-1: researcher access was competing with shareholder value.

Two observations follow from the table. First, the shutdown was not only a loss of new data. It was a loss of old data, because the archive that held the historical corpus lost the credential it needed to keep serving what it had already collected. The object of study and the infrastructure of study collapsed together. Chapter 22 pushes you toward archival deposits you control rather than ones you rent.

Second, the same data the platform withdrew from researchers was, in the same calendar year, sold to a commercial AI firm. That is the asymmetry to sit with. Enclosure is not simply "the platform closed up." It is "the platform closed *you* out while opening further to capital." Acker and Kreisberg (2019) saw this pattern coming in their work on social-media archives; Suzor and colleagues (2018) locate it within a broader problem of platform governance legitimacy.

In the Public Interest

What is at stake here is **openness** as defined in Chapter 6: the condition under which a public can scrutinize the systems that affect it. Openness is not a synonym for "the data is downloadable." It is a working arrangement in which researchers, journalists, regulators, and affected communities can observe enough to raise evidence-based questions. When platforms cut off independent researchers while retaining partnerships with large firms and governments, they do not

abolish observation. They reallocate it. Observation continues, but it flows toward private interests and away from public ones. Naming this as an openness failure, rather than a business decision, is the first move that lets you push back. Chapter 7 and Chapter 8 extend the argument into regulatory and distributional registers.

2.5. The asymmetry of observation

The previous two sections should have left you with an uncomfortable picture. The firms that hold the data describing contemporary public life can observe their users in extraordinary detail. They partner with other firms and with governments to extend that observation. They cut off the outside parties whose scrutiny would be inconvenient. The parties who remain (journalists, academics, civil-society groups) are left with robots.txt files, HTML-scraped fragments, and the occasional leaked dataset. The asymmetry is not between observers and the unobserved. It is between two classes of observers.

That asymmetry has consequences for what you should try to build. First, assume your access is temporary and plan for its withdrawal: Chapter 5 and Chapter 22 push you toward architectures where your data has a resolvable identifier and a home that outlives any single platform. Second, be honest about the limits of what you can observe: a study that conditions on platform-provided data is a study of what the platform chose to show you, and naming that is part of the work rather than a methods-section apology. Third, refusal is available. If the terms of access are corrosive (an NDA that prevents you from naming the firm, an exclusivity clause that bars you from sharing derived data with colleagues), you can decline, and Chapter 16 will argue that sometimes you should.

Public-interest data science cannot rely exclusively on large commercial platforms. The installed base this book is building toward includes govern-

ment open data, civic datasets, court records, FOIA-wrested documents, historical archives, and the output of researchers willing to publish collected data under permissive licenses. Each of those sources has its own pressures (Chapter 3 takes up exemption, Chapter 4 takes up erosion), but none of them has an S-1 that treats your access as lost revenue. Diversifying your substrate of observation is a strategic response to enclosure.

2.6. Exercises

Exercise 2.1 (Guided). Fetch and read the `robots.txt` files for three platforms. Recommended triple: Wikipedia, Reddit, and a large newspaper site (for example, nytimes.com). Use `urllib.robotparser` to check whether your `User-Agent` would be permitted on five paths per site. Write a 200-word comparison: which site is most permissive, which most restrictive, and which surprised you.

Exercise 2.2 (Guided). Implement the `polite_get` function from scratch, without copying the listing above. Add one feature the book's version lacks: either honor the `Retry-After` header on 429 responses, or cache successful responses to a local SQLite database keyed by URL so that repeated fetches during development do not re-hit the server. Commit the function with a docstring explaining your added feature and a test that exercises it against a public endpoint.

Exercise 2.3 (Analytic). Pick one platform whose data you expect to use in your final project. Download its current Terms of Service to a dated file (for example, `exercises/ch02_tos/reddit_2026-04-23.txt`). Annotate paragraph by paragraph with four marks: **R** for clauses that restrict research uses, **P** for clauses that permit them, **A** for ambiguity you would need counsel to resolve, and **L** for language that relies on unstable definitions ("public content," "non-commercial," "API partner"). Submit the annotated file and a 300-word summary of what a research plan can and cannot do under the current ToS.

Exercise 2.4 (Analytic). Extend the Reddit timeline. Add three rows from publicly reported events since 2024, with citations. Then write a second timeline, in the same format, for a different platform (Twitter/X, Facebook, TikTok, YouTube, or a government data portal). Cover at least eight years and at least five distinct access events. Cite the source for each date; prefer primary documents (official announcements, court filings, congressional testimony) over secondary reporting where both exist.

Exercise 2.5 (Open-ended). Revisit the project sketch from Exercise 1.5. Identify the single dataset whose availability would most determine whether your project is possible. Write a 500-word accessibility memo covering: who controls the data; under what terms it is accessible; what pathways exist (public download, API, FOIA request, institutional partnership, commercial license); the cost and timeline of each pathway; what has happened to the data's accessibility over the last five years; and what fallback datasets you would use if the primary became unavailable. Save as `exercises/ch02_access_memo.md`. You will return to this memo in Chapter 19 and Chapter 22.

2.7. Looking ahead

Enclosure is the first of three pressures this part diagnoses. Chapter 3 takes up exemption: even when data can be observed, certain actors are under no obligation to answer for what it describes. Law-enforcement agencies exempt from FOIA. Vendors whose contracts shield them from disclosure. Private actors performing public functions without the accountability attached. If enclosure asks *who can observe?*, exemption asks *who must answer?*

2.8. Further Reading and Resources

- Axel Bruns (2019), "After the 'APIcalypse': Social media platforms and their fight against critical scholarly research," *Information, Communication & Society* 22(11). The article that gave the post-2018 shift its name.
- James Boyle (2008), *The Public Domain: Enclosing the Commons of the Mind*. Duke University Press. Open-access at https://www.thepublicdomain.org/. The enclosure analogy in its fullest form.
- Robots Exclusion Protocol, RFC 9309 (2022): https://www.rfc-editor.org/rfc/rfc9309.html. A short, readable standardization most scrapers violate out of ignorance.
- Jonathan Zong and J. Nathan Matias (2024), "Data refusal from below." The complement to this chapter: when enclosure is not the problem, refusal might be.
- Reddit's API pricing announcement (June 2023) and the original Pushshift shutdown notice, both reachable from https://www.reddit.com/r/modnews/ archives. Primary documents for Exercise 2.4.
- The Internet Archive's Wayback Machine, https://web.archive.org/. When a platform retires a dataset, often the only remaining record of what was public. You will use it constantly.
- Electronic Frontier Foundation, "Coders' Rights Project," https://www.eff.org/issues/coders. Plain-language summaries of the CFAA, DMCA, and related statutes.
- Knight First Amendment Institute, https://knightcolumbia.org/research. Ongoing litigation and policy work on the legal scaffolding of observation.
- Wikimedia Foundation's research ethics and tool policies, https://meta.wikimedia.org/wiki/Research:Ethics. An instructive contrast: what a platform looks like when it wants researchers to study it.

3. Exemption: Who Must Answer?

Chapter 2 described a world in which what a researcher can observe is shrinking. This chapter is about a related but distinct problem. Suppose you *can* observe the outputs of a system. They are already on a published spreadsheet, or in a press release, or scrolling through a caseworker's terminal. Who is obliged to explain how they were produced? Who has to answer when they go wrong? In the United States in the mid-2020s, the honest answer for a remarkable share of consequential automated decisions is: no one with enforceable authority, on any clock a parent or a journalist or a state legislator can use.

Enclosure asks *who can observe?* Exemption asks *who must answer?* Enclosure operates through the terms of access to data. Exemption operates through trade-secret law, procurement contracts, qualified immunity, sovereign immunity, statutory carve-outs from freedom-of-information regimes, and the simple fact that many public agencies run proprietary software they did not build and cannot fully inspect. Pasquale (2016) named this the *black box society*: a condition in which the scoring and sorting machinery that organizes credit, insurance, hiring, and policing is legally and practically sealed off from the people it sorts. A decade later, the black box has migrated further into the machinery of the state itself. When the exempt actor is a commercial platform, you can at least pretend to buy your way in or shame your way in. When the exempt actor is the state, using a vendor's system to decide who gets a benefit, the pretense collapses.

This chapter takes up two cases. The long example is the Allegheny Family Screening Tool (AFST), a risk-scoring system used by the Allegheny County,

Pennsylvania child-welfare hotline since 2016. AFST is unusually well-documented for a government-deployed risk score, and you will build an evidence ledger for it from public sources. The short comparison is the Dutch state's Systeem Risico Indicatie (SyRI) and the childcare-benefits fraud detection system that prompted the *toeslagenaffaire* scandal. The Dutch cases matter because they are the rare examples in which a national court found an automated decision system unlawful and shut it down (District Court of The Hague 2020). Reading the U.S. case against the Dutch one tells you which claims about opacity have purchase, and in which legal climates.

3.1. Two kinds of exemption

Separate **legal exemption** from **practical exemption** before you go near a keyboard.

Legal exemption is the carved-out space where disclosure obligations do not reach. Trade-secret protections under the Defend Trade Secrets Act and state analogues travel with a vendor into a public agency's procurement file. FOIA exemptions 4 (trade secrets) and 7 (law enforcement) are routinely invoked to withhold model specifications, training data, and performance audits. The Computer Fraud and Abuse Act (CFAA) historically chilled independent testing: until *Sandvig v. Barr* (American Civil Liberties Union 2019), academic auditors faced credible threat of federal criminal prosecution for creating test accounts that violated a platform's terms. *Sandvig* narrowed that risk for academic research but did not eliminate civil exposure and did not reach state-level computer crime laws. Terms-of-service bindings tie back to Chapter 2: what the vendor's lawyers wrote at procurement time continues to govern what the public can ask later.

Practical exemption is the larger space. Even where the law would require disclosure, the route is slow, fragmented, and under-enforced. A reporter who files a state open-records request for a vendor contract waits six months

and receives a PDF with most of its text redacted. A parent whose family was screened out of services reads a denial letter citing an "assessment" without naming the model, the version, the features, or the threshold. The information exists. Nobody is obliged to aggregate it. Exemption does not require secrecy. It only requires that the cost of reconstructing the picture exceed what a citizen, a reporter, or an underfunded NGO can afford. Fragmentation is the weapon.

> **In the Public Interest**
>
> **Oversight**, as Chapter 7 will develop it, is not a property of a single document. It is a property of the *linkage* between documents: a procurement contract that names a vendor, a model card that describes that vendor's system, a FOIA response that discloses subgroup performance, a court filing that tests a denied claim. Exemption thrives on fragmentation because each artifact taken alone is plausible. The contract does not mention the model by name. The model card does not mention the contract. The denial letter mentions neither. Keegan (2026) calls this the **linkability** element of the installed base, and without it, oversight is a collection of disconnected complaints. Public-interest data science is, in part, the patient reconstruction of linkages the system was designed to leave broken.

3.2. Allegheny Family Screening Tool, as a public object

Allegheny County's Office of Children, Youth and Families receives roughly fifteen thousand child-welfare referrals a year. Since August 2016, hotline call screeners have seen, alongside the case narrative, a risk score produced by the Allegheny Family Screening Tool. The score estimates the probability that a child named in the referral will be re-referred, removed from the

home, or placed in foster care within a defined window. A screener uses the score as one input in deciding whether to open a formal investigation.

Eubanks (2018) treated AFST in *Automating Inequality* as a paradigmatic case of a predictive instrument that extends state surveillance into poor families under the rubric of decision support. The county and the research team have disputed some characterizations and affirmed others. AFST is unusual in that a reader who wants to know how the system works can assemble a nontrivial picture from public sources. It is also typical in that the picture is incomplete, and the incompleteness is not random.

3.3. The evidence ledger

The technique is deliberately unglamorous. Read every public document you can find about the system. For each, record what it tells you and, just as importantly, what it does not. The `pandas` DataFrame below is an *evidence ledger*: a structured record of provenance that makes gaps visible on the same footing as findings.

```
import pandas as pd

rows = [
    {
        "source": "Allegheny County RFP and vendor selection memo",
        "date": "2014-11", "type": "procurement",
        "what_it_tells_us": "Names Auckland University of Technology's "
            "Centre for Social Data Analytics as the selected team; "
            "lists the data systems to be integrated.",
        "what_it_doesnt": "Does not disclose the price, the model "
            "family, or the performance targets the vendor was "
            "expected to meet.",
    },
    {
```

```
        "source": "Vaithianathan et al. methodology report",
        "date": "2019-04", "type": "technical_report",
        "what_it_tells_us": "Describes feature engineering on 131 "
            "variables from the county data warehouse; reports AUC "
            "and calibration on historical data.",
        "what_it_doesnt": "Does not disclose the production decision "
            "threshold, the retraining cadence, or subgroup "
            "performance on race and ethnicity as deployed.",
    },
    {
        "source": "Chouldechova et al., FAccT 2018",
        "date": "2018-02", "type": "peer_reviewed",
        "what_it_tells_us": "Subgroup accuracy and error-rate "
            "disparity analysis on a research version of the model.",
        "what_it_doesnt": "Research version is not identical to the "
            "fielded tool; ground-truth labels are proxies, which "
            "the authors acknowledge.",
    },
    {
        "source": "Eubanks, Automating Inequality, ch. 4",
        "date": "2018-01", "type": "book",
        "what_it_tells_us": "Situates AFST within the county's "
            "broader data integration project; documents community "
            "concerns and the surveillance/referral feedback loop.",
        "what_it_doesnt": "Cannot speak to model changes after 2017 "
            "fieldwork; the tool has been revised multiple times.",
    },
    {
        "source": "Associated Press / Ho and Burke",
        "date": "2022-04", "type": "press",
        "what_it_tells_us": "Reports DOJ civil-rights inquiry into "
            "disability-based discrimination in the tool's deployment.",
        "what_it_doesnt": "DOJ inquiry materials are not public; we "
```

```
            "learn an inquiry exists but not what it found.",
    },
]

evidence = pd.DataFrame(rows)
evidence["date"] = pd.to_datetime(evidence["date"])
evidence = evidence.sort_values("date").reset_index(drop=True)
# => 5 rows, columns: source, date, type, what_it_tells_us, what_it_doesnt

gaps = evidence[["date", "source", "what_it_doesnt"]]
gaps.to_csv("afst_gaps.csv", index=False)
```

Five rows is not a lot. That is the point. A mature ledger for a deployed municipal system might contain thirty or forty rows, organized by procurement, technical documentation, peer review, press, litigation, and community record. The two columns matter. `what_it_tells_us` is the affirmative claim you can source-cite. `what_it_doesnt` is the negative space that tells you what you would still need to file, subpoena, or discover in order to make a stronger claim. A one-column ledger is a bibliography. A two-column ledger is the beginning of an accountability argument.

Two observations follow. The most technically detailed document (Vaithianathan and colleagues' methodology report (Vaithianathan et al. 2019)) is useful for understanding the research-time model but silent on the deployed threshold and retraining schedule. The most politically charged finding (the DOJ civil-rights inquiry reported by Ho and Burke (2022)) is the one where the underlying record is least available: we know an inquiry opened, not what it concluded.

> **The Missing Manual**
>
> The model card does not say. The methodology report does not say. The procurement memo does not say. Those three silences are them-

selves data. A standard failure mode of algorithmic-accountability writing is to treat the absence as a rhetorical flourish ("notably, the county does not disclose…") and move on. Do not move on. Write the absence into the ledger as an object with a date, a source, and a `what_it_doesnt` entry. You will use it later when you file a records request, when you testify, or when you draft a rulemaking comment. An absence that has a citation is an absence you can litigate. An absence you only gestured at in prose is a complaint.

3.4. Reading the deployed model from the outside

The ledger tells you what documentation exists. It does not tell you how the model is behaving today. For a deployed system, three kinds of external inference are available without breaking any law.

The first is **decision-record inspection**. Some jurisdictions, including Allegheny County at various points, release aggregate screening statistics: calls screened in, screened out, and referred, broken down by some subset of demographic variables. Those aggregates let you compute, crudely, whether the score's introduction shifted selection rates overall. They do not let you recover individual decisions.

The second is **disparate-impact analysis on outcomes**. If you can obtain downstream placement data (via FOIA, a collaborator inside the agency, academic partnership), you can estimate whether scored referrals have different outcomes by race, age, or disability than unscored referrals. The Chouldechova and colleagues FAccT paper (Chouldechova et al. 2018) is an example of this kind of work on a pre-deployment version.

The third is **audit-by-adversarial-use**: submitting test cases through the live interface. CFAA exposure still matters here. You would need counsel's sign-off, institutional cover, and a clear understanding of which *Sandvig*

protections reach your facts. Most municipal child-welfare systems do not expose an API you could query, which is why audit-by-use is rare in this domain and common in hiring and advertising contexts.

```
# Illustrative skeleton only. Do not run against a live agency system.
def compute_selection_rates(decisions):
    """Aggregate screen-in rates by demographic group.

    Input: one row per referral with columns
        {referral_id, group, score, screened_in}.
    """
    g = decisions.groupby("group").agg(
        n=("referral_id", "count"),
        screened_in=("screened_in", "sum"),
    )
    g["rate"] = g["screened_in"] / g["n"]
    return g.reset_index()

# Expected output columns: group, n, screened_in, rate
```

None of this code is the audit. The code is the scaffolding you build so that, when a data-sharing agreement is in place or an agency publishes aggregates, you have somewhere to put the numbers. The substantive work is upstream: negotiating the sharing agreement, filing the records request, obtaining IRB approval. Chapter 10 takes up the legal scaffolding in detail.

3.5. When the state is the adversary: SyRI and after

The Dutch case inverts a habit U.S. readers bring to these problems. In the United States, the usual adversary in an algorithmic accountability story is a commercial vendor whose system has been adopted by a public

agency. The vendor holds the trade-secret claim; the agency carries the disclosure obligation; advocates play them against each other. In SyRI, the adversary was the state itself.

Systeem Risico Indicatie was a cross-agency data-integration tool that scored households for fraud risk across welfare, taxation, labor, and housing data. It was deployed in targeted neighborhoods beginning in 2014. In February 2020, the District Court of The Hague ruled that SyRI violated Article 8 of the European Convention on Human Rights because the state's interest in fraud detection did not justify the opacity and breadth of the system (District Court of The Hague 2020). The ruling did not turn on the accuracy of the model. It turned on the state's failure to make the system legible enough for citizens to understand and challenge how they were being assessed. Dutch scholars reading the aftermath (Arruda and van Schendel 2024) note that successor systems have adapted the legal wrapper without fully abandoning the underlying practice, and that the *toeslagenaffaire* childcare-benefits scandal exposed a parallel, more damaging pattern in the tax administration.

Read the childcare-benefits scandal alongside AFST. Amnesty International's report (Amnesty International 2021) documents how the Dutch tax authority used a risk-classification model that flagged dual-nationality applicants at elevated rates, triggering aggressive clawbacks that pushed tens of thousands of families into debt and, in many cases, into losing their children to state care. The Rutte cabinet eventually resigned over it. Opacity was not incidental to the harm: families could not learn why they had been flagged, which made it impossible to contest individual decisions or to surface the pattern until journalism and parliamentary inquiry reconstructed it from outside (Geiger and Braun 2023).

Three contrasts with AFST matter. First, the Dutch regime provided a forum (the Article 8 claim) in which system-level opacity could be litigated directly, rather than through the U.S. habit of challenging individual denials. Second, the affected class was organized enough, through civil-society groups and parliamentary allies, to press the claim to a national

court. Third, the remedy was shutdown, not disclosure: the court did not order SyRI to open its model. It ordered the state to stop running it. Chapter 10 asks under what U.S. doctrinal conditions an equivalent remedy would be available, and the answer is: under very few, and none at the federal level as of this writing.

> **The Missing Manual**
>
> When the state is the exempt actor, strategies that assume the state is a neutral arbiter between citizen and vendor stop working. Most algorithmic-accountability tooling (model cards, datasheets for datasets, fairness audits) is designed for a world in which the state is the party demanding accountability from a vendor. In the SyRI and *toeslagenaffaire* cases, the party deploying the system, the party holding the data, and the party refusing disclosure were all the same ministry. The governance textbook does not cover that case, because the governance textbook was written by people employed by the ministry. Notice when you are inside that scenario, and notice that your methods have to change.

3.6. Practical exemption as a design choice

Exemption is not fate. It is a set of design and procurement choices that can be made differently. The European Union's 2024 AI Act, New York City's Local Law 144, and a growing list of state-level algorithmic-accountability bills are attempts to legislate a floor under transparency. Whether any produces working oversight depends on the same six elements Chapter 5 will name: linkable records, interpretable categories, persistent evidence, safe scrutiny, authority that can act, and remedy that reaches the affected. A statute that requires a "model card" without specifying linkage to deployment records produces compliance artifacts and no oversight.

The public-interest response to exemption has two parts. The first is the patient work this chapter has rehearsed: building evidence ledgers that make absences visible, linking artifacts across silos, naming the gaps in terms a lawyer or a legislator can use. The second is designing forward: when you have authority to shape a procurement, a contract, or a rule-making, writing linkability in as a requirement rather than a nice-to-have. Chapter 7 and Chapter 22 develop both moves.

3.7. Exercises

Exercise 3.1 (Guided). Extend the AFST evidence ledger from five rows to at least twelve. Candidate sources: the Allegheny County impact evaluation reports published between 2019 and 2024, at least one state-level audit or oversight report, the research protocol that governed the tool's introduction, any litigation related to the tool, and community-organization responses. For each new row, fill both `what_it_tells_us` and `what_it_doesnt`. Save as `exercises/ch03_afst_ledger.csv` and write a 300-word summary of the three largest gaps.

Exercise 3.2 (Guided). Pick a different deployed automated decision system: a state's Medicaid eligibility system, a city's pretrial risk instrument, a school district's early-warning system, a federal agency's fraud-detection tool. Build an evidence ledger with at least eight rows using the same schema. Save as `exercises/ch03_<system>_ledger.csv`. In 400 words, compare the shape of the absences to the AFST ledger: what documents exist for one system but not the other, and what does that asymmetry suggest about which actors face the most pressure to produce records?

Exercise 3.3 (Analytic). Read the SyRI judgment (ECLI:NL:RBDHA:2020:1878), available in English translation through Rechtspraak.nl. Identify the specific legal theory the court used to find the system unlawful. In 500 words, summarize the theory in your own words, name the treaty article or statute on which it rested, and explain whether an equivalent

argument would be available in a U.S. federal court. If you are not sure, name the sources of your uncertainty.

Exercise 3.4 (Comparative). Pair a U.S. automated decision system and a European one addressing similar policy problems (child welfare, unemployment fraud, immigration risk, predictive policing). Build a side-by-side transparency comparison covering: vendor disclosure, model documentation, performance reporting, subgroup auditing, notice to affected persons, appeal pathways, external oversight, and publicly documented harms. Submit as `exercises/ch03_comparison.md`. Identify the single transparency commitment that would be most transformative if transplanted from one regime to the other, and say what would prevent that transplant.

Exercise 3.5 (Open-ended). For the system you chose in Exercise 3.2, propose what would need to change for it to be *observable* in this chapter's sense. Write a 1,000-word memo addressed to a named accountability actor (a specific state legislator, a city auditor, a named reporter, an advocacy organization). Specify which changes require legislation, which require agency rulemaking, which require litigation, and which could be achieved through procurement reform alone. Save as `exercises/ch03_observability_memo.md`. You will reuse this memo in Chapter 10 and, revised, as a public comment in Chapter 21.

3.8. Looking ahead

Enclosure takes data out of reach. Exemption keeps the people and institutions that use data out of reach. Chapter 4 takes up the third pressure in Part I: **erosion**. Even when data is accessible and decision-makers are nominally accountable, the record of what happened can dissolve. Databases are reorganized without migration paths. Web pages are silently edited. Legacy systems are decommissioned faster than their contents can be archived. If enclosure asks *who can observe?* and exemption asks *who must answer?*, erosion asks *what can be remembered?*

3.9. Further Reading and Resources

- Frank Pasquale (2016), *The Black Box Society*. Harvard University Press. The foundational treatment of scored decision-making and the legal architecture of its opacity.
- Virginia Eubanks (2018), *Automating Inequality*. St. Martin's Press. Chapter 4 treats AFST in ethnographic depth. Read with and against the research team's published responses.
- Amnesty International (2021), *Xenophobic Machines*, https://www.amnesty.org/en/documents/eur35/4686/2021/en/. The canonical documentation of the *toeslagenaffaire*.
- District Court of The Hague (2020), SyRI judgment, ECLI:NL:RBDHA:2020:1878. Read with an English translation alongside the Dutch original.
- Lighthouse Reports / WIRED (2023), "Inside the Suspicion Machine," https://www.lighthousereports.com/investigation/inside-the-suspicion-machine/. Reverse-engineered feature importance for Rotterdam's welfare-fraud scoring.
- Allegheny County Department of Human Services reports, https://www.alleghenycountyanalytics.us/. The county's own documentation, read critically and in date order.
- AI Now Institute, *Algorithmic Accountability for the Public Sector*, https://ainowinstitute.org/publication/algorithmic-accountability-for-the-public-sector. Comparative survey of procurement, disclosure, and audit regimes.
- Knight First Amendment Institute, *Sandvig v. Barr* case page, https://knightcolumbia.org/cases/sandvig-v-sessions. Primary documents for the CFAA ruling that shaped current U.S. research-access conditions.
- Ada Lovelace Institute, *Regulatory Inspection of Algorithmic Systems*, https://www.adalovelaceinstitute.org/report/inspecting-algorithms/. A European comparative perspective on inspection regimes.

4. Erosion: What Can Be Remembered?

A public record that cannot be found is not a public record. A footnote whose URL returns a 404 is a claim without a witness. A training corpus that has been quietly withdrawn is a model whose inputs the public can no longer inspect. This chapter is about the third pressure in Part I, the one that works by subtraction. Chapter 2 asked *who can observe?* Chapter 3 asked *who must answer?* This chapter asks *what can be remembered?* and describes the slow, less visible process by which the evidence needed for any answer simply dissolves.

Erosion is rarely dramatic. An agency reorganizes a website during a transition and the old paths return 404s. A journal publisher migrates and the DOIs still resolve but the supplementary materials do not. A civic-tech nonprofit runs out of funding and its dashboard goes dark. A machine-learning dataset that trained a generation of models is withdrawn after researchers document abusive content inside it, and the weights of every model trained on it remain in circulation. Any one event looks like an operational hiccup. Taken together, they are the texture of a public record thinning as fast as anyone can thicken it.

Plantin, Lagoze, Edwards, and Sandvig (2016) argue that the platforms the public has learned to depend on operate on a different clock than public infrastructures do. A municipal water system is designed to last decades and is maintained by a workforce whose job is to keep it running. A cloud storage service or a dataset release is designed to last a product cycle and is maintained by whoever still has a quarterly incentive to care. Karasti

and Blomberg (2018) call this the mismatch between **project time** and **infrastructure time**: projects end; infrastructures are supposed not to. When the public record lives inside projects, it ends with them.

4.1. The mundane politics of retention

There is a temptation to tell erosion stories as conspiracies. Sometimes they are. Much more often they are what happens when nobody's job description includes long-term stewardship. A sysadmin retires. A contract lapses. A platform closes its API and thirty years of researcher-accumulated corpora become unciteable overnight. Gaffney and Matias (2018) documented this for a single, widely-used Reddit corpus, showing that silent data loss had produced an entire body of published work built on samples missing, in predictable ways, parts of the thing they claimed to describe. The missingness was not announced. It was visible only when someone looked.

Acker and Kreisberg (2019) describe this as the problem of archiving in an API-driven world. Classical archival theory assumes a record exists somewhere, that custody can be established, and that preservation is a matter of migration. The contemporary record does not sit still long enough to be taken into custody. Denton, Hanna, and colleagues (2021) trace the same problem for machine-learning datasets: ImageNet, the canonical benchmark for a decade of computer-vision research, was assembled from a pipeline of choices (scraping Flickr, filtering through WordNet, paying crowdworkers to label) that is undocumented in any single place. When a dataset is questioned, the record of how it was made has often evaporated faster than the dataset itself.

The model-era instances are bigger and not meaningfully different in kind. LAION-5B, the image-text dataset used to train Stable Diffusion and many of its descendants, was taken offline in late 2023 after researchers at Stanford's Internet Observatory documented verified child sexual abuse

material inside it (Thiel 2023). The withdrawal was the right call. It was also, for research purposes, a form of erosion: every paper that trained on LAION-5B now cites a dataset the reader cannot audit. Common Crawl, which underlies much of the open web's representation in large language model training, retains its crawls but its long-term archival status is a question nobody has answered on the record.

Keegan (2026) names **continuity** as one of the six elements of the installed base. The point is not that every artifact should live forever. The point is that when the public loses the ability to return to a source, the ability to audit what was built on that source goes with it. Erosion is not a failure of tools. It is a failure of incentives.

4.2. A link-rot audit you can run this afternoon

You need to see this happen in a corpus you can touch. The corpus to start with is the one you are reading: this book's own bibliography plus the "Further Reading" URLs at the end of each chapter. If you do not have that list handy, the EPA's climate-change press-release archive and NOAA's hurricane-research page are excellent substitutes, because both agencies have been reorganized enough times since 2016 that you will find real rot within minutes.

Walk a list of URLs, classify each response, record the result in a DataFrame. A 404 is obvious rot. A 301 redirect to a generic landing page is a worse and more common failure: the link resolves, returns a 200, and gives you nothing.

```
import pandas as pd
import requests
from urllib.parse import urlparse

HEADERS = {
```

```
    "User-Agent": "PublicInterestDataScience/0.1 (your-email@example.edu)",
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0
}

def classify_url(url, timeout=15):
    """Fetch a URL and classify the response.

    Returns a dict with url, status_code, final_url, category, notes.
    Categories: ok, redirect_same_host, redirect_offsite,
                client_error, server_error, timeout, dns_fail, other.
    """
    try:
        r = requests.get(url, headers=HEADERS, timeout=timeout,
                         allow_redirects=True)
    except requests.exceptions.Timeout:
        return {"url": url, "status_code": None,
                "final_url": None, "category": "timeout", "notes": ""}
    except requests.exceptions.ConnectionError as e:
        return {"url": url, "status_code": None,
                "final_url": None, "category": "dns_fail",
                "notes": str(e)[:120]}

    start_host = urlparse(url).netloc
    end_host = urlparse(r.url).netloc
    if r.status_code == 200 and r.url == url:
        cat = "ok"
    elif r.status_code == 200 and start_host == end_host:
        cat = "redirect_same_host"
    elif r.status_code == 200:
        cat = "redirect_offsite"
    elif 400 <= r.status_code < 500:
        cat = "client_error"
    elif 500 <= r.status_code < 600:
```

```
        cat = "server_error"
    else:
        cat = "other"

    return {"url": url, "status_code": r.status_code,
            "final_url": r.url, "category": cat,
            "notes": f"bytes={len(r.content)}"}


def audit(urls):
    rows = [classify_url(u) for u in urls]
    df = pd.DataFrame(rows)
    return df
```

Point this at a real list of URLs and you will get something like the distribution below. The shape is representative, not exact; rerun it and the numbers will drift.

```
urls = [
    "https://www.epa.gov/climate-change",
    "https://www.noaa.gov/climate",
    "https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing
    "https://wikimedia.org/api/rest_v1/",
    # ... twenty more from your syllabus or bibliography
]

report = audit(urls)
report.groupby("category").size()
# => ok                     11
#    redirect_same_host      4
#    redirect_offsite        3
#    client_error            2
#    server_error            0
```

```
#    timeout                 1
#    dns_fail                0
#    other                   0
```

The `redirect_offsite` row is the one to look at first. A federal agency URL that now redirects to a generic "archive" subdomain, or to a White House press page on a different topic, is a link that technically resolves but practically does not. Whenever you cite a government source in a paper, a report, or a public comment, assume the URL will be reorganized at the next administration transition. Plan for it.

> **The Missing Manual**
>
> Status codes lie. A page that returns HTTP 200 may still be useless: a content management system that issues a "soft 404," a WAF page saying "request blocked," a single-page application that loads a JavaScript shell and fetches its real content from an API you are not calling. You can partly guard against this by checking response size and content against a known baseline, but there is no substitute for opening a sample of the "ok" rows in a browser. The Wayback Machine has the same problem from the other direction: its Availability API will cheerfully return a capture that was itself a 404 page archived at the moment of rot. If the service your audit uses does not let you inspect the content of a "successful" response, your audit is not done. Always verify a subset by eye.

4.3. Recovering the rot with the Wayback Machine

The Internet Archive's Wayback Machine is the closest thing the public has to a backup of the open web. It is not complete. It is not permanent (the Archive itself is a nonprofit dependent on donations, lawsuits, and

the continued forbearance of its adversaries). It is the best you have. The Availability API is a single, small endpoint that takes a URL and an optional timestamp and returns the closest capture.

```
GET http://archive.org/wayback/available?url=<url>&timestamp=<YYYYMMDD>
```

The response shape is compact:

```
{
  "url": "https://www.epa.gov/climate-change",
  "archived_snapshots": {
    "closest": {
      "status": "200",
      "available": true,
      "url": "http://web.archive.org/web/20161119000000/https://www.epa.gov/climate-change",
      "timestamp": "20161119000000"
    }
  }
}
```

If there is no capture, `archived_snapshots` is an empty object. A recovery helper:

```
def find_capture(url, timestamp=None):
    api = "http://archive.org/wayback/available"
    params = {"url": url}
    if timestamp:
        params["timestamp"] = timestamp
    r = requests.get(api, params=params, headers=HEADERS, timeout=30)
    r.raise_for_status()
    data = r.json()
    snap = data.get("archived_snapshots", {}).get("closest")
```

```
    if not snap or not snap.get("available"):
        return None
    return {"wayback_url": snap["url"],
            "captured_at": snap["timestamp"],
            "status": snap.get("status")}


def recover_rotten(report):
    """Augment a link-rot report with Wayback captures for broken rows."""
    broken = report[report["category"].isin(
        ["client_error", "server_error", "timeout", "dns_fail",
         "redirect_offsite"])]
    recoveries = []
    for url in broken["url"]:
        cap = find_capture(url)
        recoveries.append({"url": url, **(cap or {})})
    return pd.DataFrame(recoveries)

recovered = recover_rotten(report)
# => columns: url, wayback_url, captured_at, status
```

Two details bear emphasis. First, a Wayback URL is citable: if the original source is gone, cite the Wayback capture with its timestamp. Second, the Archive allows you to *request* a capture through a separate endpoint (`https://web.archive.org/save/<url>`), which you should treat as a standard move whenever you cite a page you expect to change. Saving a copy at the moment of citation shifts you from hoping the URL will still work to knowing a snapshot exists.

> **In the Public Interest**
>
> When you archive a source at the moment of citation, you are exercising a form of ownership the law does not require of you. The page is not yours. You did not write it. You do not host it. What you hold is a responsibility for the evidence your argument rests on. Chapter 8 will develop this claim: in a public-interest practice, **ownership** is stewardship over time, not private control. The link you save today is a small instance of that stewardship. Chapter 22 teaches the full form, where you deposit datasets and documentation with resolvable identifiers (Zenodo DOIs, Internet Archive items) that outlive any one organization. Between the two is the working habit this chapter asks of you: treat every URL you cite as perishable, and treat capture-at-citation as the default, not the exception.

4.4. Datasheets as a retention practice

A link-rot audit tells you what is missing from the outside. A **datasheet** tells the next reader what the thing you produced actually was, so that when pieces of it erode, there is a record of what to look for. Gebru and colleagues (2021) proposed datasheets for datasets as a deliberate import from electronics engineering: every component ships with a spec sheet documenting what it is, how it was made, what it is for, and what it is not. Machine-learning datasets routinely ship with nothing, and the absence produces exactly the retroactive reconstructions Denton and colleagues (2021) had to do for ImageNet.

A datasheet is not a README. It is a structured document with seven sections, and the discipline of filling each one out is what makes it useful:

1. **Motivation.** Who made this, for what purpose, funded by whom? What gap did it fill?

2. **Composition.** What does an instance look like? How many are there? Are there labels, and how were they produced? Are there known subgroup imbalances?
3. **Collection process.** Where did the data come from? When? Through what mechanism? Were the subjects informed? Was consent obtained? What ethical review, if any?
4. **Preprocessing, cleaning, labeling.** What transformations were applied between raw and released? Is the raw version available? What filtering removed what?
5. **Uses.** What has this been used for? What should it not be used for? Are there tasks for which it is actively unsuitable?
6. **Distribution.** How is it distributed? Under what license? With what restrictions? Through what channel, and what is the expected lifetime of that channel?
7. **Maintenance.** Who maintains it? How will errata be communicated? Is there a versioning scheme? When is the next review?

The maintenance section will almost always be blank on the first pass, which is the point: if you cannot name the maintainer or the review cadence, you are publishing a dataset with no plan for its erosion. That is worth knowing before you publish.

```
DATASHEET_TEMPLATE = {
    "motivation": {
        "purpose": "",
        "creators": "",
        "funding": "",
        "gap_addressed": "",
    },
    "composition": {
        "instance_type": "",
        "n_instances": None,
        "labels": "",
        "subgroup_imbalances": "",
```

```
        "sensitive_attributes": "",
    },
    "collection_process": {
        "source": "",
        "period": "",
        "mechanism": "",
        "consent": "",
        "ethical_review": "",
    },
    "preprocessing": {
        "transformations": "",
        "raw_available": False,
        "filtering": "",
    },
    "uses": {
        "prior_uses": "",
        "inappropriate_uses": "",
    },
    "distribution": {
        "license": "",
        "channel": "",
        "restrictions": "",
        "expected_lifetime": "",
    },
    "maintenance": {
        "maintainer": "",
        "errata_channel": "",
        "versioning": "",
        "next_review": "",
    },
}
```

Fill this in once, by hand, for a dataset you have actually used. The first

datasheet will take several hours and will be worse than you expected. By the third, you will notice yourself asking questions at dataset-selection time that you did not ask before, which is the actual point of the practice.

4.5. Why "just put it on GitHub" is not a preservation strategy

Students and, increasingly, agencies treat GitHub as a preservation layer. It is not one. GitHub is a hosting service owned by Microsoft, with no long-term commitment to any particular repository and a history of taking down content for reasons ranging from DMCA claims to policy disputes. Repositories can be made private, renamed, force-pushed over, or deleted by their owner. The commit hash is stable; the URL that resolves it is not. Software Heritage mirrors much of public GitHub, which is good, but public awareness of Software Heritage is limited.

The same logic applies to Google Drive links in syllabi, Dropbox links in footnotes, and Notion pages in institutional documentation. Each is a commercial service on a project-time clock. For anything you need to cite, publish through a channel designed for infrastructure time: a Zenodo deposit with a DOI, an Internet Archive item, an institutional repository your university has committed to maintaining. Chapter 22 walks through the mechanics.

4.6. Closing the triptych

Chapter 2 asked who can observe; Chapter 3 asked who must answer; this chapter asks what can be remembered. Together they are Part I's working diagnosis. Access has narrowed, accountability is exempt, and the record that would let us reason about either is eroding faster than the institutions charged with preserving it can match.

Part II turns to the values that make a response coherent. Chapter 5 names the installed base explicitly, the six-element framework Keegan (2026) proposes and that this book has been gesturing at since Chapter 1. The link-rot audits and datasheets this chapter asks you to write are the first practical instances of what the next part will call **continuity**.

4.7. Exercises

Exercise 4.1 (Guided). Pick a document with at least twenty external URLs: a syllabus, a paper's reference list, a policy brief, a government report. Run the link-rot audit from this chapter over its URLs and save the DataFrame as `exercises/ch04_audit.csv`. In 200 words, describe the distribution of outcomes and name the single most consequential broken link.

Exercise 4.2 (Guided). For three of the broken or offsite-redirecting URLs from Exercise 4.1, use the Availability API to locate captures. Record the original URL, the Wayback URL, the timestamp, and a one-sentence note on whether the capture actually contains what the original cited. Submit as `exercises/ch04_recoveries.md`. Where no capture exists, use the `web.archive.org/save/` endpoint to request one of the current state for future reference.

Exercise 4.3 (Analytic). Using the Gebru and colleagues template, write a full datasheet for a dataset you have used. Fill every section. Where you do not know the answer, say so explicitly rather than leaving the field blank. Save as `exercises/ch04_datasheet_<dataset>.md`. Minimum 1,500 words; the maintenance section must name a human or an institution.

Exercise 4.4 (Comparative). Pick a government webpage whose content you care about (EPA climate-change, a CDC surveillance page, a BLS methodology page). Use the Wayback Machine to locate its earliest capture and its most recent capture before an administration

transition (January 2017 or January 2021 are useful inflection points). Compare the two and, in 500 words, describe what changed: content, framing, linked resources. Do not editorialize; describe. Save as `exercises/ch04_agency_comparison.md`.

Exercise 4.5 (Open-ended). Identify one dataset, corpus, or data portal that you believe will not survive the next decade in publicly accessible form. Write a 600-to-800-word memo naming the resource, the mechanisms you expect to erode it, and a single concrete intervention a student or small nonprofit could take now. The intervention must be something doable in a semester. Save as `exercises/ch04_endangered.md`. You will revisit it in Chapter 22.

4.8. Looking ahead

Part I has now laid out its three pressures. Part II begins with Chapter 5, **The Installed Base**, which names the six-element framework that the rest of the book uses as a working definition of public interest. The link-rot function, the Wayback recovery helper, and the datasheet template you built in this chapter are the first instances of a pattern Chapter 5 will generalize: public-interest data science is less a set of analyses than a set of maintenance practices, carried out in public, by people who are willing to do them when nobody's quarterly review depends on it.

4.9. Further Reading and Resources

- Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford (2021), "Datasheets for datasets," *Communications of the ACM* 64(12): 86–92. The canonical template. Read it before you write your first datasheet; refer back to it every time you write the next.

- Emily Denton, Alex Hanna, Razvan Amironesei, Andrew Smart, and Hilary Nicole (2021), "On the genealogy of machine learning datasets: A critical history of ImageNet," *Big Data & Society* 8(2). Essential counterweight to treating datasheets as a solved problem.
- Jean-Christophe Plantin, Carl Lagoze, Paul N. Edwards, and Christian Sandvig (2016), "Infrastructure studies meet platform studies in the age of Google and Facebook," *New Media & Society* 20(1): 293–310. The paper that names the mismatch between platform time and infrastructure time most clearly.
- Helena Karasti and Jeanette Blomberg (2018), "Studying infrastructuring ethnographically," *Computer Supported Cooperative Work* 27: 233–265. The source for project time versus infrastructure time as analytical categories.
- The Internet Archive Wayback Machine Availability API documentation: https://archive.org/help/wayback_api.php. Short, readable, and the endpoint your audits will lean on.
- David S. H. Rosenthal, *LOCKSS: Lots of Copies Keep Stuff Safe*: https://www.lockss.org/. The library-community preservation infrastructure that most of the web could learn from.
- Software Heritage: https://www.softwareheritage.org/. The mirror-of-record for public code repositories, including substantial GitHub coverage. Useful when you need to cite a commit whose origin repository has disappeared.
- David S. H. Rosenthal's blog at https://blog.dshr.org/. Two decades of patient, unglamorous writing on the economics and mechanics of digital preservation. Start with any post whose title mentions "bit rot."
- Environmental Data and Governance Initiative (EDGI), *Website Monitoring Reports*: https://envirodatagov.org/website-monitoring/. The civil-society project that tracked federal environmental website changes across the 2017 and 2021 administration transitions. An existence proof that the audit practice in this chapter scales.
- Stanford Internet Observatory (2023), "Identifying and eliminating CSAM in generative ML training data and models": https://pu

rl.stanford.edu/kh752sm9123. The technical report that led to the withdrawal of LAION-5B and a case study in what dataset deprecation actually involves.

Part II.

Values

5. The Installed Base

Part I described three pressures on public knowledge: enclosure, exemption, erosion. Each names something that goes wrong. Part II turns to the arrangements that go right when accountability works, or that have to be repaired when it fails. Chapter 6 develops **openness**. Chapter 7 develops **oversight**. Chapter 8 develops **ownership**. Before any of those chapters, you need a shared vocabulary for what those values operate on, and why "add more dashboards" is not a sufficient answer.

That vocabulary is the installed base. Star and Ruhleder (1994) used the phrase to describe the way a new piece of infrastructure never lands on a blank site. It lands on top of prior commitments, prior wiring, prior data formats, prior labor. A new system inherits its predecessors' choices the way a neighborhood inherits its streets: you can pave them, but you cannot move them without moving everything connected to them. Karasti and Blomberg (2018) pick up the idea to show that keeping infrastructure running ("infrastructuring," in their phrasing) is ongoing alignment between what the current system promises and what the installed base will tolerate. Aanestad and colleagues (2017) show the same dynamic in healthcare: national digital-health programs that ignore the installed base of paper forms, clinician workflows, and regional labs fail in the same way, every time.

Keegan (2026) extends the concept. For a public-interest data practice, the installed base is not just the technical stack. It is the socio-technical settlement that makes accountability possible: the records, the access rules, the interpretive conventions, the legal protections, the delegated authorities, and the remedies available to the people affected. When any of those is

missing, the visible parts of the system (the dashboard, the API, the model card) are working on borrowed time. When they are in place, a dashboard is a tool. When they are absent, a dashboard is a performance.

5.1. Six elements

Keegan's framework names six elements. Together they are the minimum conditions under which "public interest" can be said to be served by a data system, rather than invoked on its behalf.

Linkability. Each record the system depends on has a stable identifier that allows it to be referenced, compared, and joined across time and across systems. A case number that changes when the case is moved between databases is not a case number. A dataset version that cannot be pinned is not a version. Linkability is what makes a record a record rather than an observation.

Interpretability. The categories, units, and transformations the data encode are documented well enough that a qualified outsider can reason about what the numbers mean. Bowker and Star (2000) is the canonical argument that classification schemes are never neutral; interpretability is the installed-base requirement that whoever uses the data can see the classification and argue with it. Without it, audits produce numbers without findings.

Continuity. Evidence persists across political cycles, vendor changes, funding gaps, and administrative reorganizations. Chapter 4 gave you the technical shape: link-rot audits, datasheets, Wayback captures. Continuity is the property Karasti and Blomberg call "infrastructure time," maintained against the pull of project time.

Safe scrutiny. Researchers, journalists, auditors, and affected communities can examine the system without being sued, arrested, deplatformed, or retaliated against. Safe scrutiny is the installed-base counterpart to the

Computer Fraud and Abuse Act cases Chapter 7 will treat in detail. In its absence, openness is a trap: the data are technically available, but looking too hard is legally or institutionally punished.

Authority. Someone, a named person or a named office, has the responsibility to act on what scrutiny reveals. An audit that lands on no one's desk is a training exercise. Authority turns a finding into a decision, a decision into a policy change, a policy change into an enforcement action. It is also the element public-interest technologists most often forget to design for, because the technologist's habit is to build the artifact and assume the institution will catch it.

Remedy. The people affected have a path to contest decisions, correct the record, and receive reparation when the system has harmed them. Remedy is the element that most often has no software layer at all. It lives in ombuds offices, administrative appeals, class-action complaints, and community organizing. Any data system that offers no remedy is, for the people at the rough end of it, a system of unilateral power.

Linkability, interpretability, continuity, safe scrutiny, authority, remedy. The order matters: later elements presuppose earlier ones. You cannot offer remedy over records that cannot be linked, and you cannot wield authority over findings nobody can interpret. The list also bakes in a theory of what the next three chapters are for. **Openness** (Chapter 6) operationalizes linkability, interpretability, and parts of continuity. **Oversight** (Chapter 7) operationalizes safe scrutiny and authority. **Ownership** (Chapter 8) operationalizes remedy, and the stewardship that keeps continuity from collapsing into somebody's weekend hobby.

5.2. What goes wrong when an element is missing

The framework earns its keep diagnostically. Consider the Allegheny County Family Screening Tool, introduced in Chapter 3 and picked up in Chapter 7. Linkability is good: each hotline call is identified and

joined to administrative records. Interpretability is partial: features are documented; weights are not always easy to reason about. Continuity is maintained by a named team at the county and at Auckland University of Technology. The system is weakest at the tail. Safe scrutiny is contingent on the county's willingness to cooperate with outside researchers, which has varied. Authority lives with call-screeners who can override the score but report social pressure not to. Remedy for a family flagged and investigated is almost nonexistent: you cannot unring a caseworker visit, and the administrative record persists. This is an unusually documented system by the standards of public-sector ML. The framework lets you name precisely where the weakness lives.

Consider a contrasting case. A city publishes a real-time "open data" portal with dozens of datasets updated nightly. Linkability and interpretability are strong within each dataset. Continuity is weak: nobody has committed to preserving pre-publication raw data, the portal's vendor contract is up for rebid every three years, and every URL path will change the next time the vendor changes. Safe scrutiny is strong in the abstract and untested in practice. Authority is diffuse: the datasets are "owned" by a half-dozen departments, none of whom see responding to outside findings as their primary job. Remedy is absent. A glance at the portal would give you a five-star "open data" rating. The installed base underneath would give you a shrug.

The framework does not tell you what to do about either failure. It tells you where to look, so that the intervention lands somewhere that will hold it.

> **In the Public Interest**
>
> The installed base is where openness, oversight, and ownership stop being rhetorical and become operational. **Openness** (Chapter 6) is not a slogan about transparency; it is the working combination of linkability, interpretability, and the portions of continuity that let

> outsiders join and compare records over time. **Oversight** (Chapter 7) is not a regulator's aspiration; it is the working combination of safe scrutiny and authority. **Ownership** (Chapter 8) is not private property over data; it is the stewardship that sustains continuity and the remedy paths that connect records to people. When you see the three values named abstractly elsewhere, translate back to the six elements. That translation is the discipline this chapter teaches.

5.3. A minimum viable installed base

Enough conceptual work. The rest of the chapter builds a concrete installed base for a small civic dataset. You cannot talk seriously about any of the six elements without doing a version of this work at least once.

The dataset is a procurement log for a fictional small city the book provides: 500 rows of vendor, amount, award date, and award type. The book ships it with two deliberate failures. Some rows are missing identifiers. One transformation step, a mid-pipeline currency conversion, is undocumented. Your job is to fix both and, in doing so, touch every one of the six elements.

5.3.1. Linkability: persistent identifiers

First, look at what you have.

```
import pandas as pd

df = pd.read_csv("data/smallcity_procurement.csv")
df.head()
# => columns: record_id, vendor, amount, award_date, award_type
#    record_id is populated for 472 of 500 rows; 28 rows are blank.
```

```
df["record_id"].isna().sum()
# => 28
```

The 28 blank identifiers are the first gap. Two reasonable choices: a random UUID, or a hash derived from the record's own content. A UUID is opaque and will not change if the record is corrected; a content hash is deterministic and detects silent changes, but it shifts the moment anyone fixes a typo.

For civic records, where correction is expected and auditability matters, a compromise works. Assign a UUID as the canonical identifier. Also compute a content hash, store it in a separate column, and keep a history of hash values as a change log.

```
import hashlib
import uuid

def content_hash(row):
    payload = f"{row['vendor']}|{row['amount']}|{row['award_date']}|{row['awa
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()[:16]

missing = df["record_id"].isna()
df.loc[missing, "record_id"] = [str(uuid.uuid4()) for _ in range(missing.sum
df["content_hash"] = df.apply(content_hash, axis=1)
df.to_csv("data/smallcity_procurement_v2.csv", index=False)
```

You now have a stable reference for every record and a side channel that flags when a record's substance has changed. That is linkability in minimum viable form. The social work of linkability, keeping the ID stable across vendor migrations and administrative reorganizations, is not solved by the code.

The Missing Manual

UUIDs are the easy part. Every language ships a generator. The hard part is durable reference, and durable reference is social work. Somebody has to maintain the resolver (the service that turns an identifier into a record), negotiate the naming authority (who is allowed to mint identifiers), and make the migration commitment (when we move systems, we carry these identifiers forward). DOIs work because Crossref and DataCite exist as institutions, not because the string `10.1234/...` is clever. ORCIDs work because the ORCID consortium has a funding model. Your procurement log's identifiers will work if, and only if, some office commits to carrying them across the next three vendor contracts. If nobody has signed up, you have a UUID, not an identifier. The difference is everything.

5.3.2. Interpretability: a provenance document

Now the undocumented transformation. Inspect the amount column and you will notice something:

```
df["amount"].describe()
# => min is suspiciously close to zero, max is an order of magnitude above
#    reasonable small-city contracts. The column mixes USD and some EUR rows
#    from a pass-through transformation nobody wrote down.
```

An outsider cannot know what the numbers mean. An auditor who assumes USD will misreport. The fix is not code; it is a provenance document committed alongside the data, in a format a human and a machine can both read.

```yaml
# data/provenance.yaml
dataset: smallcity_procurement
version: "2.0"
upstream_source:
  name: Small City Department of Finance, SAP export
  accessed: 2026-03-12
  contact: finance@smallcity.gov
transformations:
  - step: 01_extract
    description: CSV export from SAP AP module, fields mapped per schema v4.
  - step: 02_currency_normalize
    description: >
      Rows with vendor country code in {DE, FR, IE, NL} carry EUR amounts.
      Converted to USD using ECB monthly reference rate for award_date month
    rate_source: https://www.ecb.europa.eu/stats/policy_and_exchange_rates/
    applied_by: bkeegan
    applied_on: 2026-03-13
  - step: 03_identifier_assignment
    description: UUID minted for 28 rows with missing record_id; content_hash
  - step: 04_publication
    channels:
      - public_portal
      - authenticated_api
      - restricted_view
known_issues:
  - Pre-2022 rows inherited from legacy ERP; vendor spelling unnormalized.
  - EUR-to-USD conversion uses monthly averages; daily rates not preserved.
```

This is the interpretability layer. Without it, the dataset is a column of numbers that behaves badly. With it, the dataset carries its own explanation. Commit the YAML to the same repository as the data.

5.3.3. Continuity: a retention policy

Continuity without a retention policy is an act of faith. Write the policy down, in a format that would survive the person who wrote it leaving the job.

```yaml
# data/retention.yaml
dataset: smallcity_procurement
schedule:
  raw_extracts:
    retain_for: 10_years
    storage: city_records_archive
    medium: encrypted_s3_with_offline_tape_copy
  published_versions:
    retain_for: indefinite
    storage: zenodo_deposit_plus_ia_mirror
    minimum_review: annual
  derivation_scripts:
    retain_for: indefinite
    storage: city_gitlab
  access_logs:
    retain_for: 2_years
    storage: city_siem
    purpose: audit_of_restricted_queries_only
responsible_office: Department of Finance, Data Stewardship Lead
review_cadence: annual
escalation_contact: clerk@smallcity.gov
```

The values chapters that follow will press on each field. What matters here is that the document exists, names a responsible office, and survives independently of the person who set it up.

5.3.4. Safe scrutiny and authority: tiered access

Not every field in a public dataset should be equally public. Vendor names and contract totals are public records in almost every jurisdiction. Bank account numbers and unredacted home addresses of sole proprietors often are not. A small, honest access tier lets you publish openly without publishing recklessly.

Use SQLite views to express the tiers. The underlying table is the full record; each view is a lens on it, and access is governed by an application-layer credential.

```
import sqlite3

con = sqlite3.connect("data/smallcity_procurement.db")
con.executescript("""
DROP VIEW IF EXISTS v_public;
DROP VIEW IF EXISTS v_authenticated;
DROP VIEW IF EXISTS v_restricted;

CREATE VIEW v_public AS
SELECT record_id, vendor, amount, award_date, award_type
FROM procurement;

CREATE VIEW v_authenticated AS
SELECT record_id, vendor, amount, award_date, award_type,
       contact_email, contract_reference
FROM procurement;

CREATE VIEW v_restricted AS
SELECT *
FROM procurement;
""")
con.commit()
```

In production you would enforce the tiering with row-level security, OAuth scopes, or an API gateway. For a pedagogical installed base, the view definitions make the commitments legible. Anyone reading the schema can see what a public user, an authenticated user, and a restricted internal user are authorized to see. The access tier is the instrument that makes safe scrutiny possible without collapsing into either "publish everything" or "publish nothing."

5.3.5. Remedy: an escalation log

Access without escalation is a suggestion box with no one watching it. The last piece is a record of who gets notified when which kind of query arrives. Keep it boring.

```yaml
# data/escalation.yaml
routes:
  - trigger: public view query volume > 10k/hour
    notify: [data_stewardship_lead, comms]
    purpose: anticipate press or scraping
  - trigger: restricted view accessed
    notify: [records_officer, audit_log]
    purpose: per-access review
  - trigger: correction request received
    notify: [records_officer, affected_department]
    sla: 10_business_days
    remedy: issue corrected row, log in change_history
  - trigger: removal request received (privacy or safety)
    notify: [city_attorney, records_officer]
    sla: 5_business_days
    remedy: redaction + tombstone record retained
```

The point of the escalation log is to make explicit, in one file under version control, what happens after a query runs. Without it, the installed base

routes everything to nobody. With it, the path from observation to remedy has at least been drawn.

5.4. What this demonstrates

You have just built, in roughly a hundred lines of code and markup, a minimum viable installed base. Linkability: UUIDs and content hashes. Interpretability: a provenance document. Continuity: a retention policy. Safe scrutiny: tiered views. Authority and remedy: an escalation log with named offices and SLAs. Each artifact is a stand-in for a much larger professional practice you will meet again. None of them does its work without the others.

Notice what this is not. It is not a dashboard. It is not a model. It is not an "AI governance" framework. It is the boring, legible plumbing under any of those things, and the book's argument is that without the plumbing, the dashboards and models and frameworks collapse the moment anyone tries to use them for what they claim to do. Keegan (2026) calls this the infrastructuring move: treating what looks like a platform problem as an installed-base problem.

Chapter 6 begins the values half of Part II by asking what openness actually requires, starting from linkability and interpretability as its operational components. The procurement dataset you have just instrumented will reappear there, extended into a Datasette-published artifact with a public URL. Chapter 7 will come back to the same dataset through the lens of oversight, asking what it takes for a researcher or journalist to scrutinize it without legal risk. Chapter 8 will return once more, asking who owns it, who stewards it, and how a person harmed by a procurement decision reaches a remedy.

The installed base is how those three chapters connect. Carry the vocabulary forward.

5.5. Exercises

Exercise 5.1 (Guided). Download the book's `smallcity_procurement.csv` dataset. Identify the 28 rows missing identifiers and assign a UUID to each, following the pattern in this chapter. Add a `content_hash` column for every row. Save the result as `exercises/ch05_procurement_v2.csv`. In 150 words, explain why you chose UUID rather than content hash as the canonical `record_id`.

Exercise 5.2 (Guided). Write the `provenance.yaml` for the procurement dataset. Include at least the five transformation steps shown in this chapter, and add one additional step of your own (for example, a vendor name normalization pass, or a deduplication). Save as `exercises/ch05_provenance.yaml`. The document must name a responsible office and at least one known issue.

Exercise 5.3 (Guided). Draft a retention policy for the same dataset. Use the template in this chapter as a starting point and adapt at least three values (retention periods, storage locations, responsible office) to a city or organization you know something about. Save as `exercises/ch05_retention.yaml`. In 200 words, justify the retention periods you chose, citing at least one real records-retention schedule (your city, your state, or a federal agency).

Exercise 5.4 (Applied). Implement the three-tier access scheme as SQLite views in a real database built from the procurement CSV. For each of three example queries (a total by vendor, a lookup by contract reference, a full row export) say which tier can run it and why. Save the database as `exercises/ch05_procurement.db` and the writeup as `exercises/ch05_access.md`.

Exercise 5.5 (Open). Design an escalation pathway and a threat model for a small civic dataset of your own choosing (a school board meeting minutes scrape, a local housing-permit feed, a campus crime log). Your pathway must name at least three triggers, the offices or individuals notified,

and a service-level agreement for each. Your threat model must name at least three adversaries (not all of them malicious; an underfunded vendor is an adversary in this sense) and one failure mode for each. Save as `exercises/ch05_escalation.md`, 800 to 1,200 words. You will revisit this artifact in Chapter 7.

5.6. Looking ahead

Part II continues in Chapter 6 with **openness**, the first of the three values. The chapter takes the linkability and interpretability pieces you have just built and asks what it takes to publish them in a form that an interested outsider can actually use. The procurement dataset will move from a local SQLite file to a Datasette instance with a resolvable URL, a license, and a documented schema, and the discussion will press on what "open" has to mean to count as open for public-interest purposes.

5.7. Further Reading and Resources

- Brian C. Keegan (2026), "Public interest data infrastructuring," *Information, Communication & Society*. The paper that names the six-element framework. The rest of Part II leans on it constantly.
- Susan Leigh Star and Karen Ruhleder (1994), "Steps toward an ecology of infrastructure," *CSCW 1994*, 253–264. The original "installed base" argument.
- Helena Karasti and Jeanette Blomberg (2018), "Studying infrastructuring ethnographically," *CSCW* 27: 233–265. The clearest articulation of infrastructuring as alignment work.
- Margunn Aanestad, Miria Grisot, Ole Hanseth, and Polyxeni Vassilakopoulou (2017), "Information infrastructures and the challenge of the installed base." The healthcare-systems reading that makes clear why ignoring the installed base is not a niche failure.

- Geoffrey C. Bowker and Susan Leigh Star (2000), *Sorting Things Out*. Indispensable for the interpretability element.
- Ben Hutchinson and colleagues (2021), "Towards accountability for machine learning datasets," *FAccT 2021*. A complementary framework for ML dataset governance.
- The DataCite schema documentation: https://schema.datacite.org. A production-grade persistent-identifier standard backed by an institutional community.
- UK Government Digital Service, "Service Manual," https://www.gov.uk/service-manual. An institution treating the installed base as a design problem.
- The 18F "Before You Ship" checklist, https://before-you-ship.18f.gov/. A reference for what a minimal production installed base looks like.

6. Openness

Chapter 5 argued that the six elements of the installed base are the conditions under which "public interest" can be said to be served by a data system rather than invoked on its behalf. This chapter takes the first three of those elements (linkability, interpretability, and the openness-facing portion of continuity) and turns them into a practice. The practice has a name most readers will already have heard. It is called open data, and the name is doing too much rhetorical work.

Open data as a programmatic idea is roughly twenty years old. It has produced real goods: the federal data.gov portal, municipal 311 feeds, FEC disbursement extracts, the census ACS API. It has also produced a great deal of what practitioners now call, with a mixture of resignation and contempt, the data dump. A data dump is a civic file dropped onto a portal with a permissive license, no documentation, inconsistent columns, encoding errors, and a filename like `final_final_v3.xlsx`. It is technically open. It satisfies the ordinance. For any public that does not already work inside the agency, it is functionally unusable.

The distinction this chapter insists on is between **formal openness** and **durable legibility**. Formal openness is what a compliance officer can certify: the file exists at a URL, the license permits redistribution, the API returns a 200. Durable legibility is what a non-expert can actually do with the thing six months from now: load it, understand its columns, cite it, compare it to last year's version. Formal openness without legibility reproduces exemption under a different name. The agency has technically satisfied disclosure while ensuring that nobody outside can use the disclosure to act.

6.1. Posting is not publishing

An analogy with scholarly publication is useful. A preprint PDF on a researcher's hard drive is not a publication. A PDF uploaded to a webserver with no metadata is not a publication either, though it meets some naive definition of "posted." A publication is the combination of a document, a record about the document (title, author, date, version, license), a citable handle (a DOI, an ISBN, a versioned URL), and a commitment from some institution that the handle will continue to resolve. Publication is a social act, not a file-system operation.

The same distinction applies to data. `df.to_csv("out.csv")` is a file-system operation. It produces a file. It does not produce a publication. A published dataset requires, at minimum, provenance (where did this come from, what was done to it), documentation (what the columns are and what they mean), a license, a citation, and a hosting commitment (where it will live, and for how long). The FAIR principles (findable, accessible, interoperable, reusable) name four of these without quite naming the last. This chapter adds legibility-to-publics as the extension FAIR understates: a dataset that is machine-interoperable and human-inscrutable is FAIR and useless. Hutchinson and colleagues (2021) make the adjacent argument for machine-learning datasets, borrowing from software engineering the habit of treating a dataset as an artifact with a lifecycle, not a file.

Zuiderwijk and Janssen (2014) reviewed national open-data policies and found the same pattern across countries: the policies are clearest on what agencies must post and weakest on what they must sustain. You can write an open-data ordinance in an afternoon. You cannot write, in any afternoon, the ongoing labor that keeps a civic dataset findable, legible, and citable three budget cycles from now. That labor is what Star and Ruhleder (1994) named as the installed base, and what Karasti and Blomberg (2018) insisted had to be studied as work rather than assumed as context. Openness, in this chapter's operational sense, is the configuration of artifacts and

commitments that makes that labor repeatable by someone other than its originator.

6.2. Licensing as a governance choice

One short detour before the tutorial. An open license is not a technical checkbox. It is a small act of governance. The license you attach to a civic dataset tells downstream users what they may do, what they must do in return, and what you expect when your dataset is recombined with others.

Three license families come up repeatedly. **CC-BY 4.0** (Creative Commons Attribution) permits any reuse as long as attribution is given. It is appropriate for most municipal datasets where the agency wants credit trails but does not want to constrain reuse. **CC0** waives copyright entirely, placing the dataset as close to the public domain as a contract can reach. It is appropriate when the dataset is already the product of public work and should have no friction at all. **PDDL** (the Open Data Commons Public Domain Dedication and License) is CC0's database-specific cousin, designed for jurisdictions where databases have a separate copyright status from their contents. For U.S. civic data, CC-BY 4.0 or CC0 will cover almost every case.

What to avoid: "non-commercial" restrictions (CC-BY-NC). Journalists and small newsrooms sell their work. A non-commercial clause tells the local paper it cannot use the dataset without a lawyer's letter. That is not openness. It is a defensive crouch dressed up in a compliance costume.

6.3. The running example: Boulder campaign finance

The tutorial uses a campaign finance extract from a city open-data portal. Campaign finance is the most politically interesting of the common civic

datasets because it sits at the intersection of accountability and messiness. The file tends to be a CSV with predictable problems: dates as strings, currency with dollar signs and commas, donor names inconsistently capitalized ("SMITH, JOHN" on one row, "John Smith" on the next, "j smith" on a third), employer and occupation fields sometimes blank and sometimes filled with "SELF" in several creative spellings, and an `amount` column that occasionally contains negative values for refunds without documenting that convention.

You start from the raw file, `boulder_campaign_finance_raw.csv`, which the book provides in its data folder. In practice you would download the equivalent from the city's portal. The portal will offer a filename like `Campaign_Finance_Filings.csv` with no versioning and no changelog; that is the first problem you will document. Treat the raw file as immutable. Do not edit it in Excel. Excel will silently coerce columns and you will not find out until a reader writes to tell you the dates are off by a month.

6.3.1. Ingest and inspect

```
import pandas as pd

raw = pd.read_csv(
    "data/boulder_campaign_finance_raw.csv",
    dtype=str,            # read everything as string first
    keep_default_na=False # do not turn empty strings into NaN yet
)
raw.shape
# => (3421, 11)

raw.columns.tolist()
# => ['Filing Id', 'Contributor', 'Address', 'Employer',
#     'Occupation', 'Amount', 'Date', 'Election',
```

```
#     'Candidate', 'Committee', 'Filing Type']

raw.head(3)
```

Reading everything as string on first pass is a habit worth keeping. Pandas's default type inference is confident and frequently wrong: it will read a zip code with a leading zero as an integer, or a dollar amount as a string if one row has a stray character. Taking inference off the table lets you see what is actually there.

6.3.2. Clean

```
import re

df = raw.copy()

# Lowercase column names, snake_case
df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]

# Dates: the portal gives mm/dd/yyyy; parse and keep ISO 8601
df["date"] = pd.to_datetime(df["date"], format="%m/%d/%Y", errors="coerce")
df["date"].isna().sum()
# => 4   four rows had a blank or malformed date; keep them, flag them

# Currency: strip $ and , then cast to float
df["amount_raw"] = df["amount"]
df["amount"] = (
    df["amount"]
    .str.replace(r"[\$,]", "", regex=True)
    .replace("", pd.NA)
    .astype("Float64")
```

```
)

# Donor names: title-case, collapse whitespace
def normalize_name(name):
    if not isinstance(name, str):
        return pd.NA
    name = re.sub(r"\s+", " ", name).strip()
    # "SMITH, JOHN" -> "John Smith"
    if "," in name:
        last, first = [p.strip() for p in name.split(",", 1)]
        return f"{first.title()} {last.title()}"
    return name.title()

df["contributor_clean"] = df["contributor"].apply(normalize_name)

# Preserve the original; do not destroy evidence
assert "contributor" in df.columns
```

Three habits are worth naming. First, every destructive transformation keeps the original column. `amount_raw` survives next to `amount`; `contributor` survives next to `contributor_clean`. If a reader disputes your normalization, they can recover what was in the source. Second, the date parse uses `errors="coerce"` rather than silently dropping failures. The four malformed dates stay visible in the frame so a datasheet can mention them. Third, the normalization is documented inline, because "we title-cased the donor names" is a choice future you will have forgotten in six weeks.

6.3.3. Normalize into an analysis-ready frame

```
keep = [
    "filing_id", "date", "contributor_clean", "contributor",
    "address", "employer", "occupation",
    "amount", "amount_raw",
    "election", "candidate", "committee", "filing_type",
]
clean = df[keep].sort_values("date").reset_index(drop=True)
clean.to_csv("data/boulder_campaign_finance_clean.csv", index=False)
clean.to_parquet("data/boulder_campaign_finance_clean.parquet")
```

Two formats, intentionally. CSV is the compatibility format. Parquet preserves the types you just fought to recover, so whoever reads it six months from now does not repeat your work. Publish both.

6.4. Writing the README

A dataset's README is the first thing a non-specialist will read and often the only thing. Write it as prose, not as bullet points of field names. At a minimum, it needs six sections:

1. **What this is.** One paragraph, plain language, for a reader who has never heard of the dataset.
2. **Provenance.** Where the raw data came from, with a URL and a retrieval date. What laws or ordinances require its publication.
3. **What was done to it.** A short narrative of the cleaning steps, with a pointer to the script that produced the cleaned file.
4. **Column documentation.** Every column, one line each, in file order. Units. Missing-value conventions.
5. **License.** Named (e.g., CC-BY 4.0) with a link to the canonical text.
6. **Canonical citation.** How the dataset should be cited. This is where a DOI goes once you have one; see Chapter 22 for the Zenodo workflow. Until then, a versioned URL and a date.

A README that fits on a single page and covers those six sections outperforms, in practice, an 80-page PDF "data dictionary" that nobody reads.

6.5. The datasheet

A README is for users. A **datasheet** is for auditors, downstream reusers, and future maintainers who need to know how the dataset came to exist and what it is and is not fit for. Gebru and colleagues (2021) proposed the form, adapting it from hardware datasheets. Chapter 4 explained why every dataset worth preserving needs one. This chapter is where you write one.

A compact datasheet for the campaign finance file has seven sections:

- **Motivation.** Why was this dataset created? Who funded it?
- **Composition.** What do the rows represent? What are the columns? Are there sensitive fields (addresses, employer names) and what is their status?
- **Collection process.** How was the data gathered? By whom? Under what legal authority?
- **Preprocessing.** What cleaning, filtering, or normalization has been applied between the raw source and this release? (Point to the script.)
- **Uses.** What has this dataset been used for? What uses are appropriate? What uses should be approached with caution?
- **Distribution.** Under what license? At what URL? With what permanence commitment?
- **Maintenance.** Who is responsible for updates? How should errors be reported?

Two pages is plenty. The point is not comprehensiveness. The point is that a reader who will never meet you can, from the datasheet alone, decide whether to trust the file for their purpose.

> **In the Public Interest**
>
> Openness without interpretive legibility reproduces exemption under a different name. Chapter 5 names six installed-base elements; this chapter's work touches three. **Interpretability** is what the README and datasheet produce: the file's categories, units, and transformations become legible to a reader who did not build it. **Continuity** is what versioned, citable hosting produces: the dataset exists under a handle that will still resolve after the next vendor contract or administrative reorganization. **Linkability** is what the stable filing identifiers and ISO-8601 dates preserve: a row in this release can be joined to the same row in next year's release. A civic dataset that skips the README, the datasheet, the versioned citation, and the long-run hosting commitment has satisfied the ordinance and failed the public. That failure is precisely what the six elements were designed to name.

6.6. Publishing with Datasette

Datasette is Simon Willison's open-source tool for publishing SQLite databases as browsable, queryable websites. It is the smallest plausible piece of infrastructure that turns a cleaned CSV into something a non-expert can explore. You point it at a SQLite file; it gives you a web UI with faceted search, a read-only SQL console, a JSON API, and a handful of export formats. For civic datasets in the single-gigabyte range, it is close to ideal.

First, convert the cleaned file to SQLite and serve it locally.

```
pip install datasette sqlite-utils datasette-publish-fly

# Build a SQLite database from the cleaned CSV
```

```
sqlite-utils insert boulder.db contributions \
  data/boulder_campaign_finance_clean.csv --csv --detect-types

# Add metadata: title, license, source, column descriptions
# (metadata.yml author writes this once; datasette reads it)
datasette serve boulder.db \
  --metadata metadata.yml \
  --port 8001
```

Open `http://localhost:8001/` and click around. You should see the table, the column facets, and a search box. If you typed a column description into `metadata.yml`, it will show up as hover text on the column header. If you did not, you will see why you should: the UI will read "column `employer`, no description provided" and your reader will learn nothing.

The `metadata.yml` is where your README's column documentation lives inside Datasette. A minimum version:

```yaml
description_html: |
  <p>Campaign finance disclosures for Boulder municipal elections.
  See <a href="README.md">README.md</a> and
  <a href="datasheet.md">datasheet.md</a> for provenance.</p>
license: CC-BY 4.0
license_url: https://creativecommons.org/licenses/by/4.0/
source: Boulder Open Data Portal
source_url: https://open-data.bouldercolorado.gov/
databases:
  boulder:
    tables:
      contributions:
        description: "One row per reported contribution, post-cleaning."
        columns:
          filing_id: "Unique identifier assigned by the city filing system."
```

```
          date: "ISO 8601 date of the contribution."
          contributor_clean: "Donor name, normalized to Title Case."
          amount: "Contribution amount in USD. Negative values are refunds."
```

6.6.1. Deploying to free hosting

Local Datasette is useful for development. For the class, you want a URL someone else can visit. `datasette publish` wraps a handful of cloud providers; the Fly.io path is the cheapest currently viable.

```
# One-time setup
fly auth signup           # or: fly auth login
# Datasette wraps Dockerfile creation and deploy
datasette publish fly boulder.db \
  --app boulder-campaign-finance \
  --metadata metadata.yml \
  --install datasette-vega \
  --install datasette-cluster-map
```

The `--install` flags pull in two plugins worth knowing. `datasette-vega` gives quick in-browser charts. `datasette-cluster-map` geocodes rows that include latitude and longitude. Neither is essential; both turn a queryable table into something a city council staffer will actually browse.

> **The Missing Manual**
>
> `df.to_csv("out.csv")` is not publication. Publication requires metadata, a citation, a license, and a place the file is committed to live. Until you have all four, you have a file. Two traps in the Datasette workflow are worth flagging before you hit them. First, `datasette publish fly` will cheerfully try to ship a SQLite file larger than the free tier's 512 MB container limit and silently fail with an error that

> reads, to the uninitiated, like a network problem. If your database is big, filter it down, move to a paid tier, or split it into a separate "archive" database you publish less often. Second, the free tier's machine sleeps after inactivity, so the first visitor each morning waits ten seconds for a cold start. None of this is in the quickstart. All of it will happen to you.

6.6.2. Registering a citation

A live Datasette URL is a useful demo. It is not, by itself, a citation. URLs drift, organizations shut down, Fly.io apps get renamed. For a dataset you want to cite in a report or a peer-reviewed paper, you want a DOI, the citation infrastructure the research world has settled on. The forward reference is Chapter 22, which walks through depositing on Zenodo and getting back a DOI you can include in the README's "canonical citation" section. The layered model is: Datasette is the interactive front door, a versioned Zenodo deposit is the permanent record, and the README ties them together with a clear "cite this" block. Compare with Chapter 8 on access tiers when some of the underlying data cannot be fully open.

Before you publish, one last sanity pass. Chapter 16 makes the case for a category audit before release: read the columns again with the question "is there anyone whose interests are harmed by the way this is named, categorized, or made joinable?" Campaign finance is less fraught than, say, 311 calls or housing-code violations, but even it exposes addresses and employers. The public-interest practice is not to stop publishing. It is to publish with the audit documented.

6.7. Exercises

Exercise 6.1 (Guided). Download the provided raw campaign finance CSV (or pull an equivalent file from your own city's open-data portal). Reproduce the ingest, clean, and normalize steps in this chapter. Commit the cleaning script and both output formats (CSV and Parquet). Write 150 words describing one decision you made during cleaning that a reasonable person could have made differently.

Exercise 6.2 (Guided). Write a README and a datasheet for the dataset you produced in 6.1, using the section templates in this chapter. Target length: one page of README, two pages of datasheet. Save both as Markdown in `exercises/ch06_openness/`. Include a license choice (CC-BY 4.0 or CC0) and justify it in one sentence.

Exercise 6.3 (Technical). Install Datasette locally. Build a SQLite database from your cleaned file. Write `metadata.yml` with title, license, source, and column descriptions. Run `datasette serve` and browse the result. Submit a screenshot of the column-description hover text working.

Exercise 6.4 (Deploys to the internet). Deploy your Datasette instance to a free hosting tier (Fly.io with `datasette publish fly`, or an equivalent you document). Share the URL with the class. In 200 words, describe one thing you changed between the local and deployed versions, and one thing that broke at deploy time.

Exercise 6.5 (Diagnostic). Find a published civic dataset in the wild: a municipal portal, data.gov, a state disclosure site. Diagnose how it fails at least one of Chapter 5's six installed-base elements. Write 400 words. Name the dataset, the agency, and the specific element; describe what a reader trying to use the data would encounter; and propose one concrete change that would close the gap. Good candidates tend to involve continuity (the dataset updates in place with no versioning) or interpretability (columns are documented by field code and nothing else).

6.8. Looking ahead

This chapter built the openness side of the installed base. Chapter 7 takes up **oversight**: what happens when openness has done its job and somebody starts asking inconvenient questions of the data. Oversight depends on openness having been done well (you cannot oversee what you cannot read) but is not reducible to it. Safe scrutiny and authority are distinct problems with their own tools. Chapter 7 treats disaggregated audits, model cards, and the legal environment in which outside scrutiny is conducted. Bring the published Datasette with you. We will put it under an auditor's gaze.

6.9. Further Reading and Resources

- Simon Willison's Datasette documentation: https://docs.datasette.io/. The canonical reference. Read "getting started" and "metadata" before the deploy section.
- Datasette Cloud: https://www.datasette.cloud/. Willison's hosted option when you do not want to run Fly.io yourself.
- Simon Willison's weblog: https://simonwillison.net/. Running commentary on Datasette, `sqlite-utils`, and the surrounding ecosystem.
- The FAIR Guiding Principles: https://www.go-fair.org/fair-principles/. The four-letter scaffold this chapter extends.
- Creative Commons license chooser: https://creativecommons.org/choose/. A three-question form that produces an appropriate license and boilerplate.
- Open Knowledge Foundation, *The Open Data Handbook*: https://opendatahandbook.org/. The movement's original playbook; still the clearest short treatment of what "open" means.
- Open Data Commons: https://opendatacommons.org/. PDDL and ODbL, for cases where Creative Commons does not quite fit.

- Sunlight Foundation archives: https://sunlightfoundation.com/. Ongoing critique of data.gov's drift and the U.S. open-government movement.
- Gebru et al. (2021), "Datasheets for datasets," *Communications of the ACM* 64(12): 86–92. The source for this chapter's datasheet template.

7. Oversight

In 2018, Joy Buolamwini and Timnit Gebru published "Gender Shades," a disaggregated audit of three commercial gender-classification systems (Buolamwini and Gebru 2018). The headline finding is familiar: the systems misclassified darker-skinned women at rates up to 34 percentage points higher than lighter-skinned men. The methodological finding is less famous and more important. Aggregate accuracy had hidden the disparity. Every vendor's marketing materials had reported a single accuracy number, and every single-number report was technically true and substantively misleading. The audit did not discover a new failure mode. It discovered a disclosure regime designed not to show it.

That is what oversight is for. It is the work of making a system's behavior visible to parties other than its builders, in a form those parties can use. "Visible" is a demanding word. A datasheet on a vendor website is not visibility if nobody with authority to act on it ever reads it. A one-off academic audit is not visibility if the vendor is free to change the model the next day. Buolamwini and Gebru's paper mattered not only because of what it found, but because it was followed by sustained pressure, by IBM's and Microsoft's decisions to pause face-recognition contracts, by the organizing that Raji and colleagues (2020) later formalized into an end-to-end internal auditing framework, and by legislation that cited the work by name. An audit is a technique. Oversight is an arrangement.

This chapter is about the difference. You will learn to produce a disaggregated audit using `fairlearn`, and you will learn what kinds of audit regimes hold up under political pressure versus collapse into what Goodman and Trehu (2023) call *audit-washing*: the performance of accountability in

place of its substance. The technical skill transfers. The political judgment about where the skill is worth deploying does not transfer automatically, and that is the harder half of the chapter.

7.1. Routine auditability versus episodic audits

Two arrangements get called "audits" and they are not the same thing.

Routine auditability is built into a system. Logs are kept in a form an outsider can parse. Model versions are signed and archived. Performance metrics are recomputed and published on a schedule. When a disaggregated regression is run, the inputs it needs already exist, and the person running it does not need anyone's permission. This is the regime that public accounting aspires to in principle and sometimes achieves in practice. Baker's (2005) history of "the public interest" in American accounting traces how this aspiration has been narrowed and co-opted over a century, and his warning is worth keeping in view: audit professions do not stay public-interested on their own.

Episodic audits are one-off events. A regulator commissions a report. A researcher publishes a paper. A journalist files a records request and runs numbers. These can be powerful, but their power is bounded by the moment and by the willingness of the audited party to cooperate a second time. Gender Shades was episodic. The follow-up work that turned it into sustained pressure was closer to routine.

Most of what the 2020s legislative wave has produced sits uncomfortably between the two. New York City's Local Law 144, which requires "bias audits" for automated employment decision tools, is the instructive case. The law took effect in 2023. It mandates annual disaggregated impact ratios for employers using covered tools. It does not specify what counts as an adequate auditor, how data must be collected, or what happens when the audit finds a problem. Goodman and Trehu (2023) and a wave of empirical studies have documented the result: a compliance rate estimated

at under 20 percent in the first year, audits of wildly varying rigor, and a cottage industry of vendors offering "LL144 bias audits" in formats that would not survive scrutiny by anyone outside the vendor ecosystem. The law created the appearance of routine auditability without the authority to enforce it. What you get is episodic audits pretending to be routine.

The European Union's AI Act, which entered staged application in 2024 and 2025, is a more ambitious attempt. It requires conformity assessments for high-risk systems, post-market monitoring, and documentation that must survive into the market-surveillance phase. Whether it holds up is a question the rest of this decade will answer. Early signs, from the machinery directive and GDPR enforcement histories, are that conformity assessments without vigorous market-surveillance authority become paperwork factories. Openness without oversight (Chapter 6) publishes information nobody acts on. Oversight without remedy (Chapter 5) produces reports nobody can use.

In the Public Interest

Oversight, in the installed-base framework, depends on two elements that the audit itself cannot supply: **authority** and **remedy**. A perfect audit nobody acts on is theater. When you read an "audit report" for a public-sector system, your first two questions should not be about method. They should be: who has the authority to act on this finding, and what remedy is available to the people harmed by the behavior it documents? If the answers are "nobody" and "none," you are reading audit-washing. The technical work in this chapter is a precondition for oversight. It is not oversight.

7.2. A disaggregated audit in `fairlearn`

The rest of the chapter works through a concrete audit. You will use **folktables**, which wraps the American Community Survey (ACS) into tasks with documented protected attributes, and **fairlearn**, which provides disaggregated metric computation. The task is income prediction: given a respondent's demographic and employment features, predict whether their annual income exceeds $50,000. A deliberately banal task. The point is to practice the audit on a problem where you already have intuitions about where disparities are likely to appear.

Install the packages if you have not:

```
# pip install folktables fairlearn scikit-learn pandas
```

Fetch a state-level slice and define the task:

```
from folktables import ACSDataSource, ACSIncome
import pandas as pd

data_source = ACSDataSource(survey_year="2022", horizon="1-Year", survey="per
ca_data = data_source.get_data(states=["CO"], download=True)

features, label, group = ACSIncome.df_to_pandas(ca_data)
# features: demographic/employment columns
# label:    Series of booleans, income > $50k
# group:    Series with racial category codes (RAC1P in the ACS schema)

print(features.shape, label.mean())
# => (roughly 30000 rows, 10 columns), positive rate ~0.45
```

`folktables` returns the RAC1P race code as its default group. You will want a second protected attribute for cross-tabulation. Sex (SEX, 1 =

male, 2 = female in ACS coding) is already in the feature matrix; extract it alongside.

```
sex = features["SEX"].map({1: "male", 2: "female"})
race_map = {
    1: "White",
    2: "Black",
    3: "AIAN",
    4: "Asian",
    5: "NHPI",
    6: "Other",
    7: "Two or more",
    8: "Hispanic (recoded)",
    9: "Other (recoded)",
}
race = group.map(race_map).fillna("Unknown")
```

A few notes on what you just did. You flattened a complex racial classification into a small number of buckets for display. The ACS RAC1P variable itself already did a lot of flattening upstream. Hispanic origin in the ACS is coded separately from race (HISP), and whether you treat "Hispanic" as a race-level category depends on what you are trying to show. There is no technically correct answer, only an answer you can defend to a skeptical reader. Write down which one you made and why, in the audit output.

Fit the simplest possible model and get predictions:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

X_train, X_test, y_train, y_test, race_train, race_test, sex_train, sex_test = (
    train_test_split(
```

```
        features, label, race, sex,
        test_size=0.3, random_state=42, stratify=label
    )
)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("lr", LogisticRegression(max_iter=1000)),
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

A logistic regression is not the state of the art. It is deliberately chosen. If your audit only works on the model you trained, it is not an audit. It is a commentary on your own pipeline. The audit should be method-agnostic.

Now the disaggregated performance analysis:

```
from fairlearn.metrics import (
    MetricFrame,
    selection_rate,
    false_positive_rate,
    false_negative_rate,
    demographic_parity_difference,
    equalized_odds_difference,
)
from sklearn.metrics import accuracy_score, precision_score, recall_score

metrics = {
    "accuracy": accuracy_score,
    "precision": precision_score,
    "recall": recall_score,
    "selection_rate": selection_rate,
```

```
    "fpr": false_positive_rate,
    "fnr": false_negative_rate,
    "count": lambda y_true, y_pred: len(y_true),
}

sensitive = pd.DataFrame({"race": race_test, "sex": sex_test})

mf = MetricFrame(
    metrics=metrics,
    y_true=y_test,
    y_pred=y_pred,
    sensitive_features=sensitive,
)

print(mf.overall)
# => accuracy, precision, recall, etc. over the full test set

print(mf.by_group)
# => the same metrics, broken out by (race, sex) cells
```

`MetricFrame.by_group` returns a `DataFrame` indexed by the cross-product of your sensitive features. This is the centerpiece of the audit. The `count` metric is not optional. It is the column that tells your reader how many people are actually in each cell, which is the number that tells them whether any of the other numbers mean anything.

> **The Missing Manual**
>
> `fairlearn`'s defaults will not tell you which parity metric is politically defensible. That choice is yours, and you have to justify it. Demographic parity asks whether positive predictions are allocated at the same rate across groups. Equalized odds asks whether error rates are balanced across groups, conditional on the true label. These metrics

> can disagree. They often do. And they cannot both be satisfied in the presence of unequal base rates, which is a mathematical result, not a political one. Worse: most audit reports bury the per-subgroup sample size in a footnote, and the FPR you are reporting for the smallest cell is computed on thirty people. That number is noise. Report the count alongside every metric, refuse to compute metrics for cells below a documented threshold, and do not let the absence of a small-sample warning hide the fact that your audit has nothing to say about the group it looked hardest for.

Compute the two standard parity summaries and compare:

```
dpd = demographic_parity_difference(
    y_true=y_test, y_pred=y_pred,
    sensitive_features=sensitive,
)
eod = equalized_odds_difference(
    y_true=y_test, y_pred=y_pred,
    sensitive_features=sensitive,
)

print(f"Demographic parity difference: {dpd:.3f}")
print(f"Equalized odds difference:     {eod:.3f}")
```

A non-trivial difference between these two numbers is informative, not a bug. It tells you that the fairness question your dataset is asking you to answer is not reducible to a single statistic.

7.3. Writing the audit for a non-technical reader

The `by_group` table is raw material. The audit is what you write around it. A useful template has four sections.

1. **What the model is, and what it is not.** A logistic regression trained on one year of ACS data for one state. A teaching artifact, not a procurement recommendation. The reader should not leave the first paragraph confused about what decisions, if any, the model is authorized to make.

2. **Who is in the data.** The per-subgroup counts, with the smallest cells called out in prose. "Our test set includes 412 Black men and 34 American Indian and Alaska Native women. The latter is below our reporting threshold of 100, so we do not report per-cell metrics for that group, but we note its absence." Audits that silently drop small cells pretend to a completeness they do not have.

3. **What the metrics show.** The disaggregated table, with a plain-language reading. Demographic parity difference is 0.21; equalized odds difference is 0.11. In the voice of the audit: *the model predicts high income for men at a rate 21 percentage points higher than for women, and this gap is not fully explained by differences in true-label rates. Error rates also vary across racial groups, with the false positive rate roughly twice as high for X as for Y.*

4. **What assumptions the audit rests on.** The racial categories were flattened. Hispanic origin was coded as a race for display. The underlying ACS data are self-reported and subject to nonresponse. The model was evaluated on a 30 percent holdout from a single year. Any of these choices could be defended differently and would produce a different audit. Name them.

Section 4 is where most real audits fail. An audit that does not document its assumptions is not reproducible by a second reader, which is the minimum condition for routine auditability. If the next auditor cannot replicate your disaggregation and get the same table, the audit is not an instrument of oversight. It is a press release.

7.4. What survives political change

You can run the code above on any public model and produce a disaggregated audit. What you cannot do, alone, is make it matter.

Three things distinguish audit regimes that survive political change from those that collapse into audit-washing. The first is **authority**: a named institution, with a budget and a mandate, that can demand the audit and act on it. Gender Shades moved IBM and Microsoft because it was one input into a field of advocacy organizations (the Algorithmic Justice League chief among them), investigative journalists, and eventually regulators. Raji and colleagues' (2020) internal auditing framework presumed an organization that wanted to know. The AI Act presumes a market-surveillance authority willing to use its enforcement powers. When no such authority exists, the audit is an artifact looking for a reader.

The second is **remedy**. If the audit finds a problem and there is nothing the people harmed can do about it, the audit is diagnostic without being therapeutic. This is the core defect of LL144 as implemented: the law requires disclosure of disparate impact but provides no individual right of action for a candidate screened out by a tool with an adverse impact ratio of 0.6. Remedy can be restorative (reinstating a denied benefit), structural (decommissioning a tool), or financial (damages). Routine auditability without a remedy pathway produces an archive of grievances, not a correction mechanism.

The third is **scope protection**. Audit regimes erode when their scope narrows without anyone defending the frame. The narrowing happens through definitional work: tightening what counts as an "automated employment decision tool," exempting "advisory" systems that are decisive in practice, carving out "pilot programs" that run for a decade. Chapter 3 treated this as the logic of exemption (Chapter 3). An audit regime whose scope is a negotiated floor rather than an expanding ceiling will be negotiated downward by the entities it is supposed to constrain.

Keegan's (2026) framework names authority and remedy as two of the six elements of the installed base. Oversight, in this sense, is not an additional value. It is the working configuration in which those elements meet. A technically excellent `MetricFrame` output and no authority to act produces a finding that goes into a drawer. A regulator with authority and no technical capacity produces enforcement actions built on misread metrics. The point of learning `fairlearn` well is not that the library does oversight. It is that the library does the part of oversight the auditor cannot delegate upward, so that what the auditor produces can be a reliable input to the arrangement the rest of the installed base has to sustain.

7.5. Exercises

Exercise 7.1 (Guided, reproduction). Reproduce the disaggregated audit from Buolamwini and Gebru (2018) on a smaller scale. Using the Fitzpatrick Skin Type dataset (or a publicly documented substitute), compute accuracy disaggregated by the intersection of skin type and sex for any open-source gender-classification model you can run on a laptop. You do not need to reproduce the magnitude of their findings. You need to reproduce the shape of the analysis: the aggregate-versus-disaggregated contrast, the intersectional cell, and the inclusion of per-cell sample sizes. Submit your notebook and a one-paragraph comparison of your result to the paper's.

Exercise 7.2 (Applied). Run the `folktables` audit above on a different state, or on a task other than income (employment, public-coverage receipt, and travel time are all in the package). Produce a `MetricFrame` with at least two sensitive features. Write a 500-word audit report in the four-section structure above. Document at least three assumptions in the assumptions section, and name at least one subgroup your data cannot support conclusions about.

Exercise 7.3 (Analytic). Using the model from Exercise 7.2, compute both `demographic_parity_difference` and `equalized_odds_difference`. Write 300 words comparing the two numbers. Which one is larger? Which failure mode does each describe in plain language? If you had to pick one as the headline number for your audit report, which would you pick, and what are you choosing not to emphasize by that choice? There is no right answer; there are defensible answers.

Exercise 7.4 (Critical reading). Find one NYC Local Law 144 bias audit report from the public record. Employer listings and linked audit reports are available through the `on.nyc.gov/ll144` hub and a growing archival collection at the Center on Race, Inequality, and the Law. Read the audit end to end. Write a 500-word critique using the four-section template. Where does the audit succeed? Where does it hide its sample sizes, define impact ratios in non-standard ways, or omit an assumptions section? Does the audit name a remedy pathway? Would a non-technical reader come away knowing what, if anything, they could do about an adverse finding?

Exercise 7.5 (Open-ended). Take the audit you wrote in Exercise 7.2 and answer this question in 400 to 600 words: what would change about the audit, its structure, its disclosures, its refusal to answer some questions, if it had enforcement authority attached? Pick a specific authority: the EEOC, a state attorney general's office, the EU AI Act market-surveillance authority, or a named procurement officer at a city government. Write as if your audit could trigger their intervention. What thresholds would you set? What would you decline to say without a formal proceeding? What part of the current audit would you have to strengthen to survive a challenge?

7.6. Looking ahead

Oversight without ownership is oversight without a subject. You can audit a model all day, but if it is unclear who owns the system, who owns the

training data, and who has standing to demand changes, the audit cannot close a loop. Chapter 8 (Chapter 8) takes up ownership as the third value in the installed-base trio, and asks what it takes to make ownership claims legible and enforceable in data infrastructures built largely out of other people's work. Chapter 11 (Chapter 11) returns to the accounting lineage this chapter invoked in passing; the professional history there is a cautionary tale about what happens to oversight regimes when their professional base loses the argument about what "public interest" means. Chapter 16 (Chapter 16) will ask whether some audits should not be performed at all, a question the technical toolkit here cannot answer on its own.

7.7. Further Reading and Resources

- Joy Buolamwini and Timnit Gebru (2018), "Gender shades: Intersectional accuracy disparities in commercial gender classification," *Proceedings of FAccT 2018*. The foundational disaggregated audit. Read it alongside the follow-up in Raji et al. (2020).
- Inioluwa Deborah Raji et al. (2020), "Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing," *Proceedings of FAccT 2020*. The paper that turns an audit technique into an organizational framework.
- `fairlearn` user guide: https://fairlearn.org/main/user_guide/. Pay particular attention to the section on interpreting `MetricFrame`, and to the documentation on disagreement between parity metrics.
- `folktables` repository and paper (Ding et al., NeurIPS 2021): https://github.com/socialfoundations/folktables. The package used in this chapter, with documentation on the tradeoffs involved in treating ACS tables as fairness benchmarks.
- NYC Department of Consumer and Worker Protection, Local Law 144 rules and guidance: https://www.nyc.gov/site/dca/about/automated-employment-decision-tools.page. Read the final rule, not

just the FAQ.

- Bryce Goodman and Julia Trehu, "AI audit washing and accountability" (2023). The term "audit-washing" in its useful form.
- EU AI Act consolidated text (Regulation 2024/1689): https://eur-lex.europa.eu/eli/reg/2024/1689/oj. Skim Articles 43, 72, and 77 for the conformity-assessment and post-market-monitoring scaffolding.
- Algorithmic Justice League: https://www.ajl.org/. The organizing vehicle that turned the Gender Shades paper into a campaign.
- Upturn's "Help Wanted" report and subsequent work on hiring tools: https://www.upturn.org/work/. A model of audit-adjacent advocacy that takes remedy seriously.
- C. Richard Baker (2005), "What is the meaning of 'the public interest'? Examining the ideology of the American public accounting profession," *Accounting, Auditing & Accountability Journal*. The history lesson. Read it before you assume professional norms will hold the line.

8. Ownership

A commercial data broker will tell you, if pressed, that it *owns* the nine-hundred-variable dossier it sells about you. Ownership here means a property claim: rights to exclude, alienate, and monetize, backed by contract law. The dossier was assembled, often without your knowledge, from loyalty programs, voter files, credit header data, and the exhaust of a hundred free apps. You do not own it. The broker does.

This chapter argues that ownership in public-interest data science is a different word, and most of the trouble in the field comes from not noticing. The broker's claim is ownership-as-property. The claim a public-interest steward has to make is ownership-as-stewardship: a maintenance obligation that requires labor, authority, and continuity across political cycles. You do not own a public dataset the way a broker owns a dossier. You own it the way a municipal archivist owns the city's property records: accountable for its condition, usability, and survival, and unable to sell it off when the grant runs out.

Chapter 5 named continuity and safe scrutiny among the six elements of the installed base. Chapter 6 built the publishing pipeline. Chapter 7 built the audit. This chapter closes Part II by asking who holds the pipeline and the audit when the original builder leaves, and what it takes to structure that holding so it does not collapse into abandonment or capture.

8.1. Two senses of the word

Ownership-as-property treats data as an asset with a rights-holder who can license, sublicense, sell, or destroy it. The vocabulary is real estate and copyright: chain of title, residual rights, assignment. The commercial data broker sits here. So, uncomfortably, does a great deal of "open" infrastructure when examined closely. A university's IRB-governed research dataset may be open to collaborators, but the technology transfer office still treats it as an asset with a chain of title.

Ownership-as-stewardship treats data as an ongoing obligation. The steward is responsible for maintenance, documentation, remediation, and succession. The vocabulary is closer to trusteeship: obligations of care, prudence, and loyalty that cannot be disposed of without arranging for someone else to take them up. A municipal records manager is a steward. So is the volunteer who keeps a Wikipedia article on a regional dialect accurate for a decade. Neither owns the data in the property sense. Both are responsible for it in a way the property owner is not.

The distinction matters because most public-interest infrastructure is built on labor that presumes stewardship while operating inside institutions that default to property. Karasti and Blomberg (2018) studied this in long-running scientific infrastructures: the system works until the person doing the work moves, retires, or quits, and then it stops working in ways that take years to become visible. Chapter 14 returns to this as ghost work (Chapter 14); notice here that "open" infrastructure has a maintenance budget, usually paid in uncredited hours by people who cannot put the work on their performance review.

Schwartz and Cook (2002) made the argument for archives two decades before data science rediscovered it. Archives are not neutral containers: what gets kept, who decides, and who is near the keeping apparatus are political choices all the way down. Stoler (2002), studying colonial Dutch archives, took it further: the archive does not record power neutrally but is itself an instrument of power, and reading it requires reading the

silences the archivist left behind. Translate to data infrastructure: there is no ownership-as-stewardship that is not also a politics of whose records survive, whose categories are used, and whose hands the maintenance falls to.

8.2. Data trusts, briefly

One response to this mismatch is the **data trust**: a legal structure in which a trustee holds rights over a dataset on behalf of beneficiaries, with fiduciary duties enforceable in court. The Ada Lovelace Institute's review (Ada Lovelace Institute 2021) found trusts are still more proposed than operational, but the proposal itself forces the questions: who the beneficiaries are, what the trustee may and may not do, what remedies are available if the trust is breached. Delacroix and Lawrence (2019) argue for bottom-up trusts in which users themselves constitute the beneficiary class.

You will not stand up a data trust in the tutorial. You will stand up something more modest: tiered access with a documented threat model and decision rules. The trust idea is the horizon; tiered access is the minimum you can maintain before one exists.

8.3. Indigenous data sovereignty as a foundational challenge

One caveat that will recur. "Stewardship with fiduciary duties to beneficiaries" presumes the beneficiaries can be identified, consented, and represented within the legal system governing the trustee. For many datasets that fails in ways the stewardship vocabulary is not equipped to solve. Te Mana Raraunga (Te Mana Raraunga 2018), the Maori Data

Sovereignty Network, argues that Maori data is not a resource to be stewarded on Maori behalf by a non-Maori institution. It belongs to Maori, under Maori governance, as sovereignty rather than trust. The CARE Principles (Carroll et al. 2020) extend this to Indigenous data governance generally: collective benefit, authority to control, responsibility, and ethics are not add-ons to FAIR, they are a different frame.

Chapter 16 (Chapter 16) takes this up in detail, including when the right answer is not tiered access but refusal. Notice here that the tools you are about to build are not universally applicable. They work when a public has delegated stewardship to an institution. They do not work when a people has not delegated anything, and the question of whether the institution ever had standing remains open.

8.4. The tutorial: tiering the Boulder campaign finance file

Return to the campaign finance dataset from Chapter 6. You published a cleaned file with donor names, amounts, dates, and candidates. Imagine the portal also included two columns you suppressed: donor home address and donor occupation with employer. Those are part of the public filings record in most jurisdictions, which is to say they are legally public. Legal publicness and safe republishability are not the same thing. An address plus a name plus an employer is enough to enable harassment, doxxing, and targeted advertising well beyond what a 1974 disclosure law contemplated.

You will build three access tiers over one SQLite database:

- **Public tier.** Aggregates only: totals by candidate, zip code, filing period. No individual rows. Anyone with a browser.
- **Researcher tier.** Individual rows with name, amount, date, candidate, occupation. No street address. Gated by a shared secret,

logged at the query level. For credentialed journalists and academic researchers with a documented project.

- **Restricted tier.** Full individual rows including address. Rare, case-by-case, granted by a named human, logged in a public register.

8.4.1. The schema and the views

Start with a single SQLite file, loaded from the Chapter 6 clean output with two columns appended: `donor_address` (street, city, state, zip) and `donor_occupation_employer` (free-text from the filing).

```
import sqlite3, pandas as pd

df = pd.read_parquet("data/boulder_campaign_finance_extended.parquet")
con = sqlite3.connect("campaign_finance.db")
df.to_sql("filings_raw", con, if_exists="replace", index=False)
```

Now define the access tiers as SQL views. Views grant query access to a projection of the base table without granting access to the table itself.

```
-- Public tier: aggregates only.
CREATE VIEW public_totals_by_candidate AS
SELECT candidate,
       COUNT(*)   AS n_contributions,
       SUM(amount) AS total_amount,
       MIN(date)   AS first_date,
       MAX(date)   AS last_date
FROM filings_raw
GROUP BY candidate;

CREATE VIEW public_totals_by_zip AS
SELECT substr(donor_address, -5) AS zip,
```

```sql
       COUNT(*)   AS n_contributions,
       SUM(amount) AS total_amount
FROM filings_raw
GROUP BY zip
HAVING n_contributions >= 5;  -- k-anonymity floor

-- Researcher tier: individual rows, no address.
CREATE VIEW researcher_filings AS
SELECT filing_id,
       contributor_clean,
       amount,
       date,
       candidate,
       committee,
       donor_occupation_employer
FROM filings_raw;

-- Restricted tier: the full table, with a trigger logging access.
-- (No view; access is to filings_raw itself, gated upstream.)
```

Three things to notice. The public view enforces a **k-anonymity floor** of five per zip, so a zip with a single donor is not re-identified by an aggregate. The researcher view strips the address but keeps occupation and employer, on the judgment that those are newsworthy in ways street addresses rarely are. The restricted tier has no view; it is the base table, controlled at the application layer because SQLite has no per-row permissions.

8.4.2. Credentials and environment variables

Read the researcher-tier shared secret from an environment variable, not from code.

```
import os
RESEARCHER_TOKEN = os.environ.get("CAMPAIGN_FINANCE_RESEARCHER_TOKEN")
if not RESEARCHER_TOKEN:
    raise RuntimeError("Set CAMPAIGN_FINANCE_RESEARCHER_TOKEN in the deploy environment.")
```

A thin Flask wrapper in front of Datasette gives you a basic-auth layer for the researcher endpoint. What matters is the pattern.

```
from flask import Flask, request, abort, jsonify
import sqlite3, hmac, datetime, json

app = Flask(__name__)
DB, AUDIT_LOG = "campaign_finance.db", "access_audit.jsonl"

def log_query(tier, user, query, ip):
    entry = {"ts": datetime.datetime.utcnow().isoformat() + "Z",
             "tier": tier, "user": user, "query": query, "ip": ip}
    with open(AUDIT_LOG, "a") as f:
        f.write(json.dumps(entry) + "\n")

@app.route("/public/totals")
def public_totals():
    rows = sqlite3.connect(DB).execute(
        "SELECT * FROM public_totals_by_candidate").fetchall()
    log_query("public", "anonymous", "totals_by_candidate", request.remote_addr)
    return jsonify(rows)

@app.route("/researcher/query", methods=["POST"])
def researcher_query():
    token = request.headers.get("X-Research-Token")
    if not token or not hmac.compare_digest(token, RESEARCHER_TOKEN):
        abort(401)
    user = request.headers.get("X-Research-User", "unknown")
```

```
    q = request.json.get("query", "")
    if not q.strip().lower().startswith("select"):
        abort(400)
    if "filings_raw" in q.lower():
        abort(403)
    rows = sqlite3.connect(DB).execute(q).fetchall()
    log_query("researcher", user, q, request.remote_addr)
    return jsonify(rows)
```

This is not a hardened production system. A determined attacker can evade the two string checks; a motivated researcher can reconstruct sensitive joins. The point is making the access tiers legible as code, with each tier's permissions visible in one file to a code reviewer.

The Missing Manual

Environment-variable credentials are not security. They are a social contract with your deployment pipeline, and the pipeline leaks them in ways most tutorials never mention. A token printed to a container log because someone left `DEBUG=true` on in production is not a secret anymore. A token captured in a stack trace that a Sentry integration forwarded to a third-party error tracker is not a secret either. A token committed to git history in a `.env` file two refactors ago and never rotated is, for practical purposes, the plaintext it started as. Treat the deployment pipeline as part of the attack surface. Rotate tokens on a schedule, not only after incidents. Audit logging configuration for what it captures on exception. "The token is in an environment variable" buys you roughly as much safety as a locked car in a rough neighborhood.

8.4.3. Decision rules in YAML

The code enforces the tiers. The decision rules say who belongs in which tier, and they are the part that will be contested. Keep them out of the code and in a versioned document.

```yaml
# access_policy.yaml
version: 2026-04-15
dataset: boulder_campaign_finance
owner: Boulder Open Data Working Group

tiers:
  public:
    audience: anyone
    contents: aggregates, k >= 5
    authentication: none
    logging: request-level, 30 days
  researcher:
    audience:
      - credentialed journalists with published affiliation
      - academic researchers with IRB approval or equivalent
      - civic-tech projects with a named public deliverable
    contents: individual rows, no street address
    authentication: shared token, user-identified
    logging: per-query, 1 year, publicly aggregated monthly
    review: quarterly, stewards plus one external reviewer
  restricted:
    audience: case-by-case, rare
    contents: full rows including address
    authentication: named human approval, logged in public register
    review: per request, two stewards plus one community member

revocation:
  grounds: [misuse, undisclosed affiliation change, repeated scope violation]
```

```yaml
  process: notice, 14-day response, appeal to full steward group
  public_notice: revocation counts published quarterly, reasons anonymized
```

The YAML does nothing by itself. It is a document you can point to when a request comes in or a classmate peer-reviews your design. Its value is forcing you to answer the questions before answering them feels like backtracking.

8.4.4. The threat model

The last artifact is the threat model: a structured description of who might attack this system, what they would be after, and what the impact would be if they succeeded. Most "security" documentation in data science projects is either absent or performative. A workable threat model is a page.

```markdown
# Threat model: Boulder campaign finance, tiered access
Version: 2026-04-15. Review cadence: every 6 months.

## Assets
- A1. Individual donor records (name, amount, date).
- A2. Donor home addresses (within A1, higher sensitivity).
- A3. Aggregate totals (published openly).
- A4. Access logs (reveal who is researching whom).
- A5. Researcher tokens (credential).

## Actors and motivations
- T1. A harasser targeting donors to a specific candidate. Seeks A2 via A1.
- T2. A commercial data broker enriching a dossier. Seeks A1 at scale.
- T3. A political opposition researcher building an intimidation list. Seeks
- T4. A careless researcher exfiltrating A1 onto an unsecured laptop.
- T5. A state actor subpoenaing A4 to identify who accessed whose records.
```

```
## Attack paths
- P1. Token leak via logs or git history (T2, T3).
- P2. SQL that evades view restrictions to reach filings_raw (T1, T2).
- P3. Joining public aggregates with external data to re-identify small-zip donors (T1).
- P4. Social-engineering a steward into granting restricted access (T3).
- P5. Compelled disclosure of A4 (T5); no technical mitigation, only retention policy.

## Mitigations (current)
- M1. Tokens in env vars, rotated every 90 days (P1, partial).
- M2. Query whitelist and view restriction (P2, partial).
- M3. k >= 5 floor on aggregates (P3).
- M4. Two-person rule for restricted grants; public register (P4).
- M5. Access logs retained 12 months, then rotated (P5).

## Gaps
- G1. No rate limiting on researcher endpoint.
- G2. No differential privacy on small-zip aggregates.
- G3. No legal review of what logs must be produced under CORA.
```

The threat model is boring to write and instantly valuable. It turns "we took security seriously" into a specific list of worries, mitigations, and open gaps.

8.5. Why "just use GitHub" is stewardship outsourcing

There is a tempting shortcut. Rather than stand up your own tiered access, put aggregates in a public GitHub repo, individual rows in a private one, done. GitHub handles authentication. GitHub handles logging. GitHub is free.

GitHub is also a platform owned by Microsoft, operated under Terms of Service it may unilaterally change, subject to U.S. export controls and sanctions, hosting content at its discretion. Over the past decade it has deplatformed projects for reasons ranging from copyright disputes to sanctions compliance, and has had outages long enough to take live public dashboards offline for a working day. None of this makes GitHub unusable. It does mean "just use GitHub" is not a stewardship decision. It is a stewardship *outsourcing* decision, with terms you did not negotiate.

The same argument applies to Google Sheets, Airtable, Notion. Each is a reasonable tool inside a stewardship arrangement. None of them, alone, *is* the stewardship arrangement. The political economy question Part II keeps returning to: who maintains the infrastructure your project depends on, and what happens when they stop.

In the Public Interest

Ownership-as-stewardship turns the continuity element of the installed base (Chapter 5) from a slogan into a job. A dataset that is not owned by anyone, in the stewardship sense, loses continuity within two organizational transitions: the original maintainer leaves, the replacement does not know the file exists, the permalink breaks. Safe scrutiny suffers the same fate: an access tier that is not actively maintained degrades into either a free-for-all (nobody rotates credentials) or a locked vault (nobody remembers who is authorized). Treat stewardship as a named position with a budget, not a gesture on a funder's report. If your project cannot name the steward, by role if not by person, it does not yet have one. Chapter 22 (Chapter 22) picks this up as custodianship.

8.6. Exercises

Exercise 8.1 (Guided). Extend the Chapter 6 Boulder campaign finance dataset with two new columns, `donor_address` and `donor_occupation_employer`, using the synthetic fields in `data/boulder_campaign_finance_extended.parquet`. Implement the three SQL views. Stand up the Flask wrapper locally, run one query against each tier, and capture the resulting audit log entries. Submit the SQL, the Flask app, and the first ten lines of your audit log.

Exercise 8.2 (Analytic). Write the `threat_model.md` and `access_policy.yaml` for Exercise 8.1 as applied to your own context. Name three actors specific to your setting, three attack paths, and three mitigations. Identify at least two gaps you are not yet mitigating. Commit both files alongside the code.

Exercise 8.3 (Peer review). Exchange your Exercise 8.2 artifacts with a classmate. Review their threat model and access policy against two questions: is there an attacker missing that you would add, and is there a decision rule in the YAML more ambiguous than it reads? Write a 300-word review identifying two concrete gaps. You will not be graded on whether the gaps were real; you will be graded on whether they were specific enough to act on.

Exercise 8.4 (Applied research). Find a real-world data trust, data cooperative, or data commons operating (not merely proposed) in 2026. Candidates include UK Biobank, Salus Coop in Barcelona, SAGE Bionetworks' governance frameworks, or cases tracked by the Ada Lovelace Institute. Document its governance in 500 to 800 words: who the trustees are, who the beneficiaries are, what rights beneficiaries have, what happens if the trust is wound up, and where the money comes from. Cite the governance documents, not journalistic summaries.

Exercise 8.5 (Open-ended, project-linked). Return to your Exercise 1.5 project sketch. For each infrastructure your project depends on (hosting,

data source, library, registry, funder), name the steward in the ownership-as-stewardship sense. For two of those stewards, write one paragraph on what happens to your project if they stop maintaining their part tomorrow. If your answer is "I would be fine," revisit whether you correctly identified the dependency. Add the result to your sketch file.

8.7. Looking ahead

Part II ends here. You have three values in hand: openness (Chapter 6) as the configuration that makes public data legible, oversight (Chapter 7) as the arrangement that makes system behavior visible to parties with authority to act, and ownership (this chapter) as the stewardship obligation that makes both survive political change. None is sufficient alone. Each, as Keegan (2026) argues, is a working configuration of the installed base rather than a principle to be invoked on its own.

Part III turns from values to lineages: the professional traditions that have done this work longer than data science has existed. Chapter 9 begins with journalism, because journalists have been wrestling with ownership-as-stewardship over source material, documents, and leaked data for more than a century, and they have working answers to questions the data science field is still formulating. The move from Part II to Part III is not theory to practice. It is a move across fields that have been practicing already, so you can import their methods without pretending you invented them.

8.8. Further Reading and Resources

- Helena Karasti and Jeanette Blomberg (2018), "Studying infrastructuring ethnographically," *CSCW*, 27: 233 to 265. The reference this chapter leans on for what maintenance labor actually is.

- Joan M. Schwartz and Terry Cook (2002), "Archives, records, and power," *Archival Science*, 2: 1 to 19. The foundational argument for archives as instruments of power.
- Ann Laura Stoler (2002), *Along the Archival Grain*. A full-length study of reading archives as political objects; the methodology generalizes to civic datasets.
- Ada Lovelace Institute, "Exploring legal mechanisms for data stewardship" (2021): https://www.adalovelaceinstitute.org/report/legal-mechanisms-data-stewardship/.
- Te Mana Raraunga, "Principles of Maori data sovereignty" (2018): https://www.temanararaunga.maori.nz/tutohinga. The foundational statement of Indigenous data sovereignty in Aotearoa New Zealand.
- Stephanie Russo Carroll et al. (2020), "The CARE Principles for Indigenous data governance," *Data Science Journal*, 19: 43. The complement and corrective to FAIR.
- Sasha Costanza-Chock (2020), *Design Justice*. Stewardship as community-led rather than institutionally-delegated; read chapter 4 with this chapter in view.
- Open Data Institute, "Data stewardship: What it is and why it matters": https://theodi.org/insights/guides/data-stewardship/.
- Sylvie Delacroix and Neil D. Lawrence (2019), "Bottom-up data trusts," *International Data Privacy Law*, 9(4): 236 to 252.
- OWASP, "Threat Modeling Cheat Sheet": https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html.

Part III.

Lineages

9. Journalism

Part II closed with ownership as stewardship. Part III opens with a profession that has been stewarding sensitive public documents since well before data science named itself. A reporter who sits on a leaked database for six months while corroborating the identities of the people inside it is doing, as routine practice, the work the last four chapters spent pages trying to name. The tradition is called journalism, and if you want to make data science useful to the public, journalism is the lineage most worth reading as a set of instructions rather than as inspiration.

This chapter is not a history of journalism. It is a prompt to notice things the data science field, including its academic algorithmic-accountability wing, continues to rediscover a decade late. Newsrooms worked out methods for verifying records against sources, for publishing under legal threat, for collaborating across borders on datasets no one outlet could analyze alone, and for extending right of reply to the subjects of an investigation before the article ran. These are verification routines that, when you import them into academic or civic data practice, raise the evidentiary floor of the work in ways the field's internal conversations about "reproducibility" rarely manage on their own.

The chapter's running case is the one that pulled the fairness-in-machine-learning conversation into public view in 2016: ProPublica's *Machine Bias* investigation of the COMPAS recidivism risk score (Angwin et al. 2016). You will download ProPublica's published data and notebook, reproduce one of its central statistics, compare your numbers to the article's, and read the methodological rejoinder from Flores, Bechtel, and Lowenkamp

(2016). The point is not to adjudicate the dispute. The point is to notice what made the dispute possible, and what it cost ProPublica to invite it.

9.1. The watchdog role, operationalized

The "Fourth Estate" is a metaphor that carries more than it can cash out. A fourth branch of government exists nowhere in any constitution. What does exist, across jurisdictions, is a loose professional arrangement: newsrooms with libel insurance, public-records statutes that force documents into the open, editors who can say no to a story, and a readership that funds the operation, imperfectly and with worsening economics, through subscriptions and advertising. The watchdog function is that arrangement working well enough to subject powerful institutions to scrutiny they would prefer to avoid.

Data journalism is the computational specialization of that function. Diakopoulos (2019) documents how algorithmic accountability reporting emerged as newsrooms learned to treat software as a beat: not a tool for telling stories faster, but a subject of investigation in its own right. The investigative journalism tradition he chronicles, institutionalized in organizations like Investigative Reporters and Editors (IRE) and its data-focused conference NICAR, developed verification standards that predate and, in important respects, outpace the academic literature on algorithmic audits.

Four of those standards bear naming here, because you will use them in the tutorial.

Sourcing means a claim in a published story is traceable to a document, a dataset, or a named human. A reporter's notebook is a provenance chain. When a reporter writes that the city's benefits portal denied 12 percent of applicants for a missing signature, someone can ask which 12 percent, from which table, queried on which date, and the answer exists.

Corroboration means a claim that matters rests on more than one independent piece of evidence. The dataset says X. A former employee also said X. A public filing from a different year also said X. Single-source claims are not forbidden, but they are flagged and carefully worded.

Right of reply means the subjects of an investigation, including the institutions whose data is under scrutiny, receive the findings before publication and have an opportunity to respond. The response may be a correction, a denial, a contextual statement, or silence. Each has an editorial consequence.

Publication with documentation means the story travels with its evidence, or with enough of its evidence that readers and other newsrooms can follow the inference. Not every dataset can be released. The inference chain, the decisions made in cleaning the data, the variables selected, and the judgment calls made are documented somewhere readers can inspect.

Academic researchers use none of these consistently. Peer review substitutes, poorly, for the last and makes almost no demands on the first three. One of this chapter's claims is that academic data science, when it engages public systems, should steal from journalism rather than reinvent.

9.2. FOIA as infrastructure

One piece of journalism's installed base deserves separate mention because Chapter 10 (Chapter 10) takes it up in detail: the Freedom of Information Act and its state equivalents. FOIA turns the watchdog function from a polite request into a procedural right. A reporter writes, the agency has a statutory deadline to respond, and a denial can be appealed to a court. The infrastructure is imperfect, underfunded, and routinely abused through delay and overbroad exemption. It is also the mechanism by which much of the material a data journalist analyzes comes into the world in the first place.

For the purposes of this chapter, notice two things. First, FOIA is what Chapter 5 (Chapter 5) calls safe scrutiny with a statutory backbone: the reporter does not have to trust the agency's goodwill, because the statute provides a lever. Second, the datasets journalists release through their reporting are often FOIA products that the agency itself never voluntarily published. ProPublica's COMPAS data is not a FOIA release (it came through a public-records request to Broward County, Florida, and a handful of correspondences with the vendor), but its genealogy sits in the same family of documents-prised-loose-through-procedure that the next chapter will train you to file.

9.3. Cross-border collaboration as a scaling strategy

A single newsroom can handle a dataset of several thousand rows. It cannot handle 11.5 million documents. The Panama Papers leak, published by the International Consortium of Investigative Journalists (ICIJ) in 2016, distributed the analysis across more than a hundred newsrooms in dozens of jurisdictions, each working on the documents relevant to their country, under coordinated publication embargoes. The Paradise Papers (2017) and the Pandora Papers (2021) used the same template. Forbidden Stories, a separate consortium, exists specifically to continue the work of journalists who have been killed or imprisoned, distributing their unpublished material among member newsrooms so that silencing any single reporter does not silence the story. Lighthouse Reports, based in the Netherlands, has built a collaborative model for algorithmic-accountability reporting across European outlets, including the Rotterdam welfare-algorithm investigation you saw in Chapter 3.

The method is worth importing to academic data work. A multi-country welfare-algorithm analysis is not, structurally, a different problem from a multi-site clinical trial. What cross-border journalism has worked out that academic consortia often have not is the governance stack that makes the collaboration survive leaks, legal threats, and deadline pressure: secure

document-sharing platforms, shared style guides, coordinated fact-checking, joint legal review, and, critically, the right of any newsroom to pull out without burning the rest of it down. Zeng, Chan, and Schafer (2022) show how much of this infrastructure is culturally located: what works in a consortium of Western European outlets is not portable to a newsroom operating under tighter state control, and an academic collaboration needs to understand transferability before it assumes the model scales.

9.4. The running example: reproducing *Machine Bias*

ProPublica's 2016 investigation of the COMPAS risk-assessment tool, used by courts across the United States to inform pretrial and sentencing decisions, argued that the score was biased against Black defendants. The finding that drew the most attention was a disparity in false-positive rates: among defendants who did not reoffend within two years, Black defendants were roughly twice as likely as white defendants to have been classified as high-risk by COMPAS. The article ran with its dataset and Jupyter notebook posted to GitHub at `propublica/compas-analysis`, an act of methodological transparency that was, in 2016, unusually generous by newsroom standards.

The tutorial walks you through reproducing the two-year recidivism-rate statistic and comparing your numbers to the article's. You will find small divergences. You will then read Flores and colleagues' rejoinder (2016) and identify which divergences matter for the argument.

9.4.1. Pulling the data

The dataset lives in the repository as `compas-scores-two-years.csv`. You can clone the repo or download the file directly.

```
import pandas as pd

URL = (
    "https://raw.githubusercontent.com/propublica/"
    "compas-analysis/master/compas-scores-two-years.csv"
)

df = pd.read_csv(URL)
df.shape
# => (7214, 53)

df[["sex", "race", "age", "priors_count",
    "decile_score", "two_year_recid"]].head()
```

ProPublica applied filters to this file before running their analysis. The notebook documents them. Cases where the COMPAS assessment was not drawn within 30 days of arrest were excluded, along with cases missing key fields. Replicate the filter before you replicate the statistic:

```
mask = (
    (df["days_b_screening_arrest"] <= 30) &
    (df["days_b_screening_arrest"] >= -30) &
    (df["is_recid"] != -1) &
    (df["c_charge_degree"] != "O") &
    (df["score_text"] != "N/A")
)
filtered = df[mask].copy()
filtered.shape
# => (6172, 53)   # matches the ProPublica notebook's post-filter count
```

9.4.2. Reproducing the statistic

The headline finding compared false-positive and false-negative rates across racial groups. A false positive here is a defendant labeled medium or high risk by COMPAS who did not reoffend within two years. Recode the score and compute the contingency table:

```
filtered["high_risk"] = (
    filtered["score_text"].isin(["High", "Medium"]).astype(int)
)

by_race = (
    filtered.groupby("race")
            .apply(lambda g: pd.Series({
                "n": len(g),
                "fp_rate": (
                    ((g["high_risk"] == 1) & (g["two_year_recid"] == 0)).sum()
                    / (g["two_year_recid"] == 0).sum()
                ),
                "fn_rate": (
                    ((g["high_risk"] == 0) & (g["two_year_recid"] == 1)).sum()
                    / (g["two_year_recid"] == 1).sum()
                ),
            }))
            .reset_index()
)
by_race.query("race in ['African-American', 'Caucasian']")
# => African-American  fp_rate ~ 0.45, fn_rate ~ 0.28
# => Caucasian         fp_rate ~ 0.23, fn_rate ~ 0.48
```

9.4.3. Checking for divergence

Your numbers will land close to the published rates but not identical. Possible sources of divergence, all worth documenting:

- Pandas has changed `.apply` semantics between versions, which can alter group ordering and rounding.
- The underlying CSV has been touched since 2016; you can pin to a specific commit hash if your analysis needs to be stable.
- The "Medium or High" threshold is one of two reasonable ways to binarize `score_text`; the notebook uses this one, but the article's prose occasionally shifts definitions.
- Rows with missing `two_year_recid` flags are handled inconsistently across replications in the literature.

Write a brief methods note, in plain prose, documenting what you did: the commit hash you pinned to, the filter you applied, the binarization rule you chose, and the precise numbers you got. This note is not a formality. It is the artifact that lets a future reader distinguish a substantive disagreement with ProPublica's argument from a transcription error in your pipeline.

> **The Missing Manual**
>
> When a newsroom publishes a methods notebook, the interesting methodological disputes are always in the details the article's prose does not cover. Read the notebook before you read the article, not after. You will see which filters were applied, which variables were dropped, and which binarization decisions the reporter made. These are the decisions the methodological rejoinders will land on. The practice of "right of reply" that journalists offer to the subjects of an investigation is not merely a courtesy; it is a verification mechanism. An institution given a chance to point out an error before publication often will, and the correction improves the final story. Academic researchers should steal this. Send your draft analysis to the institution

whose system you are auditing, with a specific deadline and specific questions. Do it even when you are under no obligation to.

9.5. Flores, Bechtel, and Lowenkamp: the rejoinder

Flores and colleagues (2016) published a direct response to ProPublica in *Federal Probation*, arguing that ProPublica's framing confused several distinct fairness criteria and that, on the criteria relevant to how risk assessments are actually used in court, COMPAS performed reasonably across racial groups. Their critique turned on the distinction between equal false-positive rates (ProPublica's focus) and equal positive predictive value (the vendor's focus). Subsequent literature, especially Chouldechova (2018) and the broader fair-ML community, established that these criteria cannot simultaneously hold when base rates differ across groups. The impossibility result is not an escape hatch. It is a forcing function: choose which fairness criterion matters for the decision the tool informs, and own the choice.

Read the rejoinder alongside your reproduction. Notice which of its criticisms your replication can speak to (the binarization choice, the filter selection) and which require different data than ProPublica released. Notice also that the rejoinder exists at all because the data was public. A privately held investigation of a proprietary tool with a gag order on the dataset would not have generated the follow-up literature that now occupies six hundred entries in the FAccT bibliography.

In the Public Interest

Chapter 6 (Chapter 6) argued that openness is not the absence of friction but the presence of a legible publication. ProPublica's decision to publish the COMPAS dataset alongside the *Machine Bias* article is the move that made the dispute with Flores and colleagues possible.

> An enclosed analysis, with the data held internally and the findings communicated only through the article, would have foreclosed the rejoinder. The methodological disagreement would have remained a matter of he-said-she-said between a newsroom and a vendor, and the fair-ML literature that grew from the case would not have had an empirical object to work on. Openness here does not just serve ProPublica's credibility. It serves the capacity of the field to argue with ProPublica and improve on its work. Keegan (2026) names this configuration as safe scrutiny reinforced by linkable provenance. Enclosure in the name of protecting a finding is a defensive posture that damages the finding's standing over time. Chapters 7 (Chapter 7) and 8 built on this: oversight requires an object to oversee, and the object has to survive long enough to be overseen.

9.6. What academic researchers can learn

Two practices from the newsroom side will not fit on a single syllabus, but both belong in your working repertoire.

The first is provenance as a default, not an exception. A reporter who cannot cite a claim cannot publish the claim. An academic researcher who cannot trace a number back through the cleaning pipeline to a source row is, in the newsroom's frame, not yet ready to publish either. The reproducibility frameworks you will meet in Chapter 11 (Chapter 11) encode this habit formally. The journalism version is older and more severe: you do not publish unless you can show your work, and "show your work" is not a virtue, it is the floor.

The second is the right of reply, already named in the callout above. When you audit a public system, send a draft of your findings to the institution responsible for the system, with a specific deadline and specific questions, and document whatever response you receive. You will get three kinds of

reply. A silence, which you note. A correction, which you incorporate. A substantive disagreement, which you either argue with in the text or adopt as a limitation. All three improve the final work. The institutional pressure against this practice, in academic settings, is that it can look like giving subjects a veto. It is not a veto. It is a verification step that produces a better paper and a defensible defense when the paper is challenged.

9.7. Looking ahead

Chapter 10 (Chapter 10) takes up the legal infrastructure that journalism has been navigating for a century and a half and that data science, when it engages public systems, is now learning to navigate. FOIA and its state equivalents, open-meetings statutes, research exceptions to the Computer Fraud and Abuse Act, and the threat environment documented in *Sandvig v. Barr* all constrain what you can observe, publish, and discover. Journalism's tradition is older than academic algorithmic accountability and more politically mature, and the political maturity is inseparable from decades of litigation about what a reporter is allowed to do. Chapter 17 (Chapter 17) picks up the translation question from the other side: how to turn a data investigation into a piece of public writing that a non-specialist reader can act on.

9.8. Exercises

Exercise 9.1 (Guided). Clone `propublica/compas-analysis` from GitHub. Run the filter described above against `compas-scores-two-years.csv` and reproduce the false-positive and false-negative rates by race. Write a one-page methods note documenting the commit hash you pinned to, the filter you applied, the binarization rule you used, and the numbers you obtained. Save the note as `exercises/ch09_compas_methods.md`. Submit a link to your git commit.

Exercise 9.2 (Analytic). Read Flores, Bechtel, and Lowenkamp (2016). In 500 to 700 words, identify the three criticisms the rejoinder makes that your Exercise 9.1 replication can speak to, and the criticisms it makes that would require different data. Your reply does not need to take a side; it needs to distinguish what the released dataset can and cannot settle.

Exercise 9.3 (Comparative). Find a data journalism investigation from a non-U.S. outlet published in the last three years. Candidates include Forbidden Stories, Lighthouse Reports, Der Spiegel's data desk, Suddeutsche Zeitung, CORRECTIV, or the ICIJ. In 400 to 600 words, analyze the piece's structure: what is the central claim, what data supports it, what was published alongside the story, which collaborators are credited, and what verification practices are visible in the methodology note (if any). Compare the published methods transparency to the ProPublica standard.

Exercise 9.4 (Research). Locate a methods notebook from a recent investigation by The Markup, ProPublica, The Pudding, Reveal, or Bellingcat. Document the notebook in 300 words: which filters it applied, which decisions it records in comments, and one decision it makes without documenting. Submit the notebook URL and your documentation together. The point is to practice reading a notebook as a reviewer, not just as a user.

Exercise 9.5 (Open-ended, project-linked). Identify a story from your own community, city, county, campus, or professional field, that would benefit from data journalism methods. In two to three paragraphs: name the story, name the data that would be required, identify the likely source (FOIA, scraping a public portal, a dataset already open), and name one person or institution who would be the subject of right-of-reply. Add the result to your `ch01_project_sketch.md` file. If the story overlaps with something you are already planning for a later chapter's exercise, say so.

9.9. Further Reading and Resources

- Julia Angwin, Jeff Larson, Surya Mattu, and Lauren Kirchner (2016), "Machine Bias," ProPublica: https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing. Read the article, then read the companion methodology at https://www.propublica.org/article/how-we-analyzed-the-compas-recidivism-algorithm.
- ProPublica's `compas-analysis` repository: https://github.com/propublica/compas-analysis. The Jupyter notebook in the repository root is the canonical object for this chapter.
- Anthony W. Flores, Kristin Bechtel, and Christopher T. Lowenkamp (2016), "False positives, false negatives, and false analyses: A rejoinder to 'Machine Bias,' " *Federal Probation*, 80(2): 38 to 46.
- Nicholas Diakopoulos (2019), *Automating the News: How Algorithms Are Rewriting the Media*. Harvard University Press. The standing academic treatment of algorithmic accountability reporting.
- The Markup's "Show Your Work" series: https://themarkup.org/series/show-your-work. Each investigation publishes its methodology as a standalone piece; read three of them back to back.
- Investigative Reporters and Editors (IRE) and the National Institute for Computer-Assisted Reporting (NICAR): https://www.ire.org. The professional home of U.S. data journalism; the NICAR conference proceedings are an underappreciated methods literature.
- International Consortium of Investigative Journalists (ICIJ): https://www.icij.org. The Panama, Paradise, and Pandora Papers investigations, with extensive documentation of the cross-border collaboration model.
- Lighthouse Reports: https://www.lighthousereports.com. European consortium specializing in algorithmic-accountability investigations; the Rotterdam welfare-algorithm piece is a useful companion to Chapter 3.
- Forbidden Stories: https://forbiddenstories.org. The consortium that continues the work of journalists who have been silenced.

- *The Data Journalism Handbook 2*, edited by Jonathan Gray and Liliana Bounegru (2021), open-access at https://datajournalism.com/read/handbook/two. The field's living textbook, with chapters by practitioners on verification, scraping, collaboration, and ethics.
- The Pudding: https://pudding.cool. For a lighter complement: visual essays whose source code ships with the story.

10. Law

Chapter 9 (Chapter 9) ended on a newsroom practice that depends on a statute. A reporter can ask politely for a document, an agency can decline politely, and without public-records law that is the end of the matter. With public-records law, the reporter writes a different kind of letter: one that cites a specific section of code, imposes a response deadline, and preserves the right to appeal. The letter is still a request. What makes it effective is the legal infrastructure behind it.

This chapter treats law as a lineage. Not because data scientists should become lawyers, but because the professions that have thought hardest about pursuing the public interest under adversarial conditions work inside courtrooms and statutes. You will read two law and society scholars on what "public interest lawyering" is, draft and track a records request, and sit with the fact that the same legal infrastructure that enables accountability work can be weaponized against the people it is supposed to serve. The running example is a class-wide Colorado Open Records Act (CORA) tracker, built on SQLite, that you and your classmates populate across a semester. By the end, you will have a reusable schema, a drafted request you can send, and a small corpus of released documents analyzed with tools no more exotic than Tesseract and pandas.

10.1. Public interest lawyering as a profession

Albiston and Nielsen (2017) studied the U.S. public interest law field through a pair of surveys spanning three decades. Public interest lawyering

is not a specialty like tax or real estate. It is a *stance* toward practice: representing interests the market will not pay to represent, often on behalf of clients whose stake in a legal outcome is structural rather than individual. Cause lawyering, the term Nielsen and Albiston (2006) develop in earlier work, treats a case as an instrument for changing a rule rather than a dispute between two parties.

Their empirical argument is that the field has settled into a two-tier structure. A small number of well-funded organizations (the ACLU, the NAACP Legal Defense Fund, the Electronic Frontier Foundation) work at the appellate-strategy and legislative-advocacy level. A much larger tier of legal aid societies and clinical programs absorbs front-line demand in family court, eviction proceedings, and benefits appeals. The first tier has strategic leverage. The second has caseload and direct contact with the communities served.

The analogy for data science is not strained. A small set of civil-society organizations works at the policy and publication tier. A much larger set of journalists, clinicians, union researchers, community organizers, and municipal IT staff does the front-line work of unpicking a broken benefits portal or a mis-calibrated risk score. The field's self-understanding is weighted toward the first tier. The actual work is weighted toward the second. This chapter's tools aim at the second.

10.2. FOIA, CORA, and the statutory geometry

The Freedom of Information Act (FOIA), enacted in 1966 and amended in 1974, 1996, and 2016, establishes a presumption that records held by federal executive-branch agencies are public unless they fall within nine enumerated exemptions. Chapter 3 (Chapter 3) already introduced the exemption list as a topology of what the government can decline to answer.

Every state has a parallel statute. Colorado's is CORA, codified at C.R.S. § 24-72-201 et seq. CORA's custodian-response deadline is three working

days, extendable to ten for "extenuating circumstances," shorter than FOIA's twenty working days. Its fee schedule is capped for staff research and retrieval time. It has a distinct regime for criminal justice records under the Colorado Criminal Justice Records Act (CCJRA), which you need to know exists because a request blending administrative and criminal justice material will come back bifurcated, with the CCJRA portion subject to a "contrary to the public interest" balancing test the custodian applies at their discretion.

That discretion matters. A CORA request asking for "all emails from the Chief of Police mentioning ShotSpotter" will produce a different response than the same request phrased as "administrative records concerning the acquisition and deployment of ShotSpotter technology, including procurement correspondence, training materials, and use-of-force memoranda." The first invites the custodian to classify the material as criminal justice records. The second names administrative categories and makes that classification harder to justify. This is the specificity a reporter brings to a sourcing interview, translated into statutory language.

The Missing Manual

Most records-request software assumes your request succeeds the first time. It does not. A well-managed request produces a negotiation: the first response will be partial, redacted, mis-scoped, or delayed with a fee estimate designed to make you withdraw. Design your tracker for iteration. Log the followups. Log the appeals. Log the reasons the custodian gave for redactions. Redactions are themselves metadata: the pattern of what the agency blacks out tells you what its legal theory is before it articulates one. If three of four releases redact the same paragraph under the same exemption, you have a de facto policy you can write about, cite in an appeal, or hand to a lawyer.

10.3. The CFAA and the chilling effect

Legal infrastructure is not only a resource for accountability work. It also constrains what accountability work is permitted to try. The Computer Fraud and Abuse Act (CFAA), enacted in 1986, criminalizes "unauthorized access" to a "protected computer." For most of its life, the statute's reach was interpreted broadly enough that violating a website's terms of service (creating fake accounts to audit a housing platform, for example) exposed the researcher to federal felony liability. Academic researchers reasonably declined to do the research.

The *Sandvig v. Barr* litigation, brought by the ACLU on behalf of Christian Sandvig and co-plaintiffs (American Civil Liberties Union 2019), challenged that interpretation. The plaintiffs were researchers who wanted to audit online employment and housing platforms for race and gender discrimination, using paired accounts that technically violated the platforms' terms of service. They sought a pre-enforcement declaration that the CFAA did not criminalize their research. In 2020, the D.C. District Court ruled in their favor on statutory grounds, narrowing the "exceeds authorized access" clause to exclude terms-of-service violations. The Supreme Court's *Van Buren* decision in 2021 confirmed the narrowing in a separate case.

The chilling effect *Sandvig* addressed is the kind of infrastructure problem this book takes seriously. A statute that criminalizes a research method does not need to prosecute a single researcher to work. It only needs to produce enough ambient uncertainty that IRBs decline to approve, general counsel decline to defend, and graduate students rationally choose a different dissertation. *Sandvig* is a counter-example: an audit population that existed because a civil society organization absorbed the legal cost of clarifying the rule.

10.4. SyRI, discovery, and weaponization

Chapter 3 (Chapter 3) already introduced the Dutch SyRI case as the successful civil society challenge to an opaque welfare-fraud algorithm. It is worth returning to here as a *litigation strategy*. The coalition (NJCM together with welfare-rights organizations and two individual plaintiffs) structured the suit around Article 8 of the European Convention on Human Rights. The District Court of The Hague ruled the SyRI statute incompatible with Article 8 in February 2020. In Albiston and Nielsen's vocabulary, this was cause lawyering at its most strategic. The ruling did not end algorithmic welfare surveillance in the Netherlands; subsequent systems have appeared with adjusted frameworks. But it established a precedent the next iteration had to work around, which is what a win looks like here.

Legal discovery is a linkability mechanism the textbook framing often misses. A civil suit compels production of internal records no records request would dislodge, places human sources under oath, and produces interrogatory answers the party is legally bound to stand behind. The cost, for the researcher, is that discovery material is typically subject to a protective order: usable in the case, not releasable without the court's permission. Chapter 16 (Chapter 16) will argue the instinct to refuse release is sometimes the right one; discovery is where the instinct is not optional.

Public records laws are neutral on their face. In practice, they can be turned against the populations they were built to protect. Immigration enforcement agencies have used state and local records requests to obtain residency, employment, and vehicle-registration data on non-citizens. School board minutes, including names of public commenters on immigration-related agenda items, have been harvested and forwarded to federal enforcement. The openness that lets a journalist investigate a sheriff's office also lets a federal agency subpoena the sheriff's record of who was booked, who posted bail, and who picked them up. This is not an argument against open

records. It is an argument for thinking about records-request work the way Chapter 8 thought about stewardship: the same infrastructure that enables one use case enables every use case the statute does not exclude.

10.5. Amicus briefs and MuckRock

Data scientists can contribute to public-interest legal work without becoming lawyers, through the amicus curiae ("friend of the court") brief: a third-party submission from an organization with subject-matter expertise, offering the court context the named parties have not developed. In technology cases, amicus briefs from researchers have become a recognized genre: computer scientists explaining what an algorithm does, statisticians explaining what a disparate-impact measure measures. The intellectual demand is within the reach of a well-prepared graduate student working with an attorney. Chapter 20 will return to the rhetoric of writing for a decision-making audience; the amicus is a close cousin.

MuckRock, the nonprofit records-request platform founded in 2010, is the closest thing the U.S. records-request ecosystem has to shared infrastructure. It handles drafting, tracking, agency routing, followups, and public release. Its archive is itself a research resource: many of the documents you would otherwise FOIA have already been FOIA'd. Its public-by-default posture has costs, because some agencies respond differently to MuckRock than to an individual requester. The running example below builds a lighter-weight, self-hosted version. A classroom, a newsroom, or a research group needs a tracker even for requests it is not ready to expose publicly, and a SQLite database is the right scale.

10.6. Tutorial: a CORA tracker in SQLite

The schema has four tables. Each request can have multiple responses (partial releases, supplementary productions after appeal). Each response can include multiple documents. Followups are requester-initiated actions (clarifying letters, appeals, fee-waiver requests) that shape the record without producing a response themselves.

```
import sqlite3
from pathlib import Path

DB_PATH = Path("cora_tracker.sqlite")

SCHEMA = """
CREATE TABLE IF NOT EXISTS requests (
    id INTEGER PRIMARY KEY,
    agency TEXT NOT NULL,
    date_submitted TEXT NOT NULL,
    description TEXT NOT NULL,
    status TEXT NOT NULL
        CHECK (status IN ('open','partial','closed','denied','appealed')),
    cora_or_foia TEXT NOT NULL
        CHECK (cora_or_foia IN ('CORA','FOIA','CCJRA','other')),
    associated_project TEXT
);
CREATE TABLE IF NOT EXISTS responses (
    id INTEGER PRIMARY KEY,
    request_id INTEGER NOT NULL REFERENCES requests(id),
    date_received TEXT NOT NULL,
    redactions_noted TEXT,
    documents_count INTEGER,
    notes TEXT
);
```

```
CREATE TABLE IF NOT EXISTS followups (
    id INTEGER PRIMARY KEY,
    request_id INTEGER NOT NULL REFERENCES requests(id),
    date TEXT NOT NULL,
    type TEXT NOT NULL
        CHECK (type IN ('clarification','fee_waiver','appeal','reminder','wit
    content TEXT
);
CREATE TABLE IF NOT EXISTS documents (
    id INTEGER PRIMARY KEY,
    response_id INTEGER NOT NULL REFERENCES responses(id),
    filename TEXT NOT NULL,
    sha256 TEXT NOT NULL,
    size_bytes INTEGER
);
"""

with sqlite3.connect(DB_PATH) as conn:
    conn.executescript(SCHEMA)
```

Inserting a request, recording a response, and querying open requests older than thirty days:

```
from datetime import date, timedelta

def insert_request(agency, date_submitted, description,
                   cora_or_foia="CORA", associated_project=None,
                   status="open"):
    with sqlite3.connect(DB_PATH) as conn:
        cur = conn.execute(
            """INSERT INTO requests
               (agency, date_submitted, description, status,
                cora_or_foia, associated_project)
```

```
                VALUES (?, ?, ?, ?, ?, ?)""",
            (agency, date_submitted, description, status,
             cora_or_foia, associated_project),
        )
        return cur.lastrowid

def record_response(request_id, date_received, documents_count=0,
                    redactions_noted=None, notes=None):
    with sqlite3.connect(DB_PATH) as conn:
        cur = conn.execute(
            """INSERT INTO responses
               (request_id, date_received, redactions_noted,
                documents_count, notes)
               VALUES (?, ?, ?, ?, ?)""",
            (request_id, date_received, redactions_noted,
             documents_count, notes),
        )
        return cur.lastrowid

def open_and_stale(days=30):
    cutoff = (date.today() - timedelta(days=days)).isoformat()
    with sqlite3.connect(DB_PATH) as conn:
        conn.row_factory = sqlite3.Row
        rows = conn.execute(
            """SELECT id, agency, date_submitted, description
               FROM requests
               WHERE status='open' AND date_submitted < ?
               ORDER BY date_submitted""",
            (cutoff,),
        ).fetchall()
    return [dict(r) for r in rows]

rid = insert_request(
```

```
    agency="Boulder Police Department",
    date_submitted="2026-02-03",
    description=("Administrative records concerning procurement "
                 "and deployment of automated license plate readers "
                 "(ALPRs), 2023-01-01 to present."),
    associated_project="alpr_audit_spring26",
)
stale = open_and_stale()
# => list of dicts you can iterate into followup emails
```

A stale-request query wired into a followup mailmerge has an outsized effect on response rates. The three-day deadline is not self-enforcing. The requester has to send the reminder.

10.7. Drafting the letter

A CORA letter is four short paragraphs. Tone is formal, specificity is load-bearing, statutory citations do real work.

> Dear Records Custodian,
>
> Pursuant to the Colorado Open Records Act, C.R.S. § 24-72-201 et seq., I respectfully request the following records maintained by the Boulder Police Department:
>
> (1) All contracts, statements of work, and data-sharing agreements between the Department and any vendor of automated license plate reader (ALPR) technology, executed between January 1, 2023 and the date of this request;
>
> (2) Training materials and standard operating procedures governing ALPR deployment by Department personnel during that period;

> (3) Internal memoranda, including supervisor correspondence, concerning ALPR use in investigations during that period.
>
> For the purposes of C.R.S. § 24-72-205(6), I request a fee waiver on the basis that these records concern a matter of public interest and I intend to disseminate the information through a published research report. Should estimated fees exceed $25, please contact me first. Should any portion be withheld, please cite the basis with specificity under C.R.S. § 24-72-204, and release any segregable non-exempt portions.

Four things in that letter do work. The statutory citation establishes that you know the law. The enumerated categories make the request specific enough that the custodian cannot lawfully refuse it as "overly broad," the most common informal denial. The fee-waiver paragraph cites the actual subsection and invokes the "public interest" standard. The segregability clause forces the custodian to release non-exempt material rather than withholding an entire document because part of it is exempt.

10.8. Analyzing a small corpus of released records

A typical CORA release is a PDF, often scanned from paper, with redactions applied as black rectangles. Tesseract OCR produces readable text from the scans; pandas holds the metadata.

```
import subprocess, hashlib
import pandas as pd
from pathlib import Path

def ocr_pdf(pdf_path, out_txt):
    tmp = Path(out_txt).with_suffix("")
    subprocess.run(["pdftoppm", "-r", "300", str(pdf_path),
                    str(tmp), "-png"], check=True)
```

```
    pages = sorted(tmp.parent.glob(tmp.name + "-*.png"))
    text = []
    for p in pages:
        r = subprocess.run(["tesseract", str(p), "-", "--psm", "6"],
                           capture_output=True, text=True, check=True)
        text.append(r.stdout)
    Path(out_txt).write_text("\n\f\n".join(text))

def sha256_of(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 16), b""):
            h.update(chunk)
    return h.hexdigest()

records = []
for pdf in Path("releases/").glob("*.pdf"):
    txt = pdf.with_suffix(".txt")
    if not txt.exists():
        ocr_pdf(pdf, txt)
    records.append({
        "filename": pdf.name,
        "sha256": sha256_of(pdf),
        "text_length": len(txt.read_text()),
        "redaction_marks": txt.read_text().count("[REDACTED]"),
    })
df = pd.DataFrame(records)
df.sort_values("redaction_marks", ascending=False).head()
```

The “redaction marks” column is crude. The real signal comes from patterning: which document types have the heaviest redactions, which exemption codes the custodian cited, and which phrases recur near the redactions. A semester’s worth of pooled class requests is enough to describe

the agency's redaction practice with a confidence a single requester cannot reach alone.

> **In the Public Interest**
>
> Chapter 5 (Chapter 5) named **remedy** as the installed base's final element, and Chapter 7's oversight framework (Chapter 7) put it starkly: evidence only matters if it can trigger action. A records request that produces an unreadable scan of a redacted contract, and then sits on a researcher's laptop, is not oversight. It is raw material for oversight. The tracker, letter template, and OCR pipeline above are the minimum scaffolding that turns that material into something a journalist can publish, an advocate can cite in a council meeting, an attorney can attach to a complaint, or a commissioner can reference in a rulemaking (Keegan 2026). The tools here advance **oversight** when they close the loop to remedy. They advance **openness** in a weaker sense even when they do not: a well-tracked request produces a record of what the agency said.

10.9. Looking ahead

Public records work is slow. The typical timeline from a first CORA letter to a usable dataset is measured in months, and an appeal can extend it by a year. Reporters and lawyers budget for this. Data scientists, trained on the responsiveness of a public API, often do not, and the gap between what a researcher expects to take a week and what actually takes a semester is the biggest reason accountability work stalls. File requests you will not use for months. Chapter 21 (Chapter 21) takes up a different records-based genre, the public comment: similar infrastructure, opposite direction, you adding to the record rather than extracting from it.

Chapter 11 (Chapter 11) turns to accounting and auditing. Auditors, like

public-interest lawyers, have spent a century working out methods for producing credible claims about institutions under adversarial conditions. What journalism contributes through sourcing and right of reply, and what law contributes through discovery and statutory procedure, accounting contributes through the audit opinion and its scaffolding: workpapers, materiality thresholds, and the reviewer who signs off on the signoff.

10.10. Further Reading and Resources

- Colorado Freedom of Information Coalition: https://coloradofoic.org/. The clearest practitioner guide to CORA, with sample letters and records of notable appeals (Colorado Freedom of Information Coalition 2024).
- MuckRock: https://www.muckrock.com/. File requests, track them, and search prior releases before reinventing (MuckRock Foundation 2024).
- Reporters Committee for Freedom of the Press, *Open Government Guide*: https://www.rcfp.org/open-government-guide/. State-by-state comparative reference (Reporters Committee for Freedom of the Press 2024).
- Electronic Frontier Foundation, CFAA reform page: https://www.eff.org/issues/cfaa (Electronic Frontier Foundation 2024).
- *Sandvig v. Barr* filings and briefs at ACLU: https://www.aclu.org/cases/sandvig-v-barr-challenge-cfaa-prohibition-uncovering-racial-discrimination-online (American Civil Liberties Union 2019).
- Text of the Freedom of Information Act, 5 U.S.C. § 552: https://www.law.cornell.edu/uscode/text/5/552 (US Congress 1966). Reading the nine exemptions in order is thirty minutes well spent.
- Colorado Open Records Act, C.R.S. § 24-72-201 et seq.: https://leg.colorado.gov/colorado-revised-statutes (Colorado General Assembly 1968).

- Sunshine Week: https://sunshineweek.org/. The annual March observance that catalyzes most of the open-records coverage worth reading (News Leaders Association 2024).
- Albiston and Nielsen (Albiston and Nielsen 2017) on the two-tier structure of U.S. public interest law.
- Samantha Warren, "Where public records meet the public interest: A reporter's field guide" (Warren 2025).

10.11. Exercises

Exercise 10.1 (Guided). Draft a CORA or FOIA request on a topic you care about. Boulder, Denver, and Colorado state agencies are all reasonable targets; if you are not in Colorado, use your state's equivalent statute. Your instructor will help scope the topic. Submit the request. Paste the confirmation into your class tracker.

Exercise 10.2 (Guided). Implement the SQLite schema. Insert your Exercise 10.1 request. Write a function that emits a formatted reminder email to any agency whose open request is older than the statutory deadline plus a grace period you justify in a comment. Commit the code to the class repository.

Exercise 10.3 (Analytic). Obtain a small corpus (five to ten documents) of released records, from MuckRock's archive if your own request has not returned yet. OCR them. For each document, record the exemption codes cited, the types of content redacted, and the date. Produce a one-page analysis describing the redaction pattern. Do not over-interpret: five documents is not a population.

Exercise 10.4 (Reading). Read the *Sandvig v. Barr* complaint, the government's motion to dismiss, and the March 2020 opinion. In 400 words, summarize the plaintiffs' legal theory, the government's theory, and the narrow statutory ground on which the court ruled. Identify which plaintiffs' arguments the court did not reach, and speculate briefly about why.

Exercise 10.5 (Open-ended). Identify a public-interest question you could answer with a records request but cannot answer from public datasets or scraping alone. Write two pages: the question, the records you would request, the agencies, the anticipated exemption battles, and the theory of change for what you would do with the records. Save it as `exercises/ch10_foia_plan.md`. The best of these will become Exercise 10.1 filings for next year's class.

11. Accounting and Auditing

In October 1929, the New York Stock Exchange lost roughly a quarter of its value in a week. Congress spent the next four years arguing about why, and by 1934 it had produced two statutes (the Securities Act of 1933 and the Securities Exchange Act of 1934) that reshaped the American professional landscape. The new law required publicly traded companies to file financial statements, and those statements to be examined by an independent auditor. Overnight, the external audit went from a discretionary service a firm might commission to reassure its banker, to a legally mandated act of public verification. The profession that conducted those audits acquired, as part of the bargain, something it had not previously had: a public interest mandate. A certified public accountant's client is the company paying the invoice. The CPA's duty of care runs to shareholders, lenders, regulators, and everyone downstream of the company's claims about itself.

That bargain has been renegotiated several times since. Arthur Andersen, one of the "Big Five" audit firms of the 1990s, signed off on Enron's financial statements for more than a decade before Enron collapsed in 2001, taking roughly $63 billion of market capitalization and twenty thousand jobs with it. Andersen did not survive the fallout. Its consulting practice had been earning more from Enron than its audit practice, and its own workpapers, shredded in a Houston office as federal investigators approached, became the emblematic image of audit failure. Congress responded with the Sarbanes-Oxley Act of 2002, which tightened auditor independence rules, required public companies to certify the accuracy of their filings, and established the Public Company Accounting Oversight Board to audit the auditors. The

profession's public interest mandate was not abolished. It was recertified under closer supervision.

This chapter treats the audit tradition as a central lineage public interest data science should claim. Carnegie and Napier (1996) argue that accounting is not a neutral record-keeping technology but a practice whose conventions and disclosure rules are the sediment of specific historical bargains. Baker (2005), examining the American public accounting profession's rhetoric, asks a pointed question: when the profession invokes "the public interest," whose interest is it actually defending? His answer, drawn from close reading of AICPA pronouncements, is that the profession has tended to serve the paying client while invoking the public as a legitimating abstraction. That gap matters here: algorithmic audit is about to repeat every mistake the accounting profession has already made, unless it studies the history and does otherwise.

11.1. From financial audit to algorithmic audit

Raji and colleagues (2020) propose a framework for internal algorithmic auditing that borrows from accounting, aerospace, and medicine: three professions where audit of some form has been institutionalized for decades. Their framework names five stages (scoping, mapping, artifact collection, testing, reflection) and insists on durable, dated workpapers. A good algorithmic audit produces documents that a reasonable outsider could read a year later and reconstruct what was examined, what was found, and what was decided. A bad one produces a one-paragraph attestation, or a dashboard with fairness metrics but no record of which metrics were chosen, by whom, against which thresholds.

The distinction matters because the regulatory environment is hardening around audit. New York City's Local Law 144, effective July 2023, requires employers using automated employment decision tools to commission an independent "bias audit" and publish a summary on the deploying

company's public website before any covered hiring decision. The law is narrow, the enforcement modest, and the early compliance data unflattering. Goodman and Trehu (2023) examine LL144 as a case of what they call "audit washing": audits whose institutional form looks like oversight but whose content is shallow enough to credential any system as compliant. The European Union's AI Act, in force since 2024, introduces a more ambitious conformity assessment regime for high-risk AI systems. It will also, if past is prologue, generate a cottage industry of audits whose depth varies enormously. Your job as a public interest data scientist is to know the difference, and to be able to defend the difference in writing.

In the Public Interest

Baker (2005) reads the American accounting profession's own pronouncements and finds a persistent conflation: "the public interest" names, in the profession's literature, sometimes the paying client, sometimes "the market," sometimes an abstract audience. Baker argues the defensible definition is narrower and harder: the public interest is served when audit practice protects "all those affected by a corporation's activities," not the entity signing the check. The **oversight** value from Chapter 7 (Chapter 7) asks what independent verification requires. Accounting answers: dated workpapers, separate audit and consulting functions, enforceable standards, and a credential that can be revoked. If your algorithmic audit lacks all four, it performs oversight rather than providing it. The **continuity** element of the installed base from Chapter 5 (Chapter 5) matters for the same reason: an audit that cannot be reproduced next quarter because the analyst's laptop has been wiped is, for public-interest purposes, an audit that did not happen.

11.2. The running example: wrapping the Chapter 7 audit

Chapter 7 walked you through a disaggregated `fairlearn` audit on a public dataset, with metrics broken out by protected group. That exercise produced a set of numbers. This chapter produces something different: a pipeline that produces those numbers on demand, with a record of every decision made along the way and a log of what changed between runs. The difference is roughly the difference between a paragraph and a workpaper.

Here is the directory layout you will build:

```
audit-pipeline/
  CHANGELOG.md
  Makefile
  README.md
  manifest.yaml
  audit.py
  thresholds.yaml
  data/
      .gitkeep
  models/
      .gitkeep
  reports/
       .gitkeep
```

Nine top-level items, most of them placeholders. The whole project is perhaps three hundred lines of code. What makes it an audit is not the code volume. It is the manifest, the changelog, and the discipline of running one before touching the other.

11.2.1. The manifest

`manifest.yaml` is the input contract. It says, for a given audit run, exactly what is being audited, at what version, with what decisions frozen in advance. A reviewer reading the manifest a year later should be able to reconstruct the run without guessing.

```yaml
# manifest.yaml
audit:
  name: "COMPAS recidivism disaggregated audit"
  run_id: "2026-04-23-001"
  date: "2026-04-23"
  analyst: "your.name@colorado.edu"
  purpose: "Quarterly oversight audit per org policy OV-07."

dataset:
  source: "ProPublica COMPAS two-year dataset"
  url: "https://github.com/propublica/compas-analysis"
  version: "commit bf4ce2"
  local_path: "data/compas-scores-two-years.csv"
  sha256: "b1946ac92492d2347c6235b4d2611184..."
  retrieved_on: "2026-04-20"

model:
  name: "logistic-baseline"
  version: "v0.3.1"
  path: "models/logistic-baseline-v0.3.1.pkl"
  sha256: "3a7bd3e2360a3d29eea436fcfb7e44c7..."
  hyperparameters:
    C: 1.0
    penalty: "l2"
    random_state: 42

protected_attributes:
```

```yaml
  - race
  - sex

metrics:
  - selection_rate
  - false_positive_rate
  - false_negative_rate
  - demographic_parity_difference
  - equalized_odds_difference

thresholds_file: "thresholds.yaml"
```

Every field is load-bearing. The dataset `sha256` is how a later reader verifies the same bytes. The model hash and hyperparameters verify the same model. The `run_id` is what the changelog references when numbers change. Nothing here is optional.

`thresholds.yaml` carries the decision rules separately, because they change on a different cadence than the data.

```yaml
# thresholds.yaml
demographic_parity_difference:
  warn: 0.05
  fail: 0.10
equalized_odds_difference:
  warn: 0.05
  fail: 0.10
false_positive_rate_gap:
  warn: 0.08
  fail: 0.15
notes: |
  Thresholds set by Policy OV-07, adopted 2026-02-15.
  "warn" triggers a note in the report; "fail" triggers an
  escalation to the Oversight Committee per Section 4.2.
```

Two files. Both dated, both in version control, both citing the policy that authorized them. Financial auditors call this *authority documentation*. Most algorithmic audits skip it.

11.2.2. The audit script

`audit.py` reads the manifest, verifies the hashes, runs the disaggregated audit from Chapter 7, compares results against the thresholds, and writes a structured report.

```
# audit.py
import hashlib
import json
import pathlib
import sys
from datetime import datetime, timezone

import pandas as pd
import yaml
from sklearn.metrics import confusion_matrix
from fairlearn.metrics import (
    MetricFrame,
    selection_rate,
    false_positive_rate,
    false_negative_rate,
    demographic_parity_difference,
    equalized_odds_difference,
)
import joblib


def sha256_of(path):
    h = hashlib.sha256()
```

```
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(8192), b""):
            h.update(chunk)
    return h.hexdigest()


def load_manifest(path="manifest.yaml"):
    with open(path) as f:
        return yaml.safe_load(f)


def verify_hashes(manifest):
    problems = []
    for kind in ("dataset", "model"):
        declared = manifest[kind]["sha256"]
        actual = sha256_of(manifest[kind]["path" if kind == "model" else "loc
        if declared != actual:
            problems.append(f"{kind}: declared {declared[:12]}, got {actual[
    return problems


def run_audit(manifest, thresholds):
    df = pd.read_csv(manifest["dataset"]["local_path"])
    model = joblib.load(manifest["model"]["path"])
    X = df.drop(columns=["two_year_recid", "race", "sex"])
    y_true = df["two_year_recid"]
    y_pred = model.predict(X)
    results = {}
    for attr in manifest["protected_attributes"]:
        frame = MetricFrame(
            metrics={
                "selection_rate": selection_rate,
                "fpr": false_positive_rate,
```

```
                "fnr": false_negative_rate,
            },
            y_true=y_true,
            y_pred=y_pred,
            sensitive_features=df[attr],
        )
        results[attr] = {
            "by_group": frame.by_group.to_dict(),
            "dpd": float(demographic_parity_difference(
                y_true, y_pred, sensitive_features=df[attr])),
            "eod": float(equalized_odds_difference(
                y_true, y_pred, sensitive_features=df[attr])),
        }
    return results


def evaluate_thresholds(results, thresholds):
    findings = []
    for attr, res in results.items():
        for metric_name, key in [
            ("demographic_parity_difference", "dpd"),
            ("equalized_odds_difference", "eod"),
        ]:
            value = abs(res[key])
            rule = thresholds[metric_name]
            if value >= rule["fail"]:
                findings.append(("FAIL", attr, metric_name, value))
            elif value >= rule["warn"]:
                findings.append(("WARN", attr, metric_name, value))
    return findings


def write_report(manifest, results, findings, out_dir="reports"):
```

```
    out_dir = pathlib.Path(out_dir)
    out_dir.mkdir(exist_ok=True)
    run_id = manifest["audit"]["run_id"]
    timestamp = datetime.now(timezone.utc).isoformat()
    report = {
        "run_id": run_id,
        "generated_at": timestamp,
        "manifest": manifest,
        "results": results,
        "findings": findings,
    }
    json_path = out_dir / f"{run_id}.json"
    md_path = out_dir / f"{run_id}.md"
    with open(json_path, "w") as f:
        json.dump(report, f, indent=2, default=str)
    # Human-readable companion
    with open(md_path, "w") as f:
        f.write(f"# Audit report {run_id}\n\n")
        f.write(f"- Generated: {timestamp}\n")
        f.write(f"- Analyst: {manifest['audit']['analyst']}\n")
        f.write(f"- Dataset: {manifest['dataset']['source']} "
                f"(sha256 {manifest['dataset']['sha256'][:12]})\n")
        f.write(f"- Model: {manifest['model']['name']} "
                f"{manifest['model']['version']}\n\n")
        f.write("## Findings\n\n")
        if not findings:
            f.write("No threshold violations.\n")
        for level, attr, metric, value in findings:
            f.write(f"- **{level}** on `{attr}` / {metric}: {value:.4f}\n")


def main():
    manifest = load_manifest()
```

```
    with open(manifest["thresholds_file"]) as f:
        thresholds = yaml.safe_load(f)
    problems = verify_hashes(manifest)
    if problems:
        print("Hash verification failed:", problems, file=sys.stderr)
        sys.exit(2)
    results = run_audit(manifest, thresholds)
    findings = evaluate_thresholds(results, thresholds)
    write_report(manifest, results, findings)


if __name__ == "__main__":
    main()
```

Notice what the script does that a notebook would not. It verifies hashes before running anything. It writes both a machine-readable JSON report and a human-readable Markdown companion. It exits non-zero when the inputs do not match the manifest, because a run against unverified inputs is not an audit. It records the UTC timestamp of the run. Every one of these behaviors is stolen from financial audit practice, where a workpaper exists to be legible to someone who was not in the room.

11.2.3. The Makefile

`Makefile` is the single-command interface. If a reviewer cannot run `make audit` and get a report, the pipeline is not reproducible.

```
# Makefile
.PHONY: audit verify clean

audit: verify
    python audit.py
```

```
verify:
    @python -c "import yaml; yaml.safe_load(open('manifest.yaml'))"
    @test -f $$(python -c "import yaml; print(yaml.safe_load(open('manifest.y
    @test -f $$(python -c "import yaml; print(yaml.safe_load(open('manifest.y

clean:
    rm -f reports/*.json reports/*.md
```

Three targets. `verify` confirms the manifest parses and its files exist. `audit` runs the script. `clean` removes generated reports, not inputs. You do not clean your workpapers. You archive them.

11.2.4. The changelog

`CHANGELOG.md` is the narrative companion to the numbers. It records, for each run, what changed since the last run and why.

```
# CHANGELOG

## 2026-04-23-001

- First run under Policy OV-07.
- Dataset: ProPublica COMPAS, commit bf4ce2, retrieved 2026-04-20.
- Model: logistic-baseline v0.3.1, trained 2026-04-15.
- Findings: WARN on race / equalized_odds_difference (0.071).
- No prior run to compare against.

## 2026-04-30-001

- Re-run with no changes to manifest.
- Library update: fairlearn 0.10.0 -> 0.11.0.
- equalized_odds_difference changed from 0.071 to 0.068.
```

```
  Root cause: fairlearn 0.11 changed default NaN handling
  in MetricFrame.by_group; see fairlearn CHANGELOG entry
  for issue #1237. No change to audit conclusion.
- Analyst decision: accept new value; document in report.
```

The Missing Manual

Reproducibility is not `pip freeze`. A `requirements.txt` tells you which packages were installed; it does not tell you why a metric changed between runs. A `Dockerfile` preserves a runtime; it does not preserve a decision. A signed `sha256` manifest fixes the inputs; it does not explain why those inputs were chosen. Reproducibility, in the sense an auditor means the word, is decisions, data, and narrative travelling together. Financial auditors date and initial their workpapers for a reason: an undated workpaper is deniable, and deniability is the enemy of accountability. Your audit code, your manifests, and your changelog entries should be dated and attributable. An undated audit script and a `results.csv` with no run context is not oversight. It is exhibit A in the defense's motion to dismiss.

11.3. Running the pipeline, twice

Commit the whole directory to git (`git init`, `git add .`, `git commit -m "ch-11 audit skeleton, run 001"`). Run `make audit`. Inspect `reports/2026-04-23-001.md`. Read what it says. Read what it does not say. Does the report record which columns were treated as features, and which were dropped? Does it record the split between training and test, or evaluate on the whole dataset? Both choices are defensible. Neither is defensible if it is implicit. The report's job is to make implicit choices legible.

One week later, run `make audit` again after updating the `run_id` in the manifest. With luck, the numbers will be identical, and you will have produced an uninteresting second workpaper confirming the pipeline is deterministic. More often, something will have drifted. A minor version bump of `fairlearn` or `scikit-learn` will have changed a default; a transitive dependency will have been yanked from PyPI; your OS will have updated `openssl` and changed a cryptographic default a download step depended on. The drift is usually small. The discipline is not in preventing it but in noticing it, documenting it in the changelog, and deciding whether it changes the audit conclusion. Nine times out of ten it does not. The tenth time is why you keep a changelog.

11.4. The regulatory landscape: LL144 and the EU AI Act

LL144 requires four things of a covered automated employment decision tool: a bias audit conducted by an independent auditor, performed within the last year, measuring selection rates and impact ratios by race/ethnicity and sex, with results published on the deploying company's public site. Goodman and Trehu (2023) surveyed first-year compliance and found that published audits varied wildly in depth and intellectual defensibility. Some were serious workpapers. Others were one-page PDFs attesting that the tool "did not exhibit bias" without disclosing which metric, which threshold, which population. A legal regime that accepts both as compliant is a regime whose public interest mandate has been captured by the rhetoric of audit without its substance.

The EU AI Act's conformity assessment regime, applied to high-risk AI systems under Article 43, is structurally more ambitious. It requires either an internal conformity assessment with extensive documentation, or a third-party assessment through a notified body, depending on the system's risk category and whether a harmonized standard is available. The regime

borrows from the EU's product safety tradition (CE marking, notified bodies, technical files), older than computing itself. The question, again, is whether the form will be filled with substance. The same auditors who wrote one-page LL144 attestations are already selling EU AI Act conformity packages. Buolamwini and Gebru's Gender Shades study (2018), which opened Chapter 7's discussion of oversight (Chapter 7), is a useful benchmark: a rigorous public audit of a deployed system, by researchers with no commercial relationship to the vendor, published with full methodology. Most conformity-assessment output will not look like Gender Shades. Some of it should.

11.5. Exercises

Exercise 11.1 (Guided). Build the directory and files above. Use the COMPAS dataset (or another tabular dataset with protected-attribute columns) and any scikit-learn classifier you trained in Chapter 7. Populate `manifest.yaml` with real hashes, commit the project to git, and run `make audit`. Submit the repository URL and the generated report. Graded on completeness, not on whether your model passes its thresholds.

Exercise 11.2 (Guided). One week after completing Exercise 11.1, run `make audit` again with an updated `run_id`. Compare the new report to the old one. Write a changelog entry describing what changed and why. If nothing changed, write an entry documenting that nothing changed and stating the exact versions of `fairlearn` and `scikit-learn` you ran under. Either way, commit the changelog update.

Exercise 11.3 (Analytic). NYC's Department of Consumer and Worker Protection maintains an enforcement page linking to employer disclosures under LL144. Pick one published bias audit from a real deploying company. Write 400 words analyzing it against the Raji et al. (2020) framework: is the scope named, the artifacts listed, the testing methodology described, the reflection documented? Where it falls short, be specific about which

workpaper is missing. Do not pick an audit from a company you hold stock in.

Exercise 11.4 (Comparative). Write a 500-word comparison of LL144 and the EU AI Act's conformity assessment regime along three dimensions: (1) scope (what systems are covered), (2) who performs the audit (independent third party, internal, notified body, regulator), and (3) what is published (full workpapers, summary report, certification only). Cite the text of both regulations and at least one secondary source for each.

Exercise 11.5 (Open-ended). Pick a data system you interact with weekly: your university's learning management system, your health insurer's claims portal, a dating app, your bank's fraud-detection system. Design an audit for it. Write 800 words covering what outcome variable you would measure, what protected attributes you would disaggregate by, what data you would need and how you would get it (including legal barriers), what thresholds you would set and on what authority, and who would receive the workpapers. You are not required to run the audit. You are required to make the audit thinkable.

11.6. Looking ahead

Chapter 12 turns from accounting to planning, the profession that has spent a century working out what it means to act in a public interest it did not invent. Where accounting answers "did the numbers happen the way the entity claims," planning answers "whose future is this, and who was in the room when it was decided." An audit without a plan produces compliance. A plan without an audit produces promises. Data science needs both.

11.7. Further Reading and Resources

- C. Richard Baker (2005), "What is the meaning of 'the public interest'? Examining the ideology of the American public accounting profession," *Accounting, Auditing & Accountability Journal* 18(5): 690–703. The essay this chapter leans on most heavily. Read it for the close reading of AICPA language.
- Inioluwa Deborah Raji et al. (2020), "Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing," *Proceedings of FAccT 2020*. The foundational paper on audit as a practice rather than a metric.
- Garry D. Carnegie and Christopher J. Napier (1996), "Critical and interpretive histories: Insights into accounting's present and future through its past," *Accounting, Auditing & Accountability Journal* 9(3): 7–39. The programmatic call for treating accounting as a historical practice rather than a neutral technique.
- NYC Department of Consumer and Worker Protection, Local Law 144 enforcement page: https://www.nyc.gov/site/dca/about/automated-employment-decision-tools.page. Start here to find real published bias audits. The page links to the rule text and to deploying-company disclosures.
- European Union (2024), "Artificial Intelligence Act," Regulation (EU) 2024/1689. Official text: https://eur-lex.europa.eu/eli/reg/2024/1689/oj. Conformity assessment is Article 43 and Annex VII. Read both.
- Bryce Goodman and Julia Trehu (2023), "AI audit washing and accountability." The working paper on LL144's early compliance data and on the structural incentives that produce shallow audits.
- PBS Frontline, "Bigger Than Enron" (2002). The canonical documentary on the Andersen-Enron collapse. Watchable in an evening; useful for the texture of how an audit firm loses its mandate.
- Public Company Accounting Oversight Board (PCAOB), inspection reports: https://pcaobus.org/oversight/inspections. Real workpapers

of real audit failures, dissected by the regulator. If you want to see what detailed audit criticism looks like, read a PCAOB inspection.

- Accounting, Auditing & Accountability Journal (AAAJ), the critical-accounting journal of record. Browse the archive for thirty years of scholarship on the public interest in accounting.
- Data & Society, "Algorithmic Impact Assessments" report series. Transitional reading between the accounting tradition and its algorithmic descendants.

12. Planning

Every data project is a planning document in disguise. It proposes that some slice of the world is worth measuring, some category system worth applying, some decision worth making differently once the analysis is done. Planning has spent the last century learning the consequences of that kind of proposal. Data scientists have not. This chapter borrows from the planners.

Part III treats the professions that figured out public-interest practice before data science existed. Chapter 9 handed you the journalist's habits of verification, Chapter 10 the lawyer's habits of record access, Chapter 11 the accountant's habits of audit and disaggregation. Planning, the fourth lineage, contributes the discipline of asking, before any decision is made, who is affected, who was in the room, who was not, and whose knowledge of the terrain counts as evidence.

12.1. A profession that learned from its mistakes

Planning as you encounter it today is the downstream result of a public fight it mostly lost and then, slowly, learned from. Robert Moses ran New York's public works apparatus for four decades on a specific theory: the public interest could be computed by a qualified expert, expressed as a highway alignment, and imposed on neighborhoods whose residents could not contest it. Jane Jacobs (1961), as Chapter 1 noted, took the other side, insisting that people who lived on a block knew things no master plan

could capture, and that a plan ignoring that knowledge was not a plan for the public.

The work of codifying what Jacobs had demonstrated fell to a generation of planners who took the problem seriously *for* planning, not merely *against* it. Paul Davidoff's 1965 essay "Advocacy and Pluralism in Planning" (1965) is the founding document. Davidoff argued there is no single neutral public interest a competent planner discovers through analysis. There are many publics, with conflicting interests, and the planner's job is to make those conflicts legible rather than paper over them with a unitary plan. The profession should supply advocate planners to underserved communities the way the legal profession supplies public defenders.

Advocacy planning did not become the default, but it changed what the default had to argue against. Sherry Arnstein's "ladder of citizen participation" (1969) followed four years later, giving planners a vocabulary for distinguishing real participation from its counterfeits (manipulation, therapy, information, consultation, placation, partnership, delegated power, citizen control). Marcus Lane's intellectual history (2005) traces how participation became a procedural requirement in most Western planning regimes, even when its substance was thin. Hashem Dadashpoor and Somayeh Ahani (2021) extend the inheritance in recent work on peri-urban change, insisting the forces reshaping a region are multiple and not reducible to what the master plan says.

You inherit this tradition when you work with geographic data. The choropleth expresses a theory about which boundaries matter; the demographic overlay, a theory about which categories count; the stakeholder list, a theory about whose voice will be sought. Planning's contribution is to name these theories as theories and insist they be defended.

12.2. Back to the Allegheny Family Screening Tool

Chapter 3 took up the Allegheny Family Screening Tool as a case of *exemption*: a risk-scoring system for child-welfare referrals that could not easily be audited from outside, because the administrative records feeding it were sealed. Read the same case through a planning lens and a different failure surfaces. The tool was designed by social scientists, procured by a county, and deployed on families. The families were not in the room. Neither were the front-line caseworkers whose judgment the tool was meant to augment, nor the neighborhoods whose referral rates it would amplify. The county ran a period of public comment. Comments were received. The deployment went forward.

A planner recognizes this as a participation artifact on the lower rungs of Arnstein's ladder: information transmitted, consultation held, the decision made by the same people who would have made it without consultation. A participation process that does not feed back into who holds **authority** (Chapter 7) to change the decision, and that produces no path to **remedy** (Chapter 5) when it goes wrong, is a participation process in name only.

12.3. A running example: the Central 70 project

You will practice the tools of this chapter on a real Denver decision: the Central 70 project, the reconstruction and widening of Interstate 70 through the Elyria-Swansea and Globeville neighborhoods north of downtown. The Colorado Department of Transportation published a final environmental impact statement and record of decision in 2017 (2017) and began construction shortly after. The project lowered the highway below grade, capped a four-block section with a park, and widened the traveled way. It also displaced residents, triggered lawsuits over environmental review, and intensified concerns about diesel exposure in neighborhoods that

had already absorbed decades of pollution from 1960s highway decisions that cut the same neighborhoods in half (2019).

This is a planning case with a documented stakeholder process, contested boundaries, and rich public data. It is also heavy material. Students who find it draining may substitute the Alpine-Balsam redevelopment in Boulder (2019) or another local case; the techniques transfer.

12.4. Stakeholders as a structured document

A stakeholder map lists the parties to a decision, what each wants, can do, and can see. It reflects the mapper's theory of whose interests count; writing that theory down makes it contestable.

Planners often produce stakeholder maps as tables. For a data science workflow, a structured, version-controlled document serves better. YAML reads like prose, diffs well in git, and loads into Python for downstream analysis. Here is a schema with one filled-in entry:

```yaml
# stakeholders_i70.yaml
decision:
  name: Central 70 reconstruction and widening
  lead_agency: Colorado Department of Transportation
  record_of_decision: 2017-01
  geography:
    primary_neighborhoods: [Elyria-Swansea, Globeville]
    county: Denver
    state: CO

stakeholders:
  - id: cdot
    name: Colorado Department of Transportation
    role: lead_agency
```

```yaml
    interests:
      - completing federally-funded reconstruction on schedule
      - maintaining I-70 freight and commuter capacity
    power:
      formal: high    # NEPA lead; owns the right of way
      informal: high
    access:
      in_the_room: yes
      documents_produced: [DEIS, FEIS, ROD, mitigation plan]
    notes: |
      Procedurally the decision-maker. Subject to FHWA oversight
      and court review under NEPA.

  - id: elyria_swansea_residents
    name: Elyria-Swansea residents
    role: affected_community
    interests:
      - reducing diesel particulate exposure
      - avoiding displacement
      - retaining neighborhood connectivity
    power:
      formal: low     # no veto over alignment
      informal: medium  # organized through neighborhood associations
    access:
      in_the_room: partially
      documents_produced: [public comments, litigation filings]
    notes: |
      Historically split by the 1964 I-70 viaduct. Limited
      Spanish-language outreach in the early EIS phases.
```

A complete map for Central 70 would list twenty to thirty entries: CDOT, FHWA, the city and county of Denver, Denver Public Schools (whose Swansea Elementary sits within two hundred feet of the widened roadway),

the EPA, neighborhood associations on both sides of the highway, the freight industry, commuter constituencies, public health researchers, legal aid organizations, and contractors.

Notice what the schema forces. You cannot fill in `power.formal` without deciding what "formal" means here, or `access.in_the_room` without deciding what "the room" is. For Central 70, "the room" is at minimum five rooms: NEPA scoping meetings, technical advisory committees, city council chambers, federal court, and the CDOT executive offices where alignment decisions were actually made. A stakeholder with full access to one may be absent from another.

12.5. Who was missing from the room

Once the map exists, a companion document lists whose perspectives are absent or under-represented, and says why. This "who was missing" analysis is short, uncomfortable to write, and the point of the exercise.

For Central 70, a partial list would include renters whose landlords received displacement compensation but who received nothing; undocumented residents who did not attend public meetings for reasons unrelated to interest; children whose cumulative diesel exposure was the actual health outcome at stake but who had no standing in NEPA; the ghosts of households displaced by the original 1964 viaduct, whose descendants bore the continuing costs of a decision made before most 2017 stakeholders were born; and residents of neighborhoods *not* directly touched by I-70 whose regional air quality nonetheless changed.

A "who was missing" analysis is a design input, not a ritual apology. If you can name whose perspective is missing, you can design the analysis to compensate: pulling in demographic data the formal process did not surface, weighting comments from less-represented zip codes, or publishing intermediate findings in formats a missing stakeholder could respond to.

That last move is where planning bleeds into journalism (Chapter 9) and the report-writing of Chapter 18.

12.6. The geographic imagination

Planning's other contribution is the habit of putting decisions on a map. A map is an argument about what counts as "the area," and every map excludes something the mapmaker did not think was relevant. Part of working with geographic data is learning to notice the exclusions.

The Python library `geopandas` is the default tool for this work. It extends `pandas` with a `geometry` column, so a `GeoDataFrame` behaves like a `DataFrame` that knows where each row is on the surface of the earth. Underneath, it is a mixing bowl of `shapely` (geometry), `pyogrio` or `fiona` (file I/O), and `pyproj` (coordinate reference systems). You will spend more time with the third than you expect.

```
import geopandas as gpd
import contextily as ctx
import matplotlib.pyplot as plt

# Load a Colorado county shapefile from Census TIGER/Line
counties = gpd.read_file(
    "tl_2023_08_cousub/tl_2023_08_cousub.shp"
)
# => GeoDataFrame with columns including NAME, GEOID, geometry
# => native CRS is typically EPSG:4269 (NAD83, lat/lon)

# Load a block-group demographic overlay (ACS 5-year, 2018-2022)
acs = gpd.read_file("acs_blockgroups_denver.geojson")

# Load a project boundary (traced from the CDOT FEIS project description)
i70 = gpd.read_file("central70_project_area.geojson")
```

Three layers, three files, three potential failure modes. The most common is a coordinate reference system (CRS) mismatch. A shapefile of Colorado counties in NAD83 (EPSG:4269) and a project boundary in Web Mercator (EPSG:3857) will plot together without raising an exception, because `matplotlib` does not know what a CRS is. The layers will not line up, because the underlying coordinate numbers mean different things. Before any spatial join, buffer, or distance calculation, project everything to a common CRS:

```
# Project all three layers to Web Mercator for contextily basemaps
target_crs = "EPSG:3857"
counties = counties.to_crs(target_crs)
acs = acs.to_crs(target_crs)
i70 = i70.to_crs(target_crs)

# For area or distance calculations, use a projected CRS
# appropriate to the region. For the Front Range:
#   EPSG:26913 (NAD83 / UTM zone 13N)
# Distances in meters will be correct; Web Mercator distances will not.
```

The `to_crs()` method rewrites the geometry column so every coordinate is expressed in the target system; it does not change the underlying shapes, only how they are addressed. If you skip this step, your choropleth will still render, but it will be wrong in ways you cannot see.

```
fig, ax = plt.subplots(figsize=(10, 10))

# Choropleth: percent of population below poverty line, by block group
acs.plot(
    column="pct_poverty",
    cmap="YlOrRd",
    linewidth=0.1,
    edgecolor="gray",
```

```
    legend=True,
    ax=ax,
)

# Project boundary on top
i70.boundary.plot(ax=ax, color="black", linewidth=2)

# Contextily basemap (OpenStreetMap tiles)
ctx.add_basemap(ax, source=ctx.providers.OpenStreetMap.Mapnik)

ax.set_axis_off()
ax.set_title(
    "Central 70 project area and block-group poverty rate\n"
    "ACS 5-year 2018-2022; CDOT project boundary from 2017 FEIS"
)
fig.tight_layout()
fig.savefig("ch12_i70_choropleth.png", dpi=150)
```

A choropleth with warmer colors clustering along the project corridor is not a causal claim; it is a descriptive statement about which communities surround the decision. Whether the project caused the poverty rate, was caused by historical decisions that also produced it, or is unrelated, is a separate analytical question. The map's job is to make the spatial question visible, not to close it.

The Missing Manual

`geopandas` is a mixing bowl: `shapely` handles geometry, `fiona` or `pyogrio` reads files, `pyproj` handles CRSs, and `matplotlib` draws the result. When something breaks, the traceback can come from any of four libraries you did not explicitly import. The most common failure is a CRS mismatch, because `matplotlib` will cheerfully plot layers in incompatible coordinate systems without complaining. Always

call `.to_crs()` to project every layer to a common CRS before you join, buffer, measure, or plot. The second most common failure: a shapefile arrives without its `.prj` sidecar about thirty percent of the time, leaving `geopandas` with no way to know what CRS the coordinates are in. Check for the `.prj` before you open the `.shp`; if it is missing, find out what projection the data was prepared in before you plot.

12.7. Community-informed categorization

The demographic columns you drop into the map are themselves planning artifacts. "Percent below poverty line" is a federal category with a specific history. "Hispanic or Latino" was introduced as a Census category in 1970 after organized advocacy. "Two or more races" did not exist as a Census response option before 2000. Every category is a fossil of an earlier argument about who counted and as what.

Bowker and Star (2000), whose work returns in Chapter 16, call this the "torque" of classification: category systems that once fit the world now shape what the world is allowed to look like. Community-informed categorization treats category choice as participatory. If the community you are mapping has its own neighborhood boundaries that do not match Census tract lines, include both, labeled. If a demographic question the community cares about (length of residence, primary language at home) is not in standard ACS tables, say so in the caption rather than pretending those dimensions do not exist.

This connects back to Chapter 8's treatment of ownership as stewardship. Treating the power to categorize as held in trust, rather than exercised unilaterally, is a planning ethic and an ownership ethic at once.

> **In the Public Interest**
>
> Ownership, in Chapter 8's sense, is about who has the standing and means to act as steward for data that describes them. Planning translates this into practice: the categories used to describe a place, and the boundaries used to define "the area of impact," should themselves be subject to participation. This is where the installed-base framework's **remedy** element (Chapter 5) gets geographic teeth. A community that cannot contest the map on which its displacement was planned has no path to remedy for the displacement itself. Audit-washing (Chapter 7) has a geographic cousin: participation-washing, where the map is drawn, the meeting is held, comments are noticed, and the decision proceeds as if none of it occurred. You can tell the difference by asking whether any document you produced could actually change the decision, or whether it was always going to be filed.

12.8. Exercises

Exercise 12.1 (Guided). Pick a local decision in Boulder, Denver, or your home community: a rezoning, a transit line, a school consolidation, a development approval. Build a stakeholder map in YAML using the schema above. Include at least ten entries. Commit the file to `exercises/ch12_stakeholders_<slug>.yaml`. Include a two-paragraph `README.md` that states what the decision is, what records you used to identify stakeholders, and what you could not find.

Exercise 12.2 (Analytic). For the decision you mapped in 12.1, write a 400-word "who was missing from the room" analysis. Name at least four stakeholder groups whose perspectives are absent or thinly represented in the documented process, and for each, name one document,

dataset, or outreach method that would have surfaced them. Save as `exercises/ch12_missing_<slug>.md`.

Exercise 12.3 (Technical). Produce a `geopandas` choropleth of your decision's geography. Use TIGER/Line shapefiles for the base geometry and an ACS 5-year table for the demographic overlay. Include at least three layers: the administrative base, the demographic variable, and the decision's boundary or corridor. Project all layers to a common CRS and document the CRS choice in the caption. Save the figure and the notebook to `exercises/ch12_map_<slug>/`. If a `.prj` file is missing from any shapefile, document how you determined the correct CRS.

Exercise 12.4 (Reflective). Read one chapter from Eubanks (2018) (the Allegheny chapter or the Indiana eligibility chapter works well for this purpose). Apply her analytic framework to the case you have been working with in 12.1 through 12.3. Where does the decision's theory of the problem locate the cause of the harm? Where does Eubanks locate it? What would change in your stakeholder map if you adopted her framing? 500 words.

Exercise 12.5 (Open-ended). Design a participatory process that would have produced a materially different outcome for your decision. "Materially different" means: a different alignment, a different boundary, a different set of mitigations, a different timeline, or a decision not to proceed. Specify what rung of Arnstein's ladder your process would occupy, who would hold what authority, and what the minimum infrastructure (time, money, translation, childcare, data access, legal support) the process would require. Then answer the harder question: what would have had to be true, structurally, about the lead agency, the funding source, or the political moment for this process to actually happen? 750 words. Save as `exercises/ch12_counterfactual_<slug>.md`. You will revisit this exercise in Chapter 15 when the reading pivots to procurement as planning by contract.

12.9. Looking ahead

Planning gave you the habit of asking who was in the room and whose knowledge of the terrain counts. Chapter 13 turns to engineering, which asks, after a bridge falls down, what combination of design choice, material weakness, and organizational pressure produced the failure. Engineering's lineage for public interest data science is the post-incident review, the failure taxonomy, and the discipline of treating each deployed system as a potential failure in slow motion. You will build a test suite, introduce a bug on purpose, and write the post-mortem. Chapter 15 returns to planning when it treats government procurement as planning by contract, and Chapter 18 returns to it when it asks you to write a report for a stakeholder audience that is not the one you would have picked.

12.10. Further Reading and Resources

- Paul Davidoff (1965), "Advocacy and Pluralism in Planning," *Journal of the American Institute of Planners* 31(4): 331–338. The founding essay for advocacy planning. Short, sharp, and still the clearest statement of the position.
- Sherry R. Arnstein (1969), "A Ladder of Citizen Participation," *Journal of the American Institute of Planners* 35(4): 216–224. The eight-rung ladder you will use to diagnose participation processes for the rest of your career.
- Virginia Eubanks (2018), *Automating Inequality*. The chapter on the Allegheny Family Screening Tool is directly relevant to Exercise 12.4; the Indiana eligibility and Los Angeles homelessness chapters both work as alternative readings.
- The GeoPandas user guide: https://geopandas.org/en/stable/docs.html. The CRS handling section is worth reading end-to-end before you plot anything serious.

- QGIS: https://qgis.org/. The free and open-source desktop GIS. When `geopandas` is not enough, or when you need to hand a reproducible map project to a non-Python collaborator, QGIS is the default.
- U.S. Census TIGER/Line shapefiles: https://www.census.gov/geographies/mapping-files/time-series/geo/tiger-line-file.html. Most administrative boundaries you will plot in the United States come from here.
- The Central 70 project record of decision and supporting environmental review documents, published by CDOT and FHWA. A substantial real-world planning archive, worth reading as an example of what a regulated public process produces and what it does not.
- The City of Boulder's Alpine-Balsam area plan: https://bouldercolorado.gov/projects/alpine-balsam. A smaller-scale planning process with extensive documentation of its public engagement phases. Useful as a second case for students who find the Central 70 material heavy.
- Marcus B. Lane (2005), "Public Participation in Planning: An Intellectual History," *Australian Geographer* 36(3): 283–299. The longer historical arc from which Arnstein and Davidoff are two nodes.

13. Engineering

Most bridges do not fall down. This is unremarkable until you notice how many there are, how heavy the trucks on them are, and how few civil engineers are on site at any given bridge on any given day. The low background rate of collapse is a professional accomplishment. It rests on standards, inspection schedules, conservative design margins, licensure, and the slow accumulation of what Henry Petroski (1992) calls the knowledge of failure. Engineering became a profession in the nineteenth century partly by agreeing to pay close attention to its own disasters. The Tay Bridge. The Tacoma Narrows. The DC-10 cargo door. The hospitality of an engineer's bookshelf to a failure taxonomy is one of the most important cultural facts about the field.

Data science does not yet have that bookshelf. It has conference papers on bias, a growing body of audit work (Chapter 7), and a scattered practice of post-incident reviews at the sites that run production systems at scale. What it lacks is a professional identity organized around the question "what failed, why, and what will we change so it does not fail again." This chapter, the last in Part III, proposes that engineering is the discipline most worth borrowing from on that question. From engineering, you take tests, incident reviews, and the cultural insistence that reliability is earned and paid for, not assumed.

13.1. The engineer's duty to the public

Engineering's claim to be a profession rests on codes of ethics that give the engineer a duty of loyalty to the public that overrides the duty to the client. The canonical formulation, in the National Society of Professional Engineers code, puts "hold paramount the safety, health, and welfare of the public" at the top of the fundamental canons. An engineer who signs off on a bridge they know to be unsafe is not acting unprofessionally because they disappointed a customer. They are acting unprofessionally because they violated a duty to the people who will drive over it. That duty survives the contract.

This matters for public interest data science because the closest analog, the duty of a financial auditor to the public investor (Chapter 11), is the reason accountancy works at all. Baker (2005) argues that the American accounting profession has spent decades rhetorically invoking "the public interest" while operating as a service industry for paying clients; engineering has managed the same tension slightly better, mostly because bridges are more visible than audit opinions. When you write a data pipeline that supports a public decision, you inherit a weaker version of the engineer's obligation. You cannot licensed-stamp a dataset. You can agree that the client's deadline does not overrule the public's exposure to a quietly broken number.

13.2. Normalization of deviance

The second thing you borrow is a vocabulary for how safe systems go bad. Diane Vaughan's (1996) study of the *Challenger* launch decision introduced the phrase **normalization of deviance** to describe how NASA engineers, over a series of flights, came to treat O-ring erosion as acceptable despite it never being in the original design specification. Each individual decision to fly was defensible on the data NASA had. The drift was collective

and gradual. By the night before the launch, the engineers who objected were the ones required to prove it was unsafe, rather than the other way around.

The data-pipeline analog is familiar. A nightly job fails at 2 a.m. once a week. Someone reruns it by hand. Eventually the rerun is automated as a retry loop. Eventually the retry loop masks a schema change upstream for six months. Eventually someone notices that the numbers in a public dashboard are off. Nobody did anything wrong on any individual Tuesday. The deviance normalized. Incident reviews, done well, reverse that drift: they put the question "when did we stop noticing this" on the table and keep asking it.

Karl Weick and Kathleen Sutcliffe's work on high-reliability organizations (Weick et al. 1999) generalizes the point. Organizations that run complex systems with low failure rates, like aircraft carriers and air traffic control, share cultural traits: preoccupation with failure, reluctance to simplify, sensitivity to operations, commitment to resilience, deference to expertise. No heroic engineer. No superhuman vigilance. A steady institutional refusal to relax. Charles Perrow (1984) pushed back that some systems are so tightly coupled and interactively complex that accidents in them are "normal" rather than preventable. The debate is unresolved and productive. The takeaway for the data scientist is that reliability is organizational before it is technical. Tests do not run themselves.

13.3. Software engineering as inheritor

Software engineering has sometimes claimed this tradition and sometimes been accused of squatting on the word. The Therac-25 radiation machine, the Ariane 5 rocket, the 737 MAX, the UK Post Office's Horizon accounting system: each is a software case the older engineering professions would recognize immediately as a safety-and-ethics failure. Data pipelines are a

newer cousin, usually less safety-critical individually, often more consequential in aggregate. Raji et al. (2020) make the case that internal algorithmic auditing, done with engineering-style discipline, closes the accountability gap between aspirational ethics and operational practice. Their framework reads, deliberately, like an engineering standards document.

What software and data work usually lack is the labor that sustains reliability in older engineering fields. There is no equivalent of the railway inspector walking the track. The people who maintain production data systems tend to be invisible in the way that Gray and Suri (2019) document ghost workers are invisible: present when something breaks, forgotten when it does not. Chapter 14 will return to this under the heading of sustainability. For now, notice that adding tests to a pipeline is a small, concrete way of making that invisible labor partly visible and transferable.

13.4. From the Ch 11 audit to a tested pipeline

The running example is the reproducible audit pipeline from Chapter 11. That chapter gave you a manifest, a script, a Makefile, and a changelog. Now you add what converts workpapers from a genre into a discipline: a test suite. Add a `tests/` directory at the top of the project:

```
audit-pipeline/
  manifest.yaml
  audit.py
  Makefile
  tests/
     fixtures/
        tiny_compas.csv
     test_shape.py
     test_ranges.py
     test_regression.py
```

```
.github/
    workflows/
        ci.yml
```

The `tiny_compas.csv` is a fifty-row synthetic file, committed. It looks like the real input but is not. Its job is to let every test run in under a second, on any laptop, with no network access.

13.4.1. Shape tests

The first tests you write catch the failure modes a data pipeline actually has. Not "does this function return 42." Closer to "does the table have the columns I expect, in roughly the row count I expect, with dtypes that will not silently coerce to nonsense."

```python
# tests/test_shape.py
import pandas as pd
import pytest

from audit import load_input

EXPECTED_COLUMNS = {
    "person_id": "object",
    "age": "int64",
    "race": "object",
    "sex": "object",
    "score": "float64",
    "assessment_date": "datetime64[ns]",
}

@pytest.fixture
def df():
```

```
    return load_input("tests/fixtures/tiny_compas.csv")

def test_columns_present(df):
    missing = set(EXPECTED_COLUMNS) - set(df.columns)
    assert not missing, f"missing columns: {missing}"

def test_column_dtypes(df):
    for col, expected in EXPECTED_COLUMNS.items():
        actual = str(df[col].dtype)
        assert actual == expected, f"{col}: expected {expected}, got {actual

def test_row_count_in_range(df):
    # Fixture is a small slice; production run will be larger.
    # For the fixture, we expect 40-60 rows.
    assert 40 <= len(df) <= 60
```

The row-count assertion is deliberately a range, not an equality. In production, an equality test against a moving dataset is a future flaky test waiting to be disabled.

13.4.2. Range tests

The second kind of test asserts that values fall in the ranges the domain actually permits. Percentages live in `[0, 1]` or `[0, 100]`, never both. Ages are non-negative and below some sane upper bound. Dates do not claim to be in 1899 or 2099 unless you are doing something unusual.

```
# tests/test_ranges.py
import pandas as pd
import pytest
from datetime import datetime, timedelta
```

```python
from audit import load_input

@pytest.fixture
def df():
    return load_input("tests/fixtures/tiny_compas.csv")

def test_score_is_probability(df):
    assert df["score"].between(0, 1).all()

def test_age_is_plausible(df):
    assert df["age"].between(10, 110).all()

def test_dates_within_window(df):
    earliest = pd.Timestamp("2010-01-01")
    latest = pd.Timestamp.now() + pd.Timedelta(days=1)
    assert df["assessment_date"].between(earliest, latest).all()

@pytest.mark.parametrize(
    "column,low,high",
    [
        ("score", 0.0, 1.0),
        ("age", 10, 110),
    ],
)
def test_column_bounds(df, column, low, high):
    assert df[column].between(low, high).all(), f"{column} out of [{low}, {high}]"
```

The Missing Manual

Tests for data pipelines are different from tests for web applications, and most testing tutorials teach the web-app kind. A web-app test asserts exact equality: the login endpoint returns status 200 and

> the body contains "Welcome." A data-pipeline test that asserts "there are exactly 18,472 rows" is the same test and almost always wrong, because the data moves. Your test for a historical dataset is closer to "the most recent row's timestamp is within 48 hours of the run time" or "the monthly row count is within two standard deviations of the trailing twelve-month mean." Write assertions in the shape of distributions and ranges, not exact values. And reach for `pytest.mark.parametrize` early: without it you will write fifteen near-identical "each column is in range" tests, and the fifteenth is where the bug lives. Parametrization is the feature that makes shape-and-range testing tractable. Nobody maintains the version written without it.

13.4.3. Regression tests on a fixture

The third test keeps the pipeline's outputs stable against itself. Run the whole audit on the committed fixture, compute the disaggregated metrics, and compare to a snapshot in the repository. If the snapshot changes, someone has to explain why.

```
# tests/test_regression.py
import json
from pathlib import Path
import pytest

from audit import run_audit

FIXTURE = "tests/fixtures/tiny_compas.csv"
SNAPSHOT = Path("tests/fixtures/expected_metrics.json")

def test_audit_matches_snapshot(tmp_path):
    results = run_audit_on_file(FIXTURE)
```

```
    expected = json.loads(SNAPSHOT.read_text())
    # Compare by group and metric, allow tiny float drift.
    for group, metrics in expected.items():
        for metric, value in metrics.items():
            got = results[group][metric]
            assert abs(got - value) < 1e-6, (
                f"{group}.{metric}: expected {value}, got {got}"
            )

def run_audit_on_file(path):
    # Thin wrapper around the manifest-driven pipeline; omitted for brevity.
    ...
```

The snapshot file is human-readable JSON, small enough to diff in a pull request. When the numbers change, the diff itself becomes part of the documentation. A reviewer can see at a glance whether the change is a bug fix or a regression.

13.5. Deliberately introducing a bug

You learn more from breaking the pipeline on purpose than from any amount of testing theory. Imagine the audit dataset includes a column `fine_amount_cents` and the pipeline converts it to dollars for reporting. Edit `audit.py`:

```
def to_dollars(cents):
    # Silent unit conversion error: dropped a zero.
    return cents / 10   # should be / 100
```

Commit the change on a branch with a message that honestly labels it:

```
git checkout -b demo/unit-conversion-bug
git commit -am "demo: introduce unit conversion bug (drop a zero)"
```

Run `pytest`. If the suite is doing its job, `test_audit_matches_snapshot` fails because the dollar amounts are ten times what the snapshot says. The failure message points at the specific metric and group. Now you have something concrete to investigate.

Fix the bug (`cents / 100`), rerun the tests, commit. The two-commit sequence (bug introduced, bug fixed) is the minimum evidence that the test was doing real work.

13.6. The post-incident review

The fix is not the end of the response. The review is. Engineering's specific contribution to the documentation genres of public-interest work is the post-incident review, sometimes called a post-mortem or, in aviation, an accident report. Its structure is stable across industries. A useful template has five sections.

Summary. One paragraph. What broke, when it was detected, who was affected, what was done.

Timeline. A minute-by-minute or day-by-day record, in UTC: detection, response start, each diagnostic step, mitigation, all-clear. Timestamps are non-negotiable.

Impact. Quantified where possible. How many users, records, or dollars, over what window. For a public-facing pipeline: how many published numbers were wrong, for how long, and on what downstream decisions they might have been relied on.

Contributing factors. Plural. What combination of design choice, process gap, missing test, insufficient alert, organizational pressure, and plain human error produced the failure. Blameless does not mean vague.

What we are changing. Specific, assigned, dated. New tests. Process changes. Monitoring additions. A change that is not assigned to someone is not a change.

For the unit-conversion bug, a three-page review is sufficient. The timeline shows the bug was introduced at 09:04, caught by `pytest` in a pre-merge CI run at 09:07, and never reached production. Impact is near zero. Contributing factors include the unit being encoded in the column name but not the type system, the absence of a sanity-check on dollar totals, and the pipeline's reliance on a reviewer noticing a number looked wrong. Changes include a range test on dollar totals, a refactor to a typed currency helper, and a note in `CONTRIBUTING.md` about unit discipline.

Publishing, even for a demo bug, is the habit. Cloudflare, GitLab, Google, NASA, and the NTSB all publish. Writing for an external reader forces a different standard of clarity than writing for yourself.

13.7. Continuous integration

Tests you do not run are decorations. The minimal configuration that makes `pytest` run on every push is a single YAML file.

```yaml
# .github/workflows/ci.yml
name: ci

on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
```

```yaml
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest
      - name: Run tests
        run: pytest -v
```

Push a branch. Open a pull request. GitHub Actions runs the workflow, marks the check green or red next to the commit, and blocks the merge if branch protection is on. The cost is five minutes to set up. The return is that the entire test suite runs on a clean machine, with declared dependencies, every time anyone proposes a change. A test that runs only on the author's laptop does not, for the purposes of the installed base (Chapter 5), exist.

> **In the Public Interest**
>
> Oversight, in the sense Chapter 7 developed, is not a heroic intervention. It is routine scrutiny, organized so that the routine itself makes surprises visible. Tests and CI are that routine at the pipeline layer. They are how you earn the right to make a public claim about a number. The workpapers of Chapter 11 and Raji et al.'s (2020) audit framework both presuppose that the numbers were produced by a process someone can vouch for. A tested pipeline with a CI check, a committed fixture, and a published post-incident review is what "vouching for it" looks like in code. The engineer's duty to public welfare, transposed into data work, is not larger or more dramatic than this; it is exactly this, done routinely, for every claim that leaves

the repository with your name on it. Continuity, in the installed-base sense of Chapter 5, is the practice of making sure the next person who inherits the pipeline can tell, from the tests and reviews alone, what it is for and when it is broken.

13.8. Exercises

Exercise 13.1 (Guided). Clone the audit pipeline from your work for Chapter 11. Add the three test files shown above: a shape test, a range test (with at least one `pytest.mark.parametrize` block), and a regression test against a fifty-row synthetic fixture. The suite must run in under one second. Commit to `exercises/ch13_tests/` with a `README.md` listing what each test asserts and why.

Exercise 13.2 (Guided). On a branch named `demo/bug`, introduce a deliberate silent error: a dropped-zero unit conversion, a wrong column in a groupby, a boolean reversed. Commit the introduction with a message that labels it a demo bug. Run the tests and confirm they fail. Commit the fix on a second commit. Push. Do not merge. The two-commit history is the artifact.

Exercise 13.3 (Analytic). Write a post-incident review for the bug from 13.2, using the five-section template. Three to four pages. Name the pipeline, the commit SHAs, the timestamps, the impact (even if zero), the contributing factors, and the changes. Save as `exercises/ch13_incident_<date>.md`. Write it as though a reporter might read it, because in the public-interest version of this work they might.

Exercise 13.4 (Technical). Add the `.github/workflows/ci.yml` shown above. Push a branch. Confirm the Actions tab shows the workflow running and that any test failure turns the check red. Screenshot the green check

on a passing run and the red check on the bug-introducing commit from 13.2. Save both to `exercises/ch13_ci/`.

Exercise 13.5 (Open-ended). Find a real-world data-pipeline incident review in the public record. Good sources: Cloudflare's incident archive, GitLab's post-mortems, NASA mission failure reports (Mars Climate Orbiter, Genesis), NYC 311 data-system outage reports, CERT vulnerability notes, AWS post-event summaries. In 300 words, summarize the five template sections as they appear in the source (the gaps are informative), identify one contributing factor that is organizational rather than technical, and say what this book's framework would have added. Save as `exercises/ch13_review_summary.md`.

13.9. Looking ahead

Part III set out to show that data science is not the first field to struggle with accountability to a public it cannot see directly. Journalism gave you the habit of publishing and being wrong in public. Law gave you the mechanics of compelling a record. Accounting gave you workpapers and the auditor's duty to the investor. Planning gave you the stakeholder map and the uncomfortable question of who was not in the room. Engineering closes the part by giving you the failure mode, the test, and the post-incident review: the cultural technology for treating every deployed system as something that can and will break, and for making the breakage legible when it does. The preservation end of that cultural technology, how a failure record and the data it references survive long enough to matter, is taken up again in Chapter 22.

Part IV, Adjacent Conversations, turns from lineages you inherit to fields you sit beside. Chapter 14 opens with the one most likely to be confused with this book's project: public interest technology and the "data for good" movement. Both claim the public-interest label. Both do work this book respects. Both also carry assumptions about philanthropy, the state, and

whose labor maintains the systems once the grant ends that are worth naming rather than absorbing. You will read those fields with the lineages of Part III still in hand, because sustainability is a reliability problem too. The pipeline that is not maintained is a pipeline that will eventually produce the wrong number, quietly, in a decision that matters.

13.10. Further Reading and Resources

- Henry Petroski (1992), *To Engineer Is Human: The Role of Failure in Successful Design*. The accessible entry point to engineering's culture of learning from disaster. The Tacoma Narrows chapter pairs well with this one.
- Diane Vaughan (1996), *The Challenger Launch Decision*. The canonical sociology-of-technology study of a high-consequence failure. Worth it for the "normalization of deviance" framing alone.
- Charles Perrow (1984), *Normal Accidents: Living with High-Risk Technologies*. The counterweight to high-reliability optimism. Read alongside Weick and Sutcliffe.
- Betsy Beyer et al., eds., *Site Reliability Engineering: How Google Runs Production Systems* (2016), free at https://sre.google/books/. The chapters on post-mortem culture and error budgets are directly relevant.
- *Increment* magazine's "On-Call" and "Post-mortem" issues, at https://increment.com/. Short practitioner essays on running and reviewing systems at scale.
- NTSB accident investigation reports, at https://www.ntsb.gov/investigations/. The gold standard for the post-incident review as a public document. Pick any completed report and read it end to end.
- Cloudflare's public postmortem archive, at https://blog.cloudflare.com/tag/postmortem/. A corporate example of the genre done publicly.

- GitLab's public incident postmortems, at https://about.gitlab.com/handbook/engineering/infrastructure/incident-management/. A SaaS shop that lost user data in 2017 and wrote about it in detail.
- The pytest documentation, at https://docs.pytest.org/. The `parametrize`, `fixture`, and `tmp_path` sections are the ones you will reach for.
- NSPE Code of Ethics for Engineers, at https://www.nspe.org/career-growth/ethics/code-ethics. Two pages. Read alongside Baker (2005) on accounting's parallel code.

Part IV.

Adjacent Conversations

14. Public Interest Technology and Data for Good

Part IV begins with a problem of neighbors. Public interest data science, as this book develops it, has two immediate cousins whose self-presentation is strong enough that students often arrive assuming the three terms are synonyms. They are not. Public interest technology (PIT) and data-for-good each have their own institutional history, funders, canonical texts, and characteristic blind spots. You should know what those are, because the job listings you will apply for and the foundations that will eventually pay your rent draw their vocabulary from these fields. And because, as Aula and Erkkilä (2023) argue, the philanthropic political economy that sustains PIT and data-for-good is not the same political economy that sustains journalism, law, or the archival sciences you met in Part III. Its rhythms are different. Its pathologies are different.

This chapter is sympathetic and skeptical in equal measure. PIT has built real things. GetCalFresh helped millions of Californians apply for food assistance through a form that did not hate them. Ushahidi mapped post-election violence in Kenya in 2008 and has since been deployed in dozens of countries during crises where official channels failed. The question this chapter asks is not whether these tools worked. It is whether they were built to last, who pays when the launch press cycle ends, and what labor is rendered invisible when a tool is described as "the platform." The concept that organizes the chapter is **sustainability**, understood not as a vague aspiration but as a specific configuration of code, people, money, and data that has to be maintained or it will rot. You will conduct a sustainability

audit of two real projects, one from the US PIT world and one from the global South data-for-good world, and score them against a rubric. By the end you should be able to look at a project's launch announcement and tell, within a rough range of error, how likely it is to still be there in five years.

14.1. The PIT University Network and its framing text

Public interest technology as an organized field is young. McGuinness and Schank (2021) trace its emergence to the Obama-era US Digital Service and the parallel founding of Code for America, arguing that the lesson of healthcare.gov's 2013 failure and its subsequent rescue was that government needed people who could write code, understand policy, and translate between the two. *Power to the Public* is the book that codified this framing. It is a confident book, written by practitioners, and it does real work: it names a kind of professional who does not fit the conventional civil-service or tech-industry career ladder, and it argues that such people should be trained, hired, and paid.

The Public Interest Technology University Network, coordinated by New America, took up that argument and built around it. By 2024 the network included dozens of member universities, each committing to some combination of new courses, fellowship programs, and hiring pathways (New America 2024). The network's framing is career-oriented: its promotional material emphasizes the pipeline from student to fellowship to government-or-nonprofit job. That emphasis is one of the field's genuine strengths. A generation of students now has a defensible answer to the question "what do you do with a computer science degree if you do not want to work for a surveillance advertising company."

What the PIT-UN framing tends to elide is what happens after the student becomes the fellow, the fellow becomes the hire, and the hire ships the thing.

The career pipeline metaphor implies a forward-flowing river; actual projects are more like gardens. Karasti and Blomberg (2018) call this maintenance work **infrastructuring** and argue that it is the least-narrated, least-funded phase of any sociotechnical system. It is the move Keegan (2026) makes in framing the installed base as something maintained rather than built. You met the concept in Chapter 5; you will meet it in Chapter 13 under the heading of operations. Here it becomes a diagnostic lens for any PIT project.

14.2. The rhythm of foundation funding

The pathology is structural. Most PIT and data-for-good projects are funded by philanthropic foundations on three-year grant cycles. The first year is design and pilot. The second year is refinement. The third year is "scaling." The theory implicit in that rhythm is that a pilot, if it works, will either attract a sustaining revenue stream (government contract, earned revenue, follow-on philanthropy) or be absorbed by an institution that already has one. This theory is sometimes correct. It is much more often wrong. Aula and Erkkilä (2023) describe the resulting dynamic as a churn in which foundations perpetually fund new pilots because the old pilots did not scale, with the result that a disproportionate share of field capacity is tied up in pilot projects that will be dead in five years. Chambers (2025) sharpens this critique: the "data-for-good" imaginary treats data-intensive civic projects as discrete interventions that can be completed, rather than as ongoing commitments that require ongoing labor.

You can see the pattern in the shape of project websites. A project in year one has blog posts every two weeks and a Medium series and a podcast. A project in year three has a landing page with three case studies dated from year two and a Slack invitation link that no longer works. A project in year five either has a funding home (a government line item, a membership model, fee-for-service revenue) or a GitHub repo whose last meaningful commit was the grant-close deliverable. The audit below is designed to

surface which kind of project you are looking at before you commit time to building on top of it.

14.3. Two running examples: GetCalFresh and Ushahidi

Consider two projects that have lasted a long time.

GetCalFresh was launched by Code for America in 2014 to simplify the CalFresh (California's SNAP) application process (Code for America 2024). The original application was an 18-page PDF. The GetCalFresh version took most applicants under ten minutes, and millions of Californians used it over the next decade. It became the template for a broader argument: that benefits-access technology can measurably move the needle on enrollment, and that civic tech should focus on this kind of service delivery rather than flashier interventions. In 2023 the service transitioned from a Code-for-America-operated platform to one operated by the state of California. That transition is, in sustainability terms, the best possible outcome. A civic-tech pilot became a line item in a state agency's budget. The thing you should want for any project you build is that you eventually work yourself out of a job because the state has taken over the function.

Ushahidi was launched in Nairobi in 2008 to crowdsource reports of post-election violence in Kenya, after major media outlets stopped publishing details (Ushahidi 2024). The platform was built, initially, over a weekend. It has since been deployed in over 150 countries, during earthquakes, refugee crises, elections, and epidemics. Ushahidi is an unusual case because it emerged from a global-South context and has remained headquartered there, funded through a mix of foundation grants, deployment fees, and in-kind partnerships. Its maintenance problem is also unusually visible: the codebase is open source, deployments are often led by local partners whose technical capacity varies, and the organization has been candid over

the years about the difficulty of keeping a 2008-vintage platform current against 2024-era security and mobile-device realities.

Pair these two. One US project that transitioned to a government funding home. One global-South project surviving 16 years on a hybrid funding model. Neither is a failure. Both illustrate that sustainability is possible, looks different in different institutional contexts, and is the exception rather than the rule.

14.4. A sustainability audit: querying GitHub

The technical component of this chapter is a reproducible audit you can run on any project with a public code repository. The audit asks six questions:

1. Where does the code live? Repo URL, license, last commit date, contributor count.
2. Who maintains it? Named maintainers, their employer, and how they are funded.
3. Who owns the data?
4. What happens when the current grant ends?
5. What labor is invisible in the product: translation, annotation, moderation, cleaning?
6. How does the project score against a simple rubric?

The first question is the one you can mostly answer with the GitHub REST API. The later ones require reading. The code below fetches repo metadata, the license, contributor count, open issues, and open pull requests, and returns a one-page DataFrame that you can render into a report.

```
import os
import requests
import pandas as pd
```

```
from datetime import datetime, timezone

GITHUB_TOKEN = os.environ.get("GITHUB_TOKEN")  # strongly recommended
HEADERS = {
    "Accept": "application/vnd.github+json",
    "User-Agent": "PublicInterestDataScience/0.1 (your-email@example.edu)",
    "X-GitHub-Api-Version": "2022-11-28",
}
if GITHUB_TOKEN:
    HEADERS["Authorization"] = f"Bearer {GITHUB_TOKEN}"


def gh(path, params=None):
    url = f"https://api.github.com{path}"
    response = requests.get(url, headers=HEADERS, params=params, timeout=30)
    response.raise_for_status()
    return response


def audit_repo(owner, repo):
    base = f"/repos/{owner}/{repo}"
    meta = gh(base).json()

    # Contributors: paginate to count, not to list.
    contributors = gh(f"{base}/contributors",
                      params={"per_page": 100, "anon": "true"}).json()
    contributor_count = len(contributors)

    # Open issues and PRs. GitHub counts PRs as issues; split them.
    issues = gh("/search/issues",
                params={"q": f"repo:{owner}/{repo} is:open is:issue"}).json()
    prs = gh("/search/issues",
             params={"q": f"repo:{owner}/{repo} is:open is:pr"}).json()
```

```
    # Last commit date on the default branch.
    commits = gh(f"{base}/commits", params={"per_page": 1}).json()
    last_commit = commits[0]["commit"]["committer"]["date"] if commits else None
    if last_commit:
        last_commit_dt = datetime.fromisoformat(last_commit.replace("Z", "+00:00"))
        days_since = (datetime.now(timezone.utc) - last_commit_dt).days
    else:
        days_since = None

    license_name = (meta.get("license") or {}).get("spdx_id") or "NONE"

    row = {
        "repo": f"{owner}/{repo}",
        "stars": meta.get("stargazers_count"),
        "forks": meta.get("forks_count"),
        "license": license_name,
        "last_commit": last_commit,
        "days_since_last_commit": days_since,
        "contributors": contributor_count,
        "open_issues": issues.get("total_count"),
        "open_prs": prs.get("total_count"),
        "default_branch": meta.get("default_branch"),
        "archived": meta.get("archived"),
    }
    return row


projects = [
    ("codeforamerica", "getcalfresh-site"),   # illustrative; confirm exact repo
    ("ushahidi", "platform"),
]

audit = pd.DataFrame([audit_repo(o, r) for o, r in projects])
```

```
print(audit.T)
# => one column per project, rows: stars, forks, license, last_commit,
#    days_since_last_commit, contributors, open_issues, open_prs, archived
```

Two observations. The `search/issues` endpoint is rate-limited more strictly than most others (30 requests per minute for authenticated users); for a classroom audit across five projects this is fine, for a broader study you will want to page through the issues endpoint directly. And `anon: "true"` on the contributors endpoint tells GitHub to include commits from email addresses that do not resolve to current GitHub accounts, which matters because in many PIT projects a meaningful share of historical contribution is from now-departed staff whose accounts have lapsed.

The Missing Manual

Two traps. First, a healthy-looking GitHub repo with recent commits is not evidence of sustainability. It might be one person making cosmetic updates before a grant report. Always check the *breadth* of commits: run `git shortlog -sne --since="1 year ago"` against a local clone and count how many distinct humans made substantive changes. Fifty commits in the last year from one committer is a solo project that has not yet been told. Twelve commits from seven committers is, in this field, genuinely healthy. Second, the GitHub API rate-limits unauthenticated requests to 60 per hour. If you run the audit without `GITHUB_TOKEN` set, you will get throttled before the third project and the errors will not be obvious. Create a personal access token with `public_repo` scope, put it in an environment variable, and confirm your headroom with a quick call to `/rate_limit` before you start.

14.5. Interpreting the audit

The numbers are a first cut. They do not tell you who pays the maintainers. For that you have to read the project's About page, their annual reports, and their IRS Form 990 if they are a US nonprofit. You might find that the repo with fifteen active contributors is supported by a $4M/year philanthropic grant whose cycle ends in eighteen months, while the repo with three contributors sits on a state agency's operations budget and will quietly keep running for a decade. The first looks healthier on GitHub. The second is more sustainable.

A simple rubric, scored 0 to 3 on each dimension for a 0 to 15 total, does useful disciplinary work:

- **Code health.** License present, recent commits, open issues being triaged, at least one release tag in the past year.
- **Maintenance breadth.** More than one active human contributor; more than one employer represented among contributors.
- **Labor visibility.** The project publicly names the people who do the unglamorous work: translators, annotators, moderators, support staff.
- **Data ownership.** The project has a written data policy, a stated position on whose data it holds, and a plausible answer to "what happens to the data if the project shuts down."
- **Funding model.** The project has a named current funder, a stated runway, and a plausible non-grant funding path (government contract, fee-for-service, membership, endowment).

A score of 12 or higher suggests the project will probably still exist in five years. A score below 6 suggests you should not build a dependency on it. Scores in between are where most real projects live, and the rubric's value is less the number and more the specific gaps it surfaces.

14.6. Ghost work and the invisible labor

No audit of a data-for-good project is complete without asking about labor the product does not advertise. Gray and Suri (2019) introduced the term **ghost work** for the tasks humans perform to make "artificial intelligence" appear to work: content moderation, image labeling, spam filtering, transcript cleanup, edge-case triage. Ghost work is not an implementation detail. It is the labor that sustains the system. It is also, in the PIT and data-for-good worlds, disproportionately performed by people who are paid little, credited never, and employed through layers of subcontracting that make their labor legally separate from the mission-driven organization whose logo is on the product.

Benefits-access projects depend on translation. Someone translated the GetCalFresh intake flow into Spanish, Mandarin, and Vietnamese. That someone's name is not on the About page. Crisis-mapping projects depend on verification: every Ushahidi deployment involves volunteers or paid staff who triage incoming SMS reports, assess plausibility, and decide what to publish. Those verifiers are the platform, in a way that is not true of the software. When Chambers (2025) critiques the "data-for-good imaginary," part of the critique is that it treats the technology as the thing and the labor as a regrettable overhead. Your audit should flip that framing. When the rubric asks about "labor visibility," it is asking whether the people whose work keeps the system alive appear anywhere a reader can find them.

> **In the Public Interest**
>
> In Chapter 8 you learned to distinguish data ownership as control from data ownership as stewardship. Stewardship is a verb. It implies ongoing labor. The sustainability audit here is one way of asking who is doing that labor for a project you did not build. When a project scores well on "data ownership" in the rubric, that does not mean

someone *has* the data in a legal-custody sense. It means someone is paid to care for it: to migrate it across storage platforms, to answer deletion requests, to maintain access controls, to renew the domain. Openness without stewardship is neglect with extra steps. Oversight without stewardship is performance art. The single most useful question you can ask a PIT or data-for-good project is not "is your code open source?" It is "who pays the person who will answer the phone in 2029?"

14.7. Exercises

Exercise 14.1 (Guided). Conduct a sustainability audit of one US PIT project. Recommended choices: GetCalFresh, Colorado's PEAK benefits portal, or the Benefits Data Trust enrollment tools. Apply the GitHub API audit code above if the project has a public repo; if not, note the absence and document what you can find from the project's website, annual report, and recent news coverage. Produce a filled-in rubric with a paragraph justifying each score. 500 words plus the audit table.

Exercise 14.2 (Guided). Do the same for a non-US project. Recommended choices: Ushahidi, Data4Change, TransparenciaMX, or Code for Pakistan's Civic Innovation Lab. Use the same rubric, and explicitly note where the rubric fits poorly. Global-South projects often operate under funding, governance, or regulatory conditions the US-centric rubric does not capture. Say so rather than pretending the score is comparable.

Exercise 14.3 (Analytic). Pick one project from the two exercises above and identify three forms of invisible labor it relies on. For each, name the task, describe who is likely to be doing it, estimate how many hours per year the task consumes across the organization, and propose one specific way the project could make that labor visible in its public materials (annual

report, website, credits page). Use Gray and Suri (2019) as a reference. 400 words.

Exercise 14.4 (Design). Propose a sustainability intervention for one of the projects you audited. Your proposal must include: one named role (title and fraction of FTE), a dollar figure for twelve months of salary and benefits, a specific funding source that could support it, and a paragraph on what would have to be true for the funding to actually bite. If you cannot estimate the dollar figure within a factor of two, your proposal is not done. Use public salary data (Glassdoor, Form 990 Schedule J, PayScale) to anchor it. 500 words plus a one-page budget.

Exercise 14.5 (Open-ended). Return to the final-project sketch you drafted in Exercise 1.5. What would you need to keep it alive for five years after you leave the class? Write a short sustainability plan addressing: who hosts it, who owns the domain, who has the credentials, who answers when something breaks, who pays for storage, who decides when to retire it. Be honest. "I will maintain it in my free time" is an answer, but it is a weak one, and you should say why it is weak. 400 to 600 words. You will revisit this plan in Chapter 22 when you make an archival deposit of the project.

14.8. Looking ahead

Chapter 15 turns from PIT and data-for-good to **GovTech and civic tech**, a neighboring but distinct conversation about building technology inside and around government itself. Where PIT is an academic-philanthropic formation and data-for-good is a corporate-philanthropic one, GovTech is an industry and civic tech is a movement. The sustainability problems you diagnosed here recur there in different forms: procurement cycles replace grant cycles, contractor lock-in replaces foundation churn. But the underlying question, who pays the maintenance cost, is the same.

Chapter 16 will then ask whether the right response to a project you cannot sustain is sometimes to refuse to build it at all.

14.9. Further Reading and Resources

- Tara Dawson McGuinness and Hana Schank (2021), *Power to the Public* (McGuinness and Schank 2021). The framing text for the PIT field. Read it as a manifesto, not a diagnosis.
- Mary L. Gray and Siddharth Suri (2019), *Ghost Work* (Gray and Suri 2019). The canonical account of the invisible labor behind AI systems. The argument generalizes beyond the Mechanical Turk cases they study.
- Ville Aula and Tero Erkkilä (2023), "Digital governance and the philanthropic political economy of the data-for-good" (Aula and Erkkilä 2023). The clearest academic statement of the critique this chapter draws on.
- Public Interest Technology University Network: https://www.newamerica.org/pit/university-network/. The hub page for the US academic consortium, including member institutions, fellowship listings, and the field's self-description (New America 2024).
- Code for America: https://codeforamerica.org/. The longest-running US civic tech organization. Their annual report is worth reading for its candor about what worked and what did not.
- Ushahidi: https://www.ushahidi.com/ (Ushahidi 2024). The long-running global-South counter-example. Their GitHub organization at https://github.com/ushahidi is a good place to run the audit code in this chapter.
- Carter Chambers (2025), "The trouble with the 'data-for-good' imaginary" (Chambers 2025). The sharpest recent critique of the data-for-good framing.
- GitHub REST API documentation: https://docs.github.com/en/rest (GitHub 2024). The reference for the audit code.

- Helena Karasti and Jeanette Blomberg (2018), "Studying infrastructuring ethnographically" (Karasti and Blomberg 2018). The theoretical backbone for thinking about maintenance as a first-class concern.
- Code for America's 2023 transition memo on GetCalFresh becoming a state-operated service is a useful case study in succession planning; search the organization's blog for "CalFresh transition."

15. GovTech and Civic Tech

On October 1, 2013, HealthCare.gov launched and immediately collapsed. The federal insurance marketplace had been built by a constellation of prime contractors through a procurement process whose incentives rewarded compliance with the Statement of Work rather than delivery of a working system. Six weeks of public humiliation, a rescue squad flown in from the private sector, and a Presidential apology later, two institutions were born: the United States Digital Service, housed in the Executive Office of the President, and 18F, housed in the General Services Administration. Their pitch was simple. Government cannot buy its way out of software problems if the people writing the contracts do not know what software is. Both offices were founded on the conviction that technical competence has to live inside the state, not at the end of a procurement chain.

That conviction has had a rough decade. 18F shipped a catalog of reusable components, a hiring pipeline for civic-minded technologists, and a reputation for finishing what it started. It also spent its entire institutional life as a target. Inspector General reports questioned its cost-recovery model. Successive administrations slashed its budget, restored it, and slashed it again. By 2025 the office was operating under an anti-DEI executive order that made it hard to retain the staff it had spent a decade recruiting, and its institutional future was once again being debated on a rolling basis. The United States Digital Service, rebranded in 2025 by executive order and partially repurposed, was likewise caught in the cycle. The lesson these trajectories offer is not that internal capacity is a bad idea. It is that internal capacity has no natural constituency, whereas vendor capacity has a lobbying association. Chapter 7's argument about oversight, and

Chapter 8's argument about ownership, both run through the question of where the technical knowledge sits. When it sits with a vendor, the vendor owns what the public can inspect.

This chapter is the second in Part IV. It extends Chapter 14 by asking a sharper question. If public interest technology is the professional identity, what are the organizational forms through which it actually reaches the public? Two candidate answers dominate the field. **GovTech** refers to firms that sell software and services to the government. **Civic tech** refers to technologists, often volunteer or philanthropically funded, who build alongside or outside government. The two are sometimes described as complements. They have very different accountability levers. GovTech is accountable, in principle, through procurement law, contract terms, and the audit rights retained by the buyer. Civic tech is accountable, in principle, through the communities it serves and the funders it relies on. In practice, as Bharosa (2022) argues in his research agenda on the rise of GovTech, neither accountability story holds up without intentional construction. Procurement is where governance happens or does not. And civic tech is sustained only as long as someone writes the check.

15.1. Five models on one map

It helps to fix five reference points before going further.

The **United States Digital Service** was founded in 2014, modeled on the UK's Government Digital Service, and designed to deploy small teams of technologists into agencies in crisis. It is internal capacity, funded through the executive branch, with limited authority to compel. The **18F** office, founded the same year inside the General Services Administration, was designed as a fee-for-service consultancy to other agencies. Both are US examples of what Brown et al. (2017) call the "Government as a Platform" (GaaP) ambition, where the state itself provides shared services that reduce duplication across agencies.

The **UK Government Digital Service** (GDS), founded in 2011, is the elder sibling the US offices tried to copy. Its Service Manual and Service Standard became export goods, adopted in variant form by Canada, Australia, Singapore, and others. Estonia's **X-Road**, by contrast, is not primarily about service design. It is an interoperability layer. X-Road is a secure data exchange protocol that lets one Estonian agency query another's database without either agency building a custom integration, and it sits underneath e-residency, digital signatures, and most of what people mean when they praise "Estonian e-government." Governance in the Estonian model happens through the interoperability spec. If your system does not speak X-Road, it does not get to play.

The fifth reference point is **IndiaStack**, the layered set of APIs built around the Aadhaar biometric identity system. IndiaStack is what a GaaP platform looks like at population scale, and it is what Taylor (2021) calls the hard case. Presence-less, paperless, cashless service delivery is a real improvement when it works, and a serious violation when it does not. Biometric enrollment of over a billion people was not, in any meaningful sense, consented to, because the alternative to enrollment was exclusion from subsidized food rations, banking, and in some cases public-sector employment. Rao and Nair (2019) tracks how Aadhaar's optionality eroded in practice even where it was legally preserved. The IndiaStack case does not discredit the GaaP ambition. It names the governance stakes of building at that scale without the installed base of Chapter 5 actually in place.

These five cases do not share a politics. They share a structural question: where does technical capacity sit, and what does it have to answer to?

15.2. Procurement is where the governance is

Most of a government's relationship with technology is mediated by contracts. A Request for Proposal (RFP) is not, at first glance, a governance document. It is a solicitation. Vendors read it and decide whether to bid.

But the RFP is where the buyer declares, in binding language, what the system must do, what data it must produce, what interfaces it must expose, and what happens when the contract ends. If the RFP does not require the vendor to export data in an open format on termination, the vendor owns the database. If the RFP does not require logging, there will be no logs to audit. If the RFP does not reserve the buyer's right to publish, the public cannot see what it paid for.

Brown et al. (2017) make this point at the level of platform theory: Government as a Platform succeeds or fails at the interface specifications, not at the marketing. The interface specifications live in the contract. You can read a thousand op-eds about digital government without encountering a single clause from an actual contract, which is why the op-eds tend to generalize in ways the contracts do not.

Consider a running example. The State of Colorado, like most US states, maintains a procurement portal where active solicitations are posted. Boulder's municipal purchasing division does the same at the city level. A plausible RFP type for a public-interest audit is a benefits eligibility modernization, a permit portal, or a data warehouse consolidation. Pick one. For the rest of this chapter, assume you are looking at a Colorado RFP for a permit portal that would replace the existing paper-and-PDF process in a mid-sized county. The vendor will build a web application, a backend database, an API for third-party integrations, and a payment integration. The county will pay in milestone installments over three years and then enter a five-year operations-and-maintenance (O&M) period.

Your first move, reading this RFP, is to map what artifacts the system would produce. A permit application is a record. A decision is a record. A payment is a record. A site inspection generates a record. An appeal generates a record. Every one of these is either stored in the vendor's database, forwarded to the county's records system, or both. The question the RFP has to answer, and often fails to answer, is which of these records the county owns, in what format they can be exported, and what happens to them when the contract terminates.

> **In the Public Interest**
>
> The contract is the governance. This claim is not rhetorical. If the RFP does not require the vendor to export data in a non-proprietary format at no additional cost, on demand, with documented schema, then the vendor owns the database in operational terms even if the county owns it in legal terms. You met this distinction in Chapter 8: legal title is not the same as practical control. Oversight, as Chapter 7 developed it, requires the ability to see. A clause that requires nightly exports to an S3 bucket the county controls, in Parquet or CSV plus a JSON schema file, is worth more than fifty pages of strategic planning documents. The procurement clause is where linkability, interpretability, continuity, and safe scrutiny are either bought or given up. If you take one practice from this chapter, let it be this: when a colleague tells you a public system is "well governed," ask to see the contract.

15.3. Five clauses worth writing

Here are five procurement clauses, expressed as YAML for readability, that any public-interest-oriented buyer should consider boilerplate. They are not legal drafting. They are the specification from which a legal drafter works.

```yaml
data_export:
  trigger: [on_demand, quarterly, on_termination]
  format: [csv, parquet]
  schema: json_schema_v7
  destination: buyer_controlled_storage
  delivery_sla_hours: 72
  cost: included_in_base_contract
```

```yaml
interoperability:
  apis:
    style: REST
    auth: oauth2
    spec: openapi_3_1
    rate_limits_documented: true
  identifiers:
    persistent: true
    resolvable_across_agencies: true
    scheme: [ark, doi, uuid_v4]

auditability:
  immutable_log:
    events: [read, write, delete, access_grant, access_revoke]
    retention_years: 7
    export_format: ndjson
  third_party_inspection:
    allowed: true
    notice_days: 10
    scope: [code, logs, schemas]

source_code:
  ownership: buyer
  delivery: git_repository_with_full_history
  license_on_termination: public_domain_or_cc0

vendor_transition:
  termination_assistance_months: 6
  knowledge_transfer: documented_runbooks
  successor_onboarding: required
  data_migration_tested_quarterly: true
```

Read those back. Each clause is a specific technical commitment with

a specific failure mode if you omit it. Omit `data_export` and the vendor can charge a five-figure extraction fee on termination. Omit `interoperability.identifiers.persistent` and two years later no one will be able to link a 2026 permit record to a 2028 inspection record because the IDs reset when the database was migrated. Omit `auditability.immutable_log` and you will not be able to tell, after the fact, whether a decision was overridden by a human or by a scheduled job. Omit `source_code.ownership` and you have a vendor lock-in problem that will outlast the people who wrote the contract. Omit `vendor_transition` and the moment you want to switch vendors you will discover that switching is impossible, which is how vendor lock-in becomes a permanent governance condition rather than a temporary inconvenience.

The Missing Manual

Government APIs have documentation that describes what the endpoint returns. The documentation almost never describes what it does not return. The Census Bureau's American Community Survey API, for example, returns an array-of-arrays where the first row is the column headers. If you feed the raw response to `pd.DataFrame` the headers will land as a data row and your first real record will be read as column names. The fix is trivial once you know: `pd.DataFrame(data[1:], columns=data[0])`. The fix is a small tax. The tax you did not anticipate is that the list of variables an agency publishes in its metadata is not the list of variables it considers reliable, and the list of geographies the endpoint accepts is not the list of geographies for which estimates exist. Document these gaps for yourself. Then document them for the next person.

15.4. A small Census API exercise

You can run an actual government API call in a few lines. The Census Bureau's ACS 5-year endpoint is documented at `census.gov/data/developers` and returns estimates for any level of geography down to block group. Use it to pull median household income for all Colorado counties for the most recent vintage.

```
import requests
import pandas as pd

# ACS 5-year, 2022 vintage. Variable B19013_001E is median household income.
url = "https://api.census.gov/data/2022/acs/acs5"
params = {
    "get": "NAME,B19013_001E,B19013_001M",
    "for": "county:*",
    "in": "state:08",  # Colorado FIPS
}

response = requests.get(url, params=params, timeout=30)
response.raise_for_status()
payload = response.json()

# The Missing Manual move: first row is headers, the rest is data.
df = pd.DataFrame(payload[1:], columns=payload[0])
df = df.rename(columns={
    "B19013_001E": "median_income",
    "B19013_001M": "moe",
})
df["median_income"] = pd.to_numeric(df["median_income"], errors="coerce")
df["moe"] = pd.to_numeric(df["moe"], errors="coerce")
df = df.sort_values("median_income", ascending=False)
print(df[["NAME", "median_income", "moe"]].head())
```

```
# => Douglas County, Broomfield County, Elbert County at the top;
#    margin of error column tells you how much to trust each point estimate.
```

Plotting is one `matplotlib` call away.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 10))
top = df.head(20)
ax.barh(top["NAME"], top["median_income"], xerr=top["moe"])
ax.invert_yaxis()
ax.set_xlabel("Median household income (USD)")
ax.set_title("Colorado counties: ACS 5-year median income, 2022 vintage")
fig.tight_layout()
fig.savefig("ch15_acs_median_income.png", dpi=120)
```

Two things are worth noticing. First, the margin of error bars are not decorative. ACS is a sample survey; small counties have margins of error so wide that the point estimate is almost meaningless on its own. Any dashboard that publishes the point estimate without the MOE is misreading the data. Second, the API required no authentication for a small request like this one, but the production pattern is to register for an API key and pass it as `&key=...` so the Bureau can rate-limit you without guessing. The same pattern applies to the Federal Reserve's FRED API (`fred.stlouisfed.org/docs/api`), which requires a key, returns JSON with a clearer schema, and is the right endpoint if your question is about macroeconomic rather than demographic series.

15.5. Why the distinction between GovTech and civic tech matters

A GovTech vendor answers to its client. The client is the government. If the government writes good contracts, GovTech becomes a reasonable way to deliver public services. If the government writes bad contracts, GovTech becomes a mechanism for extracting rents from public budgets while producing systems the public cannot inspect. The accountability lever is procurement, amplified by the audit and inspection rights written into the contract. Bharosa (2022) raises the "Trojan horse" worry specifically: if a state's internal capacity has atrophied, it no longer has the expertise to write contracts that hold vendors accountable, and GovTech becomes structurally extractive even where individual vendors are well-intentioned.

A civic tech project answers to its funders and, in principle, to the communities it serves. If the funder is a foundation, the accountability lever is the foundation's theory of change and renewal criteria. If the users are community members, the accountability lever is their continued use, which is easier to observe than to measure. Aragon et al. (2020) make the case, in *Human-Centered Data Science*, for building with communities rather than for them. That is a methodological commitment, but it does not solve the funding problem. As Chapter 14 developed, civic tech projects are structurally prone to die between grant cycles, and the distinction you are drawing now matters because the failure modes are different. A failed GovTech contract produces a locked-in system that no one can afford to replace. A failed civic tech project produces a dead GitHub repo and a handful of volunteers who do not get paid.

Neither failure mode is inevitable. Both become more likely when the people commissioning the work do not know what they are looking at. Reading a procurement document with the clauses above in mind, or reading a civic tech project's governance page with Chapter 14's sustainability rubric in mind, is part of the craft. The craft is not glamorous. Chapter 12 argued that planning's lineage contributes rigor about who bears costs and who

bears benefits. GovTech and civic tech inherit that question. The contract, or the grant agreement, is where the answer gets written down.

15.6. Exercises

Exercise 15.1 (Guided). Go to the Colorado state procurement portal or Boulder's municipal purchasing site and find an active or recently-closed RFP for a software system. Read the Statement of Work in full. Produce a one-page map of the artifacts the winning system would generate: records (what kinds, how many per year), documents (what formats), and processes (which ones would leave a trace in the system). For each artifact, write one sentence saying whether the RFP requires that artifact to be exportable by the buyer, and in what format.

Exercise 15.2 (Applied). Take the RFP from Exercise 15.1. Draft three to five additional procurement clauses, in YAML or markdown, that would close the gaps you found. Each clause should name a specific data type, format, retention period, and access condition. Share your draft with a classmate and defend each clause against a skeptical vendor's likely objection.

Exercise 15.3 (Technical). Pull a dataset from either the Census ACS API or the FRED API. Choose a question you actually want the answer to. Document the call, the response structure, and at least one gotcha you encountered that was not in the published documentation. Produce one chart. Write 200 words on what the chart shows and what it cannot show.

Exercise 15.4 (Comparative). Write a 600-word comparison of two "government as a platform" initiatives drawn from this list: USDS, 18F, GDS UK, Estonia X-Road, IndiaStack. Your comparison should address funding model, accountability structure, and scope of authority. Cite at least one scholarly source per case, including Bharosa (2022) or Brown et

al. (2017). Close with a paragraph on what each case teaches about where internal state capacity should sit.

Exercise 15.5 (Open). What data would you require a vendor to export before you would declare a public-sector contract complete? Write a one-page specification with a title, a scope section, five to ten required exports, a default format for each, and a definition of "complete" that a county attorney could attach to a contract. Treat this as a reusable template. You will use it again in Chapter 18 when you write the report section on public-sector procurement, and again in Chapter 21 if you ever comment on a federal IT rulemaking.

15.7. Looking ahead

Chapter 16 closes Part IV by turning the critical lens back on the book's own framing. If GovTech and civic tech are the organizational forms through which public interest technology reaches the public, critical data studies is the tradition that asks whether "reaching the public" is always the right ambition. Some of the traditions Chapter 16 takes up, including Indigenous data sovereignty, abolitionist critique, and feminist refusal, would read the procurement clauses in this chapter and ask a different question: what data should not be collected at all, and whose consent to the platform was never asked? The contract is where governance happens. It is also where extraction happens. The next chapter holds both truths at once.

15.8. Further Reading and Resources

- United States Digital Service, https://www.usds.gov/. The founding US internal-capacity office, with a Digital Services Playbook worth reading even if you never intend to work there.

- 18F, https://18f.gsa.gov/. The GSA-housed sibling, with open-source components and case studies that double as procurement worked examples.
- Estonia's X-Road, https://x-road.global/. The canonical interoperability protocol; the technical documentation is unusually honest about tradeoffs.
- Code for America, https://codeforamerica.org/. The US civic-tech anchor institution, with GetCalFresh and the Clear My Record initiative as case material.
- GOV.UK Service Manual, https://www.gov.uk/service-manual. The UK's public playbook for designing government services; still the single best operational reference in the field.
- US Census Bureau API User Guide, https://www.census.gov/data/developers/guidance/api-user-guide.html. The endpoint used in this chapter; read the "Known Issues" section before you write any code against it.
- FRED API documentation, https://fred.stlouisfed.org/docs/api/fred/. The Federal Reserve Bank of St. Louis's macroeconomic series, with a cleaner schema than Census and a more opinionated rate limiter.
- Nesta's GovTech landscape reports, https://www.nesta.org.uk/. UK-grounded but globally relevant mapping of the GovTech vendor ecosystem.
- Linnet Taylor, "The price of certainty" (2021). The IndiaStack critique in its most citable form; read alongside any breathless GaaP advocacy.
- Nitesh Bharosa, "The rise of GovTech: Trojan horse or blessing in disguise?" (2022). A research agenda that names the hard questions explicitly and refuses to resolve them prematurely.

16. Critical Data Studies, Data Justice, and Refusal

This chapter is the hinge of the book. For fifteen chapters you have been building an installed base: records that link, categories that can be interpreted, evidence that persists, researchers who can scrutinize, authorities who can act, and people who can seek remedy (Keegan 2026). The argument of Chapters 2 through 15 is that a working public-interest data practice needs all six. The argument here is that, for some of the people this installed base is supposed to serve, any one of those six can also be the mechanism by which harm arrives. A record that links is a record that surveils. A category that can be interpreted is one that was once imposed. Evidence that persists can be subpoenaed a decade later by an agency that did not exist when the data was collected. Scrutiny can be predatory. Authority can be carceral. Remedy can be a pretext for further data collection.

Critical data studies holds these possibilities in view. Data justice asks who decides, under what conditions, on whose terms. Refusal is what you do when the answer to those questions is that the data should not exist in the form proposed. This chapter introduces all three and asks you to do two things with the same municipal dataset: audit its classification schema, and write a refusal specification for it. The pair is the point. Neither alone does the work.

Part IV has been about neighbors: public interest technology, data-for-good, GovTech, civic tech. This final chapter sits next to a neighbor who is less comfortable. Critical data studies does not build platforms and is not

in the career pipeline Chapter 14 describes. But the practice you will learn in Part V is stronger when it can absorb the critique honestly, including the parts that undercut this book's own starting framework.

16.1. The starting canon: boyd and Crawford

Begin with boyd and Crawford (2012), "Critical questions for big data." Still the most cited essay in the field because it named six provocations every chapter of this book has implicitly answered. Big data changes the definition of knowledge. Claims to objectivity are misleading. Bigger is not always better. Out of context, big data lose meaning. Accessibility is not ethicality. Limited access creates new digital divides.

The provocation that matters most here is the fifth. Accessibility is not consent. The fact that a record exists in a public dataset and is legally available for analysis does not settle whether you should use it. Homeless Management Information System (HMIS) data, the running example for this chapter, is a tight case. HMIS exists because the US Department of Housing and Urban Development requires it as a condition of Continuum of Care funding (US Department of Housing and Urban Development 2024). The records are real, detailed, and, under certain conditions, available to researchers. The people documented in them did not consent in any sense a research ethics board would recognize. They sought shelter, and documentation was the price.

16.2. The political work of classification

Bowker and Star (2000), *Sorting Things Out*, is the second book you need. Classification is never neutral. Every category system is a historical artifact produced by some authority, for some purpose, at some moment. The ICD encoded homosexuality as mental illness until 1973. The US Census's race

categories have been rewritten roughly every decade since 1790, each time to serve a different administrative project. Classification does not describe the world. It imposes a shape on it, and the shape tends to serve whoever did the imposing.

The consequence for data science is specific. When you call `value_counts()` on a column labeled `race` or `gender` or `veteran_status` and see five or six buckets, you are not seeing the distribution of human variation. You are seeing the surviving output of a classification pipeline that, at some earlier stage, had more, fewer, or different categories with boundaries that did not match the ones you now see. The flattening happened upstream, usually at intake, often because a reporting template required a fixed list. Your descriptive statistics inherit the politics of that template whether or not you read it.

Denton et al. (2021) make this concrete for machine learning. Their genealogy of ImageNet traces how a dataset assembled in 2006 to 2009, using WordNet's 1980s synset hierarchy, carried into the 2010s a taxonomy of human kinds that encoded racial pseudoscience, categories for "failure" and "loser," ethnic slurs, and an implicit theory that the visual world is a tree of discrete kinds. When ImageNet became the benchmark for a generation of computer vision models, those decisions propagated into production systems that never revisited the WordNet assumptions. The dataset was a historical artifact. The benchmarks trained on it made the artifact load-bearing.

16.3. Data justice and the question of for whom

Taylor (2017) puts a different question on the table. The fairness literature that Chapter 7 reviewed asks whether an algorithm treats different groups equivalently. Data justice asks something prior: what counts as justice in this context, for whom, through what mechanisms? An algorithm can satisfy demographic parity across race categories and still be unjust,

because the deeper question is whether the decision system itself should exist, whether the people subject to it had any say in its design, and whether the categories used to sort them are categories they recognize. D'Ignazio and Klein (2020), *Data Feminism*, extends this framing into seven principles, of which "examine power" and "elevate emotion and embodiment" are the ones most likely to make an engineering audience uncomfortable, and therefore the ones most likely to be doing real work.

You met openness in Chapter 6 and ownership in Chapter 8. Data justice says those values are not sufficient. Openness can be the mechanism by which a community's vulnerability becomes indexable by anyone with a laptop. Ownership can be a contract that extracts data from people under conditions they had no power to negotiate. Neither value is wrong; both are incomplete without the prior question of for whom.

16.4. Refusal as design

Zong and Matias (2024) and Garcia et al. (2022) develop the concept this chapter is built around. Refusal is not the absence of data practice. It is a design move. To refuse is to specify what would not be collected, what would be collected differently, what would be collected only under conditions the affected community controls, and who gets to make each call. A well-written refusal spec is as detailed as a data management plan and does similar work: it commits authors to particular operations and can be held to account if they are violated. The output is not a dataset. It is an absence that has been designed.

The key move, which students miss the first time, is that refusal is generative. A refusal spec is not a protest document. It says: if we were not collecting this, what *would* we do to answer the underlying question? That substitution is where the intellectual work lives.

16.5. The installed base, revisited

This is where the book's framework binds. The installed base is built around the premise that records should link, persist, be interpretable, and be available for scrutiny. Carroll et al. (2020) and the Te Mana Raraunga network propose something different. The CARE Principles for Indigenous Data Governance (Collective Benefit, Authority to Control, Responsibility, Ethics) were articulated as a companion, and corrective, to FAIR (Findable, Accessible, Interoperable, Reusable), the default standard for scientific data stewardship (Wilkinson et al. 2016). CARE does not reject FAIR. It says FAIR is necessary and insufficient. A dataset can satisfy every letter of FAIR and still be in the wrong relationship with the community it describes.

Māori data sovereignty, as Te Mana Raraunga articulates it (Te Mana Raraunga 2018), begins with the premise that data about Māori communities are a Māori responsibility. Who holds the data, who decides what questions get asked of it, who benefits from the use, are not methodological concerns downstream of the research question. They are the question. The US Indigenous Data Sovereignty Network (United States Indigenous Data Sovereignty Network 2024) has made a parallel argument in the United States context.

This is the honest moment. The installed-base framing assumes the default should be linkable, persistent, interpretable records under conditions of scrutiny. CARE and Indigenous data sovereignty say that default is itself a political choice reflecting the settler colonial state's interest in legibility more than any universal principle of good governance. You can take the critique seriously without abandoning the framework. You cannot take it seriously while pretending the framework is neutral.

16.6. Running example: Denver HMIS

The Denver metro region, like every US Continuum of Care, operates an HMIS as a condition of HUD funding (US Department of Housing and Urban Development 2024). HMIS records client-level data for anyone entering homelessness services: intake demographics, shelter stays, case notes, exits, returns. A researcher working with a CoC partner can, under a Data Use Agreement, obtain de-identified extracts.

The tension is stark. Without HMIS, continuums cannot document who they served, cannot qualify for HUD funding, and cannot produce the Point-in-Time counts that drive resource allocation. With HMIS, people experiencing homelessness are among the most heavily documented populations in the United States per capita, and that documentation is potentially available to immigration enforcement, family court, criminal justice agencies, and any other state actor with a sufficiently motivated subpoena (Eubanks 2014). The audit is not academic. The refusal is not rhetorical.

The Missing Manual

The HMIS Data Standards manual names the categories. It does not name the decisions that produced them. The "Race" field allows American Indian or Alaska Native, Asian, Black or African American, Native Hawaiian or Pacific Islander, White, and Client Doesn't Know / Client Refused / Data Not Collected. When you run `df["race"].value_counts()` and see six buckets, there were almost certainly more at intake. A client who identified as Oaxaqueño, Somali Bantu, Rohingya, or Dakota was collapsed upstream into one of the HUD-allowable codes by the intake worker, because the HMIS validation rules reject anything else. The documentation will tell you what the codes mean. It will not tell you what was lost when the intake worker had three seconds to pick one.

16.7. Technical component 1: the category audit

The first artifact is a notebook that inventories the categorical fields, traces each to an administrative source, and documents what the collapsing does.

```
import pandas as pd

df = pd.read_csv("data/hmis_extract.csv", low_memory=False)

CATEGORICAL_FIELDS = [
    "race", "ethnicity", "gender", "veteran_status",
    "project_type", "destination", "disabling_condition",
    "prior_living_situation",
]

def audit_field(series):
    counts = series.value_counts(dropna=False).to_frame("n")
    counts["pct"] = (counts["n"] / len(series) * 100).round(2)
    counts["field"] = series.name
    return counts.reset_index(names="value")

audit = pd.concat([audit_field(df[f]) for f in CATEGORICAL_FIELDS])
audit.to_csv("out/category_audit.csv", index=False)
# => One row per (field, value) pair with counts and pct.
```

That gives you the frequencies. It does not give you the provenance. For each field, write a one-paragraph provenance note that answers four questions. Who defined the categories, when, for what administrative need, and what changed from the prior version? The HMIS manual is your source for the first three; HUD's Federal Register notices from each update cycle are your source for the fourth.

```
PROVENANCE = {
    "race": {
        "authority": "OMB Directive 15 (1997), implemented in HMIS 2014",
        "known_flattening": (
            "Collapses Indigenous identities (Dine, Dakota, Lumbee) "
            "into 'American Indian or Alaska Native'. Collapses South, "
            "Southeast, and East Asian into 'Asian'. No MENA category."
        ),
        "purpose": "Federal civil-rights reporting, not service-matching.",
    },
    # Repeat for each field.
}
```

Cross-tabulate categories against missingness. A provenance note is worth more when you can show the rate at which "Data Not Collected" is selected and how that rate varies by intake site.

```
missing_by_site = (
    df.assign(race_missing=df["race"].isin(
        ["Client doesn't know", "Client refused", "Data not collected"]
    ))
    .groupby("intake_site")["race_missing"]
    .mean()
    .sort_values(ascending=False)
)
# => High "Data Not Collected" rates often track sites where clients
# distrust documentation; the distrust is itself data.
```

Notice what that last snippet did. It treated absence as a finding. D'Ignazio and Klein (2020) call this counting what is missing, and counting why. A site with 40% missingness on race is not a site with poor data quality. It is a site where 40% of clients, faced with a form, chose not to answer. That refusal is a signal. A competent audit surfaces it.

16.8. Technical component 2: the refusal specification

The second artifact is a structured refusal specification. The spec is a markdown document with a YAML preamble. The YAML commits the authors to specific operations; the markdown explains each. Below is a skeleton you can adapt.

```yaml
---
dataset: denver_metro_hmis_2024
governance_body: denver_lived_experience_advisory_board
review_cycle: annual

would_not_collect:
  - field: full_legal_name
    rationale: |
      Legal name enables cross-system linkage with criminal-
      justice and immigration databases. Replace with a
      salted pseudonymous ID generated at intake.
  - field: social_security_number
    rationale: |
      HUD requires SSN only for specific subpopulations.
      Defaulting to SSN collection exceeds the federal
      minimum and creates a subpoena target.

collect_differently:
  - field: race
    current: HUD six-category closed list
    proposed: |
      Free-text self-identification, with HUD codes as an
      administrative mapping applied only when required
      for federal reporting. The raw self-id is retained
      under community control; only the mapping leaves the
```

```yaml
      Continuum.

collect_under_community_governance:
  - field: case_management_notes
    governance: |
      Notes are retained by the direct-service provider,
      not uploaded to the shared HMIS instance. Aggregate
      statistics derived from notes are reviewed by the
      Lived Experience Advisory Board before release.

decision_process:
  who_decides: |
    The Lived Experience Advisory Board, which includes at
    least five currently- or recently-unhoused members with
    voting rights equal to agency staff.
  by_what_process: |
    Changes require a majority vote at a noticed public
    meeting. Emergency additions can be enacted by the
    chair pending ratification at the next meeting.
  appeals: |
    A client may request removal of their own record from
    any downstream analytic extract. The request is honored
    within 30 days and logged in aggregate.
---
```

The structure is load-bearing. Each section answers a specific question: what would you not collect, what would you collect differently, what would you collect only under community-controlled conditions, who makes the call when the spec needs to change, by what process, and with what appeals. A refusal spec that skips the decision-process section is a wish list, not a governance document.

The deliverable is the pair. The audit shows what is in the data and what the categories are doing. The refusal shows what would be different, and

under whose authority, if the audit's findings were taken seriously. Neither on its own forces the question. Together they do.

16.9. Refusal in the wild: the Algorithmic Ecology

The Stop LAPD Spying Coalition, in partnership with the Free Radicals collective, published "The Algorithmic Ecology" in 2020 (Stop LAPD Spying Coalition and Free Radicals 2020). Its method is to map an algorithmic system (LAPD's predictive policing program at the time) as an ecology: community, institutional, and political-economic levels. The refusal it articulates is not "fix the algorithm" but "dismantle the conditions that make the algorithm legible as a solution to anything." The coalition's earlier "Before the Bullet Hits the Body" (Stop LAPD Spying Coalition 2018) did the groundwork by showing how LAPD's data-driven programs routed back to the same neighborhoods regardless of the model's claims to neutrality. When LAPD discontinued PredPol in 2020 and the LASER program shortly after, the coalition's organizing was a proximate cause.

That is refusal as generative, in the sense Zong and Matias (2024) intend. It did not settle for a seat at the table. It specified the conditions under which the table should not exist in its current form, and built the political capacity to enforce the specification. Your spec in Exercise 16.2 is modest by comparison. The move is the same.

> **In the Public Interest**
>
> Chapter 6 argued openness is interpretive legibility: data you can read, re-read, and link. Chapter 8 argued ownership is the authority to determine use, including the authority to *not* be owned. This chapter asks what happens when those two framings conflict. A dataset can be maximally open, perfectly interpretable, and exhaustively linkable, and still violate CARE because the community it describes never

had authority over whether it should exist. Openness as you met it in Chapter 6 breaks down when the community is not the author of the opening. Ownership as you met it in Chapter 8 breaks down when it can only be asserted by entities the installed base recognizes. Indigenous data sovereignty names both breakdowns. Your refusal specification is where you do something about them, not by rejecting openness or ownership, but by rebuilding both on the prior question of authority.

16.10. Exercises

Exercise 16.1 (Guided). Obtain a public HMIS-derived dataset. Many Continuums of Care publish de-identified aggregate extracts, HUD AHAR reports contain analyzable tables, and the Denver Metro CoC publishes dashboards with extractable tables. If no HMIS analog is accessible, use Denver Police stop data or Boulder's eviction-filings feed. Produce the category audit: a table of field frequencies, a provenance note for each field, and a missingness cross-tab by at least one stratifying variable. Commit as `audit.ipynb`.

Exercise 16.2 (Applied). Write a refusal specification for the same dataset, following the YAML skeleton. Include at least two `would_not_collect` entries, two `collect_differently` entries, one `collect_under_community_governance` entry, and a full `decision_process` section. Each entry needs a rationale you would defend in a public meeting. Commit as `refusal.md`.

Exercise 16.3 (Reading). Read the CARE Principles (Carroll et al. 2020) and FAIR (Wilkinson et al. 2016). Write 400 words comparing them on three dimensions: who each treats as the data-governance authority, what each says about reuse, and where they conflict in practice. Do not pretend they are compatible where they are not.

Exercise 16.4 (Analytic). Pick a refusal in the wild: the Algorithmic Ecology from Stop LAPD Spying, the Detroit Community Technology Project's work on Project Green Light (Detroit Community Technology Project 2019), a recent Indigenous data sovereignty initiative tracked by USIDSN, or another case your instructor approves. Write 500 words on what is being refused, who is doing the refusing, what alternative the refusal proposes, and what made it effective or not. Cite at least three primary sources.

Exercise 16.5 (Open-ended). Return to the project sketch from Exercise 1.5. Identify one question where refusal is the right answer. Write two paragraphs: what you would not collect or analyze, and what substitutes for the refused collection. The substitute may be a different research question, a different unit of analysis, a governance structure, or a partnership. Save as `ch16_project_refusal.md`. Carry this forward into Part V.

16.11. Looking ahead

Part V begins in Chapter 17 (Chapter 17) with the op-ed, the first of six genres through which data scientists contribute to public conversations. The audit and refusal you produced here are internal artifacts, the kind of document a research team circulates to set its own terms. The op-ed is their public counterpart: a form that addresses an audience that is not a research team, in a language that is not YAML, with a claim that is not hedged. Part V is the pivot from seeing to doing. This chapter is the last moment to settle what you see.

16.12. Further Reading and Resources

- Bowker and Star (2000), *Sorting Things Out: Classification and Its Consequences*. MIT Press. Start here.

- D'Ignazio and Klein (2020), *Data Feminism*. MIT Press. Open-access at https://data-feminism.mitpress.mit.edu/. Seven principles and the case studies that make them mean something.
- The CARE Principles for Indigenous Data Governance: https://www.gida-global.org/care. Short, specific, a direct complement to FAIR.
- Te Mana Raraunga, "Principles of Māori Data Sovereignty": https://www.temanararaunga.maori.nz/tutohinga.
- Stop LAPD Spying Coalition, "The Algorithmic Ecology" (2020): https://stoplapdspying.org/the-algorithmic-ecology-an-abolitionist-tool-for-organizing-against-algorithms/. A short essay and a diagram that will change how you read algorithmic systems.
- Audra Simpson (2014), *Mohawk Interruptus*. Duke University Press. The book that made refusal a theoretical term in contemporary Indigenous studies.
- Simone Browne (2015), *Dark Matters: On the Surveillance of Blackness*. Duke University Press. The prehistory of the surveillance systems this chapter's audit targets.
- Safiya Umoja Noble (2018), *Algorithms of Oppression*. NYU Press. A canonical treatment of search as a site of racial formation.
- US Indigenous Data Sovereignty Network: https://usindigenousdata.org/.
- HUD HMIS Data Standards manual: https://www.hudexchange.info/resource/3824/hmis-data-dictionary/. Read the introduction and the Race field documentation before Exercise 16.1.

Part V.

Genres

17. Op-Eds

Parts I through IV have been about learning to see. You have traced the structural pressures that constrain public knowledge (Chapter 2, Chapter 3, Chapter 4), named the countervailing values that hold accountability together (Chapter 6, Chapter 7, Chapter 8), studied the professional lineages that have done public-interest work for longer than data science has existed (Chapter 9 through Chapter 13), and triangulated your practice against the fields it is not (Chapter 14, Chapter 15, Chapter 16). That work has produced a diagnosis and a vocabulary. It has not yet produced a reader.

Part V is the pivot from seeing to doing. Each of the next six chapters takes a distinct genre of public writing or making, traces its history and its audience, and walks you through producing a real artifact in that form. By the end of Part V you will have, on disk and ready to submit, an op-ed, a report, a research proposal, a testimony statement, a public comment on a live rulemaking, and an archival deposit with a resolvable identifier. The chapters are independent enough that you can read them out of order, but the artifact convention is not optional. The chapter is not done when you have read it; the chapter is done when you have sent the draft. This one starts with the shortest form and the widest audience.

17.1. Why the op-ed matters, and why it does not

The op-ed is the 800-word argument you write for a stranger. It is not a report (Chapter 18), which assumes a policy audience willing to read

40 pages. It is not testimony (Chapter 20), which assumes a subpoena-strength reason for you to be at the microphone. It is a piece of persuasion, addressed to a general reader who has approximately four minutes and two competing browser tabs, written in the voice of a person who thinks something should happen. Most of what you have built in previous chapters (the audit, the dataset, the scraper) is illegible to that reader. The op-ed is what makes it legible.

It is also the genre public-interest data scientists most frequently skip. The report goes to the funder. The paper goes to the committee. The dashboard goes to the client. None of them reach the person whose benefits were denied, the council member whose zoning vote is next Tuesday, or the reporter who might pick up the thread if they knew the thread existed. An op-ed that is published in a regional paper is read by more members of the affected public than any peer-reviewed article you will write this year. That is an uncomfortable fact, and one the profession has mostly refused to sit with.

17.2. A short history of a strange page

The op-ed page (literally, the page opposite the editorial) is younger than most of the datasets you work with. *The New York Times* ran its first recognizable op-ed page on September 21, 1970, under editorial page editor John B. Oakes and op-ed editor Harrison Salisbury (Oakes 1970). The idea was to make room for outside voices on the page facing the paper's unsigned institutional editorials: voices that did not agree with the *Times*, that came from outside journalism, and that addressed the day's questions with a specificity the editorial page was too institutional to risk.

The op-ed page spread from the *Times* to other broadsheets through the 1970s and 1980s, and with it a set of conventions that still govern most pitches today: roughly 750 words, an argument pegged to the news cycle, a voice the paper's own columnists cannot reliably produce. In 2021, the

Times renamed the form "guest essays" on the grounds that the page-layout metaphor no longer matched the digital product (Kingsbury 2021). Most outlets have quietly followed, though the genre itself is stable. Writers still call them op-eds.

What has changed since 1970 is the ecosystem in which the op-ed sits. The legacy broadsheets still publish op-eds, but their slush piles are brutal and their turnaround on unsolicited pitches is measured in weeks. Around them has grown a second tier that is, for most public-interest data scientists, more realistic and in some ways more useful: *The Conversation US*, which commissions op-eds from academic authors and republishes them under Creative Commons to hundreds of regional outlets; *Inside Higher Ed* and *The Chronicle of Higher Education*, which publish arguments on research policy; *MIT Technology Review* and *Wired*, which publish longer technology op-eds with reporting attached; regional dailies like the *Boulder Daily Camera* and the *Colorado Sun*, whose readers include the city councils and county commissioners your audit is actually addressed to; and the newsletter tier (Substack, Ghost, individual blogs) where you publish the same 800 words to a self-selected audience with zero gatekeeping.

17.3. Who gets published where

The op-ed page is not a neutral meritocracy. The Op-Ed Project, a nonprofit founded by Catherine Orenstein in 2008, has for fifteen years documented the demographic distribution of bylines in major U.S. op-ed pages. Their byline surveys consistently find that roughly 80 percent of opinion writers in the legacy broadsheets are men, and that writers of color are underrepresented at ratios that have improved slowly and unevenly since tracking began (The Op-Ed Project 2022, 2024). The explanation is not that underrepresented writers pitch less often, though they do; it is that the pitch economy runs on weak-tie networks (Rolodexes, conference circuits, editor DMs) and those networks are the ones the profession has

always had. If you are reading this book, you have reasons to pitch that are exactly as legitimate as anyone else's. Pitch anyway.

The political economy also shapes what counts as an "op-ed-able" argument. A piece that says a city's predictive-policing vendor is overfitting to historical arrests is op-ed-able; a piece that says policing is itself the problem is, at most outlets, not. The constraint is not a conspiracy. It is a product of what editors believe their median reader will finish. You will do better work if you understand the constraint, write around it where you can, and name it when you cannot.

17.4. The anatomy of an 800-word argument

An op-ed is not a short paper. It is an argument with a particular load-bearing structure, and editors can diagnose whether the structure is there within sixty seconds of opening your attachment. Five parts:

The hook is the first sentence, or at most the first two. It is the news peg, the anecdote, the surprising number. Its job is to earn the next paragraph.

The nut graf (from journalism; the paragraph that says "in a nutshell") is where you tell the reader what your argument is. Not what the piece is about. What you want the reader to believe by the end. If you cannot write your nut graf as a single declarative sentence, the argument is not ready.

The argument is two to four paragraphs that develop the claim. Each paragraph should do one thing. "One paragraph, one move" is the rule you will hear from every op-ed coach, and it is the rule most first drafts violate.

The evidence is where your data lives. One chart, one number, one case, a named source. You are not proving the argument the way a paper would;

you are giving the reader enough to trust you. A single well-chosen statistic with a live hyperlink is worth a paragraph of hedged prose.

The call to action is the last paragraph. What should happen, and who should make it happen? "Policymakers should consider" is the call to action of a piece that has no call to action. "The Boulder City Council votes Tuesday on an ordinance that would require X; it should vote yes" is a call to action. Specificity is the whole game.

Hook, nut graf, argument, evidence, call. Print it on an index card. Pin it above your monitor.

17.5. Pitching is half the work

Editors do not read op-eds they have not been pitched. This is the single most common failure mode among first-time authors: they write a brilliant 800 words, attach it to an email that says "Please consider the attached for your opinion page," and wonder why they never hear back. The attached draft is the easy half. The three sentences in the email body are the hard half.

A working pitch email has a specific shape. The subject line names the genre and the angle: "Op-ed pitch: what Boulder's eviction data actually shows." The body opens with the argument in one sentence. The second sentence says why the piece is timely (there is a vote, a ruling, an anniversary, a report). The third sentence says why you are the person to write it: your credential is not your degree, it is the work you have already done on the question. Paste the full draft below a horizontal rule in the email body. Attach it as well, ideally as `.docx` and `.txt`. Put your phone number in the signature. If the editor wants the piece, they want to call you today.

The Missing Manual

Op-ed editors want a pitch email and a draft that is one click from publishable. Submitting a Google Doc with Track Changes, or a PDF, or a link to a Notion page, is a rookie mistake that tells the editor you do not know the genre. Paste the full text into the email body. Attach the draft as `.docx` for the copy desk. Include a one-sentence bio and a phone number. Do not send a 500-word cover letter. Do not apologize for the length of your pitch. Do not ask whether they would be interested before sending the draft: by the time they decide, they have already moved on. The other piece of infrastructure most authors skip: a single embedded chart. Outlets that can publish a chart (and most can, even the online editions of print dailies) report that op-eds with one chart get substantially more social-share traffic than op-eds without one. Make the chart non-optional.

17.6. A worked example: the Boulder audit as an op-ed

Take the dataset you have been building since the earlier chapters. It does not matter which one: the HMIS audit from Chapter 7, the campaign-finance scrape from Chapter 9, the Alpine-Balsam housing analysis from Chapter 12, or your own. You have a finding. You have evidence. You have, almost certainly, a moment when a public body is about to decide something your finding is relevant to. An op-ed is the translation.

Suppose your audit found that the Boulder County homelessness response system (as documented in HMIS) routes single adults to shelter beds in a way that systematically disadvantages households with recent non-Colorado addresses. You have the numbers. You have the code. You have Chapter 7's disaggregated accuracy report. What you need is a 800-word piece for the

Boulder Daily Camera or the *Colorado Sun*, pegged to the next Boulder City Council agenda item on homelessness services funding, that gives a reader who will never open your notebook a reason to call their council member.

Your hook is the upcoming vote, or the specific person turned away. Your nut graf says, in one sentence, that the scoring system is producing a pattern its designers did not intend and the Council should pause the contract renewal to investigate. Your argument is two paragraphs on how the scoring works and one paragraph on what the data shows. Your evidence is the chart below. Your call is: postpone the December vote, commission an external audit, and release the vendor's scoring logic under CORA (the Colorado Open Records Act; see Chapter 10).

Here is the chart. One chart. You will not get two.

```
import pandas as pd
import matplotlib.pyplot as plt

# Rates are illustrative; replace with your audit's actual figures.
df = pd.DataFrame({
    "group": ["In-state, 1+ yr", "In-state, <1 yr", "Out-of-state"],
    "referral_rate": [0.58, 0.41, 0.23],
    "n": [812, 294, 187],
})

fig, ax = plt.subplots(figsize=(6.5, 3.8))
bars = ax.bar(df["group"], df["referral_rate"],
              color=["#2b6cb0", "#2b6cb0", "#c53030"])
for bar, n in zip(bars, df["n"]):
    ax.text(bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.015,
            f"n={n}", ha="center", fontsize=9)
ax.set_ylabel("Share referred to shelter within 7 days")
ax.set_ylim(0, 0.75)
```

```
ax.set_title("Boulder County HMIS referral rates by prior-address tenure,\n"
             "January 2023 through June 2025")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
fig.tight_layout()
fig.savefig("ch17_hmis_referral.png", dpi=160, bbox_inches="tight")
```

Two things about that chart are deliberate. The title is a complete sentence, because a newsroom reader looks at the title first and the axes second. The colors code the argument (one group is the outlier), because a newsroom reader will not parse a legend. The sample sizes are on the bars, because a newsroom reader who does not see them will assume the disparity is noise. The chart earns its place in 800 words because it does in one image what a paragraph of prose would fail to do.

A note on rendering. Editors accept `.docx`, `.txt`, and pasted-into-email plain text. They do not accept `.qmd`, `.ipynb`, or `.tex`. Render your draft to Word with `quarto render draft.qmd --to docx`, and keep a plain-text version in the email body so the editor can read it without an attachment. For citations, the web convention is in-text hyperlinks rather than footnotes: link the phrase "the county's own data" to the dataset or the audit. Do not include a bibliography. If the editor wants a source list, they will ask.

17.7. Why this matters for the book's argument

An op-ed is, among other things, an exercise in openness as argument rather than openness as data release. Chapter 6 taught you to publish a dataset with a data dictionary and a durable identifier. That is necessary and insufficient. A dataset nobody reads, like a report nobody reads, is openness in form without openness in effect. An op-ed is how you make the

dataset's implication publicly legible; it is the reason the data dictionary existed in the first place.

> **In the Public Interest**
>
> Transparency is not self-executing. Chapter 6 built an installed base of documented, citable data; this chapter is where that base meets a reader. An op-ed that lands in the *Daily Camera* the week before a council vote does more work for the public interest than the same finding published as a 40-page report the council staff will skim in a briefing (compare Chapter 18). You will write both. The difference between a data release that changes a vote and one that does not is almost always a 750-word argument somebody pitched to an editor who picked up the phone.

17.8. Exercises

Exercise 17.1 (Analytic). Pick three recent op-eds on AI or data policy, at least one from a data scientist writing for a general audience. Start with Emily Bender (Bender 2023) on large language models, Deborah Raji (Raji 2020) on how data encodes racism, and Meredith Broussard (Broussard 2023) on algorithmic bias; substitute equivalents if those links have rotted. For each, identify the hook (first sentence), the nut graf (one sentence that states the argument), and the call to action (what the author wants the reader to do, and who the reader is). Produce a two-page annotated comparison. Note: at least one of your three op-eds will not have a crisp nut graf. Say so.

Exercise 17.2 (Core artifact, part 1). Draft a 750 to 1,000 word op-ed based on the audit, dataset, or analysis you built earlier in the book. Target a specific, named outlet: the *Boulder Daily Camera*, the *Colorado Sun*, *The Conversation US*, *Inside Higher Ed*, or an

instructor-approved equivalent. Your draft must include a hook, a nut graf, a specific call to action, and at least one in-text hyperlink to source data. Save it as `exercises/ch17_oped_draft.md` and render to `exercises/ch17_oped_draft.docx`. Word count is enforced. A 1,200-word op-ed will not be published; a 600-word op-ed is not yet finished.

Exercise 17.3 (Core artifact, part 2). Write the pitch email that accompanies your draft. Three sentences in the body: argument, timeliness, credential. Paste the full draft below a horizontal rule. Include an appropriate subject line, a one-sentence bio, and a phone number. Save it as `exercises/ch17_pitch_email.md`. The email must be one a real editor at your target outlet could read in under ninety seconds and decide to act on.

Exercise 17.4 (Core artifact, part 3). Produce one embedded chart for the op-ed. Build it in matplotlib, `seaborn`, or `plotnine`; export at a size that will survive being shrunk to 500 pixels wide. The chart's title must be a full declarative sentence. The chart must carry its sample sizes. The chart must work in grayscale (test by saving as black-and-white and confirming the argument survives). Save as `exercises/ch17_oped_chart.png` at 160 dpi.

Exercise 17.5 (Open, moment-of-publication). Op-eds live or die by timing. Watch the local and national news cycle for a week. Identify the moment, real or plausible, when your op-ed would be most likely to land: a council vote, a court ruling, an anniversary, a federal action. Rewrite the lede (the hook plus the first paragraph) for that moment. Save both ledes (the original and the retimed version) and write 200 words on what changed and why. If the moment arrives within the window of the course, send the pitch.

Exercises 17.2, 17.3, and 17.4 together constitute the Part V artifact for this chapter. At the end of the week you will have a file in the course's submission repository that is, without further work, a complete op-ed package you could send tomorrow. Some of you will send it. Some of yours

will be published. The draft that is not published goes into the portfolio alongside the five artifacts from the remaining Part V chapters.

17.9. Looking ahead

The op-ed reaches audiences the report never will. The report, for a different audience, makes a case the op-ed cannot. Chapter 18 takes up the longer form: how to structure a 20-to-40 page argument for a policy reader, how to keep the figures reproducible under a build system, and how to write the two-page executive summary that is, in practice, the only part most readers will read. The skills carry forward. The pitch-mindset (know your reader, make every element earn its place) carries forward most of all. Chapter 20 and Chapter 22 extend it into genres where the audience is a committee chair and a permanent record, respectively, and the stakes of getting the framing right are proportionally higher.

17.10. Further Reading and Resources

- The Op-Ed Project, https://www.theopedproject.org/. Handbook, byline survey, and pitch-coaching network. If you read only one external resource from this chapter, read this one.
- *The Conversation US*, pitch guidelines: https://theconversation.com/us/pitches. The most accessible outlet in the United States for academic data scientists. Pieces are published under Creative Commons and routinely picked up by regional papers.
- Emily M. Bender, representative guest essay on large language models: (Bender 2023). A model of how to write a data-science op-ed for a general audience without flattening the technical argument.
- Inioluwa Deborah Raji, "How our data encodes systematic racism," *MIT Technology Review*, December 2020: (Raji 2020). A model of

a data-scientist op-ed that leads with a case and a number, not a methodology.

- Meredith Broussard, guest essay on algorithmic bias: (Broussard 2023). See also her book (Broussard 2018) for the longer argument.
- *The Associated Press Stylebook* (Associated Press 2024). The house style used by most U.S. newsrooms. Read the entries on numbers, dates, and attribution before your first pitch.
- Poynter, op-ed how-to resources: (Poynter Institute 2023). Craft-level advice from the journalism industry's in-house trainer.
- Kathleen Kingsbury, "Why we're retiring the term op-ed," *The New York Times*, April 2021: (Kingsbury 2021). Context on the genre's current self-understanding at the paper that invented it.
- ProPublica's "Machine Bias" series (Angwin et al. 2016). Not op-eds, but the kind of reporting that the best data-scientist op-eds cite and translate for a general reader.
- Keegan (2026), "Public interest data infrastructuring." The framework this book extends. An op-ed is the smallest unit of public infrastructure you will build in this course; read the paper for the larger unit.

18. Reports

The op-ed you drafted in Chapter 17 travels light: it reaches a commuter, a council member, or a reporter already circling the story, and gets one decision out of them before the next tab loads. The report is the opposite instrument: long, footnoted, designed to be excerpted and cited three years later in a brief you will never see, and written because some institutional reader (a program officer, a legislative staffer, a regulator's policy advisor, an advocacy research director) needs a document they can put in a folder and hand to their general counsel. The op-ed is a knock on the door. The report is the evidence you leave behind after being let in.

This chapter treats the report as a distinct genre with its own history, political economy, and load-bearing conventions. The chapter's artifact is a 10-to-15-page report draft, with a real executive summary, one completed section, at least one reproducible figure built from a committed notebook, and a one-paragraph cover note addressed to a named institutional reader. The starting material is the topic you pitched as an op-ed in Chapter 17; the audience shifts, but most of the argument does not.

18.1. A short political economy of the think-tank report

The modern think-tank report is a twentieth-century invention shaped by a funding ecology of foundation grants, corporate sponsorship, occasional government contracts, and a small number of large philanthropic commitments to AI and internet policy. The shops you will cite most

often (AI Now, Data & Society, EPIC, CDT, Mozilla, Brookings, New America, Ada Lovelace) sit between academic research and advocacy, and each calibrates that space differently: AI Now is argumentative, Data & Society explanatory, Brookings academic with a policy preamble, Mozilla a designed magazine with citations. Read three before you write your own.

They share a structure: a one-to-two-page executive summary, findings organized by claim, methods folded in or in an appendix, recommendations aimed at a specific institutional actor, and a bibliography. Proportions matter: AI Now spends half its pages on argument and a quarter on evidence; Brookings inverts those; Mozilla spends more on design and less on methods than either. Before you draft, look at where your model report spends its pages. That is where its authors believe the argument lives.

Each report has a reader in mind who is not you: a Brookings paper on generative AI in the federal workforce is written for a senior civil servant at OMB; a Data & Society primer on worker surveillance for a program officer at a labor-focused foundation; an Access Now report for a UN Special Rapporteur's staff. Reports that read as if written for a general audience are almost without exception worse reports.

18.2. Reports that inform and reports that mobilize

There is a useful distinction most authors do not draw explicitly: reports whose job is to inform versus reports whose job is to mobilize. An informing report surveys a field, names the actors, and leaves the reader better oriented. A mobilizing report builds to a demand: a rule the regulator should write, a line-item the legislator should add, a disclosure the company should make. Both are legitimate, and they are not interchangeable. A mobilizing report that reads as merely informing will be ignored by those who could have acted on it; an informing report that reads as mobilizing will be discounted by those who trusted it as a survey.

You can usually tell the difference by reading the recommendations. An informing report hedges ("policymakers should consider a range of approaches"); a mobilizing report is specific ("the Federal Trade Commission should open a Section 6(b) study into the five named vendors within 180 days"). If your recommendations could be photocopied into a different report on a different topic without changing meaning, you have written a report that will not cause anything to happen. That may be the correct choice. Be sure it is a choice.

Non-US exemplars clarify the range. Access Now's #KeepItOn report is mobilizing, giving UN human rights mechanisms and national-level litigants a shared evidentiary record. Derechos Digitales (Santiago) publishes reports that work as regional advocacy, academic reference, and inputs to Inter-American Court proceedings. Paradigm Initiative (Lagos) publishes Londa to equip African civil society with country-by-country baselines (Paradigm Initiative 2024). The political economy differs: less foundation money, more distributed authorship, tighter integration with litigation and legislative work.

18.3. The executive summary is the report

Most readers will read exactly three things: the title, the executive summary, and the recommendations. Some will glance at the figures. Almost none will read the methods. This is not a failure of the readers: a foundation program officer reads fifteen reports a week, a legislative staffer thirty. They are triaging, and the executive summary is where the triage happens.

The implication is direct. The executive summary is not a summary of the report; it is the argument of the report, compressed to one and a half pages, with the evidence pointers inline. It must stand alone: if a reader forwards only the executive summary to their general counsel, the general counsel should be able to decide whether to recommend a meeting. The recommendations belong in the executive summary, not saved for

a climactic reveal. If they are not in the executive summary, they are invisible.

Headings are the report's second skeleton. A skimming reader reads section headings and figure captions, and if the headings read as a coherent outline, the reader will trust the document enough to read further. "Findings" is a weak heading; "Three ways the scoring system diverges from its stated criteria" is a strong one. Write headings as if they were the nut grafs of the sections they introduce.

> **The Missing Manual**
>
> Most readers will read the executive summary, the headings, and the charts. Write those three layers as if they were the whole report. The executive summary is not a summary; it is the argument compressed to one and a half pages, with evidence pointers inline. If recommendations are not in the executive summary, they are invisible. Test this before sending: print the executive summary, the table of contents, and the figure captions, and hand them to a colleague who has not seen the report. If they can tell you what you found, what you want done, and who should do it, the three-layer skeleton works. If they cannot, your argument lives in places no one will read.

18.4. Methods transparency as a credibility move

A report is not a peer-reviewed article and does not need to be. What it needs is methods transparency a skeptical reader can audit in an afternoon: a one-page description of how you obtained your data, when, from which endpoint, and with which parameters; a link to a public code repository with the scripts that produced each figure; a datasheet-style note for each dataset you relied on, following the template proposed by Gebru and colleagues (Gebru et al. 2021); and a clear statement of what the analysis

cannot support. The last item separates reports people cite from reports people weaponize.

Methods transparency connects to the accountability infrastructure of Chapter 11 and the authority-and-remedy framework of Chapter 7. A report whose claims can be reproduced against public data by a third party survives contact with a hostile reader; a report whose claims rest on private datasets, undocumented pipelines, or one-off notebooks lasts exactly as long as no one asks. You have already learned how to produce the first kind; the report is where you show that you did.

> **In the Public Interest**
>
> Oversight requires documentation that can travel, and reports are the documentation that travels. The op-ed in Chapter 17 gave a reader a reason to care; the report gives an institution something it can act on, cite, attach to a brief, and hand to counsel. This is **authority** and **remedy** in the sense of Chapter 7: a report without methods transparency gives a reader no grounds to trust it, and a report without specific recommendations gives no actor a path to act. The reproducibility discipline of Chapter 11 (documented notebooks, committed code, pinned dependencies, datasheets) is not academic ornament; it is the load-bearing infrastructure that lets your findings travel from your laptop to a rulemaking record to a brief to a ruling without degrading at each step. The credibility of the field depends on that infrastructure holding.

18.5. The translation gap

You already have the findings and the evidence. Writing a report is overwhelmingly the work of translating those findings into terms an institutional

reader can act on, and that translation has three moves first-time authors consistently underestimate.

First, naming the actor. "Someone should regulate this" is not a recommendation; it is a complaint. The recommendation is: the Colorado Department of Regulatory Agencies should open an investigation under C.R.S. section X into the vendor's scoring practices; the legislative staffer should draft language for the next session's HB-25-XXXX. Name the body, the authority, and the calendar.

Second, naming the remedy. An institutional reader cannot act on "transparency"; they can act on a specific disclosure requirement, a specific records release, or a specific audit provision added to a procurement contract. Your recommendation should be copy-pasteable into a draft rule or draft bill. If it is not, it is not yet a recommendation.

Third, naming the trade-off. Every institutional reader has someone in the next room whose job is to poke holes in the recommendation. Your report is stronger if it names the most plausible objection, treats it seriously, and explains why the recommendation still stands. Reports that pretend their recommendations are free lunches get dismissed by the people who know the lunches are not free.

18.6. Quarto as a report stack

Most of the technical apparatus is what you already know. A report is a Quarto project with its own YAML header (not the book's), a main `report.qmd`, one or more `figures.qmd` partials, a `references.bib`, and a directory of committed notebooks that generated each figure. Quarto renders the same source to HTML, PDF (for program officers who print everything), and, via Pandoc, `.docx` (for the policy staffer who runs Track Changes on your recommendations).

A minimal project layout:

```
report/
  _quarto.yml
  report.qmd
  figures.qmd
  references.bib
  notebooks/
    fig_01_referral_rates.ipynb
    fig_02_geographic.ipynb
  data/
    hmis_extract_2024q3.csv
    datasheet.md
  README.md
```

A minimal `_quarto.yml`:

```yaml
project:
  type: default
  output-dir: _output

author:
  - name: "Your Name"
    affiliation: "INFO 46XX, University of Colorado Boulder"
date: "2026-05-15"

format:
  html: {toc: true, embed-resources: true, theme: cosmo}
  pdf: {documentclass: scrartcl, toc: true, number-sections: true}
  docx: {reference-doc: templates/report-template.docx}

bibliography: references.bib
csl: https://www.zotero.org/styles/chicago-author-date
execute: {echo: false, warning: false}
```

A minimal `report.qmd` with an executive-summary callout and an included figures partial:

```
# Scoring disparities in Boulder County's homelessness response system

::: {.callout-important title="Executive summary"}
This report documents three ways the vendor's scoring system used by
Boulder County's Coordinated Entry process diverges from its stated
criteria. Using HMIS records from July 2023 through June 2024, we find
that households with recent out-of-state addresses are referred to
shelter beds at 23 percent, compared to 58 percent for in-state
households with more than a year of residency (@fig-referral-rates).
We recommend that the Boulder City Council pause the December 2026
contract renewal vote, commission an external audit, and release the
vendor's scoring logic under CORA.
:::

## Three ways the scoring system diverges from its stated criteria

{{< include figures.qmd >}}

## Methods {#sec-methods}

The HMIS extract was obtained through a CORA request (BC-2024-0912).
The pipeline is committed at
`https://github.com/yourname/boulder-coord-entry-audit`; the
datasheet follows the template of @gebru2021.

## Recommendations

1. Pause the December 2026 contract-renewal vote.
2. Commission an external audit with a disaggregation specification
   modeled on @sec-ch-07-oversight.
3. Release the vendor's scoring logic under CORA.
```

```
## References {.unnumbered}
```

The `figures.qmd` partial loads and labels reproducible charts produced by notebooks in `notebooks/`:

```
#| label: fig-referral-rates
#| fig-cap: "Shelter referral rate by residency group, Boulder County
#|   Coordinated Entry, July 2023 to June 2024. Notebook:
#|   notebooks/fig_01_referral_rates.ipynb"

import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("data/referral_rates_by_group.csv")
fig, ax = plt.subplots(figsize=(6.5, 3.8))
ax.barh(df["group"], df["referral_rate"])
ax.set_xlabel("Referral rate")
ax.set_xlim(0, 1)
fig.tight_layout()
```

To render as HTML, PDF, and `.docx` in one pass: `quarto render report.qmd --to html,pdf,docx`. The `.docx` output is the one the policy staffer runs Track Changes on; build for it from day one, since retrofitting a Word-compatible template at the last minute is a Friday-afternoon surprise you can avoid.

18.7. Institutional audiences, named

The report is not written for "policymakers" any more than the op-ed was written for "the public." It is written for specific humans in specific roles. Four audiences recur.

Foundation program officers fund the work that follows your report. They read for whether the argument fits the portfolio, whether you are the right grantee, and whether the recommendations are actionable. Time budget: ten minutes. First read: executive summary and recommendations.

Legislative staffers draft the bills and committee questions. They read for statutory language, a named committee with jurisdiction, and a witness list. Time budget: under ten minutes. First read: executive summary and whatever sidebar most resembles a witness list.

Regulatory policy advisors work inside agencies considering a rulemaking or responding to a petition. They read for whether your report supplies evidence that would survive a challenge in the rulemaking record. Their time budget is longer (they will read the methods) but their standard is stricter. They are auditing you.

Advocacy organization research directors are the readers most likely to build on your work, reading for fit with their litigation, campaign, or next report, and whether you are a potential collaborator. Their time budget is the longest, and they are most likely to ask for your data.

Identify which of these four your report is primarily for before you draft. Write the executive summary and recommendations to that reader. You cannot serve all four as the primary reader.

18.8. Reports across the lineages

The report does not belong to data science. It is a shared form across the lineages of Part III: an accounting report (Chapter 11) documents trust through procedural rigor, a planning report (Chapter 12) documents a decision through stakeholder mapping, and a public-interest technology report (Chapter 14) documents a deployment through user research. When your report borrows from all three, say so. A report visibly self-aware about its genre reads as adult.

18.9. Exercises

Exercise 18.1 (Structural analysis, US). Pick a recent AI Now or Data & Society report (2023 or later). Map its structure: how many pages are executive summary, findings, methods, recommendations, appendices? Where does the argument live? Who is the implied institutional reader, and what evidence tells you? Write 400 words. Save as `exercises/ch18_structural_analysis_us.md`. You may use McGuinness and Schank (2021) as a framing reference for what "public interest technology" reports are trying to accomplish and how.

Exercise 18.2 (Structural analysis, non-US). Do the same for a recent report from Access Now, Derechos Digitales, or Paradigm Initiative. Pay particular attention to where the report assumes a reader who is not the US federal government. Compare to your US report from Exercise 18.1 in one paragraph at the end: what conventions differ, and what does that tell you about the political economy of each report's audience? Save as `exercises/ch18_structural_analysis_nonus.md`.

Exercise 18.3 (Executive summary, outline, one full section). Take the op-ed topic you drafted in Chapter 17 and expand it into a report. Produce three deliverables: (a) a one-and-a-half-page executive summary that includes the recommendations inline; (b) a full section-by-section outline with one-sentence nut grafs for each section; (c) one section (three to five pages) written in full, with figures inline and citations against `references.bib`. Save as `report/report.qmd` in your class repository. Follow the YAML-and-layout conventions in this chapter. This is the core of your Part V artifact. See Gray and Bounegru (2021) on writing for policy audiences.

Exercise 18.4 (Reproducible figure with a committed notebook). Produce at least one figure for your report from a notebook committed to the repository at `report/notebooks/`. The notebook must: load data from a documented path, document every filter or transformation with a

comment, and render the figure to `report/figures/`. Add a datasheet-style provenance note (one page, following Gebru et al. (2021)) for the dataset the notebook uses. Commit the notebook, the datasheet, and the figure. The figure caption in `figures.qmd` should cite the notebook by filename. Tie this explicitly to the reproducibility discipline you built in Chapter 11.

Exercise 18.5 (Cover notes to three named humans). Identify three humans (by role, and by name if you can) who would need to read your report for it to matter. At least one must be a foundation program officer or an advocacy research director; at least one must be a legislative staffer or a regulatory policy advisor. For each, draft a one-paragraph cover note tailored to that reader. The cover notes should differ substantively: a program officer's note points to the portfolio fit and the follow-on work; a staffer's note points to the pending bill or hearing; a regulator's note points to the open docket or petition. Save all three as `report/cover_notes.md`. This draws directly on the stakeholder-mapping logic of Chapter 14 and Chapter 12.

Exercises 18.3, 18.4, and 18.5 together produce the Part V artifact: a report draft with an executive summary, one completed section, at least one reproducible figure built from a committed notebook with a datasheet-style provenance note, and three cover notes addressed to named institutional readers. When you have all four components in your class repository, tag the commit `ch18-artifact-v1`. The artifact is the chapter.

18.10. Looking ahead

Chapter 19 (Chapter 19) takes the genre into the academic and grantmaking ecosystems. A report documents what you found; a research proposal argues for what you will find if someone funds you. The conventions overlap and diverge in ways that catch first-time authors off guard: the specific-aims page borrows from the executive summary, but the theory of change runs

the opposite direction. The work you did on the report's recommendations has already produced most of the proposal's significance section. From there, Chapter 20 turns your recommendations into a five-minute spoken statement, and Chapter 22 gives your report and its data a permanent address.

18.11. Further Reading and Resources

- AI Now Institute, annual reports, https://ainowinstitute.org/publications. The canonical example of the mobilizing report in the US technology-policy ecosystem; read at least one cover to cover before drafting your own.
- Data & Society Research Institute, reports, https://datasociety.net/library/. A wider range of report types (primers, case studies, ethnographies). Pay attention to how each is structured for its implied reader.
- Access Now, #KeepItOn annual internet shutdowns report, https://www.accessnow.org/campaign/keepiton/. Canonical non-US advocacy report; note the integration with UN human rights mechanisms.
- Mozilla Foundation, *Internet Health Report*, https://internethealthreport.org. Design-forward, magazine-style report that rewards reading for layout and voice.
- Paradigm Initiative, *Londa* report on digital rights in Africa (Paradigm Initiative 2024), https://paradigmhq.org/londa. Distributed-authorship model with country-level reporting; the opposite of the single-institute think-tank report.
- Quarto project and book documentation, https://quarto.org/docs/projects/quarto-projects.html and https://quarto.org/docs/books/. The authoritative reference for multi-format rendering; the book you are reading is built with it.

- Pandoc's Word output documentation, https://pandoc.org/MANUAL.html#option--reference-doc. The two-page guide that fixes ninety percent of `.docx` formatting surprises, especially how to use a reference document for headings and styles.
- Tara Dawson McGuinness and Hana Schank, *Power to the Public* (McGuinness and Schank 2021), for the political economy of public-interest technology reports and the foundations that fund them.
- *The Data Journalism Handbook 2* (Gray and Bounegru 2021), particularly the chapters on report-length investigative projects; the conventions travel.
- Timnit Gebru and colleagues, "Datasheets for datasets" (Gebru et al. 2021). The provenance-note template you should adapt for every dataset your report relies on.
- Keegan (2026) (Keegan 2026), on the six elements of the installed base and why reports are the genre in which they most often travel.

19. Research Proposals

The op-ed in Chapter 17 argued for a thing that already exists; the report in Chapter 18 documented a thing that had already happened. The research proposal is stranger than either. It argues, in the present tense, for the significance of work that does not yet exist, using evidence you cannot yet produce, on behalf of findings you cannot yet guarantee, to readers who will decide whether the work should exist at all. You are asking a funder to believe in a future that is currently a document.

The hard part is not the methods, which you already have. It is translating a public-interest data question into a shape a funder can recognize, and translating your own working life into a budget that a program officer can defend to a board. By the end of the chapter you will have produced four artifacts: a one-page specific-aims document, a simple budget with a narrative justification, a data management plan (DMP), and a simulated Institutional Review Board (IRB) protocol. None is a submitted grant. All are the documents a submitted grant is made of.

19.1. The grant-proposal genre landscape

The funders you might realistically approach divide into four ecologies, each with its own vocabulary and its own review cycle.

Federal science and humanities agencies include NSF, NIH, IMLS, and NEH. They publish detailed proposal guides (NSF's PAPPG runs several hundred pages (National Science Foundation 2024)), use peer-review

panels, require formal budgets with indirect-cost recovery, and expect data management plans and broader-impacts sections. As of 2025, they are volatile ground: appropriations uncertainty, agency restructuring at IMLS, and shifting priorities mean a proposal submitted under one quarter's rules may be reviewed under the next's (Institute of Museum and Library Services 2024; National Endowment for the Humanities 2024; National Institutes of Health 2024).

National private foundations include Knight, MacArthur, Ford, and Robert Wood Johnson, plus a long tail of smaller outfits funding public-interest technology work. They run on program cycles, usually expect a letter of inquiry before a full proposal, and keep review standards less public than NSF's; you are writing for a program officer who is writing for a board (John S. and James L. Knight Foundation 2024).

Regional and community foundations include the Gates Family Foundation in Colorado, the Boettcher Foundation, the Denver Foundation, and analogues in every state (Gates Family Foundation 2024; Boettcher Foundation 2024). Grants run smaller (often $10,000 to $75,000), cycles are shorter, and priorities are locally specific. For a first public-interest research proposal, they are often the right target.

University seed grants are the quietest category and frequently the most accessible. A research development office will fund a $5,000 to $25,000 pilot with a two-page application. A pilot result is the strongest possible leverage for the next proposal up the chain.

Behind all four ecologies sits the Foundation Directory Online (Candid 2024), which maps private funders and their giving patterns. Spend an afternoon with it before you start writing.

19.2. The political economy of what gets funded

The gap between the money available for public-interest research and the number of projects that could usefully spend it is large and does not close. Foundation endowments run on five-percent payout rules; federal budgets are set by political appropriations. The median program officer manages a portfolio in which most of the projects they would like to fund will not be funded.

Two implications follow. Specificity beats scope: a proposal promising to solve algorithmic discrimination in public-benefits systems will lose to one promising to audit the adverse-action notices issued by one county's SNAP system over eighteen months, because the second is something a program officer can defend to a skeptical board. And if your specific aims are not legible in ninety seconds, your proposal will not get the second read.

Political economy also shapes what counts as "research." A community-engaged audit whose primary output is a community report is harder to fund through NSF than through a regional foundation. A refusal specification whose output is a decision not to build a dataset is harder to fund than either. Part of the proposal-writing work is finding the funder for whom your project is recognizable.

19.3. The specific-aims page as sales pitch

The specific-aims page is the single most important page of any research proposal. It is one page, the first thing a reviewer reads and often the last. For most foundation reviewers, it is the only thing they will read closely before forming an opinion that is then difficult to move. The report's executive summary from Chapter 18 is its closest sibling, but runs the opposite direction: the executive summary distills work already done; the specific-aims page promises work not yet done and argues that it is worth paying for.

A conventional specific-aims page has four moves. First, a significance paragraph naming the public-interest problem in the vocabulary the funder uses. Second, a gap paragraph naming what is not yet known, or audited, or refused. Third, two or three numbered aims, each with a one-sentence hypothesis or deliverable. Fourth, an impact paragraph naming the actors who will act on the results.

First-time authors miss the first move most often. The significance paragraph is not a general statement about why data matters; it is a claim written in the exact vocabulary of the funder's open call. If the NSF program says "trustworthy AI," your first paragraph says "trustworthy AI." If the Knight Foundation call says "local news ecosystems," your first paragraph says "local news ecosystems." This is not cynicism. It is translation.

The Missing Manual

The specific-aims page is not a summary. It is a sales pitch, written in the vocabulary of the field you are selling to, by a person who has read the funder's last three years of awards and knows what rhymes with them. Rewrite it five times. Rewrite it once in the voice of the program officer's most recent annual-report letter. Rewrite it once as if you were the skeptic on the review panel. Show each draft to a colleague who does not work on your topic and ask them what you are proposing to do and why a funder should care. If they cannot tell you, the page is not done. And: the budget narrative matters more than the budget numbers. A plausible story about personnel time wins over a precise calculation that assumes the work will take half as long as it will actually take. Reviewers have seen the half-time calculation. They are not impressed by it.

19.4. Budgets and justifications

A proposal budget has three components. **Personnel** covers salary and fringe for the people doing the work (PI, co-investigators, graduate students, hourly undergraduates, a contracted community liaison). **Direct costs** cover everything else accounted for line by line (software licenses, compute credits, travel, honoraria, transcription, modest equipment). **Indirect costs**, or facilities and administration (F&A), are overhead the institution charges to keep the lights on. Federal indirect rates at research universities run from roughly 25 to 65 percent of modified total direct costs; foundations often cap indirects at 10 to 15 percent, and some refuse them entirely. Know the rate before you start.

A simple one-year, $50,000 project budget, at a regional foundation that caps indirects at 15 percent, might look like this:

```
import pandas as pd

budget = pd.DataFrame([
    {"category": "Personnel", "item": "PI (0.10 FTE, summer)", "amount": 14000},
    {"category": "Personnel", "item": "Graduate RA (0.25 FTE, 9 mo)", "amount": 15000},
    {"category": "Personnel", "item": "Fringe benefits (22%)", "amount": 6380},
    {"category": "Direct", "item": "Community-liaison honorarium", "amount": 3000},
    {"category": "Direct", "item": "Participant honoraria (10 x $75)", "amount": 750},
    {"category": "Direct", "item": "Compute and storage (AWS / Zenodo)", "amount": 1200},
    {"category": "Direct", "item": "Travel (2 site visits)", "amount": 1500},
    {"category": "Direct", "item": "Publication and open-access fees", "amount": 1650},
    {"category": "Indirect", "item": "F&A at 15% of MTDC", "amount": 6520},
])

totals = budget.groupby("category")["amount"].sum()
print(totals)
# => Direct        8100
#    Indirect      6520
```

```
#    Personnel    35380
#    Name: amount, dtype: int64
print("Total:", budget["amount"].sum())
# => Total: 50000
```

The numbers are not the hard part; the narrative justification is. Every line needs a sentence or two in a companion document saying what the money pays for and why that level of effort is necessary. The graduate RA line is not "0.25 FTE for nine months at the university rate." It is "25 percent effort over nine academic months to conduct the FOIA log coding, build the reproducible pipeline described in Aim 2, and co-author the final community report." A reviewer should come away knowing what each person will actually do on each Tuesday afternoon.

Program officers read the budget narrative before they read the budget. If the narrative is vague, they assume the numbers are vague. Write the narrative first and fit the numbers to it.

19.5. The data management plan

Federal funders require a data management plan (DMP) in every proposal, and foundations increasingly ask for one (California Digital Library 2024). The DMP is not a formality: it tells the funder, in two pages, what data your project will produce, where it will live, who can access it, for how long, and under what conditions. The DMPTool, operated by the California Digital Library, provides templates for most major funders (California Digital Library 2024).

A serviceable DMP covers six sections.

Types of data. What you will collect or generate. Be specific. "Administrative data from the Colorado Department of Human Services under a data use agreement" is specific. "Public records" is not.

Formats and standards. CSV, Parquet, GeoJSON, JSON-LD, whatever applies. If you follow a community standard (HUD's HMIS data dictionary, or the FAIR Guiding Principles (Wilkinson et al. 2016)), name it.

Storage during the project. University-managed storage under a DUA, encrypted at rest, with access restricted to named project personnel.

Retention and destruction. How long you keep the data, and what you do with it at the end. A state-agency DUA typically requires destruction of identifiable data within ninety days of project close.

Sharing. What you will make public, and where. A de-identified derived dataset on Zenodo (CERN 2024) with a CC-BY license and a DOI; a code repository on GitHub linked from the dataset record; a datasheet following Gebru and colleagues (Gebru et al. 2021). If something will not be shared, say why.

Preservation. Where the shared outputs live after you stop paying attention. Zenodo and the Internet Archive provide preservation commitments (Chapter 22 treats these at length); a university institutional repository is another option. A GitHub repo you maintain is not a preservation plan.

The DMP is where you operationalize the installed base for your own project: linkability in formats-and-standards, interpretability in the datasheet, continuity in preservation, safe scrutiny in sharing. Draft with those in mind and the document reads as more than compliance.

19.6. The IRB protocol as a sibling genre

The IRB protocol sits beside the proposal but is not part of it. A proposal is a budget document; an IRB protocol is a risk document arguing the work, if funded, will not harm the humans it touches. Different office, different reader, different conventions.

The framing texts are the Belmont Report (National Commission for the Protection of Human Subjects of Biomedical and Behavioral Research 1978) and the revised Common Rule (US Department of Health and Human Services 2018). Belmont's three principles (respect for persons, beneficence, justice) structure every IRB form you will fill out. Your local IRB (at CU Boulder, the Office of Research Integrity (University of Colorado Boulder Office of Research Integrity 2024)) operationalizes them through a protocol template covering study population, recruitment, consent, data handling, risks, benefits, and conflict-of-interest disclosures.

Public-interest data science frequently works at the edge of the IRB's jurisdiction. An audit of administrative data under a DUA may be "not human subjects research." A community survey unambiguously requires full review. A scrape of public social-media posts lands in a contested middle ground whose answer depends on your institution, your IRB chair, and the year. When in doubt, ask. An IRB that tells you a project is exempt is a better friend than an IRB that finds out about your project in the press.

The simulated protocol you will draft here is a student-level exercise in thinking through consent, risk, and benefit for a project that is probably exempt but worth describing as though it were not.

19.7. Community-engaged research as a proposal-design problem

If the work involves a community partner (a neighborhood association, a legal-aid clinic, a union local, a tribal nation's data office), the proposal has a different shape. The partner is a collaborator, not a subject; their time is an in-kind contribution or a subcontracted line item; their name belongs on the proposal; their data governance sets the terms.

Costanza-Chock's design-justice framing (Costanza-Chock 2020) and the CARE Principles for Indigenous data governance (Carroll et al. 2020) are the two sources most worth reading before you draft a community-engaged proposal. They change the budget (honoraria become line items), the DMP (data-sovereignty provisions become load-bearing), and the aims (the community's question is Aim 1).

19.8. Broader impacts, taken seriously

NSF requires a broader-impacts section; most private foundations require something analogous. First-time authors treat these as boilerplate: we will mentor undergraduates, publish open access, present at a conference. Reviewers read past this text without registering it.

The strong version asks who will be better off if the project succeeds, in a way that would not have happened otherwise. It names community partners, open datasets, reproducible code, training opportunities for students who would not otherwise have had them, and downstream uses the research makes possible. It ties to the ownership question from Chapter 8: whose problem is this, and who gets to act on the answer? And it anticipates the sustainability question from Chapter 14: when the grant money is gone, does the infrastructure continue, or evaporate with your last invoice?

> **In the Public Interest**
>
> The broader-impacts section is where "public interest" becomes concrete and accountable. Do not treat it as boilerplate. Tie it explicitly to **ownership** as you built it in Chapter 8: the community whose data you are using is a partner with a stake in the outcomes, not a population to be studied. Tie it explicitly to **sustainability** as you interrogated it in Chapter 14: a funded pilot that dies when the grant ends has not advanced the installed base; it has displaced

> the attention of staff and community members who will be asked to engage the next funded pilot with less trust. A broader-impacts section that names the partners, names the deliverables they will own at the end, and names the maintenance plan that survives the grant is a broader-impacts section a reviewer will remember. A section that promises mentorship and open-access publication without naming who is mentored, by whom, or where the open-access version will live is a section that reads as filler.

19.9. Exercises

Exercise 19.1 (Structural reading). Find a successful NSF proposal in your subfield. Many NSF awards publish a public abstract and, under FOIA, a redacted version of the full proposal; several repositories (including NSF's own Awards Search) make this accessible. Diagram the specific-aims structure: where does significance appear, where do the aims appear, where does the impact paragraph sit, and how does the language of the aims echo the language of the program solicitation? Write 400 words. Save as `exercises/ch19_nsf_aims_diagram.md`.

Exercise 19.2 (Specific-aims page). Draft a one-page specific-aims document for a one-year, $50,000 research proposal that extends your Part II, III, or IV work (the audit, the refusal specification, the FOIA-tracked investigation, the stakeholder-mapped visualization, whichever is yours). Target a plausible regional funder: the Gates Family Foundation, the Boettcher Foundation, a relevant university seed-grant program. Read three of their recent awards before you write. Use the four-move structure (significance, gap, aims, impact) from this chapter. Rewrite five times. Save the fifth draft as `proposal/specific_aims.md`. This is the first of the chapter's four artifacts.

Exercise 19.3 (Budget and justification). Produce a simple budget spreadsheet and a one-page narrative justification for the project in Exercise 19.2. Use either the pandas example in this chapter or a plain CSV in `proposal/budget.csv`. Include personnel (with FTE and fringe), direct costs, and indirect costs at the funder's actual cap. Write the narrative as if you were the PI explaining the numbers to a skeptical program officer. Save the narrative as `proposal/budget_justification.md`. This is the second artifact.

Exercise 19.4 (Data management plan). Draft a two-page DMP for the project using the DMPTool template for your target funder (California Digital Library 2024). Cover all six sections: types of data, formats and standards, storage during the project, retention and destruction, sharing, and preservation. Name the datasheet standard (Gebru et al. 2021) and the preservation host (Zenodo, Internet Archive, or institutional repository) explicitly. Save as `proposal/dmp.md`. This is the third artifact.

Exercise 19.5 (Simulated IRB protocol). Draft a simulated IRB protocol for the project, covering consent, risk, data handling, benefits, and conflict-of-interest disclosure. Use your university's IRB template if you can find it; otherwise use the simulated template your instructor provides. Ground the consent and risk sections in the Belmont Report's three principles (National Commission for the Protection of Human Subjects of Biomedical and Behavioral Research 1978) and the revised Common Rule (US Department of Health and Human Services 2018). If the project is probably exempt, say so, and explain your reasoning. This is a learning exercise, not a submission. Save as `proposal/irb_protocol.md`. This is the fourth artifact.

Exercises 19.2, 19.3, 19.4, and 19.5 together produce the Part V artifact for this chapter: a specific-aims page, a budget with narrative justification, a data management plan, and a simulated IRB protocol. When all four files are committed, tag the commit `ch19-artifact-v1`. The artifact is the chapter (Keegan 2026).

19.10. Looking ahead

Chapter 20 (Chapter 20) turns the argument you just made in writing into one you can make out loud in five minutes in front of a legislative committee or a regulatory hearing. The specific-aims page has most of the content you will need; testimony will make you cut it to a tenth of its length and speak it to a room. Chapter 22 (Chapter 22) gives the data your funded project produces a permanent address.

19.11. Further Reading and Resources

- NSF *Proposal and Award Policies and Procedures Guide* (PAPPG) (National Science Foundation 2024), https://www.nsf.gov/publications/pub_summ.jsp?ods_key=pappg. The authoritative US federal reference. Skim once cover to cover; reread the specific-aims and broader-impacts sections before every submission.
- DMPTool, California Digital Library (California Digital Library 2024), https://dmptool.org. The simplest way to draft a data management plan that fits the conventions of most major funders.
- The Belmont Report (National Commission for the Protection of Human Subjects of Biomedical and Behavioral Research 1978), https://www.hhs.gov/ohrp/regulations-and-policy/belmont-report/index.html. The framing text for every IRB protocol you will ever write. It is short. Read it.
- NIH Office of Extramural Research (National Institutes of Health 2024), https://grants.nih.gov/. The NIH's grants portal; its policy guide is a useful counterpoint to NSF's PAPPG for projects adjacent to public health.
- Knight Foundation grants page (John S. and James L. Knight Foundation 2024), https://knightfoundation.org/grants/. Representative of the national private-foundation proposal ecology for journalism and civic-technology work.

- Foundation Directory Online (Candid 2024), via most research-library subscriptions. The map of the private-foundation landscape. Spend an afternoon with it before you start writing.
- Gates Family Foundation (Gates Family Foundation 2024) and Boettcher Foundation (Boettcher Foundation 2024) grant pages. Representative Colorado regional foundations; the closest live targets for the specific-aims page you will draft in Exercise 19.2.
- Goldsmith and Kleiman on proposal writing for public-interest technology projects (Goldsmith and Kleiman 2017); Robinson on grassroots grant-writing for small-budget public-interest projects (Robinson 2017). Two very different books, both useful.
- Revised Common Rule, 45 CFR 46 (US Department of Health and Human Services 2018), https://www.hhs.gov/ohrp/regulations-and-policy/regulations/45-cfr-46/. The federal regulations behind every IRB protocol at a covered institution. Skim the subpart A definitions once before you draft Exercise 19.5.
- Your university's research development office and its IRB template. For Colorado Boulder readers, the Office of Research Integrity (University of Colorado Boulder Office of Research Integrity 2024). The single most useful local resource, and the one most students discover too late.

20. Testimony

A hearing room is an unusual venue for data science. The lights are fluorescent. The chairs are bolted to the floor. Your laptop is closed because the committee clerk asked you to close it. Somewhere behind the dais, a staffer who drafted the bill is watching the witness table with an expression you cannot quite read. A red light blinks above the microphone when your five minutes begin. A yellow light warns you at the four-minute mark. You will not get the extra forty-five seconds you wish for.

Testimony is a genre. Like the op-ed in Chapter 17 and the report in Chapter 18, it has conventions you can learn, editors who enforce them, and a specific theory of what counts as contribution. Unlike those genres, it is also a performance, delivered in public, recorded to a transcript, and preserved as part of the legislative record. A well-argued op-ed persuades at scale. A well-argued report becomes institutional evidence. A well-delivered piece of testimony becomes *voice-in-the-record*: a durable, searchable, citeable mark that a particular person said a particular thing to a particular body on a particular day. That is a different kind of artifact than the other two, and the genre's rules follow from that difference.

This chapter treats testimony as a public-record argument. You will write a five-minute oral statement, a longer written addendum for the permanent record, and a set of Q&A responses for hostile questions you can anticipate. The running example is a mock hearing before a Colorado state legislative committee considering an AI transparency or data-broker bill; many state legislatures have been debating variations of this material in the mid-2020s, which makes the scenario both plausible and useful for practice. The mechanics transfer: US Congressional hearings, state committees,

European parliamentary inquiries, and UN Special Rapporteur submissions all share a common structure, even if the clocks and procedures differ.

20.1. What testimony is and is not

An op-ed is persuasion at scale. You do not know your reader; the outlet is the introduction. A report is institutional evidence. You do know your institutional sponsor, and the sponsor's imprint carries you. Testimony is neither. It is voice. A committee invites *you*, specifically, because of who you are and what you can say that someone else cannot. The invitation is political work. Witness selection signals which perspectives a committee wants on the record, and by implication, which it does not. The person who invited you expects you to sharpen a particular argument. The person across the aisle expects you to flinch when a hostile question arrives. Both expectations are correct, and neither is a reason to refuse.

Testimony has four features that distinguish it from the other Part V genres:

1. **Voice.** You speak in the first person. You name your affiliations, your relevant experience, and the claim you are there to make. An op-ed can be abstracted from the author; testimony cannot.
2. **Brevity.** Five minutes is a US Senate and House convention. State committees vary from two to seven. The European Parliament gives longer slots to invited witnesses and shorter ones to public submissions. The UN Special Rapporteur system takes written submissions of several pages but rarely oral testimony from outside the staff. Whatever the clock, it is shorter than you want.
3. **A specific ask.** Testimony that does not end in a specific legislative or regulatory request is not testimony. It is a lecture. Committees do not need lectures. They need someone to say, on the record, "vote yes on the amendment in line 42," or "strike the preemption clause,"

or "the definition of *automated decision system* in Section 2 should include the following four elements."

4. **Q&A.** After the five minutes, members of the committee ask questions. They are not always in good faith. You must answer in complete sentences, without notes, under time pressure. The Q&A is where testimony is won or lost.

Frances Haugen's October 2021 appearance before the Senate Commerce Subcommittee on Consumer Protection is a useful genre case study (Haugen 2021). Read the transcript twice. The first time, notice how much of it is not about Facebook. It is about Haugen. She names her training as a product manager, the specific team she worked on, the internal research she saw, and what her security clearance was. The personal voice is load-bearing. Without it, the claim that Facebook knew about Instagram's effects on teen girls becomes an abstract assertion. With it, the claim is a witness account. The second time you read, mark every sentence that contains a specific ask. There are fewer than you expect, and they are not buried. She wants Section 230 reform, SEC whistleblower protections, and independent research access. The asks are concrete enough that a staffer could draft an amendment from them that afternoon.

Contrast that performance with a congressional oversight hearing where the witness is a CEO who has been subpoenaed. The CEO has been prepped by counsel to say as little as possible. The voice is still personal, but the function is defensive: the record the hearing produces is a set of non-answers a future litigant can use in a deposition. Haugen's testimony is voice-in-the-record as *contribution*. The defensive CEO hearing is voice-in-the-record as *preservation*. Both are recognizable genre performances. You will be writing the first kind, but it helps to know the second exists, because the committee members you face will have sat through many more of the second than the first.

> **In the Public Interest**
>
> Oversight, as Chapter 7 defines it, requires testimony that can become part of the record. A brilliant argument made privately to a staffer is not testimony. It may be more effective, politically, than a public statement, but it leaves no durable mark. The installed base of public-interest work (Chapter 7) depends on a paper trail that future researchers, journalists, and litigants can cite. Written testimony is that paper trail. When you testify in a state committee and your written submission enters the record, you have created a legally preserved document (Chapter 10) that someone six years from now can cite in a brief, a FOIA request, or a dissertation. Chapter 22 will take up the archival deposit that makes sure your version survives even if the committee's website does not.

20.2. The scenario

For the rest of the chapter, imagine a real bill. Colorado SB-XX (substitute any pending number you prefer) would require state agencies to disclose when they use an automated decision system to adjudicate a benefit, permit, or license, and would give affected individuals a right to a human review. You have been invited by the bill's sponsor to testify as an outside expert. You are not a lobbyist. You are a data scientist at a university or a nonprofit who has studied automated decision systems in public-benefits contexts. The committee is the Senate Judiciary Committee. The hearing is at 1:30 p.m. on a Thursday. You are the fourth witness. The chair is sympathetic to the bill but skeptical of anything that sounds like overreach. The ranking minority member is hostile to the bill and will ask a pointed question.

You have been told your oral statement cannot exceed five minutes. You may submit a written statement of any reasonable length. The written

statement is what enters the permanent record.

20.3. A pandas detour that earns one sentence

Before you write either document, you need a concrete fact. Testimony that relies on vibes does not survive Q&A. The fact does not need to be novel; it needs to be yours, defensible, and citable. Here is a small analysis that produces the kind of sentence worth reading aloud.

```
import pandas as pd

# Hypothetical Boulder Housing Choice Voucher processing log
# Fields: applicant_id, surname, application_date, approval_date
df = pd.read_csv("hcv_processing_2023.csv", parse_dates=["application_date", "approval_date"

df["days_to_approval"] = (df["approval_date"] - df["application_date"]).dt.days

# A very rough demographic proxy: Spanish-surname list from US Census
spanish_surnames = pd.read_csv("census_spanish_surnames.csv")["surname"].str.upper().tolist(
df["spanish_surname"] = df["surname"].str.upper().isin(spanish_surnames)

overall_median = df["days_to_approval"].median()
spanish_median = df.loc[df["spanish_surname"], "days_to_approval"].median()
gap = spanish_median - overall_median

print(f"Overall median: {overall_median} days")
print(f"Spanish-surname median: {spanish_median} days")
print(f"Gap: {gap} days")
# => Overall median: 47 days
# => Spanish-surname median: 78 days
# => Gap: 31 days
```

That analysis produces a single sentence you can say out loud: "Our audit found that Boulder's Housing Choice Voucher approval time for Spanish-surname applicants was 31 days longer than the median, and that gap is larger than the federally recommended response window." The sentence is short. It is sourced. It is falsifiable. A hostile member of the committee can ask you to submit the code, and you can submit the code. The written addendum will include a footnote with the repository URL and the commit hash. The oral statement will name the fact once and move on.

This is, incidentally, the clearest answer to the question "why should I learn pandas if my goal is public-interest policy work." Because a single well-sourced sentence in the hearing record is worth more than a thousand words of unsupported assertion, and `pandas` is how you produce the sentence.

20.4. The five-minute oral statement

Here is the structure that works. It is not the only one, but it is the one you should start from and deviate deliberately.

```
[0:00-0:30]  Who you are, one sentence of credentialing,
             one sentence naming the bill and your position.
[0:30-1:30]  A concrete case or finding. Name a person,
             a place, a number. Make the abstraction visible.
[1:30-3:00]  The core argument. Why the current situation
             fails the public interest. Two or three supporting
             beats, not more.
[3:00-4:00]  What the bill does well and what it should fix.
             Specific language where possible.
[4:00-4:45]  Your ask in one sentence, repeated twice if needed.
[4:45-5:00]  Thank the chair. Stop talking.
```

The opening is the hardest to write and the most important to read aloud. Your first sentence should fit on one breath. "Chair Gonzales, Ranking

Member Liston, members of the committee, thank you for the opportunity to testify today on Senate Bill XX" is fine, but only if you then immediately say something specific. Do not spend your first minute reading your CV. You will be cut off.

> **The Missing Manual**
>
> Two things every state-legislative handbook fails to mention. First, the five minutes start when the chair says "you may begin," not when you sit down at the witness table. The hearing clock is not sympathetic to your throat-clearing. Practice your opening sentence out loud, time it, and cut it in half if you need to. Second, the written statement becomes the permanent record; the oral statement does not unless C-SPAN or the state cable channel happens to record it and happens to keep the recording. Treat the written statement as the testimony. Treat the oral statement as the advertisement that makes the committee want to read the written one.

20.5. The written addendum for the record

The written submission is what a staffer will send to the state archivist, what a future researcher will find in LexisNexis, what a journalist will quote next year, and what a lawyer will attach to a brief the year after that. Two to four pages, single-spaced, with footnotes. The oral statement says the thing; the written statement shows the work.

A structure that works for most committees:

1. **Identification.** Name, affiliation, relevant experience, the bill under consideration, and your position in one or two sentences.
2. **Summary of the problem.** Two or three paragraphs. The evidence you have that is relevant to the bill. Cite your sources.

3. **The audit finding (or analogous empirical contribution).** The fact you are there to put on the record. A figure or table if it clarifies. A link to the underlying data and code.
4. **Analysis of the bill.** Section by section. Where the bill's definitions are sound. Where they are too narrow, too broad, or unclear. Proposed amendment language where you have it.
5. **Response to likely objections.** The committee has a record of this debate. Name the most common objection and respond to it in a paragraph. You are saving the chair the trouble.
6. **The ask.** Explicit. A vote. An amendment. A floor statement. An interim study. Something the committee can actually do.
7. **Contact information and availability.** You are offering to follow up in writing.

The written statement is not a draft law review article. It is closer to a legal brief: footnoted, argumentative, and ruthlessly targeted at the decision the committee is about to make.

20.6. Anticipating Q&A

After your five minutes, the chair will thank you. Then the questions begin. Some members will lob softballs to give you another minute on the clock; others will try to discredit you. A three-sentence answer is almost always the right length. Sentence one: direct response to the question. Sentence two: the evidence or reasoning. Sentence three: the connection to your ask or a clean exit.

Here are five question types you can expect at an AI transparency hearing, with a template answer structure for each.

The "are you a lobbyist" question. "Professor, who funded the study you cited?" Disclose the funding source. Name any relevant grants or affiliations. Connect back to why the finding stands regardless.

The "are you overreaching" question. "Wouldn't your proposal shut down every automated system in state government?" Acknowledge the concern. Cite the bill's scope language or your proposed amendment. Name a specific carve-out that addresses the concern.

The "are you the right expert" question. "You're a data scientist, not a lawyer. How do you know this is the right policy?" Acknowledge the genuine limit. Name what your expertise does bring. Name the collaborators or sources whose legal expertise you are drawing on.

The "isn't this already handled" question. "Don't existing due-process protections cover this?" Name the specific gap. Cite a case or a dataset showing the gap. Explain why the bill closes it.

The "will this work" question. "How will agencies actually comply without massive new staffing?" Name a comparable jurisdiction that has implemented something similar. Cite the implementation cost data if you have it. Concede any open question honestly.

The worst thing you can do is answer a question you were not asked. Senators and state legislators can tell the difference, and they will use the pivot against you. If you do not know the answer, say so and offer to follow up in writing. The follow-up letter becomes part of the record.

20.7. Practice delivery and timing

Record yourself delivering the statement. Time it. Almost every first draft runs long. The first cut is usually an adjective you did not know you had written. The second cut is an entire paragraph you liked. The third cut is the transition sentence you kept because it sounded smart. After three cuts, you are under five minutes.

A note on reading versus speaking. Senior lobbyists speak without notes. Most witnesses read from a printed statement. Either is fine. What is not fine is looking down at your laptop throughout. Print the statement. Mark

the breaths in pencil. Underline the one or two phrases you must deliver cleanly.

20.8. Closing and what comes next

Testimony is a small, unusual genre within public-interest data science. Most students will testify once or twice in their careers; a few will testify regularly. For both groups, the habit of *producing the paper trail* matters more than the live performance. The written statement, the audit code, the dataset, and the follow-up letter all become searchable, citable artifacts in a legislative record that outlasts the hearing.

Chapter 21 takes up public comments on federal rulemakings: a related genre with a different clock (30 to 60 days instead of five minutes), a different venue (Regulations.gov instead of a hearing room), and a different kind of reader (a docket officer at an agency, not a senator). Many of the same habits transfer. You will still need a specific ask, a concrete finding, and a written record that survives.

20.9. Exercises

Exercise 20.1 (Guided). Read Frances Haugen's October 2021 Senate Commerce testimony (the transcript is widely available through the committee's archive). Mark every passage where she uses personal voice, every concrete empirical claim, and every specific legislative ask. Count each category. Write 200 words on what the ratio tells you about the genre.

Exercise 20.2 (Core artifact). Write a five-minute oral statement (~750 words) for the mock Colorado hearing scenario described in this chapter, or substitute a real pending bill in your state. Include the introductory sentence, one concrete finding from an analysis you actually run (see the

pandas example in this chapter), and a specific legislative ask. Save it as `artifacts/ch20_oral_statement.md`.

Exercise 20.3 (Core artifact). Write a two-to-four-page written addendum for the permanent record. Follow the seven-part structure above. Include at least three footnotes with real sources, at least one table or figure from your analysis, and proposed amendment language for one section of the bill. Save it as `artifacts/ch20_written_statement.pdf`.

Exercise 20.4 (Core artifact). Draft three-sentence answers to five anticipated hostile questions, using the five question types in this chapter as a starting set. Save them as `artifacts/ch20_qa_prep.md`. Read the answers aloud and confirm that each fits in under twenty seconds.

Exercise 20.5 (Open-ended). Record yourself delivering the oral statement. Time it. If it runs long, cut it down. Once it fits, upload the written version (Exercise 20.3) to the course's Internet Archive collection with a meaningful title, a clear abstract, and at least three topical tags. Submit the `archive.org` item URL. This previews the archival deposit work of Chapter 22, where you will learn the full metadata stack.

20.10. Further Reading and Resources

- Frances Haugen, testimony before the US Senate Committee on Commerce, Science, and Transportation, Subcommittee on Consumer Protection, Product Safety, and Data Security, October 5, 2021. The canonical example for this chapter (Haugen 2021). Watch the full hearing if you can; the Q&A is as instructive as the prepared statement.
- Joy Buolamwini, testimony before the US House Committee on Oversight and Reform on facial recognition technology, May 2019. Paired with the *Gender Shades* research (Buolamwini and Gebru 2018), it is a model of how an audit becomes a hearing statement. TODO: verify canonical transcript URL.

- Inioluwa Deborah Raji, various congressional and state-legislative appearances on AI auditing and accountability, 2020 onward. TODO: confirm specific hearing transcripts; her FAccT work (Raji et al. 2020) is the technical backbone.
- Safiya Umoja Noble, Senate Judiciary Committee comments on algorithmic amplification and platform accountability. TODO: verify specific hearing and date; her book (Noble 2018) is the long form.
- Congress.gov hearings archive: https://www.congress.gov/committees. Browse recent Commerce, Judiciary, and Energy and Commerce hearings for current AI and data-broker testimony.
- Senate.gov testimony archive: https://www.senate.gov/committees/hearings_meetings.htm. Committee-by-committee listings of upcoming and archived hearings.
- UN Office of the High Commissioner for Human Rights, submissions to Special Rapporteurs: https://www.ohchr.org/en/special-procedures-human-rights-council. A different form of testimony, written and international, with its own conventions.
- Common Cause, "How to testify at a hearing": https://www.commoncause.org/. A plain-English guide for first-time witnesses. TODO: pin to current URL.
- Public Citizen's advocacy resources: https://www.citizen.org/. Includes sample testimony and witness-preparation materials.
- Keegan (2026), *Public interest data infrastructuring* (Keegan 2026), on why voice-in-the-record is part of the installed base rather than a rhetorical flourish.

21. Public Comments

Federal rulemaking is where many of the country's most consequential data-policy fights are decided, and almost nobody you know has written a comment on one. The op-ed in Chapter 17 reaches a general reader. The report in Chapter 18 reaches a funder. The testimony in Chapter 20 reaches a committee that invited you in advance. A public comment on a proposed rule reaches the federal agency drafting the rule, on the agency's schedule, in a form the agency is legally required to read, without anyone inviting you first. The door is open. Most of the profession walks past it.

This chapter treats the public comment as a genre. You will read a Notice of Proposed Rulemaking (NPRM), find the specific questions the agency is asking, draft a substantive comment that cites the NPRM's section numbers, and (with instructor approval) submit it through regulations.gov. The running example is one of the mid-2020s' large algorithmic-accountability dockets: the Federal Trade Commission's Commercial Surveillance and Data Security ANPRM, or the National Institute of Standards and Technology's AI Risk Management Framework comment cycles. Both produced thousands of pages of public comment, much of it substantive, all of it now part of the administrative record. Pick either, or find a current docket that overlaps your project; a TODO note later explains how.

21.1. What a public comment is

The Administrative Procedure Act of 1946 is the legal scaffolding under which most federal regulation happens. Its most-cited provision for our

purposes is 5 U.S.C. § 553, "Rule making," which requires that when an agency proposes a substantive rule, it must publish notice in the Federal Register, give the public an opportunity to participate "through submission of written data, views, or arguments," and consider what it receives before issuing a final rule (US Congress 1946). That sentence is short, but the obligations it creates are load-bearing. An agency that skips the public comment step, or that ignores comments raising a "significant" issue, risks having its rule vacated in court as "arbitrary and capricious." The comment period is the mechanism by which federal rules are made legally durable.

The mechanics are standardized. An agency publishes an NPRM in the Federal Register. The NPRM names a docket number, describes the proposed rule, explains the agency's legal authority, summarizes the evidence, and almost always asks the public a series of specific questions. It announces a comment period, typically 30 to 90 days. During that window, anyone, US citizen or not, may submit comments through regulations.gov or directly to the agency. At the close, the agency reviews comments, addresses significant issues in the preamble, and publishes the final rule.

Three features make comments worth your attention. They are written, which suits data scientists. They are on the agency's clock, not yours. And the substantive ones become part of the *administrative record*: the bundle of documents a reviewing court will examine if the rule is later challenged. A comment you wrote in 2024 may be cited in a 2029 opinion. That is an unusual half-life for a weekend's work.

21.2. Why data scientists underuse this genre

Comments are lower-friction than testimony and higher-leverage than op-eds, and the profession submits shockingly few. Three reasons recur.

First, nobody invited you. A congressional committee selects its witnesses; a newspaper commissions its op-eds. Rulemaking is the opposite: the agency publishes the NPRM, and the comment box sits waiting for whoever shows

up. The lack of an invitation is the point. It is also, for a profession trained on peer review, psychologically disorienting.

Second, the form is unfamiliar. Agencies do not publish style guides for comments the way newspapers publish pitch guidelines. You have to figure out on your own that a substantive comment cites the NPRM's section numbers, attaches supporting evidence, and ends with specific proposed changes. The conventions exist; they are just not written down where a student would find them.

Third, the outcome is uncertain. You submit, and the agency either adopts your suggestion, partially adopts it, ignores it, or rejects it in a paragraph of the final rule's preamble. For a profession that likes metrics, a genre whose principal effect is cumulative can feel unrewarding. It is, nonetheless, how most federal regulation is shaped.

21.3. State and local equivalents

Federal rulemaking is the best-documented system, but not the only one. Colorado's state rulemaking process runs through the Secretary of State's office, which publishes the *Colorado Register* and maintains a rules-in-process portal [TODO: confirm canonical URL]. Colorado attorney general opinions, Public Utilities Commission dockets, and Department of Regulatory Agencies rulemakings all follow an analogous notice-and-comment rhythm. City governments run their own processes: Boulder City Council hearings accept written comment before meetings, Denver zoning amendments go through formal periods, Boulder Valley School District policy revisions allow written comment before the board votes. The mechanics differ. The genre is the same: a written, time-bounded, substantive argument submitted into a governmental record.

If federal rulemaking feels too abstract for your project, a state or city comment is a reasonable first artifact. The drafting discipline transfers, and the local audience is often more responsive.

21.4. The substantive comment standard

Agencies distinguish, informally but consistently, between *substantive* and *form-letter* comments. A substantive comment does at least three things: it identifies itself as responding to specific provisions of the NPRM (by section and paragraph), it offers evidence or analysis the agency did not already have, and it proposes specific changes to the rule. A form letter repeats, verbatim, text that a campaign asked supporters to paste into the comment box. Both are counted, but differently. The administrative record treats a thousand identical form letters as evidence of public sentiment; it treats one substantive comment as potentially dispositive evidence the agency must address in the final rule. Courts reviewing a rule for arbitrariness will look for whether the agency engaged with substantive comments. They will not expect the agency to engage, paragraph by paragraph, with a form-letter campaign (Congressional Research Service 2023).

This is good news for the public-interest data scientist. You cannot out-organize a thousand-member advocacy group at volume. You can produce one comment the agency is legally obliged to take seriously, which is a much better return on your weekend.

The Missing Manual

"Comment flooding" is what happens when a campaign tool posts the same text a thousand times under a thousand names. Agencies treat identical or near-identical comments as one comment with a thousand signatories, not a thousand substantive arguments. This cuts both ways: your one carefully written comment is not drowned out, but if your comment looks like a flood entry (no section citations, no evidence, no specifics), it is processed accordingly regardless of your credentials. Separately, the regulations.gov API has changed endpoints and authentication mechanisms several times across the past decade. Before writing code against it, open https://open.gsa.g

ov/api/regulationsgov/ and confirm the current version matches what this chapter describes. TODO for students: verify the documented API at the start of the semester and note divergences in your exercise writeup.

21.5. The regulations.gov API

Regulations.gov is the shared federal portal where most agencies post dockets and collect comments. It exposes a REST API through api.data.gov, which requires a free API key you request by email (US General Services Administration 2024). The three endpoints you will use most are the document search (`/v4/documents`), the docket retrieval (`/v4/dockets/{id}`), and the comment submission endpoint (which has had different names across API versions; verify before writing against it).

The documentation has shifted. Older tutorials point at `/api/v3/documents` or at regulations.gov's internal API, neither of which is the current canonical entry point. Treat the GSA open API documentation as authoritative and any code sample older than a year as provisional.

An illustrative search for open dockets matching a keyword:

```
# TODO: verify endpoint and auth against https://open.gsa.gov/api/regulationsgov/
# before running. The API has changed at least twice since 2020.

import os
import requests

API_KEY = os.environ["REGULATIONS_GOV_API_KEY"]
BASE = "https://api.regulations.gov/v4"

HEADERS = {
```

```
    "X-Api-Key": API_KEY,
    "Accept": "application/vnd.api+json",
    "User-Agent": "PublicInterestDataScience/0.1 (your-email@example.edu)",
}

def search_dockets(keyword, page_size=25):
    params = {
        "filter[searchTerm]": keyword,
        "filter[documentType]": "Proposed Rule",
        "page[size]": page_size,
        "sort": "-postedDate",
    }
    r = requests.get(f"{BASE}/documents", headers=HEADERS,
                     params=params, timeout=30)
    r.raise_for_status()
    return r.json()["data"]

hits = search_dockets("algorithmic accountability")
for h in hits[:5]:
    attrs = h["attributes"]
    print(attrs["docketId"], attrs["title"][:70],
          attrs.get("commentEndDate", "no end date"))
# => FTC-2022-0053 Trade Regulation Rule on Commercial Surveillance ...
# => NIST-2023-0009 AI Risk Management Framework ...
```

The payload uses the JSON:API convention, nesting attributes under `attributes` and relationships under `relationships`. Read it once with `json.dumps(..., indent=2)` before writing parsing code against it.

21.6. Fetching and reading an NPRM

The NPRM is usually a long PDF with a parallel HTML page. The Federal Register publishes a structured XML version easier to parse than the PDF; the regulations.gov API exposes it via the document's `fileFormats` attribute.

```
def fetch_document(document_id):
    r = requests.get(f"{BASE}/documents/{document_id}",
                     headers=HEADERS, timeout=30)
    r.raise_for_status()
    return r.json()["data"]

doc = fetch_document("FTC-2022-0053-0001")
formats = doc["attributes"].get("fileFormats", [])
```

An NPRM has a predictable structure: preamble, background on statutory authority, alternatives analysis, proposed regulatory text, and almost always a numbered list of questions the agency is asking the public to address. Those numbered questions are the most valuable part of the document for a commenter. If the agency asked a question and you answered it with evidence, your comment is prima facie substantive.

A routine to extract question paragraphs from the HTML version:

```
# TODO: the Federal Register's HTML structure varies by document;
# adjust selectors after inspecting the specific docket.

from bs4 import BeautifulSoup
import re

def extract_questions(html_text):
    soup = BeautifulSoup(html_text, "html.parser")
```

```
    questions = []
    for p in soup.find_all(["p", "li"]):
        text = p.get_text(" ", strip=True)
        if re.search(r"\?\s*$", text) and len(text) > 40:
            questions.append(text)
    return questions

# Expect roughly 20-100 explicit questions in a substantive NPRM.
# The FTC commercial surveillance ANPRM asked 95 numbered questions.
```

Ninety-five questions is a lot. You are not obliged to answer all of them. You are asking which two or three overlap your evidence base, and answering those with the best argument you can muster.

21.7. Tracking the docket over time

A rule you are following will accumulate documents over months: the NPRM, comment-period extensions, supplemental notices, industry responses, occasionally a second NPRM. The SQLite tracker from Chapter 10 adapts here with a small extension: add a `rulemakings` table and a `comments` table alongside the CORA schema.

```
import sqlite3
from pathlib import Path

DB_PATH = Path("rulemaking_tracker.sqlite")

SCHEMA = """
CREATE TABLE IF NOT EXISTS rulemakings (
    docket_id TEXT PRIMARY KEY,
    agency TEXT NOT NULL,
```

```
    title TEXT NOT NULL,
    posted_date TEXT,
    comment_end_date TEXT,
    status TEXT NOT NULL
        CHECK (status IN ('open','closed','final','withdrawn')),
    notes TEXT
);
CREATE TABLE IF NOT EXISTS comments (
    id INTEGER PRIMARY KEY,
    docket_id TEXT NOT NULL REFERENCES rulemakings(docket_id),
    submitter TEXT,
    date_submitted TEXT,
    substantive INTEGER,   -- 0 or 1
    url TEXT,
    notes TEXT
);
"""

with sqlite3.connect(DB_PATH) as conn:
    conn.executescript(SCHEMA)
```

Reuse the insert-and-query patterns from Chapter 10. A weekly cron that polls the docket, inserts new documents, and flags rules with `comment_end_date` within 14 days will stop you missing windows. Missing the window is the single most common failure mode for first-time commenters.

21.8. The shape of a substantive comment

A comment has a header, an introduction, a body organized by the sections of the NPRM it addresses, and a conclusion with specific proposed changes. The first page looks like this, roughly:

```
[Your name, affiliation, and contact information]
[Date]

Re: Comment on [Agency] [Docket Number]
    [Full title of the NPRM]
    [Federal Register citation, e.g., 87 Fed. Reg. 51,273]

Introduction

I submit this comment on the [Agency]'s notice of proposed rulemaking
("NPRM") concerning [topic]. I am a [researcher, data scientist,
student] whose work focuses on [area]. I write in my individual
capacity; the views expressed are my own and not those of my employer.
```

The introduction establishes standing, the sections you intend to address, and a one-paragraph summary of your position. The body then addresses specific questions:

```
Response to Question 12 ("How should the Commission define 'automated
decision system' for purposes of this rule?")

The Commission's proposed definition in Section 2(a)(4) excludes
systems that are "advisory" rather than dispositive. This exclusion is
too broad. [Evidence paragraph citing a public audit, a peer-
reviewed
study, or a dataset.] I propose the following revised text:

  (a)(4) "Automated decision system" means any software, including
  machine-learning models, that is used to make or materially support
  a decision affecting a legally protected interest, whether the
  decision is final or advisory.
```

Three things that paragraph does: names the question, cites the section, proposes concrete redlined text. Agency staff can draft a revised section

from that paragraph without interpolation. That is the standard to aim for.

The conclusion restates the proposed changes as a short numbered list and provides contact information for followup. Attach evidence as exhibits. A comment with four exhibits (an audit report, a dataset card, a letter of support, a bibliography) reads as materially different from one with none.

21.9. Coalition comments and individual voice

Many of the most influential comments on the mid-2020s AI rulemakings were coalition submissions: ACLU with EFF and Upturn; the Leadership Conference on Civil and Human Rights with a dozen signatories; academic working groups coordinated through the AI Now Institute or the Center for Democracy & Technology. A coalition comment has institutional weight and authorial dilution. An individual comment has the opposite profile: thinner institutional authority, sharper voice.

Neither dominates. If your argument rests on a technical audit you conducted, your individual signature is load-bearing and you should sign it yourself. If your argument matches thirty other civil-society groups' arguments, your signature adds to a coalition letter. A heuristic: if what you are saying has not been said in this exact form by any current coalition letter, submit it under your own name.

> **In the Public Interest**
>
> A public comment becomes part of the administrative record the moment the agency accepts it. That record is not an archive of opinions; it is the evidentiary base on which future litigation depends. When a rule is challenged in the D.C. Circuit, the court reviews the record for whether the agency adequately considered "significant"

comments. A substantive comment you submit in 2026 may be cited in a 2030 opinion vacating a rule for failure to respond. This is oversight as Chapter 7 defined it: the authority-and-remedy pair wired into the legal architecture of Chapter 10. Your comment is not a speech act but a durable artifact that gives courts, journalists, and scholars a reason to look at what the agency did and did not consider. Write it so it reads well out of context six years from now, because that is when it will be read.

21.10. A note on timing and the regulatory calendar

The comment period is non-negotiable. A comment filed after the window closes is, as a legal matter, absent from the record. Watch the close date. Watch the Federal Register for extension notices (agencies sometimes extend after pressure, occasionally by weeks). Watch for supplemental NPRMs that reopen comment on a narrower issue. ProPublica and the Administrative Conference of the United States maintain rulemaking trackers that surface the high-impact dockets without you scanning the Federal Register daily.

If you are drafting for a live federal docket, verify the docket is still open before you start. TODO for students: confirm your chosen docket has not closed, been withdrawn, or moved to final rule. The FTC commercial surveillance docket and the NIST AI RMF cycles cited here are from the 2022-2024 period and may no longer be open; your instructor will help you pick a current replacement if so.

21.11. Exercises

Exercise 21.1 (Guided). Register for a regulations.gov API key through api.data.gov. Use the search endpoint to find three currently-open dockets

relevant to your project's topic. Record the docket ID, agency, title, and comment end date for each in a short table. Paste the table into your exercise writeup.

Exercise 21.2 (Guided). Pick one of the three dockets from Exercise 21.1 and read the NPRM carefully. Extract every numbered question the agency asks the public. List them in a document, grouped by the section of the NPRM they appear in. Write one sentence per question describing what kind of evidence would be responsive to it. Expect somewhere between 20 and 100 questions; if you find fewer than 10, your extraction is probably incomplete.

Exercise 21.3 (Analytic, Part V artifact). Draft a substantive comment that answers at least two of the questions you identified in Exercise 21.2. Your comment must: (1) identify itself by docket number and NPRM section numbers; (2) cite at least one external source beyond the NPRM, such as a peer-reviewed paper, a public audit, or an agency dataset; (3) propose concrete revised regulatory text; and (4) fall between 800 and 2,500 words. Save the draft to `exercises/ch21_comment.md`. This is your Part V artifact for this chapter.

Exercise 21.4 (Analytic). Retrieve three comments already submitted to your chosen docket. Classify each as (a) substantive, (b) form-letter, or (c) off-topic, using this chapter's substantive-comment criteria. Write 150 words per classification justifying your judgment with quotations from the comment.

Exercise 21.5 (Open-ended, optional). With explicit instructor approval, submit your Exercise 21.3 comment through regulations.gov. Archive the submission confirmation (comment tracking number, date, acknowledgment email) in your exercise folder. Record the URL at which your comment appears on the docket once posted; agencies typically take 1-5 business days. Add the comment to your `rulemaking_tracker.sqlite` database. This exercise is optional because a submitted comment enters the permanent regulatory record under your name.

21.12. Looking ahead

You now have, once Exercise 21.3 is drafted, the fifth of six Part V artifacts: a substantive comment on a live rulemaking, cited to section numbers, with concrete proposed text. Chapter 22 closes Part V and the book. The op-ed, the report, the proposal, the testimony, and the comment are durable only if someone can find them after your employer's website is reorganized, your university decommissions your faculty page, and the agency moves its docket to an archival system nobody remembers how to search. Chapter 22 teaches you to deposit each artifact into archival infrastructure (the Internet Archive and Zenodo) with resolvable identifiers, metadata, and a plan for the next decade. It is the final step in turning a portfolio of work into a public record.

21.13. Further Reading and Resources

- Regulations.gov main portal: https://www.regulations.gov/. The shared federal docket system. Browse open dockets by agency and close date.
- Federal Register's "How to comment effectively" guide: https://www.federalregister.gov/uploads/2011/01/the_publics_guide.pdf. TODO: verify current URL; this is the widely-cited public participation handbook. A short and practical primer on drafting substantive comments.
- Congressional Research Service, *A Brief Overview of Rulemaking and Judicial Review*, CRS Report R41546 (Congressional Research Service 2023). The best short orientation to how notice-and-comment rulemaking produces legally durable federal regulation and where comments fit in the administrative record.
- Regulations.gov API documentation at the GSA open data portal: https://open.gsa.gov/api/regulationsgov/ (US General Services Ad-

ministration 2024). The canonical developer documentation. Verify against the version posted at the start of your semester.

- The Office of Information and Regulatory Affairs (OIRA) at OMB: https://www.reginfo.gov/public/. OIRA reviews "significant" federal rules before publication. Its public dashboard shows which rules are currently under review and where the bottlenecks are.
- ProPublica's rulemaking tracker: https://projects.propublica.org/represent/. TODO: verify canonical URL for the current rulemaking tracker. ProPublica and the Administrative Conference of the United States maintain running indices of high-profile rulemakings.
- FTC Commercial Surveillance ANPRM docket, FTC-2022-0053: https://www.regulations.gov/docket/FTC-2022-0053. An exemplar docket for algorithmic-accountability commenters. Read two or three of the substantive academic comments filed; they are a usable template for your own.
- NIST AI Risk Management Framework comments: https://www.nist.gov/itl/ai-risk-management-framework. A second exemplar docket, useful for students whose evidence base leans toward technical standards rather than enforcement.
- *Administrative Procedure Act*, 5 U.S.C. § 553 (US Congress 1946). The two paragraphs of statute every commenter should have read at least once.

22. Archival Deposits

Every other chapter in Part V has addressed an audience that already exists. The op-ed (Chapter 17) argues to a reader holding tomorrow's paper. The report (Chapter 18) answers a funder in the room. The proposal (Chapter 19) speaks to reviewers convened this fiscal year. The testimony (Chapter 20) addresses a committee that invited you. The public comment (Chapter 21) reaches an agency on a docket open for sixty days. Each of those publics was reachable because each is alive.

An archival deposit is different. You are contributing to a conversation with a public that does not yet exist. A graduate student in 2048 reading old EPA Toxics Release Inventory data to reconstruct an environmental-justice claim. A city staffer in 2035 looking for Colorado groundwater trends that USGS stopped publishing in 2026. A journalist in 2060 writing a retrospective on federal climate portals reorganized during this decade. None of those readers can be pitched to, invited, or surveyed. They will find what you deposited if, and only if, it is still there.

This is the final chapter, and the genre it teaches is the one in which openness, oversight, and ownership (Chapter 6, Chapter 7, Chapter 8) come to rest in a single physical-digital object: a bitstream, a checksum, a resolvable identifier, a provenance record, held by an institution committed to keeping them. Everything the book has built points here. The datasheet from Chapter 4, the cleaned dataset from Chapter 6, the tiered-access matrix from Chapter 8, the category audit from Chapter 16, and the five Part V artifacts can all be archived as one collection. You finish the chapter with that portfolio, deposited and citable.

22.1. Archives and climate, archives and memory

It is tempting to treat archival deposit as a technical afterthought: you did the work, now you put it somewhere. The archival profession has been saying for a decade that this is wrong. Tansey (2015) argues that climate change is an archival problem because the floods, fires, and storms that threaten records are the events that make those records most urgently needed. Mazurczyk and colleagues (2018) extend the argument quantitatively, mapping American archives against projected sea-level rise and showing how much institutional memory sits in places the country's institutions are not planning to protect. Blanke (2024) takes it digital: the Anthropocene is also a problem for server rooms, for migrations across storage media, and for the economics of keeping bits online for a century.

Bluntly: the federal agencies whose datasets document the climate crisis are the same agencies whose data-retention budgets are most exposed to it. An EPA dataset is at risk from two directions. The weather can take the building. A single administration can take the dataset. The archive hedges both.

Ghaddar (2016) and Robinson-Sweet (2018) describe a second horizon: truth-and-reconciliation futures in which publics will want to examine what present governments recorded, classified, suppressed, or destroyed. What you deposit, and what you decline to deposit, is part of what a later public can say about us. Someone will go looking. Either your work is there or it is not.

22.2. The institutional landscape

A "durable" archive is not a synonym for "a server you rent." Durability is an institutional property: a mandate to preserve, a budget funded across political cycles, a succession plan, and federated backup so the loss of one

building does not destroy the collection. A handful of institutions meet that standard. Several more claim to and do not.

The Internet Archive operates the Wayback Machine and a general items repository (Internet Archive 2024). It accepts almost anything, mints durable URLs, and exposes a Python client. Its legal status has been repeatedly challenged in court, a sign of how much it matters.

Zenodo, operated by CERN, accepts datasets, code, and papers up to 50GB, mints a Digital Object Identifier (DOI) on publication, and preserves what it holds on CERN's long-term infrastructure (CERN 2024). A Zenodo DOI is the shortest path from a student laptop to a citable identifier.

Figshare and **OSF** play similar roles; OSF integrates tightly with research-project workflows (Center for Open Science 2024). **ICPSR** at the University of Michigan is the oldest social-science data archive and the standard for survey data and restricted-use materials (Inter-university Consortium for Political and Social Research 2024). **Dryad** is the preferred repository for the life sciences (Dryad 2024). **Dataverse** is a federated platform at dozens of universities; **DASH** at NCAR is the domain archive for atmospheric data.

Software Heritage archives source code at scale (Software Heritage 2024). **LOCKSS** ("Lots of Copies Keep Stuff Safe") is the federated-preservation backbone used by libraries (Stanford University Libraries 2024). **Perma.cc** captures web pages for legal citation (Harvard Library Innovation Lab 2024). **EDGI** coordinates volunteer archiving of federal environmental data at risk of removal (Environmental Data and Governance Initiative 2024).

Notice what is not on this list. GitHub is not a preservation institution; its retention guarantees are a product roadmap, not a mandate. Hugging Face is the same. Google Drive, Dropbox, S3, and personal websites are the same. They can be links in a preservation chain; they cannot be the last link. If the only copy of your dataset lives on a platform whose survival depends on next quarter's earnings, it is not archived. It is stored.

22.3. Federated archiving as political strategy

That list is also a political arrangement. LOCKSS's founding claim is in its name: safety comes from redundancy across independent custodians (Stanford University Libraries 2024). When the EPA's climate pages were reorganized in 2017, EDGI's volunteer "data rescue" events moved copies of at-risk datasets into Internet Archive items before the changes took effect. No single institution was going to do that; a federation did.

An archival deposit is therefore political as well as technical. Depositing the same dataset in two independent repositories, in two jurisdictions, hedges against any one failing, being captured, or being compelled. The exercise below asks you to deposit to both the Internet Archive and Zenodo. That redundancy is not theater; it is the minimum viable federation.

22.4. The running example: EPA Toxics Release Inventory

The Toxics Release Inventory (TRI), mandated by the Emergency Planning and Community Right-to-Know Act of 1986, collects annual self-reports from industrial facilities on releases of listed toxic chemicals to air, water, and land (US Environmental Protection Agency 2024). It is one of the most-cited environmental-justice datasets in the United States and has run continuously since 1987. Like every federal environmental dataset, it is subject to threshold changes, definitional revisions, and occasional quiet removal of historical files when schemas are updated.

Pick a TRI reporting year's facility-level file (the 2022 Basic Data File is small enough for a laptop), download it, keep the original bytes. The workflow below transfers to any at-risk federal environmental dataset: NOAA Climate Data Online, USGS groundwater, EPA AirData, NOAA GHCN-D.

22.5. Preservation-appropriate formats

Before any upload, convert your artifacts into formats still readable in fifty years. This is the highest-leverage decision in the workflow. The Library of Congress's "Sustainability of Digital Formats" registry rates formats on openness, adoption, transparency, and self-documentation (Library of Congress 2024). The short version:

- **Tabular data**: CSV (UTF-8, RFC 4180) as primary; Parquet as companion for large tables. Not Excel; its binary format is semi-proprietary and its date handling is notorious.
- **Documentation**: plain text or Markdown. Not Word. A `.docx` opened in 2045 will render in ways neither you nor your reader can predict.
- **Images**: TIFF or PNG. Not JPEG; every re-save degrades it.
- **Geospatial**: GeoJSON or GeoPackage primary; Shapefile as companion (its 10-character field-name limit truncates columns).
- **Code**: plain-text source files, not notebooks, with an HTML export attached. A `.py` is more likely to parse in 2050 than an `.ipynb`.

You will work harder than you want on this step. Do it anyway. Format choice is the variable most strongly associated with whether your deposit is legible in thirty years.

22.6. Using the Internet Archive client

The `internetarchive` Python client and its `ia` command-line companion are the fastest way to deposit (Internet Archive 2024). Run `ia configure` once to store your credentials, then upload. The item identifier is a short unique string that becomes part of the URL (`https://archive.org/details/<identifier>`). Identifiers are forever; choose one that describes the content.

```
from internetarchive import upload, get_item

metadata = {
    "title": "EPA Toxics Release Inventory, 2022 Basic Data File (mirror)",
    "creator": "Your Name, University of Colorado Boulder",
    "date": "2026-04-23",
    "subject": ["environment", "toxics", "EPA", "TRI",
                "environmental justice"],
    "language": "eng",
    "licenseurl": "https://creativecommons.org/publicdomain/mark/1.0/",
    "description": (
        "Mirror of the US EPA Toxics Release Inventory 2022 Basic "
        "Data File, retrieved from epa.gov on 2026-04-20. See the "
        "attached preservation plan."
    ),
    "mediatype": "data",
    "collection": "opensource",
}

files = ["tri_2022_us.csv", "tri_2022_us.parquet",
         "datasheet.md", "README.md", "LICENSE.txt",
         "preservation_plan.md"]

upload("epa-tri-2022-basic-mirror", files=files, metadata=metadata)
# => [<Response [200]>, <Response [200]>, ...] on success

# Verify the deposit. Compare server SHA-1 to what you computed locally.
item = get_item("epa-tri-2022-basic-mirror")
for f in item.files:
    print(f["name"], f["size"], f.get("sha1"))
```

HTTP 200 does not guarantee the files are actually there. Fetch the item back and confirm each file has non-zero size and a matching SHA-1. That

verification is what most tutorials skip.

> **The Missing Manual**
>
> The `internetarchive` client returns success codes on uploads that silently failed: the item is created, the files are listed with size 0, and the HTTP layer looks fine. Always verify by fetching the item back with `ia metadata <item_id>` or `get_item(id)` and comparing the server's SHA-1 to yours. Separately, Zenodo DOIs take up to an hour to resolve after minting; do not panic when a fresh DOI returns "not found" ten seconds after publishing. The other silent killer is format choice. The difference between a file that survives fifty years and one that does not is usually not storage redundancy but format: CSV beats Excel, TIFF beats JPEG, Markdown beats Word, plain source beats a bundled notebook. Storage is easy. Legibility is hard.

22.7. Minting a DOI with Zenodo

Zenodo's REST API issues a DOI on publication, which is what makes it the right second archive in the federated pair (CERN 2024). Request a personal access token from `zenodo.org/account/settings/applications/`, scope it to `deposit:write` and `deposit:actions`, and treat it like a password.

```
import os, requests

ZENODO = "https://zenodo.org/api"
PARAMS = {"access_token": os.environ["ZENODO_TOKEN"]}
HEADERS = {"Content-Type": "application/json"}

# 1. Create an empty deposition.
r = requests.post(f"{ZENODO}/deposit/depositions",
                  params=PARAMS, headers=HEADERS, json={})
```

```
r.raise_for_status()
dep = r.json()

# 2. Upload files into the deposition bucket.
for path in ["tri_2022_us.csv", "tri_2022_us.parquet",
             "datasheet.md", "README.md", "preservation_plan.md"]:
    with open(path, "rb") as fh:
        requests.put(f"{dep['links']['bucket']}/{os.path.basename(path)}",
                     data=fh, params=PARAMS).raise_for_status()

# 3. Attach Dublin Core metadata.
meta = {"metadata": {
    "title": "EPA Toxics Release Inventory, 2022 Basic Data File (mirror)",
    "upload_type": "dataset",
    "description": "Mirror of the US EPA TRI 2022 Basic Data File...",
    "creators": [{"name": "Your Name",
                  "affiliation": "University of Colorado Boulder"}],
    "keywords": ["environment", "toxics", "EPA", "TRI",
                 "environmental justice", "at-risk data"],
    "license": "cc-zero", "language": "eng",
    "related_identifiers": [
        {"identifier": "https://archive.org/details/epa-tri-2022-basic-mirror
         "relation": "isIdenticalTo", "resource_type": "dataset"},
        {"identifier": "https://www.epa.gov/toxics-release-inventory-tri-prog
         "relation": "isDerivedFrom", "resource_type": "other"},
    ],
}}
requests.put(f"{ZENODO}/deposit/depositions/{dep['id']}",
             params=PARAMS, headers=HEADERS, json=meta).raise_for_status()

# 4. Publish. This mints the DOI and makes the record immutable.
r = requests.post(
    f"{ZENODO}/deposit/depositions/{dep['id']}/actions/publish",
```

```
    params=PARAMS)
print(r.json()["doi"])  # => 10.5281/zenodo.XXXXXXX
```

The `related_identifiers` field is where federation becomes legible. You are telling Zenodo that its record is identical to the Internet Archive item and derived from the EPA source. A future user landing on any of the three can follow the chain to the others. Before going live, test against `sandbox.zenodo.org`: the API is identical, the DOIs are not real, the mistakes are free.

22.8. A preservation plan

A deposit without a preservation plan is a bet that someone else will figure out what you meant. A plan makes the bet explicit by answering four questions.

Retention. How long do you intend the deposit to remain accessible, and under what conditions would it be withdrawn? "Indefinite, absent legal compulsion" is acceptable. So is "Ten years, then re-evaluate." Specify something.

Migration. What happens when the format becomes unreadable? Parquet in 2026 will not be Parquet in 2056. State whether migration is the depositor's responsibility (usually yes for the first generation), the repository's (sometimes yes for major archives), or unresolved. Name a review interval; five years is reasonable.

Custodianship. Who is responsible when you move, change institutions, or leave the work? Karasti and Blomberg (2018) call this "maintenance across decades," and their infrastructure studies (Chapter 5) is the backbone of the answer. Designate a successor, an institution, or a community. "My GitHub account" is not an answer. "The University of Colorado Boulder

libraries' digital preservation program, with EDGI as federated redundancy" is.

Access and refusal. Tie back to the tiered-access matrix from Chapter 8. Open access or tiered (DUA, embargo windows)? Refusal conditions from Chapter 16 under which portions should be withdrawn? Make the rules legible before a future dispute requires them.

A plan that answers those four questions, committed alongside the dataset in the same deposit, is a preservation plan and a governance document. Someone will need it.

> **In the Public Interest**
>
> The public interest includes people who do not yet exist. That is not a metaphor. The graduate student in 2048, the council staffer in 2035, the journalist in 2060: they are the public your deposit serves, and the archive is how you speak to them. This is the chapter where the book's three values come to rest. **Openness** (Chapter 6) lets future readers find the bits, read them, and cite them. **Oversight** (Chapter 7) lets them hold present institutions accountable using records the institutions did not destroy. **Ownership** (Chapter 8) binds the deposit to a custodianship chain that does not evaporate with a funding cycle. The installed base (Chapter 5) was never only about the present; it was always about whether the future could read us.

22.9. Weaving the book together

The previous twenty-one chapters each contribute a piece of the portfolio. The datasheet from Chapter 4 is the provenance document. The cleaning pipeline from Chapter 6 produced the CSV and Parquet versions. The tiered-access matrix from Chapter 8 sits inside the preservation plan.

The tests from Chapter 13 evidence that the pipeline still runs. The sustainability reflection from Chapter 14 is why you chose a federated pair. The category audit from Chapter 16 sits in the README as a note about what the dataset does and does not enumerate.

The five Part V artifacts cluster around the deposit. The op-ed argues the dataset matters. The report is the analysis a future reader will consult. The proposal is the forward-looking plan. The testimony is the record of saying so in public. The public comment is the administrative record. All five, alongside the dataset, archive as one collection. Exercise 22.4 asks you to build it.

22.10. Verifying and citing

A deposit that cannot be cited is a backup. A deposit that can be cited is a contribution. After publication, resolve the DOI and confirm it lands. Paste the Internet Archive URL into a fresh logged-out session and confirm you can download the files. Compute their SHA-256 and compare to what you deposited. Then write the citation in the form your future readers will paste:

> Your Name (2026). *EPA Toxics Release Inventory, 2022 Basic Data File (mirror)* [Dataset]. Zenodo. `https://doi.org/10.5281/zenodo.XXXXXXX`. Also at Internet Archive: `https://archive.org/details/epa-tri-2022-basic-mirror`.

That citation, on your CV and project pages, is the shortest route from "I did the work" to "here is the permanent link." It is the minimum unit of public-interest infrastructure Keegan (2026) describes. Write it.

22.11. Exercises

Exercise 22.1 (Guided). Install the `internetarchive` client and register at archive.org. Pick an at-risk federal environmental dataset (EPA TRI 2022, a NOAA Climate Data Online file, or a USGS groundwater extract). Download it, convert to CSV and Parquet, write a one-page datasheet following Chapter 4, and deposit the bundle as an Internet Archive item with full Dublin Core metadata. Verify with `ia metadata <item_id>` that every file has non-zero size and a matching SHA-1. Submit the item URL and verification output.

Exercise 22.2 (Guided). Create a Zenodo account, request an access token, and test the deposit flow against `sandbox.zenodo.org` before going live. Then deposit to production Zenodo with a `related_identifiers` entry pointing back to your Internet Archive item. Verify the DOI resolves (allow up to an hour). Submit the DOI, the resolved URL, and a note on how long resolution took.

Exercise 22.3 (Reflective). Write a two-to-three-page preservation plan for the dataset. Address retention, migration, custodianship, and access and refusal (tying back to Chapter 8 and Chapter 16). Commit it into the deposit as `preservation_plan.md` and update the metadata to reference it.

Exercise 22.4 (Portfolio synthesis). Archive your op-ed (Chapter 17), report (Chapter 18), proposal (Chapter 19), testimony (Chapter 20), and public comment (Chapter 21) as a coherent collection alongside your dataset deposit. Write a collection-level README introducing the project, summarizing each artifact in two to three sentences, giving full citations with DOIs or permanent URLs, and explaining how the artifacts relate. Deposit to both the Internet Archive and Zenodo. This is your end-of-book portfolio.

Exercise 22.5 (Open-ended). Propose a public-interest data infrastructure that should exist in 2035 but does not. A federated archive of state

benefits-administration audit logs, a repository of municipal algorithmic impact assessments, a climate-resilient mirror of at-risk federal datasets: anything you have wanted to see while working through this book. Three to five pages. Address what it is, what publics it serves, what the first five years of building it look like year by year, what an archival-deposit strategy for its outputs entails, and who bears the maintenance labor across decades. You are not committing to build it; you are demonstrating that you can imagine one and describe what imagining it seriously requires.

22.12. A coda

The book began with three line plots of Wikipedia pageviews and a claim that "public interest" is operational rather than a slogan. It ends with a dataset, a DOI, a preservation plan, and five other artifacts that describe a project you can point to. That is the difference between reading about the practice and entering it.

What you have built, if you did the exercises honestly, is a working theory of yourself as a public-interest data scientist, tested against a named dataset, a real docket, a real funder, a real rulemaking, carrying a citation chain. You will change your mind about parts of it; the practice assumes you will. But you now have a first version, committed to the record, that a future version of you can read and argue with.

There is more work. The federal datasets keep disappearing. The dockets keep opening and closing. The benefits algorithms keep being procured. Someone has to keep doing this, and the book's argument from page one is that "someone" includes you. Not alone, not as a hero, but as a practitioner inside a federation, in conversation with journalists, lawyers, accountants, planners, and engineers, with the critics who sharpen the practice and the refusers who bound it. Do the work. Deposit the artifacts. Cite the chain. Invite the next person in. That is what the installed base needs, and what the next public is counting on.

22.13. Further Reading and Resources

- Internet Archive, *Python library documentation*: https://archive.org/developers/internetarchive/. The canonical reference for the client used throughout this chapter.
- Zenodo, *REST API documentation*: https://developers.zenodo.org. Authoritative for the deposit flow; test in the sandbox at https://sandbox.zenodo.org before going live.
- ICPSR, *Guide to social science data preparation and archiving*: https://www.icpsr.umich.edu/files/deposit/dataprep.pdf. Sixty years of social-science data stewardship, distilled.
- Open Science Framework, *OSF help documentation*: https://help.osf.io/. The project-scaffolded alternative to Zenodo.
- Software Heritage: https://www.softwareheritage.org/. The Internet Archive of source code; if your deposit includes code, cite its SWHID alongside the DOI.
- LOCKSS, *Program documentation*: https://www.lockss.org/. The federated-preservation backbone the chapter's federation claim leans on.
- Environmental Data and Governance Initiative: https://envirodatagov.org/. Volunteer-driven federal environmental archiving; join a data-rescue event.
- Digital Preservation Coalition, *Digital Preservation Handbook*: https://www.dpconline.org/handbook. The practitioner reference for everything this chapter only sketches.
- Library of Congress, *Sustainability of Digital Formats*: https://www.loc.gov/preservation/digital/formats/. The registry that tells you, for any format, how scared to be about using it.
- Perma.cc: https://perma.cc/. For preserving web citations in a form that will resolve when the original URL rots.

A. Appendix A: AI Assistance Disclosure

A.1. What was AI-assisted

Draft zero of this book was produced with substantial assistance from Claude, an assistant made by Anthropic. The assistant generated first-pass prose for all twenty-two chapters and the index, following a detailed editorial prompt written by the author. The author then reviewed, revised, and (in many places) rewrote the output. The book's framework, chapter scope, running examples, and exercise design originate with the author. The prose does not.

This disclosure exists because a book about public interest data science should not hide how it was made. The same principle that asks a model card to document training data asks an author to document drafting practice.

A.2. What was not AI-assisted

The bibliography's selection reflects the author's reading, not the assistant's. The running examples (Boulder and Denver municipal records, a synthetic worker-log fixture, a class-wide FOIA tracker, the regulations.gov walkthrough) are authorial choices grounded in the companion course. The "Missing Manual" and "In the Public Interest" callout convention is the

author's. The Part V genre structure, and particularly its commitment to producing submittable artifacts, is the author's pedagogical argument.

The assistant was asked not to invent citations. Where a suggested reference could not be verified, the bibliography contains a `TODO:` flag rather than a fabricated entry. Readers who find a citation that cannot be located should open an issue.

A.3. How to verify

- The prompt used to generate draft zero is preserved in the repository's issue history under the label `prompt-archive`.
- Each chapter's first commit is the AI-generated draft; subsequent commits are human revisions. Use `git log --follow ch-NN-slug.qmd` to inspect a chapter's editorial history.
- Chapter-by-chapter revision notes are maintained in `claude.md` and in pull-request discussions.

A.4. Why disclose

Because readers deserve to know. Because other authors deciding whether to use similar tools need honest reports of what worked and what did not. Because a book that argues for documentation of automated systems must document its own use of one. And because the professional lineages this book draws on (journalism, law, accounting, planning, engineering) all insist on provenance: whose work, whose hand, whose judgment.

A.5. Versioning

This disclosure was last updated on the date of the most recent commit to this file. See `git log appendix-ai-disclosure.qmd` for the authoritative version history.

B. Appendix B: Further Resources

This appendix collects external resources for readers who want to go further. Each chapter has its own curated "Further Reading and Resources" section; those lists are the place to start. This appendix aggregates the cross-cutting tools, communities, and training venues that appear in more than one chapter, plus a few that did not fit anywhere.

B.1. Cross-cutting tools

- **Datasette** (datasette.io). Simon Willison's toolkit for publishing SQLite databases as inspectable web applications. Used in Chapters 6, 10, 11, and 22.
- **Quarto** (quarto.org). The rendering framework this book is built with. Used implicitly throughout and explicitly in Chapter 18.
- **The Wayback Machine** (archive.org/web). The Internet Archive's web capture service. Used in Chapters 4 and 22.
- **regulations.gov** (regulations.gov). The federal rulemaking docket. Central to Chapter 21.
- **DMPTool** (dmptool.org). Templates for data management plans. Central to Chapter 19.
- **Zenodo** (zenodo.org). CERN's general-purpose research archive. Central to Chapter 22.
- **fairlearn** (fairlearn.org). Python toolkit for disaggregated performance evaluation. Central to Chapters 7 and 11.
- **geopandas** (geopandas.org). Geographic data for pandas users. Central to Chapter 12.

B.2. Communities and networks

- **Public Interest Technology University Network** (PIT-UN)
- **Lines of Code for America** and local Code for X brigades
- **OpenCorporates** and the Sunlight Foundation's archived work
- **The Markup** and **ProPublica** (newsroom exemplars)
- **Data {&} Society**, **AI Now Institute**, **EPIC**, **Upturn**
- **Access Now**, **Derechos Digitales**, **Paradigm Initiative** (non-US counterparts)
- **Te Mana Raraunga** (Māori Data Sovereignty Network)
- **United States Indigenous Data Sovereignty Network** (USIDSN)

B.3. Training venues

- **IRE and NICAR**. Investigative Reporters and Editors / National Institute for Computer-Assisted Reporting.
- **ICPSR Summer Program**. Social-science data training at the University of Michigan.
- **Data Science for Social Good**. Summer fellowships hosted at various universities.
- **FAccT** and **CSCW**. Academic venues where much of this book's source material lands.

B.4. Critical readings you did not meet in this book

This book's framework focuses on records, openness, oversight, and ownership. Readers who want a sharper critique of that framing should start with:

- Simone Browne, *Dark Matters: On the Surveillance of Blackness* (2015)
- Ruha Benjamin, *Race After Technology* (2019)
- Safiya Umoja Noble, *Algorithms of Oppression* (2018)
- Sasha Costanza-Chock, *Design Justice* (2020)
- Audra Simpson, *Mohawk Interruptus* (2014) on refusal as political stance

These texts do not fit neatly into the chapter arc because they do not accept its premises. That is the point of reading them.

B.5. When things change

External links break. Tools deprecate. APIs change. Report a broken link or a deprecated tool by opening an issue against the book's repository. Maintenance of this appendix is the point, not an afterthought.

Aanestad, Margunn, Miria Grisot, Ole Hanseth, and Polyxeni Vassilakopoulou. 2017. "Information Infrastructures and the Challenge of the Installed Base." *Health Informatics Meets eHealth*.

Acker, Amelia, and Adam Kreisberg. 2019. "Social Media Data Archives in an API-Driven World." *Archival Science*.

Ada Lovelace Institute. 2021. *Exploring Legal Mechanisms for Data Stewardship*. Ada Lovelace Institute; AI Council. https://www.adalovelaceinstitute.org/report/legal-mechanisms-data-stewardship/.

Albiston, Catherine R., and Laura Beth Nielsen. 2017. "Public Interest Law Organizations and the Two-Tier System of Access to Justice in the United States." *Law & Social Inquiry* 42 (4): 990–1023.

American Civil Liberties Union. 2019. *Sandvig v. Barr: Researchers' Right to Investigate Algorithmic Discrimination*. ACLU. https://www.aclu.org/cases/sandvig-v-barr-challenge-cfaa-prohibition-uncovering-racial-discrimination-online.

Amnesty International. 2021. *Xenophobic Machines: Discrimination Through Unregulated Use of Algorithms in the Dutch Childcare Benefits Scandal*. Amnesty International. https://www.amnesty.org/en/documents/eur35/4686/2021/en/.

Angwin, Julia, Jeff Larson, Surya Mattu, and Lauren Kirchner. 2016. *Machine Bias: There's Software Used Across the Country to Predict Future Criminals. And It's Biased Against Blacks*. ProPublica. https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing.

Aragon, Cecilia, Shion Guha, Marina Kogan, Michael Muller, and Gina Neff. 2020. "Human-Centered Data Science: An Introduction." *MIT Press*.

Arnstein, Sherry R. 1969. "A Ladder of Citizen Participation." *Journal of the American Institute of Planners* 35 (4): 216–24.

Arruda, Beatriz de, and Sascha van Schendel. 2024. "After SyRI: The Post-Judgment Landscape of Algorithmic Welfare Surveillance in the Netherlands." *European Journal of Risk Regulation*.

Associated Press. 2024. *The Associated Press Stylebook*. Associated Press. https://www.apstylebook.com/.

Aula, Ville, and Tero Erkkilä. 2023. "Digital Governance and the Philanthropic Political Economy of the Data-for-Good." *Information Polity*.

Baker, C. Richard. 2005. "What Is the Meaning of 'the Public Interest'?

Examining the Ideology of the American Public Accounting Profession." *Accounting, Auditing & Accountability Journal* 18 (5): 690–703.

Bender, Emily M. 2023. *Opinion: AI Hype and the Language Model Arms Race*. The New York Times / guest essay.

Bharosa, Nitesh. 2022. "The Rise of GovTech: Trojan Horse or Blessing in Disguise? A Research Agenda." *Government Information Quarterly* 39 (3): 101692.

Blanke, Tobias. 2024. "Digital Archives and the Anthropocene." *Archival Science*.

Boettcher Foundation. 2024. *Boettcher Foundation Grants and Scholarships*. Boettcher Foundation. https://boettcherfoundation.org/.

Bowker, Geoffrey C., and Susan Leigh Star. 2000. *Sorting Things Out: Classification and Its Consequences*. MIT Press.

boyd, danah, and Kate Crawford. 2012. "Critical Questions for Big Data: Provocations for a Cultural, Technological, and Scholarly Phenomenon." *Information, Communication & Society* 15 (5): 662–79.

Boyle, James. 2017. "The Public Domain: Enclosing the Commons of the Mind." *Yale Law Journal*.

Broussard, Meredith. 2018. *Artificial Unintelligence: How Computers Misunderstand the World*. MIT Press.

Broussard, Meredith. 2023. *Opinion: The Bias Baked into AI*. The New York Times / guest essay.

Brown, Alan, Jerry Fishenden, Mark Thompson, and Will Venters. 2017. "Appraising the Impact and Role of Platform Models and Government as

a Platform (GaaP) in UK Government Public Service Reform: Towards a Platform Assessment Framework (PAF)." *Government Information Quarterly* 34 (2): 167–82.

Bruns, Axel. 2019. "After the 'APIcalypse': Social Media Platforms and Their Fight Against Critical Scholarly Research." *Information, Communication & Society* 22 (11): 1544–66.

Buolamwini, Joy, and Timnit Gebru. 2018. "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification." *Proceedings of the 1st Conference on Fairness, Accountability, and Transparency*, 77–91.

California Digital Library. 2024. *DMPTool: Data Management Planning for Researchers*. University of California. https://dmptool.org.

Candid. 2024. *Foundation Directory Online*. Candid. https://fconline.foundationcenter.org/.

Carnegie, Garry D., and Christopher J. Napier. 1996. "Critical and Interpretive Histories: Insights into Accounting's Present and Future Through Its Past." *Accounting, Auditing & Accountability Journal* 9 (3): 7–39.

Carroll, Stephanie Russo, Ibrahim Garba, Oscar L. Figueroa-Rodríguez, et al. 2020. "The CARE Principles for Indigenous Data Governance." *Data Science Journal* 19: 43.

Center for Open Science. 2024. *Open Science Framework Documentation*. Center for Open Science. https://help.osf.io/.

CERN. 2024. *Zenodo REST API Documentation*. CERN. https://developers.zenodo.org.

Chambers, Carter. 2025. "The Trouble with the 'Data-for-Good' Imaginary." *Data & Society*.

Chouldechova, Alexandra, Diana Benavides-Prado, Oleksandr Fialko, and Rhema Vaithianathan. 2018. *A Case Study of Algorithm-Assisted Decision Making in Child Maltreatment Hotline Screening*. Proceedings of the 1st Conference on Fairness, Accountability,; Transparency.

City of Boulder. 2019. *Alpine-Balsam Area Plan*. City of Boulder Planning Department. https://bouldercolorado.gov/projects/alpine-balsam.

Code for America. 2024. *GetCalFresh: A Simpler Way to Apply for CalFresh Food Benefits*. Code for America. https://www.getcalfresh.org/.

Colorado Department of Transportation. 2017. *Central 70 Project: Final Environmental Impact Statement and Record of Decision*. Colorado Department of Transportation and Federal Highway Administration. https://www.codot.gov/projects/i70east.

Colorado Freedom of Information Coalition. 2024. *CORA Resources and Sample-Letter Library*. Colorado Freedom of Information Coalition. https://coloradofoic.org/.

Colorado General Assembly. 1968. *Colorado Open Records Act, c.r.s. § 24-72-201 Et Seq.* Colorado Revised Statutes. https://leg.colorado.gov/colorado-revised-statutes.

Congressional Research Service. 2023. *A Brief Overview of Rulemaking and Judicial Review*. CRS Report R41546. https://crsreports.congress.gov/product/pdf/R/R41546.

Costanza-Chock, Sasha. 2020. *Design Justice: Community-Led Practices to Build the Worlds We Need*. MIT Press.

D'Ignazio, Catherine, and Lauren F. Klein. 2020. *Data Feminism*. MIT Press.

Dadashpoor, Hashem, and Somayeh Ahani. 2021. "Explaining Objective Forces, Driving Forces, and Causal Mechanisms Affecting the Formation and Expansion of the Peri-Urban Areas: A Critical Realism Approach." *Land Use Policy* 102: 105232.

Davidoff, Paul. 1965. "Advocacy and Pluralism in Planning." *Journal of the American Institute of Planners* 31 (4): 331–38.

Delacroix, Sylvie, and Neil D. Lawrence. 2019. "Bottom-up Data Trusts: Disturbing the 'One Size Fits All' Approach to Data Governance." *International Data Privacy Law* 9 (4): 236–52.

Denton, Emily, Alex Hanna, Razvan Amironesei, Andrew Smart, and Hilary Nicole. 2021. "On the Genealogy of Machine Learning Datasets: A Critical History of ImageNet." *Big Data & Society* 8 (2).

Detroit Community Technology Project. 2019. *A Critical Summary of Detroit's Project Green Light and Its Greater Context*. Detroit Community Technology Project. https://detroitcommunitytech.org/?q=content/critical-summary-detroits-project-green-light-and-its-greater-context.

Diakopoulos, Nicholas. 2019. *Automating the News: How Algorithms Are Rewriting the Media*. Harvard University Press.

District Court of The Hague. 2020. *NJCM et al. v. The State of the Netherlands (SyRI Judgment), ECLI:NL:RBDHA:2020:1878*. Rechtspraak.nl. https://uitspraken.rechtspraak.nl/details?id=ECLI:NL:RBDHA:2020:1878.

Dryad. 2024. *Dryad: Publish and Preserve Your Data*. Dryad Digital

Repository. https://datadryad.org/.

Electronic Frontier Foundation. 2024. *Computer Fraud and Abuse Act Reform*. Electronic Frontier Foundation. https://www.eff.org/issues/cfaa.

Environmental Data and Governance Initiative. 2024. *EDGI: Archiving Environmental Data and Documenting Changes to Federal Websites*. EDGI. https://envirodatagov.org/.

Eubanks, Virginia. 2014. "Want to Predict the Future of Surveillance? Ask Poor Communities." *The American Prospect*, January.

Eubanks, Virginia. 2018. *Automating Inequality: How High-Tech Tools Profile, Police, and Punish the Poor*. St. Martin's Press.

Flores, Anthony W., Kristin Bechtel, and Christopher T. Lowenkamp. 2016. "False Positives, False Negatives, and False Analyses: A Rejoinder to 'Machine Bias'." *Federal Probation* 80 (2): 38–46.

Gaffney, Devin, and J. Nathan Matias. 2018. "Caveat Emptor, Computational Social Science: Large-Scale Missing Data in a Widely-Published Reddit Corpus." *PLoS ONE* 13 (7): e0200162.

Garcia, Patricia, Tonia Sutherland, Marika Cifor, et al. 2022. *No: Critical Refusal as Feminist Data Practice*. Proceedings of CSCW 2022.

Gates Family Foundation. 2024. *Gates Family Foundation Grant Programs*. Gates Family Foundation. https://www.gatesfamilyfoundation.org/grants.

Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, et al. 2021. "Datasheets for Datasets." *Communications of the ACM* 64 (12): 86–92.

Geiger, Gabriel, and Justin-Casimir Braun. 2023. *Inside the Suspicion Machine*. Lighthouse Reports / WIRED. https://www.lighthousereports.com/investigation/inside-the-suspicion-machine/.

Ghaddar, J. J. 2016. "The Spectre in the Archive: Truth, Reconciliation, and Indigenous Archival Memory." *Archivaria* 82: 3–26.

GitHub. 2024. *GitHub REST API Documentation*. GitHub Docs. https://docs.github.com/en/rest.

Goldsmith, Stephen, and Neil Kleiman. 2017. *A New City O/S: The Power of Open, Collaborative, and Distributed Governance*. Brookings Institution Press.

Goodman, Bryce, and Julia Trehu. 2023. "AI Audit Washing and Accountability." *SSRN*.

Gray, Jonathan, and Liliana Bounegru, eds. 2021. *The Data Journalism Handbook: Towards a Critical Data Practice*. Amsterdam University Press.

Gray, Mary L., and Siddharth Suri. 2019. *Ghost Work: How to Stop Silicon Valley from Building a New Global Underclass*. Houghton Mifflin Harcourt.

Harvard Library Innovation Lab. 2024. *Perma.cc: Websites Change. Perma Links Don't.* Harvard Law School. https://perma.cc/.

Haugen, Frances. 2021. *Testimony Before the United States Senate Committee on Commerce, Science, and Transportation, Subcommittee on Consumer Protection, Product Safety, and Data Security*. US Senate. https://www.commerce.senate.gov/2021/10/protecting-kids-online-testimony-from-a-facebook-whistleblower.

Ho, Sally, and Garance Burke. 2022. *An Algorithm That Screens for Child Neglect Raises Concerns*. Associated Press. https://apnews.com/article/child-welfare-algorithm-investigation-9497ee937e0053ad4144a86c68241ef1.

Hutchinson, Ben, Andrew Smart, Alex Hanna, et al. 2021. "Towards Accountability for Machine Learning Datasets: Practices from Software Engineering and Infrastructure." *Proceedings of FAccT 2021*.

Institute of Museum and Library Services. 2024. *IMLS Grants*. Institute of Museum and Library Services. https://www.imls.gov/grants.

Internet Archive. 2024. *Internet Archive Python Library Documentation*. Internet Archive. https://archive.org/developers/internetarchive/.

Inter-university Consortium for Political and Social Research. 2024. *ICPSR: Advancing Research Through Data*. University of Michigan. https://www.icpsr.umich.edu/.

Jacobs, Jane. 1961. *The Death and Life of Great American Cities*. Random House.

John S. and James L. Knight Foundation. 2024. *Knight Foundation Grants and Programs*. Knight Foundation. https://knightfoundation.org/grants/.

Karasti, Helena, and Jeanette Blomberg. 2018. "Studying Infrastructuring Ethnographically." *Computer Supported Cooperative Work* 27: 233–65.

Keegan, Brian C. 2026. "Public Interest Data Infrastructuring." *Under Review*.

Kingsbury, Kathleen. 2021. *Why We're Retiring the Term "Op-Ed"*. The New York Times. https://www.nytimes.com/2021/04/26/opinion/nyt-

oped-guest-essay.html.

Lane, Marcus B. 2005. "Public Participation in Planning: An Intellectual History." *Australian Geographer* 36 (3): 283–99.

Library of Congress. 2024. *Sustainability of Digital Formats: Planning for Library of Congress Collections*. Library of Congress. https://www.loc.gov/preservation/digital/formats/.

Mazurczyk, Tara, Nathan Piekielek, Eira Tansey, and Benjamin Goldman. 2018. "American Archives and Climate Change: Risks and Adaptation." *Climate Risk Management* 20: 111–25.

McGuinness, Tara Dawson, and Hana Schank. 2021. *Power to the Public: The Promise of Public Interest Technology*. Princeton University Press.

MuckRock Foundation. 2024. *MuckRock: Public Records Requests, Tracked and Published*. MuckRock Foundation. https://www.muckrock.com/.

National Commission for the Protection of Human Subjects of Biomedical and Behavioral Research. 1978. *The Belmont Report: Ethical Principles and Guidelines for the Protection of Human Subjects of Research*. US Department of Health, Education, and Welfare. https://www.hhs.gov/ohrp/regulations-and-policy/belmont-report/index.html.

National Endowment for the Humanities. 2024. *NEH Grant Programs*. National Endowment for the Humanities. https://www.neh.gov/grants.

National Institutes of Health. 2024. *NIH Office of Extramural Research: Grants and Funding*. National Institutes of Health. https://grants.nih.gov/.

National Science Foundation. 2024. *Proposal and Award Policies and Procedures Guide (PAPPG)*. NSF. https://www.nsf.gov/publications

/pub_summ.jsp?ods_key=pappg.

New America. 2024. *Public Interest Technology University Network*. New America. https://www.newamerica.org/pit/university-network/.

News Leaders Association. 2024. *Sunshine Week*. News Leaders Association. https://sunshineweek.org/.

Nielsen, Laura Beth, and Catherine R. Albiston. 2006. *The Organization of Public Interest Practice, 1975–2004*. University of Chicago Law School.

Noble, Safiya Umoja. 2018. *Algorithms of Oppression: How Search Engines Reinforce Racism*. NYU Press.

Oakes, John B. 1970. *The New York Times Launches the Op-Ed Page*. The New York Times.

Paradigm Initiative. 2024. *Londa: Digital Rights and Inclusion in Africa Report*. Paradigm Initiative. https://paradigmhq.org/londa.

Pasquale, Frank. 2016. *The Black Box Society: The Secret Algorithms That Control Money and Information*. Harvard University Press.

Perrow, Charles. 1984. *Normal Accidents: Living with High-Risk Technologies*. Basic Books.

Petroski, Henry. 1992. *To Engineer Is Human: The Role of Failure in Successful Design*. Vintage Books.

Plantin, Jean-Christophe, Carl Lagoze, Paul N. Edwards, and Christian Sandvig. 2016. "Infrastructure Studies Meet Platform Studies in the Age of Google and Facebook." *New Media & Society* 20 (1): 293–310.

Poynter Institute. 2023. *How to Write an Op-Ed That Gets Published*. Poynter. https://www.poynter.org/.

Raji, Inioluwa Deborah. 2020. *How Our Data Encodes Systematic Racism*. MIT Technology Review. https://www.technologyreview.com/2020/12/10/1013617/racism-data-science-artificial-intelligence-ai-opinion/.

Raji, Inioluwa Deborah, Andrew Smart, Rebecca N. White, et al. 2020. "Closing the AI Accountability Gap: Defining an End-to-End Framework for Internal Algorithmic Auditing." *Proceedings of FAccT 2020*, 33–44.

Rao, Ursula, and Vijayanka Nair. 2019. "Aadhaar: Governing with Biometrics." *South Asia: Journal of South Asian Studies* 42 (3): 469–81.

Reporters Committee for Freedom of the Press. 2024. *Open Government Guide*. Reporters Committee for Freedom of the Press. https://www.rcfp.org/open-government-guide/.

Robinson, Andy. 2017. *Grassroots Grants: An Activist's Guide to Proposal Writing*. Chardon Press.

Robinson-Sweet, Anna. 2018. "Truth and Reconciliation: Archivists as Reparations Activists." *The American Archivist* 81 (1): 23–37.

Schwartz, Joan M., and Terry Cook. 2002. "Archives, Records, and Power: The Making of Modern Memory." *Archival Science* 2: 1–19.

Software Heritage. 2024. *Software Heritage: The Universal Archive of Source Code*. Inria and UNESCO. https://www.softwareheritage.org/.

Stanford University Libraries. 2024. *LOCKSS: Lots of Copies Keep Stuff Safe*. Stanford University. https://www.lockss.org/.

Star, Susan Leigh, and Karen Ruhleder. 1994. "Steps Toward an Ecology of Infrastructure: Complex Problems in Design and Access for Large-Scale Collaborative Systems." In *Proceedings of the 1994 ACM Conference on Computer-Supported Cooperative Work*.

Stoler, Ann Laura. 2002. *Along the Archival Grain: Epistemic Anxieties and Colonial Common Sense*. Princeton University Press.

Stop LAPD Spying Coalition. 2018. *Before the Bullet Hits the Body: Dismantling Predictive Policing in Los Angeles*. Stop LAPD Spying Coalition. https://stoplapdspying.org/before-the-bullet-hits-the-body-a-report-on-predictive-policing/.

Stop LAPD Spying Coalition, and Free Radicals. 2020. *The Algorithmic Ecology: An Abolitionist Tool for Organizing Against Algorithms*. Medium / Stop LAPD Spying Coalition. https://stoplapdspying.org/the-algorithmic-ecology-an-abolitionist-tool-for-organizing-against-algorithms/.

Suzor, Nicolas P., Tess Van Geelen, and Sarah Myers West. 2018. "Evaluating the Legitimacy of Platform Governance: A Review of Research and a Shared Research Agenda." *International Communication Gazette* 80 (4): 385–405.

Swansea, Ben. 2019. *The Highway That Split Denver: Race, Displacement, and the I-70 Rebuild*. Denverite / Colorado Public Radio.

Tansey, Eira. 2015. "Archival Adaptation to Climate Change." *Sustainability: Science, Practice, and Policy* 11 (2): 45–56.

Taylor, Linnet. 2017. "What Is Data Justice? The Case for Connecting Digital Rights and Freedoms Globally." *Big Data & Society* 4 (2).

Taylor, Linnet. 2021. "The Price of Certainty: How the Politics of

Pandemic Data Demand an Ethics of Care." *Big Data & Society* 7 (2).

Te Mana Raraunga. 2018. *Principles of Māori Data Sovereignty*. Māori Data Sovereignty Network. https://www.temanararaunga.maori.nz/tutohinga.

The Op-Ed Project. 2022. *Byline Survey Report*. The Op-Ed Project. https://www.theopedproject.org/byline-report.

The Op-Ed Project. 2024. *The Op-Ed Project Handbook*. The Op-Ed Project. https://www.theopedproject.org/resources.

Thiel, David. 2023. *Identifying and Eliminating CSAM in Generative ML Training Data and Models*. Stanford Internet Observatory. https://purl.stanford.edu/kh752sm9123.

United States Indigenous Data Sovereignty Network. 2024. *USIDSN: Promoting Indigenous Data Sovereignty in the United States*. Native Nations Institute, University of Arizona. https://usindigenousdata.org/.

University of Colorado Boulder Office of Research Integrity. 2024. *Institutional Review Board: Human Subjects Research*. University of Colorado Boulder. https://www.colorado.edu/researchinnovation/irb.

US Congress. 1946. *Administrative Procedure Act, 5 u.s.c. § 553*. United States Code. https://www.law.cornell.edu/uscode/text/5/553.

US Congress. 1966. *Freedom of Information Act, 5 u.s.c. § 552*. United States Code. https://www.law.cornell.edu/uscode/text/5/552.

US Department of Health and Human Services. 2018. *Federal Policy for the Protection of Human Subjects (Common Rule), 45 CFR 46*. Code of Federal Regulations. https://www.hhs.gov/ohrp/regulations-and-

policy/regulations/45-cfr-46/.

US Department of Housing and Urban Development. 2024. *HMIS Data Standards Manual*. HUD Exchange. https://www.hudexchange.info/resource/3824/hmis-data-dictionary/.

US Environmental Protection Agency. 2024. *Toxics Release Inventory (TRI) Program*. US Environmental Protection Agency. https://www.epa.gov/toxics-release-inventory-tri-program.

US General Services Administration. 2024. *Regulations.gov API Documentation*. Api.data.gov. https://open.gsa.gov/api/regulationsgov/.

Ushahidi. 2024. *About Ushahidi: Crowdsourced Crisis Mapping Since 2008*. Ushahidi. https://www.ushahidi.com/about.

Vaithianathan, Rhema, Emily Putnam-Hornstein, Nan Jiang, Parma Nand, and Tim Maloney. 2019. *Developing Predictive Models to Support Child Maltreatment Hotline Screening Decisions: Allegheny County Methodology and Implementation*. Centre for Social Data Analytics, Auckland University of Technology.

Vaughan, Diane. 1996. *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA*. University of Chicago Press.

Warren, Samantha. 2025. *Where Public Records Meet the Public Interest: A Reporter's Field Guide*. Nieman Lab.

Weick, Karl E., Kathleen M. Sutcliffe, and David Obstfeld. 1999. "Organizing for High Reliability: Processes of Collective Mindfulness." *Research in Organizational Behavior* 21: 81–123.

Wilkinson, Mark D., Michel Dumontier, IJsbrand Jan Aalbersberg, et al. 2016. "The FAIR Guiding Principles for Scientific Data Management

and Stewardship." *Scientific Data* 3: 160018.

Wu, Tim. 2021. "The Curse of Bigness: Antitrust in the New Gilded Age." *Columbia Law Review*.

Zeng, Jing, Chung-Hong Chan, and Mike S. Schäfer. 2022. "Contested Chinese Dreams of AI?: Public Discourse about Artificial Intelligence on WeChat and People's Daily Online." *Information, Communication & Society* 25 (3): 319–40.

Zong, Jonathan, and J. Nathan Matias. 2024. "Data Refusal from Below: A Framework for Resisting Institutional Data Harms." *ACM Journal on Responsible Computing*.

Zuiderwijk, Anneke, and Marijn Janssen. 2014. "Open Data Policies, Their Implementation and Impact: A Framework for Comparison." *Government Information Quarterly* 31 (1): 17–29.